TDA 1819: Descripción y simulación de una computadora de 32 bits

Autor: Scorza, Facundo Ricardo

Cátedra: Taller de Arquitectura

Contacto: zorito1994@yahoo.es

Facultad de Informática, UNLP, 50 y 120, La Plata

Supervisor: Villagarcía Wanza, Horacio Alfredo

Índice

1. Introducción 1

2. Entorno de trabajo 5

2.1. Lenguaje de programación 6

2.2. Simulador 8

2.2.1. Instalación 12

2.3. Diseño de la computadora original 22

3. TDA 1819 37

3.1. Arquitectura 45

3.1.1. Atributos generales 45

3.1.2. Directivas y repertorio de instrucciones 54

3.1.3. Códigos de operación e identificadores 60

3.1.4. Formatos de instrucciones 65

3.1.5. Estructura de la memoria principal 70

3.1.5.1. Caso de prueba 86

3.1.6. Ciclo de ejecución: segmentación del cauce 99

3.1.7. Arquitecturas von Neumann y Harvard 119

3.1.8. Riesgos de la segmentación y atascos en el cauce 125

3.1.8.1. Dependencias estructurales 145

3.1.8.2. Lectura Después de Escritura 148

3.1.8.3. Escritura Después de Lectura 163

3.1.8.4. Escritura Después de Escritura 169

3.1.8.5. Riesgos de control 181

3.1.8.6. Atascos de fin del programa 194

3.1.8.7. Programas de prueba 264

3.1.9. Ejemplo de prueba completo 301

3.2. Diseño 309

3.2.1. Modelo jerárquico de entidades 309

3.2.2. Máquina de estados 309

3.3. Programación 309

3.3.1. Ejemplos 309

3.4. Simulación 309

3.4.1. Interfaz de usuario 309

3.4.2. Pruebas generales 309

3.4.3. Estructura de la memoria principal: Ejemplo 309

3.4.4. Riesgos de la segmentación del cauce: Ejemplos 309

3.4.5. Ejemplo de simulación final 309

4. Propuesta de enseñanza 309

4.1. Módulo 1 309

4.2. Módulo 2 309

1. Introducción

Aunque actualmente la arquitectura de computadoras sea uno de los campos de estudio más relevantes de la informática, no existen en la práctica demasiadas implementaciones que describan y simulen efectivamente el comportamiento de un procesador de forma tal que pueda ser comprendido y utilizado por los estudiantes y/o profesionales del área para llevar a cabo distintos experimentos científicos y/o académicos, desde comprobar la respuesta ofrecida por la CPU durante la ejecución de diversos programas escritos en el lenguaje Assembler hasta incorporar nuevas funcionalidades a la PC que aún se encuentren pendientes de ser incluidas, ya sea algún periférico de entrada/salida o bien alguna mejora que pueda repercutir favorablemente sobre el rendimiento del procesador (segmentación, unidad de punto flotante, etc.).

Como un ejemplo de simulador existente en este campo y utilizado en el ámbito académico con fines educativos se puede mencionar el MSX88, concebido en el año 1985 en la Escuela Universitaria de Ingeniería Técnica de Telecomunicación de la Universidad Politécnica de Madrid como respuesta a las necesidades docentes en el área de la Arquitectura de Ordenadores. El procesador utilizado consiste en una versión simplificada de la CPU 8088 de Intel: arquitectura interna de 16 bits y externa de 8 bits, bus de direcciones de 16 bits, arquitectura de tipo CISC (Repertorio Complejo de Instrucciones). Otro caso podría ser el WinMIPS64, el cual posee una finalidad muy similar al anterior, pero presenta una drástica diferencia respecto a la arquitectura del procesador utilizado para la simulación, ya que aquí la CPU descripta es un MIPS64 con una arquitectura de 64 bits y de tipo RISC (Repertorio Reducido de Instrucciones). No cabe ninguna duda de que, con sus contrastes, ambos simuladores han cumplido su propósito exitosamente ya que, con algunas actualizaciones y modificaciones, aún al día de la fecha siguen siendo empleados por profesores universitarios de esta rama para introducir a los alumnos a la programación de una CPU a bajo nivel.

A modo recordatorio se presentan a continuación en modo sintético las principales diferencias entre una arquitectura CISC y una RISC. Se debe tener en cuenta que no existen procesadores cuyo diseño represente de manera fiel y pura ninguno de los dos tipos de arquitectura, ya que la mayoría es en realidad una mezcla de ambos, aunque siempre con más características de uno que de otro:

Tabla 1. Comparación entre las arquitecturas CISC (Repertorio Complejo de Instrucciones) y RISC (Repertorio Reducido de Instrucciones).

Arquitectura / Característica	CISC	RISC
Ciclos de ejecución	Cada instrucción puede requerir un número variable de ciclos de reloj para ejecutarse	Cada instrucción requiere exactamente un único ciclo de reloj para ejecutarse
Tipo de diseño	Diseño centrado en el hardware: la unidad de control realiza la mayor parte del trabajo	Diseño centrado en el software: los compiladores de alto nivel realizan la mayor parte del trabajo
Uso de la memoria RAM	Utilización eficiente del espacio disponible en la memoria RAM (menor cantidad de instrucciones por programa)	Ineficiente utilización del espacio disponible en la memoria RAM (pueden producirse cuellos de botella si la RAM es limitada)
Interfaz de Entrada/Salida	E/S independiente: espacios de direcciones y repertorios de instrucciones separados para memoria y E/S	E/S mapeada en memoria: memoria y E/S comparten un único espacio de direcciones y repertorio de instrucciones
Formato y longitud de las instrucciones	Instrucciones complejas de longitud variable	Instrucciones sencillas y estandarizadas de longitud fija
Microprogramación	Puede soportar microcódigo (microprogramación: las instrucciones complejas son tratadas como pequeños programas)	Sólo se permite una única capa de instrucciones
Cantidad de instrucciones	Gran número de instrucciones	Pequeña cantidad de instrucciones
Modos de direccionamiento	Modos de direccionamiento complejos	Número reducido de modos de direccionamiento
Cantidad de registros	Cantidad reducida de registros de almacenamiento	Elevado número de registros de almacenamiento

Segmentación	La implementación de la segmentación es complicada	La implementación de la segmentación es sencilla
Programación	La codificación de los programas es más sencilla	Los programas requieren un mayor número de líneas de instrucciones
Almacenamiento en memoria	Little Endian: los bytes se almacenan desde el menos significativo en la dirección más baja hacia el más significativo en la más alta	Big Endian: los bytes se almacenan desde el más significativo en la dirección más baja hacia el menos significativo en la más alta

Ambos simuladores mencionados anteriormente resultan extremadamente útiles para introducir a los estudiantes que están en las etapas iniciales de su carrera a la programación de una CPU y al funcionamiento de esta última durante la ejecución de diversos programas. No obstante, cuando el docente se encuentra en la presencia de alumnos más avanzados, su objetivo ahora es que los alumnos no se comporten como meros espectadores del comportamiento del procesador sino que participen aún más activamente en él, conociendo interiormente el código que se encuentra detrás del funcionamiento de cada uno de los componentes y de su interacción entre sí a fin de poder modificarlo para mejorar el rendimiento de la CPU incorporando nuevas técnicas aún no implementadas o bien agregar más funcionalidades a la PC con la inclusión de periféricos de E/S adicionales todavía faltantes en la misma.

En este apartado es donde debe destacarse la importancia de los lenguajes de descripción de hardware (HDLs, Hardware Description Languages), utilizados desde los años 70 en los ciclos de diseño de sistemas digitales asistidos por herramientas de CAD electrónico. En los años 80 aparecen los lenguajes Verilog y VHDL que, aprovechando la disponibilidad de herramientas hardware y software cada vez más potentes y asequibles y los adelantos en las tecnologías de fabricación de circuitos integrados, logran imponerse como herramientas imprescindibles en el desarrollo de nuevos sistemas. Estos lenguajes son sintácticamente similares a los de programación de alto nivel –Verilog tiene una sintaxis similar al C y VHDL a ADA– y se diferencian de éstos en que su semántica está orientada al modelado del hardware. Su capacidad para permitir distintos enfoques en el modelado de los circuitos y su independencia de la tecnología y metodología de diseño permiten extender su uso a los distintos ciclos de diseño que puedan utilizarse. Por ello, para los profesionales relacionados de alguna

manera con el diseño o mantenimiento de sistemas digitales resulta hoy en día imprescindible su conocimiento.

A partir de la información anterior se puede concluir que, al tratarse efectivamente la PC de un sistema digital, su diseño puede ser perfectamente descripto mediante cualquiera de estos lenguajes de manera tal de obtener una representación de la misma tan fiel como se desee sin requerir en un principio ningún tipo de hardware en particular para comprobar su funcionamiento. Por lo tanto, sería extremadamente útil contar con un simulador que posea características similares a aquéllos mencionados anteriormente pero que se encuentre escrito en algún lenguaje de descripción de hardware de manera tal que, además de permitir a los alumnos codificar programas en Assembler para ejecutarlos en una CPU y comprobar el funcionamiento de esta última, no los limite a contemplar la actividad del procesador de forma externa y pasiva a través de una interfaz gráfica sino que les ofrezca la oportunidad de analizar internamente todos los componentes y señales involucrados para que puedan afianzar todos los conocimientos adquiridos en cátedras anteriores vinculadas con este campo y comprender su relevancia si se desea llevar a cabo un diseño para describir una PC o CPU absolutamente original o bien mejorar el desempeño de una ya existente.

A pesar de que existen en la actualidad diversas propuestas de descripciones e implementaciones para distintos procesadores escritas en alguno de estos lenguajes de descripción de hardware, aún ninguna de ellas es tan completa ni fiable como para ser utilizada de manera universal en el ámbito académico de la manera en que sí lo son los dos simuladores previamente mencionados.

Por lo tanto, se propuso como objetivo para este proyecto desarrollar el diseño de una PC a través de un HDL basado en las descripciones ofrecidas por dichos simuladores de forma tal que los estudiantes más avanzados puedan implementar programas en el lenguaje Assembler para ejecutarlos en la CPU tal como lo hacían en cátedras anteriores en el MSX88 y el WinMIPS64 pero además sean capaces de escribir código en un lenguaje de descripción de hardware para mejorar el rendimiento del procesador o incorporar nuevas funcionalidades tanto a la CPU como a la computadora que la contiene.

Asimismo, la escritura de este informe fue llevada a cabo considerando que los alumnos para quienes el proyecto aquí desarrollado se encuentra orientado, así como ya poseen alguna noción previa de la semántica y sintaxis de Assembler, un lenguaje de programación de bajo nivel, también han interactuado durante el transcurso de al menos una de las asignaturas precedentes del plan de estudios con algún lenguaje de descripción de hardware, preferentemente el VHDL, a fin de definir el diseño de diversos circuitos digitales, aunque

indudablemente de una menor complejidad comparada con la dimensión de la PC ideada para este proyecto. A su vez, se tuvo en cuenta que, con el objetivo de comprobar el funcionamiento de los circuitos digitales diseñados, los estudiantes ya están además familiarizados con la instalación y el manejo de alguno de los simuladores ya existentes y conocidos para, tal como su nombre indica, simular el comportamiento de dichos circuitos: ModelSim, ISim (Xilinx ISE Simulator), QSim (Quartus II Simulator) o, preferiblemente, el Active-HDL. La razón que motiva la recomendación de una experiencia anterior, aunque sea básica, en el uso del lenguaje VHDL y el simulador Active-HDL en particular, se comprenderá en la próxima sección del informe, "2. Entorno de trabajo", en la cual se describirán pormenorizadamente, entre otros temas, todas las herramientas optadas para diseñar, desarrollar e implementar la nueva computadora.

Finalmente, procederá a presentarse formalmente la denominación seleccionada tanto para la PC como para la CPU diseñadas: "TDA 1819", TDA por la cátedra para la cual este proyecto fue inicialmente concebido (Taller de Arquitectura) y 1819 por el período durante el cual esta primera versión fue desarrollada (2018-2019).



Figura 2. Logo diseñado para la PC y la CPU del proyecto:
denominación "TDA 1819" (MDCCCXIX en números romanos).

2. Entorno de trabajo

En esta sección se describirán tanto el lenguaje de programación como el simulador utilizados para describir el diseño desarrollado para la PC, junto con la descripción que había sido originalmente seleccionada como punto de partida para el mismo, así como también los motivos que llevaron al autor a determinar tales decisiones. Este contenido es simplemente de carácter introductorio en caso de que el usuario aún no se encuentre familiarizado con el uso

del lenguaje de descripción de hardware y/o el simulador particularmente elegidos para este proyecto.

2.1. Lenguaje de programación

Como cabría esperarse a partir de todos los conceptos desarrollados anteriormente se optó por utilizar un lenguaje de descripción de hardware para describir el diseño de la PC. El lenguaje seleccionado en particular para este proyecto es aquél conocido como VHDL (VHSIC Hardware Description Language; VHSIC = Very High Speed Integrated Circuit). Los estudios para la creación de este lenguaje comenzaron en el año 1981, bajo la cobertura de un programa para el desarrollo de Circuitos Integrados de Muy Alta Velocidad (VHSIC), del Departamento de Defensa de los Estados Unidos. En 1983 las compañías Intermetrics, IBM y Texas Instruments obtuvieron la concesión de un proyecto para la realización del lenguaje y de un conjunto de herramientas auxiliares para su aplicación. Finalmente, en el año 1987, el lenguaje VHDL se convierte en la norma IEEE-1076 –como todas las normas IEEE, se somete a revisión periódica, por lo que en 1993 sufrió algunas leves modificaciones–.

El lenguaje VHDL fue creado con el propósito de especificar y documentar circuitos y sistemas digitales utilizando un lenguaje formal. En la práctica se ha convertido, en un gran número de entornos de CAD, en el HDL de referencia para realizar modelos sintetizables automáticamente. Las principales características del lenguaje VHDL se explican en los siguientes puntos:

- Descripción textual normalizada: El lenguaje VHDL es un lenguaje de descripción que especifica los circuitos electrónicos en un formato adecuado para ser interpretado tanto por máquinas como por personas. Se trata además de un lenguaje formal, es decir, no resulta ambiguo a la hora de expresar el comportamiento o representar la estructura de un circuito. Está, como ya se ha dicho, normalizado, o sea, existe un único modelo para el lenguaje, cuya utilización está abierta a cualquier grupo que quiera desarrollar herramientas basadas en dicho modelo, garantizando su compatibilidad con cualquier otra herramienta que respete las indicaciones especificadas en la norma oficial. Es, por último, un lenguaje ejecutable, lo que permite que la descripción textual del hardware se materialice en una representación del mismo utilizable por herramientas auxiliares tales como simuladores y sintetizadores lógicos, compiladores de silicio, simuladores de tiempo, de cobertura de fallos, herramientas de diseño físico, etc.

- Amplio rango de capacidad descriptiva: El lenguaje VHDL posibilita la descripción del hardware con distintos niveles de abstracción, pudiendo adaptarse a distintos propósitos y utilizarse en las sucesivas fases que se dan en el desarrollo de los diseños. Además, es un lenguaje adaptable a distintas metodologías de diseño y es independiente de la tecnología, lo que permite, en el primer caso, cubrir el tipo de necesidades de los distintos géneros de instituciones, compañías y organizaciones relacionadas con el mundo de la electrónica digital; y, en el segundo, facilita la actualización y adaptación de los diseños a los avances de la tecnología en cada momento.
- Otras ventajas: Además de las ventajas ya reseñadas también es destacable la capacidad del lenguaje para el manejo de proyectos de grandes dimensiones, las garantías que comporta su uso cuando, durante el ciclo de mantenimiento del proyecto, hay que sustituir componentes o realizar modificaciones en los circuitos, y el hecho de que, para muchas organizaciones contratantes, sea parte indispensable de la documentación de los sistemas.

Se puede mencionar adicionalmente que se posee el conocimiento de que los alumnos de la cátedra para la cual fue inicialmente desarrollado este proyecto ya han programado en este lenguaje para diseñar circuitos digitales en asignaturas anteriores de la carrera. Por lo tanto, cuentan con la capacidad para ponerse a estudiar y trabajar directamente sobre el código ya disponible para describir la PC sin tener que atravesar una etapa previa de aprendizaje que sería imprescindible en caso contrario al tratarse de un diseño tan complejo como éste, aprovechándose así más eficientemente el tiempo disponible de las clases para que los estudiantes puedan alcanzar los objetivos propuestos para la cátedra.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity signed_adder is
  port
  (
    aclr : in   std_logic;
    clk  : in   std_logic;
    a    : in   std_logic_vector;
    b    : in   std_logic_vector;
    q    : out  std_logic_vector
  );
end signed_adder;

architecture signed_adder_arch of signed_adder is
  signal q_s : signed(a'high+1 downto 0); -- extra bit wide

begin -- architecture
  assert(a'length >= b'length)
    report "Port A must be the longer vector if different sizes!"
    severity FAILURE;
  q <= std_logic_vector(q_s);

  adding_proc:
  process (aclr, clk)
    begin
      if (aclr = '1') then
        q_s <= (others => '0');
      elsif rising_edge(clk) then
        q_s <= ('0'&signed(a)) + ('0'&signed(b));
      end if; -- clk'd
    end process;

end signed_adder_arch;
```

Figura 3. Código escrito en el lenguaje VHDL para describir el comportamiento de un sumador con signo.

2.2. Simulador

El simulador elegido para comprobar el funcionamiento del diseño desarrollado para la PC es el Active-HDL, una solución integrada de creación y simulación de diseños desarrollada por la compañía Aldec para FPGA (Matrices de Compuertas Programables) orientada a entornos de trabajo en equipo y concebida para ser instalada sólo en el sistema operativo Microsoft Windows (XP/2003/Vista/7/8/10 32/64 bits). El IDE (Entorno de Diseño Integrado) incorporado en el Active-HDL incluye un conjunto completo de herramientas dedicadas para los diseños gráficos y la programación en lenguajes de descripción de hardware, junto con un simulador de lenguaje mixto de conexiones de compuertas lógicas para un rápido despliegue y verificación de diseños para FPGA. El gestor para el flujo del diseño invoca más de ciento veinte herramientas para EDA (Automatización de Diseños Electrónicos) y FPGA durante los flujos de diseño de entrada, simulación, síntesis e implementación y les permite a los distintos

equipos permanecer dentro de una única plataforma común a través de la totalidad del proceso de desarrollo para la FPGA. El Active-HDL ofrece soporte para los dispositivos FPGA punteros de las compañías Altera, Atmel, Lattice, Microsemi (Actel), Quicklogic y Xilinx, entre otras.

Algunas de sus principales ventajas incluyen:

- Gestor de diseño unificado para entornos de trabajo en equipo.
- Despliegue ágil y veloz de diseños con editores de texto, esquemático y máquina de estados.
- Simulador potente de lenguaje mixto de kernel común (VHDL, Verilog, SystemVerilog (Diseño), SystemC).
- Depuración (debugging) avanzada y cobertura de código.
- Verificación basada en aseveraciones: SVA (SystemVerilog Assertions), PSL (Property Specification Language), OVA (OpenVera Assertions), etc.
- Co-simulación DSP (Procesamiento Digital de Señales) con interfaz MATLAB/Simulink.
- Posibilidad de compartir diseños de una manera rápida con la herramienta de Autogeneración de Documentación del Diseño en los formatos HTML y PDF.



Figura 4. Logo utilizado por el simulador Active-HDL.

Con respecto a la tarea del diseño, el Active-HDL utiliza métodos gráficos y textuales para el Design Entry (Entrada del Diseño) e integra más de ciento veinte herramientas para EDA en una plataforma única. Las herramientas para la gestión de diseños colaboran para eliminar los problemas enfrentados por los entornos basados en trabajo de equipo durante el proceso de desarrollo del diseño para la FPGA.

Este simulador también incorpora un simulador de lenguaje mixto de kernel común con herramientas interactivas que les permite a los diseñadores una rápida depuración del código escrito para el diseño. Las herramientas de depuración como el Advanced Data Flow (Flujo de Datos Avanzado) y el Xtrace proveen a los usuarios una representación gráfica de las señales

internas del sistema, incrementando así la visibilidad y asistiendo en la depuración de diseños complejos. El Active-HDL además incluye herramientas para el análisis y la cobertura de código, permitiendo a los diseñadores incorporar una verificación orientada a métricas dentro del proceso de diseño.

Finalmente, el Active-HDL les posibilita a los diseñadores documentar de una manera veloz todos los aspectos del espacio de trabajo de su diseño para su posterior revisión, reutilización y preservación. Esto a su vez facilita la capacidad para mantener una documentación apropiada de todas las etapas del proceso de desarrollo, eliminándose así muchos inconvenientes surgidos por los entornos de diseño que involucran el trabajo simultáneo de múltiples equipos.

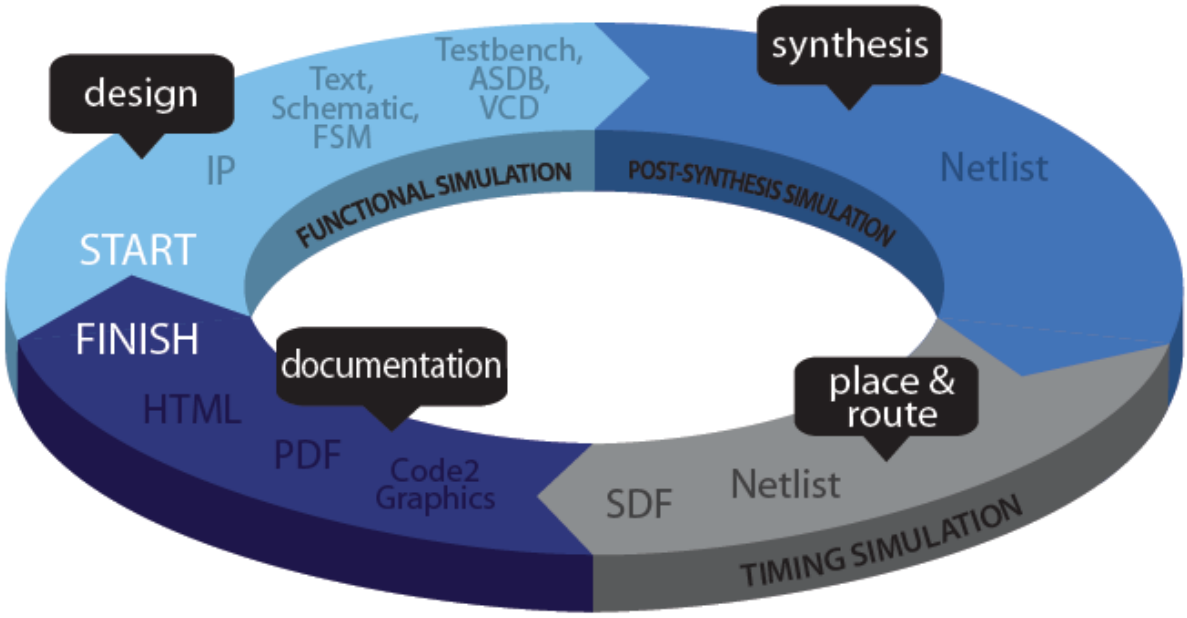


Figura 5. Flujo de Diseño para FPGA.

A continuación, se incluyen algunas definiciones para facilitar la comprensión del gráfico anterior:

- IP (Intellectual Property): Un núcleo o bloque IP (Intellectual Property) es un plan de diseño reutilizable de una unidad lógica, celda o circuito integrado (comúnmente denominado "chip") cuya propiedad intelectual pertenece a sólo un ente. Los núcleos IP pueden ser utilizados como bloques fundacionales dentro de diseños para circuitos integrados para aplicaciones específicas (ASICs) o FPGAs.

- FSM (Finite State Machine): Se denomina máquina de estados a un modelo de comportamiento de un sistema con entradas y salidas, en donde las salidas dependen no sólo de las señales de entradas actuales sino también de las anteriores. Las máquinas de estados se definen como un conjunto de estados que sirve de intermediario en esta relación de entradas y salidas, haciendo que el historial de señales de entrada determine, para cada instante, un estado para la máquina, de forma tal que la salida depende únicamente del estado y las entradas actuales. Una máquina de estados se denomina máquina de estados finitos (FSM por finite state machine) si el conjunto de estados de la máquina es finito, este es el único tipo de máquinas de estados que podemos modelar en un computador en la actualidad; debido a esto se suelen utilizar los términos máquina de estados y máquina de estados finitos de forma intercambiable.
- ASDB (A Simulation Database): El Active-HDL almacena datos de las simulaciones en un archivo ASDB. Este archivo contiene el historial de los valores de algunas señales particulares y jerarquías del diseño cuyo comportamiento fue seleccionado para ser estudiado durante el transcurso de la simulación. Para ver el contenido del archivo ASDB debe encontrarse vinculado con un archivo AWC (A Waveform Configuration), el cual a su vez contiene información respecto a la lista de los objetos que son representados, el color, el sistema numérico de representación, el orden, etc.
- VCD (Value Change Dump): Es un formato de archivo especificado en el estándar IEEE 1364-1995. El archivo VCD consiste en realidad de un archivo ASCII que contiene información de cabecera, definiciones de variables y cambios en los valores de dichas variables. Los archivos VCD almacenan información sobre cambios de valores durante las simulaciones realizadas para redes y registros. Pueden exportarse formas de onda (“waveforms”) conteniendo señales VHDL a un archivo VCD.
- Netlist: Es una descripción de la conectividad de un circuito electrónico. En su forma más simple, consiste en una lista de los componentes electrónicos de un circuito junto con otra lista de los nodos a los cuales se encuentran conectados. Una red (“net”) es una colección de dos o más componentes interconectados. La estructura, complejidad y representación de las netlists puede variar considerablemente, pero el propósito fundamental de cada netlist es expresar información sobre conectividad. Las netlists usualmente proveen únicamente instancias, nodos y quizás algunos atributos de los componentes involucrados.
- SDF (Standard Delay File): En las etapas iniciales del diseño se llevarán a cabo simulaciones “funcionales” para comprobar que la lógica en el circuito desarrollado

funcione correctamente. Después de que las herramientas FPGA seleccionadas logren generar una disposición propiamente dicha para el diseño de las compuertas lógicas, podría ser recomendable llevar a cabo una simulación final con información sobre retardos de propagación generada durante el proceso de disposición de las compuertas. Esta información es utilizada frecuentemente como una verificación final para comprobar que los retardos inesperados generados durante el proceso de disposición no creen violaciones de tiempo en el diseño. Las herramientas de disposición crearán un archivo SDF que incluya esta información. Al incluir esta última, el modelo puede ser puesto a prueba basándolo en estos retardos de propagación.

- Code2 Graphics: CODE-2 (Computador Didáctico Elemental, versión 2), y su predecesor ODE son dos procesadores ideados por los profesores Alberto Prieto y Antonio Lloris, de la Universidad de Granada, para facilitar la compresión del funcionamiento y el diseño hardware de un computador. Ambos procesadores son de tipo RISC, con un formato de instrucciones completamente regular, y con 16 instrucciones máquina. La longitud de palabra (tanto de la ALU como de la memoria) de ODE es de 12 bits, mientras que la de CODE-2 es de 16 bits. La concepción y diseño de estos ordenadores se realizó con el objetivo de que el modelo pudiese utilizarse para enseñanza del lenguaje máquina, del lenguaje ensamblador y realizar con los alumnos su diseño completo a nivel de puertas lógicas y circuitos integrados de media escala (decodificadores, multiplexores, etc.), así como con circuitos de muy gran escala (con una FPGA, por ejemplo). Existe una abundante documentación sobre CODE-2, así como material didáctico (ensamblador cruzado, emulador, etc.) definido como software libre. Además, hay una versión comercializada del mismo.

2.2.1. Instalación

La versión del simulador utilizada para este proyecto fue el Active-HDL Student Edition 2018, una alternativa sin costo alguno ofrecida por Aldec para que los estudiantes puedan utilizarla durante su aprendizaje y que también incluye una licencia que les permite comenzar a usar el simulador inmediatamente después de instalarlo, sin necesidad de ningún tipo de activación previa. La misma se encuentra disponible para su descarga en el sitio web oficial de Aldec: https://www.aldec.com/students/student.php?id=9; como se mencionó anteriormente, en forma absolutamente gratuita.

El único requerimiento exigido por Aldec para poder instalar el simulador consiste en que el usuario ingrese sus datos personales en un formulario de solicitud incluido en el sitio de descarga del mismo. Si el proceso fue exitoso, el sitio web mostrará un mensaje agradeciendo su registración e informando que un enlace de descarga para el instalador del simulador está siendo enviado a la dirección de correo ingresada en el formulario. Una vez sucedido esto, el usuario accederá a su casilla de correo electrónico personal, en la cual debería encontrar un nuevo mensaje en la bandeja de entrada de parte de "Aldec University Program (university@aldec.com)" con el asunto "Aldec Active-HDL Student Edition Download Information", el cual incluirá enlaces de descarga para las cinco partes que constituyen el instalador, cada una de las cuales deberá ser descargada para poder instalar el simulador:

Tabla 6. Nombres de los archivos que se deben descargar para instalar el simulador, junto con su tamaño y la función que cada uno de ellos desempeña en el proceso de instalación.

Nombre	Función	Tamaño
Active-HDL-Student-Edition-2018_7z_sfx.exe	Autodescompresor	190 KB
Active-HDL-Student-Edition-2018_7z_sfx.7z.001	Parte 1	800 MB
Active-HDL-Student-Edition-2018_7z_sfx.7z.002	Parte 2	800 MB
Active-HDL-Student-Edition-2018_7z_sfx.7z.003	Parte 3	800 MB
Active-HDL-Student-Edition-2018_7z_sfx.7z.004	Parte 4	392 MB

Todos los archivos descargados deberán ser preservados en una carpeta única con su nombre y extensión originales, y además los enlaces de descarga expirarán veinticuatro horas a partir del instante en el cual el mensaje fue enviado. Por lo tanto, si el usuario no lograra descargar los ficheros durante el tiempo permitido, debería repetir todo el proceso de registración desde el principio.

Una vez descargadas las cinco partes del simulador, ya no es necesario poseer una conexión a Internet para continuar con los pasos restantes del proceso de instalación. Ahora deberá ejecutarse el archivo "Active-HDL-Student-Edition-2018_7z_sfx.exe", un autoextractor encargado de descomprimir las otras cuatro partes y combinarlas en un único archivo final: el instalador propiamente dicho del simulador. Una vez abierto el descompresor, se le ofrecerá al usuario la posibilidad de seleccionar un directorio destino para la extracción de las partes y la creación del instalador. Una vez elegido, la descompresión se realizará automáticamente y finalizará sin mostrarle al usuario ningún tipo de aviso aun si el proceso fue exitoso; la única

comprobación posible consiste en verificar la existencia de un nuevo archivo en el directorio seleccionado para la extracción, cuyo nombre debería ser "Active-HDL-Student-Edition-2018.exe" y su tamaño, 2,75 GB. Éste será el instalador definitivo para el simulador.

El próximo paso será ejecutar el instalador creado anteriormente para llevar a cabo la instalación definitiva del simulador. Antes de comenzar, el asistente de instalación le ofrecerá al usuario la posibilidad de seleccionar tanto el directorio destino para la instalación como la carpeta en la cual se ubicará el espacio de trabajo para almacenar los proyectos desarrollados durante el transcurso de la utilización del Active-HDL, recomendándose en esta oportunidad mantener en ambos casos las opciones por defecto ofrecidas por el asistente ("C:\Aldec\Active-HDL-Student-Edition" y "C:\My_Designs" respectivamente). También deberán elegirse los componentes del simulador que se desean incluir en la instalación, así como los tipos o extensiones de archivos que deberían ser asociados al Active-HDL; nuevamente se aconseja conservar todas las opciones seleccionadas por defecto.

Ahora el instalador llevará a cabo el proceso de instalación del simulador. El mismo se realiza de manera automática y demorará algunos minutos. Al finalizar, se le mostrará un mensaje al usuario informando la exitosa instalación del Active-HDL Student Edition y ofreciéndole las posibilidades de leer las últimas modificaciones realizadas a la versión más reciente del programa (versión 10.5) y de agregar a PATH el directorio de instalación del simulador (PATH es una variable de entorno de los sistemas operativos POSIX y los sistemas de Microsoft, en ella se especifican las rutas en las cuales el intérprete de comandos debe buscar los programas a ejecutar). Se recomienda mantener seleccionada únicamente esta última opción.

Si el usuario ha obedecido correctamente todas las instrucciones indicadas previamente, el simulador ya debería encontrarse instalado en el ordenador. La comprobación más fácil de la exitosa finalización del proceso consiste en acceder al Escritorio de la PC y verificar que se haya creado un nuevo acceso directo con el nombre "Aldec Active-HDL Student Edition". Este archivo representa uno de los muchos caminos válidos que existen para comenzar a utilizar el simulador. No obstante, aquí es donde termina este tutorial, ya que tanto la configuración como el uso del Active-HDL variarán en función del diseño que el usuario desee desarrollar. Se ofrecerá más información sobre este apartado en capítulos posteriores, una vez que se haya descripto más detalladamente el diseño planificado para este proyecto en particular.

A continuación, se incluyen capturas de algunos de los pasos más importantes que constituyen el proceso de instalación del simulador:

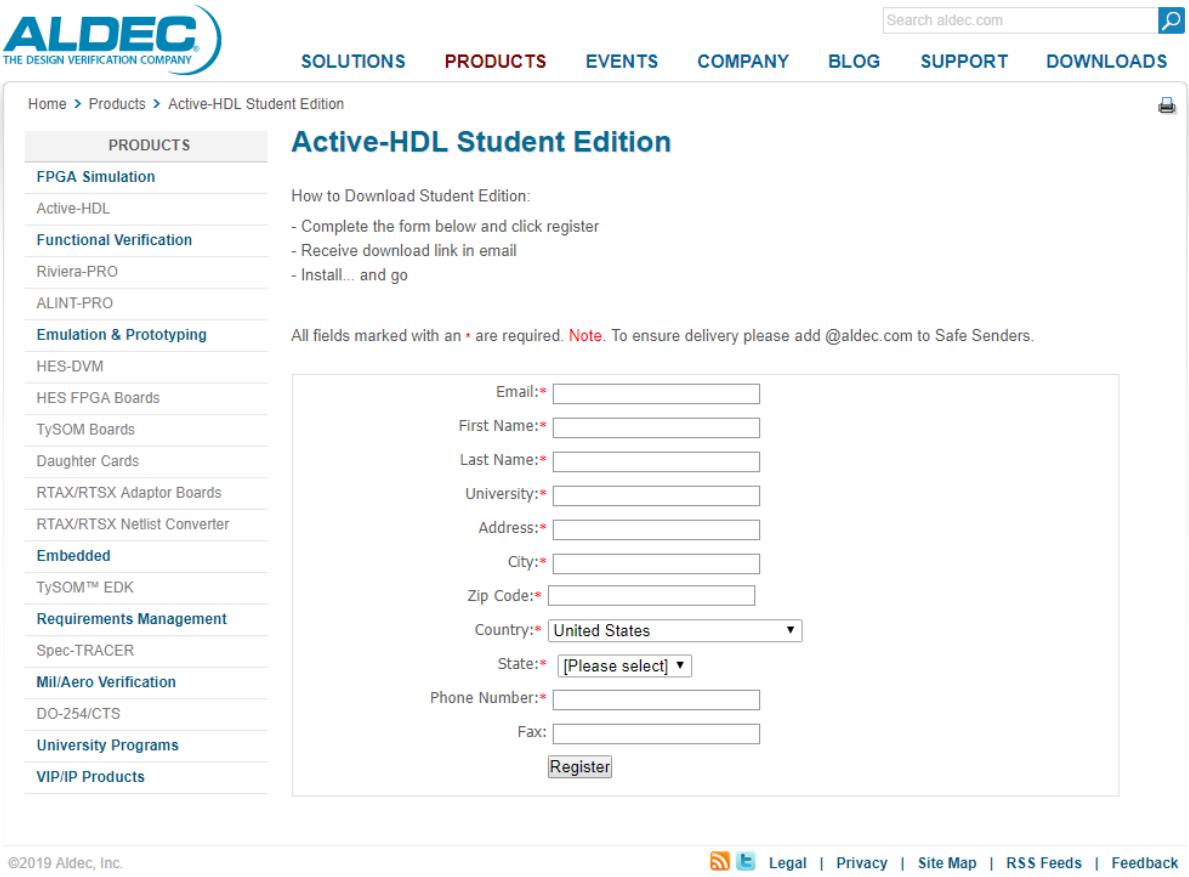


Figura 7. Formulario de registración para la descarga del Active-HDL Student Edition 2018 desde el sitio web oficial de Aldec (Zip Code = Código Postal).

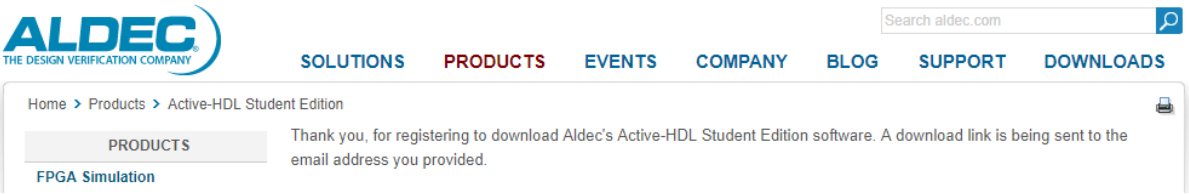
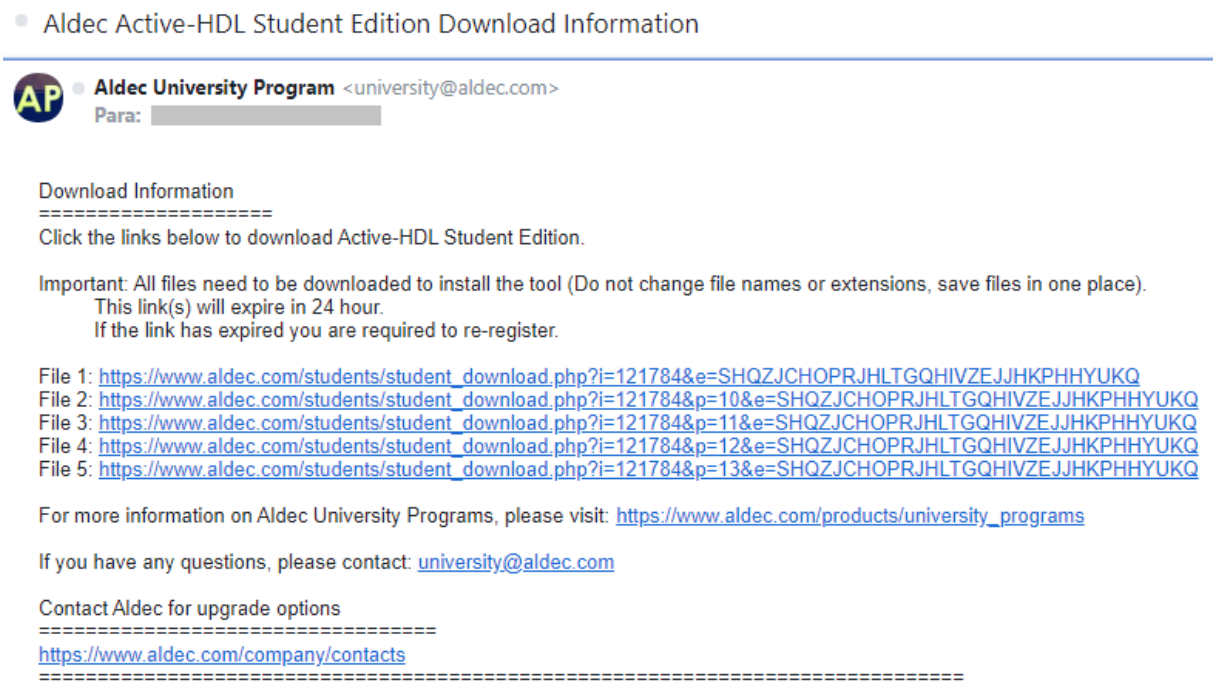


Figura 8. Mensaje informado al usuario por el sitio web oficial de Aldec luego de que el mismo haya completado y enviado exitosamente el formulario de la figura anterior.



Aldec Active-HDL Student Edition Download Information

Aldec University Program <university@aldec.com>
Para:

Download Information
==========
Click the links below to download Active-HDL Student Edition.

Important: All files need to be downloaded to install the tool (Do not change file names or extensions, save files in one place).
This link(s) will expire in 24 hour.
If the link has expired you are required to re-register.

File 1: https://www.aldec.com/students/student_download.php?i=121784&e=SHQZJCHOPRJHLTGQHIVZEJJHKPHHYUKQ
File 2: https://www.aldec.com/students/student_download.php?i=121784&p=10&e=SHQZJCHOPRJHLTGQHIVZEJJHKPHHYUKQ
File 3: https://www.aldec.com/students/student_download.php?i=121784&p=11&e=SHQZJCHOPRJHLTGQHIVZEJJHKPHHYUKQ
File 4: https://www.aldec.com/students/student_download.php?i=121784&p=12&e=SHQZJCHOPRJHLTGQHIVZEJJHKPHHYUKQ
File 5: https://www.aldec.com/students/student_download.php?i=121784&p=13&e=SHQZJCHOPRJHLTGQHIVZEJJHKPHHYUKQ

For more information on Aldec University Programs, please visit: https://www.aldec.com/products/university_programs

If you have any questions, please contact: university@aldec.com

Contact Aldec for upgrade options
==========
https://www.aldec.com/company/contacts
==========

Figura 9. Mensaje enviado por Aldec al correo electrónico indicado por el usuario en su formulario de registración con los cinco enlaces de descarga para el instalador del simulador.

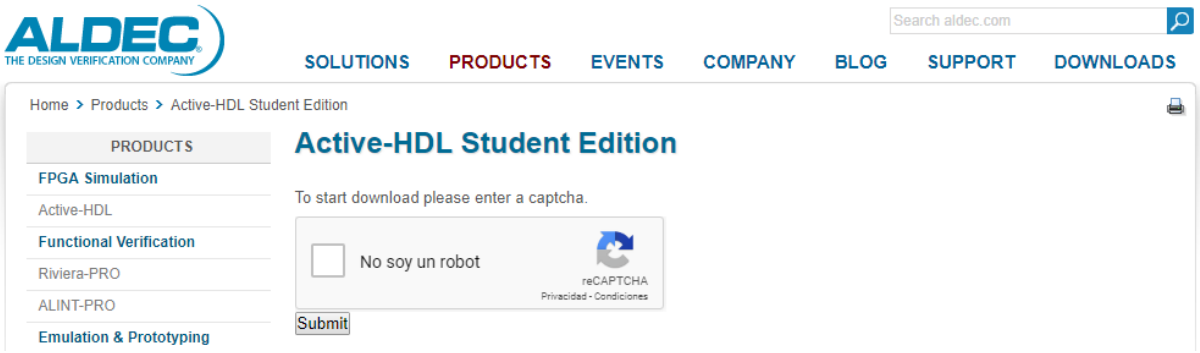


Figura 10. Pantalla de descarga para cada una de las cinco partes del instalador del simulador desde el sitio web oficial de Aldec.

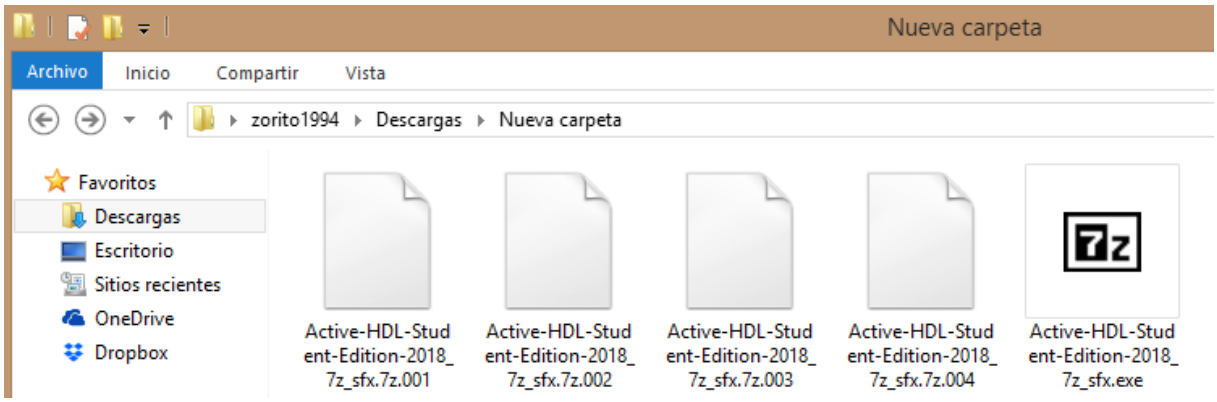


Figura 11. Archivos descargados desde los enlaces de la figura 9, necesarios para generar el instalador del Active-HDL.

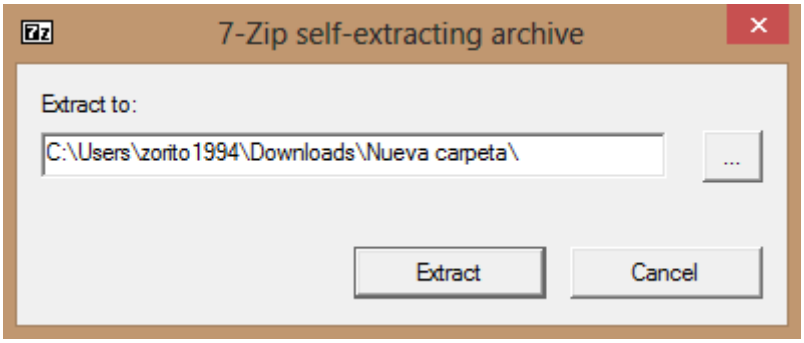


Figura 12. Selección del directorio destino para la generación del instalador del Active-HDL a partir de los archivos de la figura anterior.

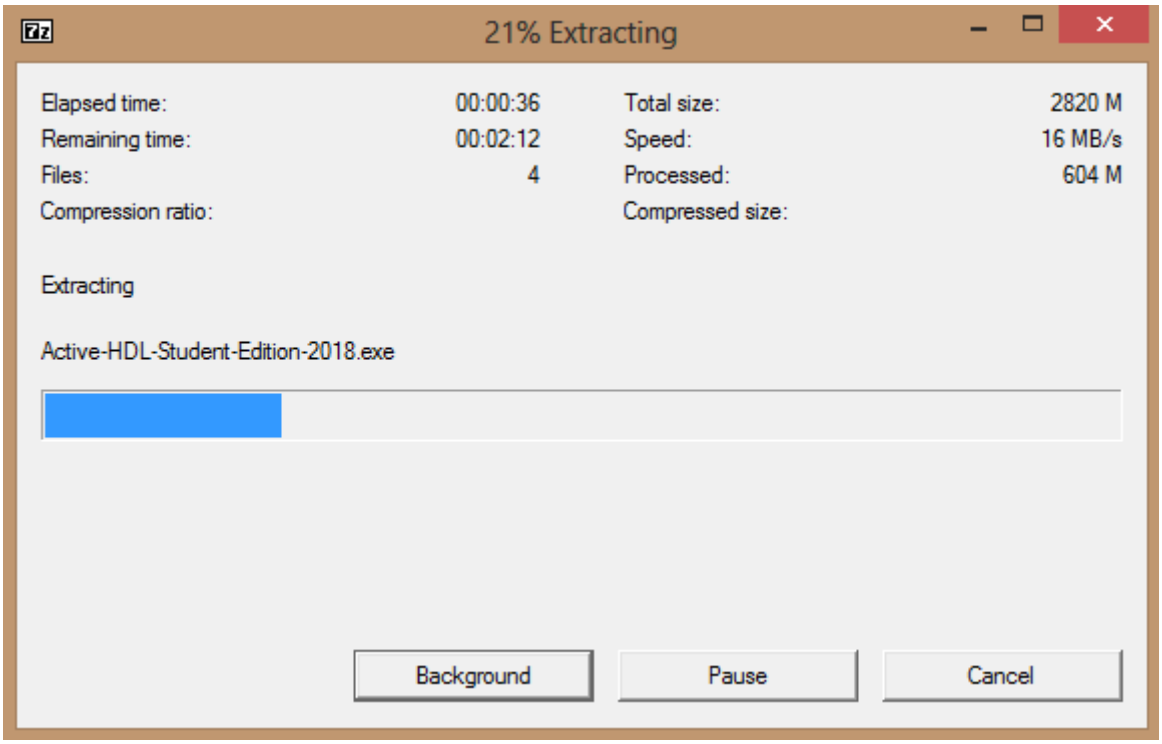


Figura 13. Ejecución del proceso de extracción de los archivos de la figura 11 para generar el instalador del simulador.

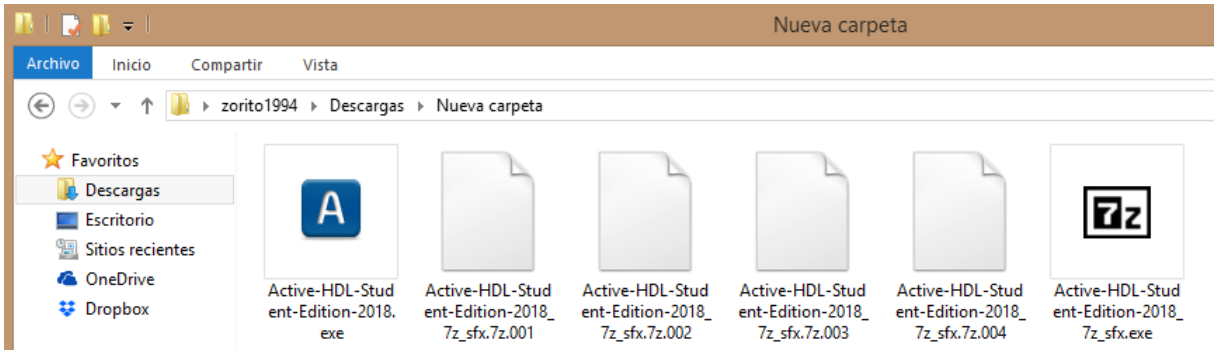


Figura 14. Generación exitosa del instalador del simulador (“Active-HDL-Student-Edition-2018.exe”) en la misma carpeta que los archivos descomprimidos.

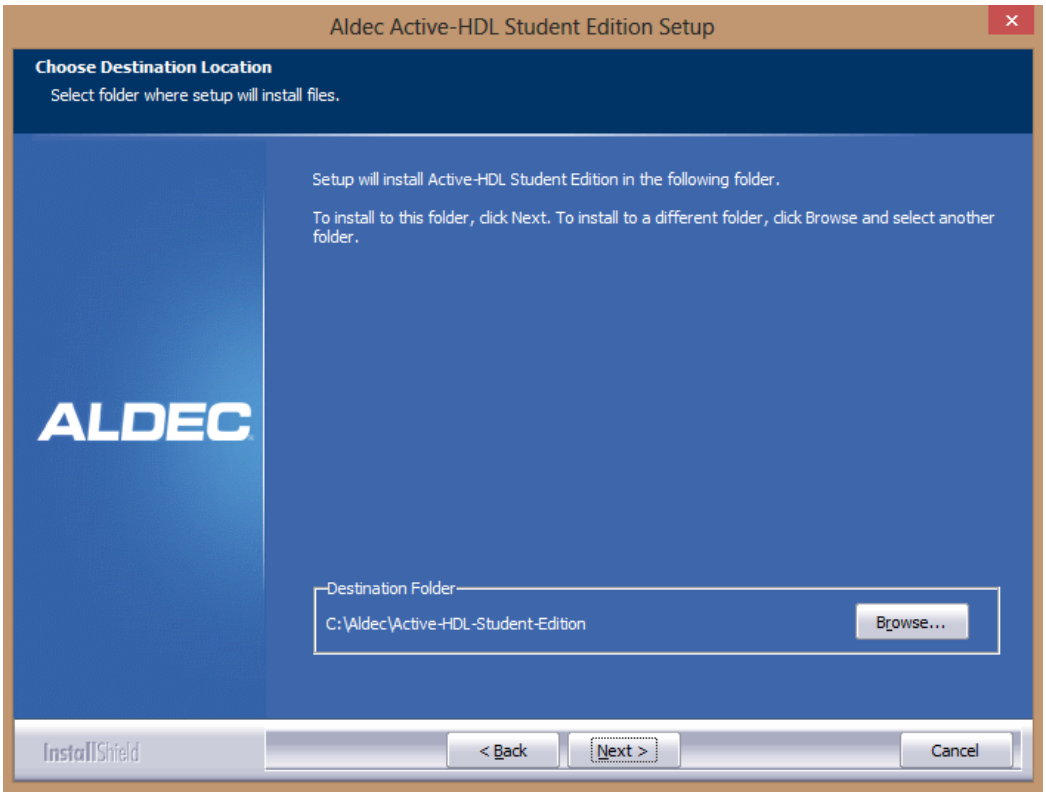


Figura 15. Selección del directorio destino para la instalación definitiva del Active-HDL mediante la ejecución del instalador de la figura anterior.

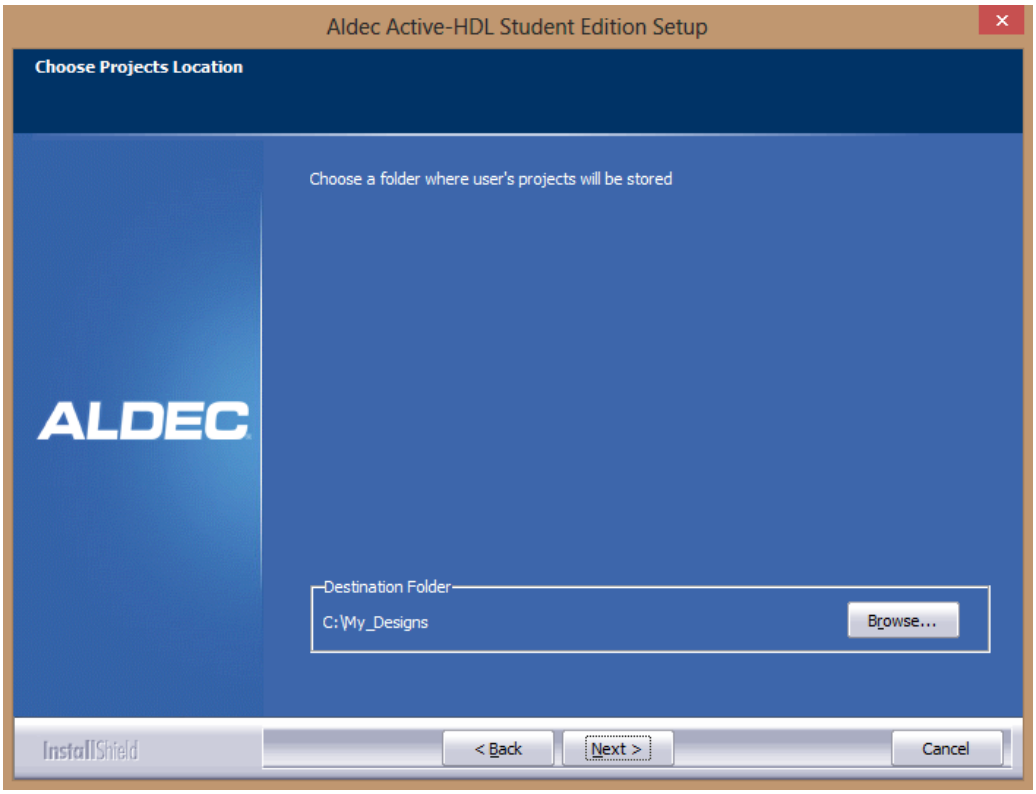


Figura 16. Selección del directorio para el espacio de trabajo destinado a almacenar los proyectos desarrollados durante el transcurso de la utilización del Active-HDL.

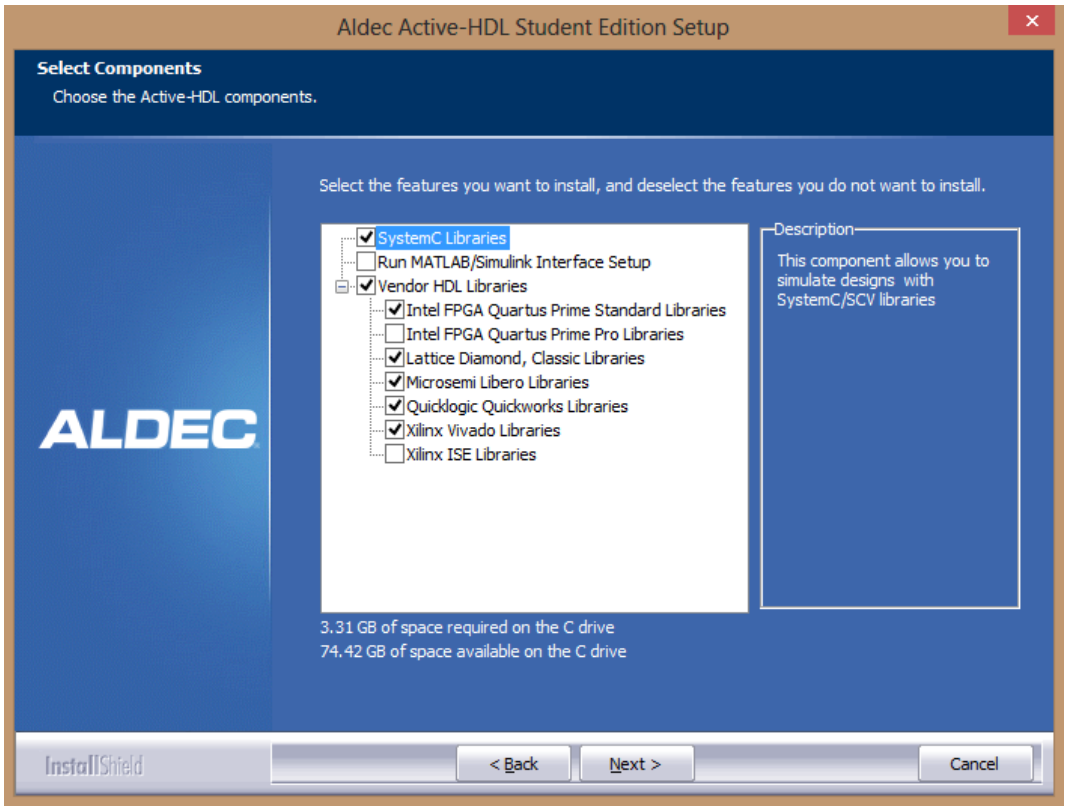


Figura 17. Selección de los componentes a incluir en la instalación del simulador.

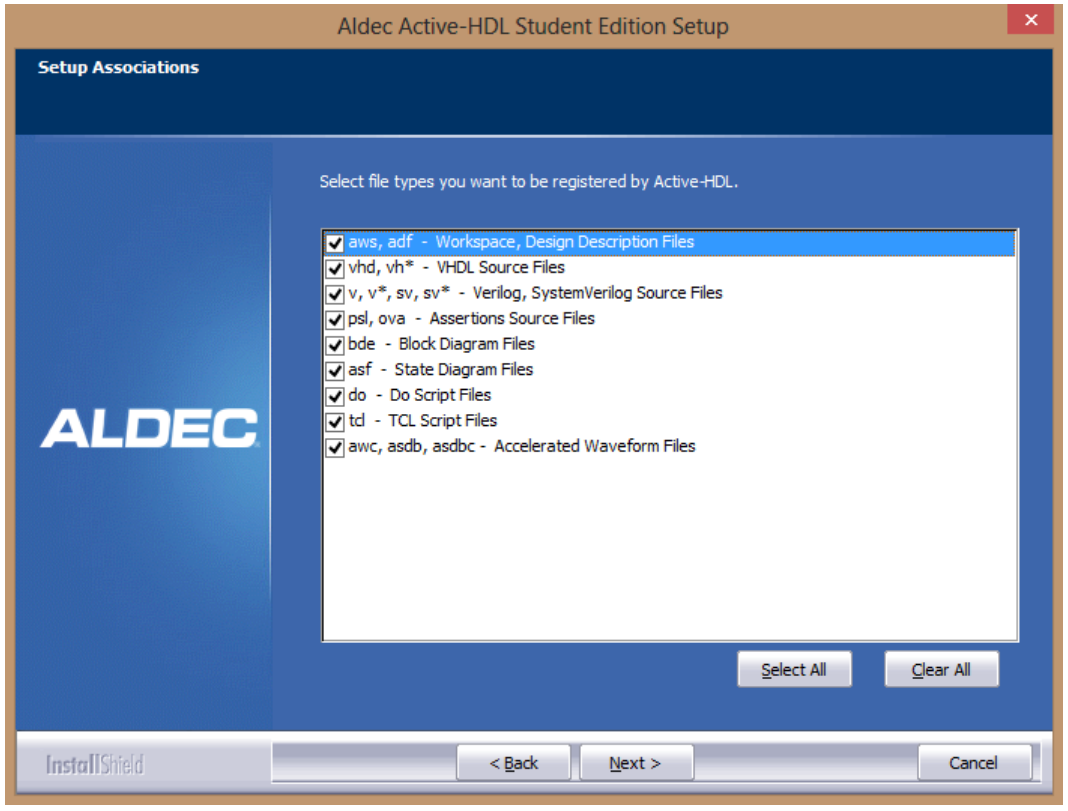


Figura 18. Selección de los tipos o extensiones de archivos que deberían ser asociados al Active-HDL.

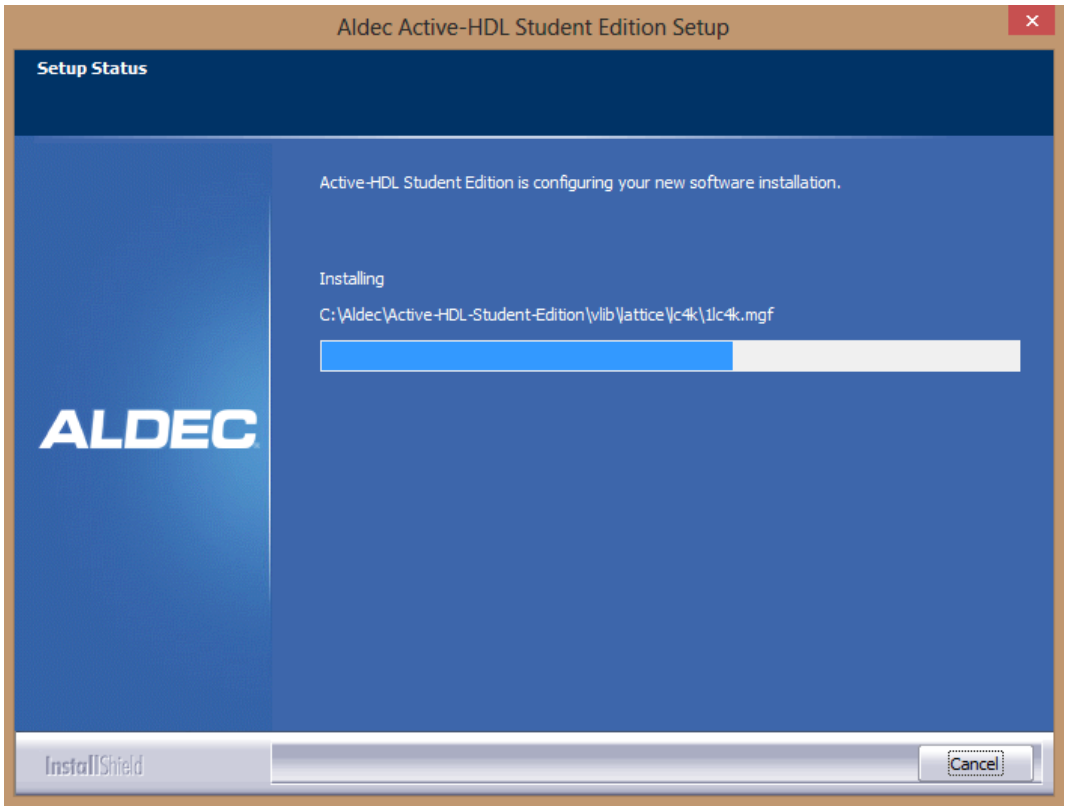


Figura 19. Ejecución del proceso de instalación definitiva del simulador.

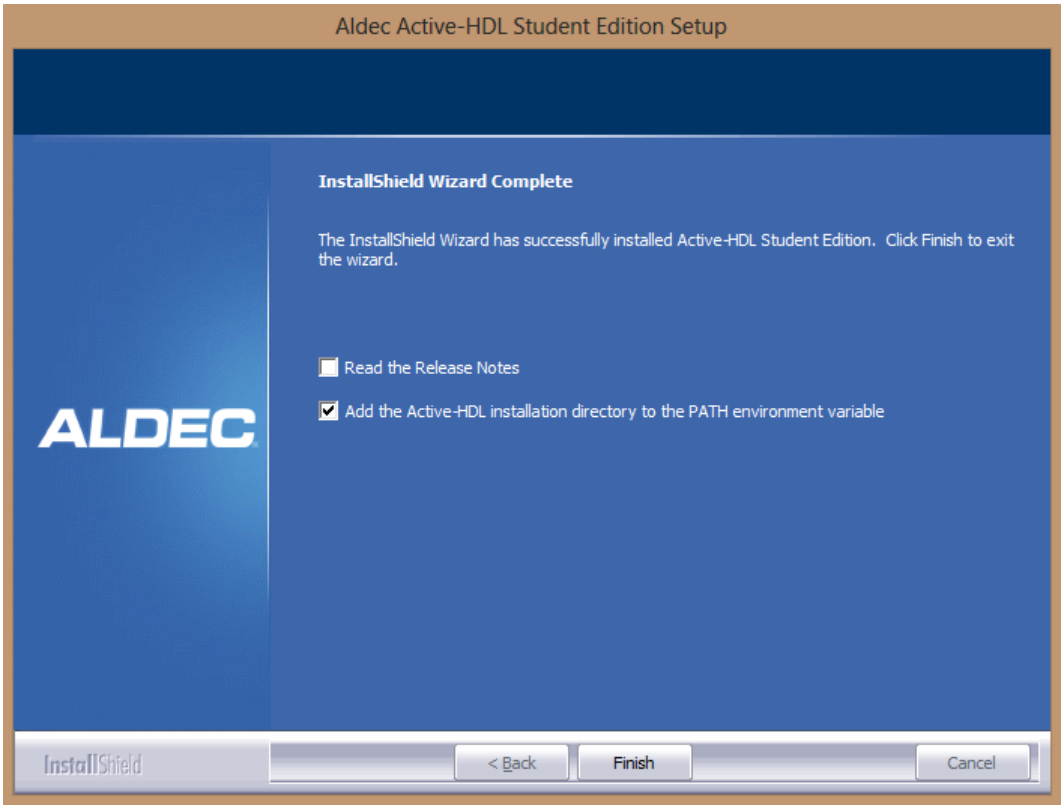


Figura 20. Selección de acciones a realizar una vez finalizado exitosamente el proceso de instalación del Active-HDL.



Figura 21. Acceso directo para la ejecución del Active-HDL, generado automáticamente en el Escritorio por el instalador del simulador una vez finalizada correctamente la instalación.

2.3. Diseño de la computadora original

Originalmente se había optado por tomar como punto de partida para el diseño del proyecto un código disponible para su descarga en la biblioteca digital oficial VHDL de la Universidad de Hamburgo, ubicada en el sitio web https://tams.informatik.uni-hamburg.de/research/vlsi/vhdl/index.php?content=06-models, el cual describe un modelo comportamental simple e incompleto del microprocesador Intel 80386.

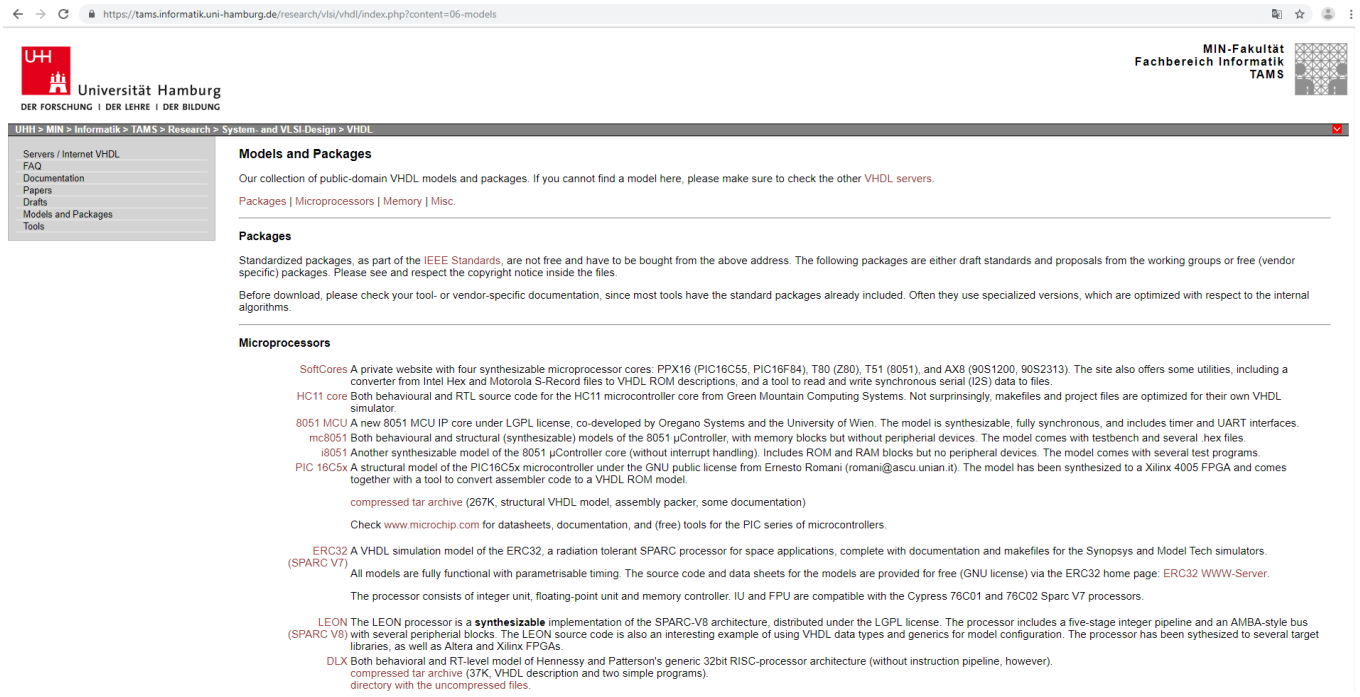


Figura 22. Biblioteca digital oficial VHDL de la Universidad de Hamburgo.

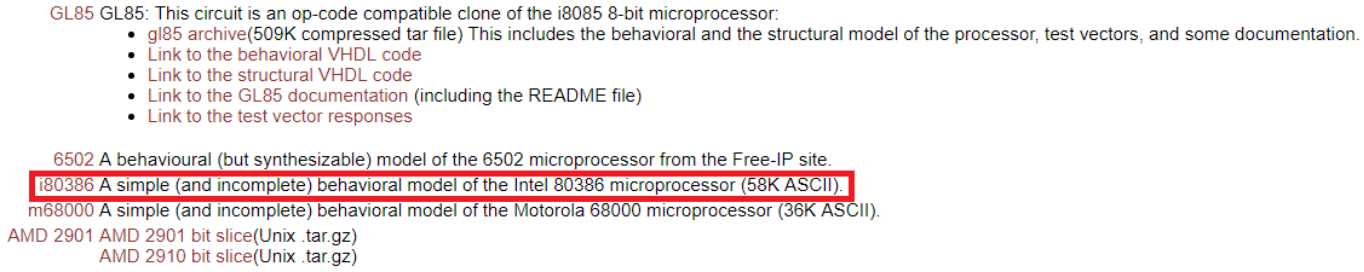


Figura 23. Enlace de descarga para el código VHDL del modelo del microprocesador originalmente seleccionado para comenzar el diseño de este proyecto.

Al no haberse podido localizar ningún tipo de manual que definiera y explicara las características principales de este modelo original, se decidió llevar a cabo un proceso de ingeniería inversa para, a partir de un análisis minucioso y exhaustivo del código, establecer cuáles habían sido la idea y el objetivo originales del diseñador y, por lo tanto, el grado de avance que tenía el desarrollo de la CPU hasta la fecha. A partir de dicho estudio, se logró extraer la información que se detalla a continuación (sólo se mencionan las funcionalidades del procesador que ya se encuentran implementadas):

Tabla 24. Atributos generales del diseño de la computadora original.

Tipo de arquitectura	CISC
Ciclos de ejecución	Cada instrucción puede requerir uno o dos ciclos de reloj para ejecutarse en función del tamaño de su operando
Interfaz de Entrada/Salida	E/S independiente
Formato y longitud de las instrucciones	Instrucciones de formato y longitud variables: uno, dos, tres o cinco bytes en función del tamaño de su operando
Frecuencia del reloj	A determinar por el usuario
Bus de datos	32 bits
Bus de direcciones	32 bits
Modos de direccionamiento	Inmediato
	Directo de Registro
	Directo de Memoria
	Indirecto con Registro
	Indirecto con Desplazamiento Relativo al IP
Registros de uso general	EAX
	EBX
	ECX
	EDX
Almacenamiento en memoria/cola	Little Endian

Tabla 25. Repertorio de instrucciones de la CPU original.

Tipo de instrucción	Instrucción	Comentario	Operación	Observación
Transferencia de Datos	MOV *dest, fuente*	Copia *fuente* en *dest*	(*dest*) ← (*fuente*)	1
	IN *dest, fuente*	Carga el valor en el puerto *fuente* en *dest*	(*dest*) ← (*fuente*)	2
	OUT *dest, fuente*	Carga en el puerto *dest* el valor en *fuente*	(*dest*) ← (*fuente*)	3
Aritmética	INC *dest*	Incrementa *dest* en una unidad	(*dest*) ← (*dest*) + 1	4
Transferencia de Control	JMP *desp*	Salta incondicionalmente a *desp* direcciones de la próxima instrucción	(IP) ← (IP) + *desp*	5
Control	NOP	Operación nula (no realiza ninguna acción)		

Observaciones:

1. Las posibilidades para *dest/fuente* son: *reg/mem*, *reg/op.inm*, *mem/reg*. *mem* sólo puede ser [BX], siendo (BX) una dirección de memoria (direccionamiento indirecto).
2. Las posibilidades para *dest*/*fuente* son: AL/*mem*, AL/DX. *mem* debe ser una dirección entre 0 y 255. Puede ser un operando inmediato o una etiqueta.
3. Las posibilidades para dest/fuente son: *mem*/AL, DX/AL. *mem* debe ser una dirección entre 0 y 255. Puede ser un operando inmediato o una etiqueta.
4. *dest* sólo puede ser *reg*.
5. *desp* sólo puede ser un operando inmediato entre -2.147.483.648 y 2.147.483.647 (valor con signo de 32 bits).

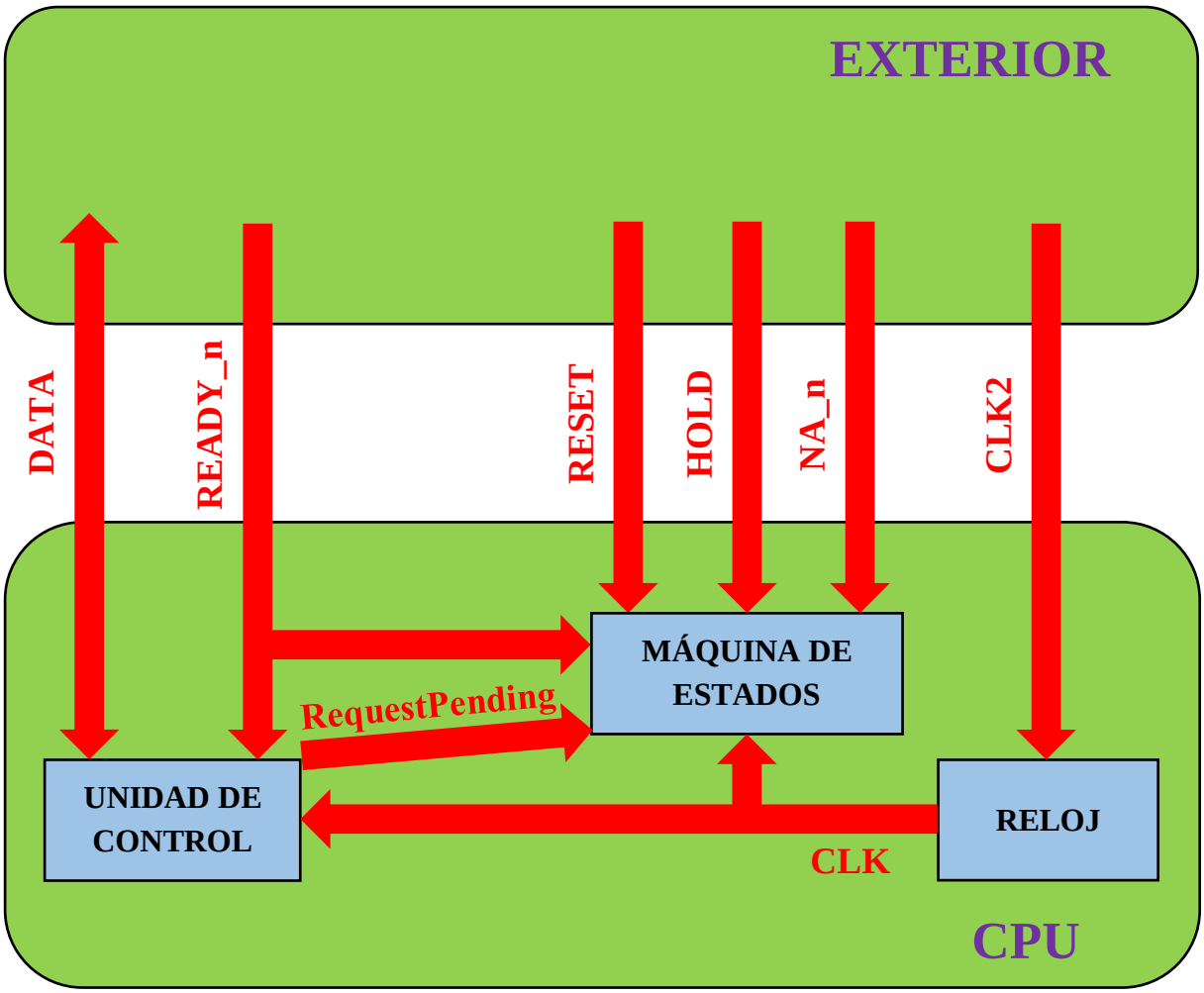


Figura 26. Diseño de los componentes y señales y su interacción para la CPU original.

Observaciones:

- Puertos de entrada/salida:
  - ❖ Data: permite al usuario indicarle al procesador el código de operación de la instrucción a ejecutar junto con el operando necesario para llevarla a cabo. Dicho operando variará en función de la instrucción ingresada (dirección de memoria a leer/escribir, valor inmediato, dirección de salto, etc.). También le informa al usuario el dato resultante una vez ejecutada la instrucción (dato leído/escrito, resultado de la operación aritmética, dirección de salto efectiva, etc.).
- Puertos de entrada:
  - ❖ CLK2: señal de reloj del usuario. La frecuencia de esta señal será el doble de aquélla correspondiente al reloj interno del procesador. Esta señal se activa por flanco ascendente.

- ❖ NA_n: permite al usuario segmentar el cauce del procesador. Esta señal se activa con un nivel bajo.
- ❖ READY_n: cumple una función similar al puerto CLK2, habilitando la ejecución de la instrucción que se encuentre pendiente de ser atendida siempre y cuando su respectivo dato haya sido cargado en el puerto Data. Esta señal se activa por flanco descendente.
- ❖ HOLD: señal de espera de reconocimiento. Esta señal se activa con un nivel alto.
- ❖ RESET: permite al usuario reiniciar el procesador. Esta señal se activa por flanco ascendente, pero sólo se desactiva por flanco descendente.

- Señales internas:
  - ❖ RequestPending: su valor es '1' (Pending) si existe alguna instrucción pendiente de ser atendida y '0' (NotPending) en caso contrario.
  - ❖ CLK: reloj interno del procesador. Su frecuencia es la mitad de aquélla correspondiente a la señal de reloj del usuario. Esta señal se activa por flanco ascendente.

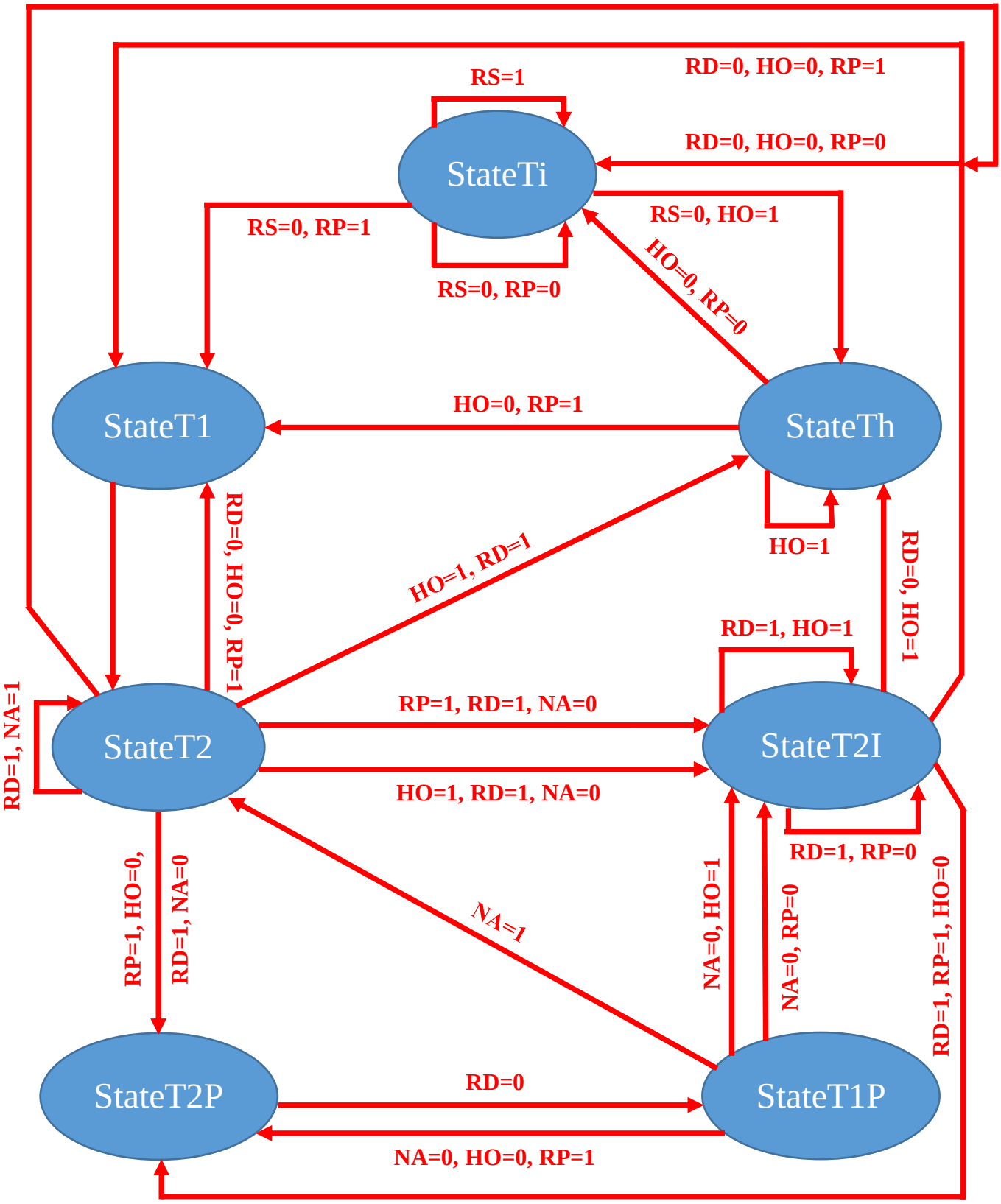


Figura 27. Funcionamiento de la CPU original representado a través de una máquina de estados (RS = RESET; RP = RequestPending; HO = HOLD; RD = READY_n; NA = NA_n).

Observaciones:

- StateTi: Estado de Reinicio (Reset State). Todos los registros del procesador pasan a poseer valores de alta impedancia ('Z') y no se realiza ninguna otra acción.
- StateTh: Estado en espera de la señal de reconocimiento.
- StateT1: Primer estado de un cauce no segmentado. Aquí se lleva a cabo la totalidad de la ejecución de cada instrucción.
- StateT2: Estado de un cauce no segmentado durante el cual el procesador permanece ocioso.
- StateT1P: Primer estado de un cauce segmentado.
- StateT2P: Estado subsecuente de un cauce segmentado.
- StateT2I: Estado subsecuente de un cauce potencialmente segmentado.

Para cerrar esta sección del informe, se expondrán las conclusiones finales obtenidas sobre el estado actual del proyecto original a partir de las tablas y figuras anteriores:

- Como habrá podido notarse, el diseño original sólo contempla la funcionalidad de la CPU, sin haber desarrollado aún ningún tipo de planificación general para el resto de los componentes de la PC con los cuales debería interactuar durante el desempeño de sus tareas. Es decir, tanto la memoria principal como los periféricos de entrada/salida no existen en absoluto en el proyecto llevado a cabo hasta la fecha, por lo cual cualquier tipo de instrucción que requiera realizar tanto accesos a memoria (transferencias de datos desde o hacia memoria, operaciones aritmético-lógicas o transferencias de control con direccionamiento directo de memoria o indirecto con registro, etc.) como operaciones de entrada/salida (lectura/escritura sobre puertos de periféricos de entrada/salida para realizar impresiones o activar barras de leds o microconmutadores, similar al MSX88, o bien leer datos ingresados por el usuario a través de la pantalla de una terminal o escribir información en ella para que pueda observarlos, al igual que en el WinMIPS64) no podrá ser actualmente implementada por el programador en el repertorio de instrucciones de la CPU hasta no haber descrito previamente de alguna manera el comportamiento de dichos componentes tanto internamente como formando parte de una PC que los contenga y vincule y en la cual deberán transmitir y recibir información del procesador ya desarrollado según corresponda.
- La ausencia de un ensamblador compatible con este procesador implica que resulta imposible para un programador de un nivel de abstracción más alto escribir programas en lenguaje Assembler que puedan ser interpretados y ejecutados por esta CPU, por lo cual la

única alternativa viable sería estudiar detalladamente el código de operación (opcode) pensado por el diseñador para cada instrucción del repertorio de manera tal que la misma resulte inteligible para la unidad de control encargada de ejecutarla. Dicho en otras palabras, para utilizar este procesador habría que escribir instrucciones directamente en código máquina (binario), sin ningún tipo de abstracción que le facilite la tarea al programador, por lo cual la labor de programar sería considerablemente más tediosa y complicada. Además, al no poseer el diseño ningún banco de pruebas (testbench), no se cuenta con un ejemplo claro tanto de la configuración previa del procesador requerida para ejecutar una instrucción como el formato de esta última y el método adecuado para que el usuario/programador pueda transmitirla a la CPU.

- A pesar de que tanto la máquina de estados de la figura 27 como la presencia de la señal NA_n sugieren la existencia de una segmentación de dos etapas ya implementada y plenamente operativa en el cauce del procesador, esto en realidad no es así, ya que la activación de la segmentación sólo implica un cambio en los estados de la máquina, pero no en la ejecución propiamente dicha de las instrucciones. Para empezar, no se ha implementado ninguna división en etapas para el proceso de ejecución de las mismas, por lo cual resultará imposible poseer más de una instrucción ejecutándose simultáneamente: cada instrucción se ejecuta en una única etapa que requiere un ciclo de reloj para completar sus acciones, por lo cual cada una deberá esperar a que finalice la anterior para comenzar a ejecutarse. En otras palabras, la ejecución de las instrucciones por parte de este procesador es estrictamente secuencial. Se concluye en este apartado que el diseño desarrollado hasta el momento propone a través de la máquina de estados una “interfaz” o “idea” para incluir la alternativa de segmentar el cauce de ejecución de las instrucciones en dos etapas para reducir el tiempo de ejecución total del programa permitiendo la ejecución de dos instrucciones al mismo tiempo. Sin embargo, la implementación de esta interfaz aún se encuentra pendiente, tarea que deberá ser llevada a cabo por el diseñador que continúe con este proyecto de manera tal que la máquina de estados propuesta refleje genuinamente en todo momento la situación actual de la CPU.
- La descripción seleccionada para el diseño original de la CPU es la comportamental: en ella se define la forma en la cual se comporta el circuito digital, teniendo en cuenta sólo las características del mismo respecto al comportamiento de las entradas y las salidas. Ésta es la forma que más se parece a los lenguajes de software ya que la descripción puede ser secuencial, además de combinar características concurrentes. No obstante, este método no es el más recomendable cuando el diseño digital se vuelve complejo o está conformado por

múltiples bloques de hardware como sería el caso de este procesador: la alternativa más adecuada podría ser una descripción estructural, en la cual se define el circuito a partir de instancias de componentes, representando cada componente a un bloque de hardware distinto. Dichas instancias conforman un diseño de jerarquías, al conectar los puertos de las mismas con las señales internas del circuito o bien con puertos del circuito de jerarquía superior o inferior. Al encontrarse actualmente la CPU diseñada en un estado tan incompleto es posible que la descripción comportamental aún se adapte al funcionamiento esperado por parte del procesador, pero a medida que este último se vuelva más complejo y se le incorporen más funcionalidades, se tornará cada vez más complicada la tarea de programar código adicional manteniendo esta forma de descripción. Por lo tanto, se sugiere replantear el diseño de la CPU con la mayor prontitud posible de manera tal que su descripción sea más estructural, analizando las funciones esperadas del procesador y procurando asignar cada una a un componente individual, el cual debería ser tan independiente como sea posible de la CPU en sí y de los restantes componentes. Esto es lo que se conoce en la programación de alto nivel como "modularización", pero aquí también será sumamente útil para evitar que el diseñador se vea abrumado y superado por la complejidad del diseño a desarrollar.

- El repertorio de instrucciones ofrecido por el diseñador se encuentra actualmente muy limitado: además de las restricciones ya mencionadas respecto a las operaciones de entrada/salida y que requieran acceder a memoria, se puede observar que aún resta implementar casi la totalidad de las instrucciones aritmético-lógicas (suma, resta, multiplicación, división, or, and, not, etc.), de desplazamiento de bits (a izquierda o derecha) y de transferencia de control (salto condicional). De todas maneras, este no sería un inconveniente relevante ya que la implementación e inclusión de las instrucciones faltantes no debería en un principio implicar modificaciones radicales en el diseño original: simplemente habría que agregar los códigos de operación asociados a las nuevas instrucciones a la unidad de control de la CPU para que pueda interpretarlos y determinar las próximas acciones a realizar.
- La ausencia en este diseño de una memoria principal a la cual la CPU pueda acceder para leer o escribir datos obviamente conlleva que la sección de memoria conocida como pila tampoco se encuentre complementada ni mucho menos implementada, al igual que su manejo y acceso por parte de instrucciones específicas para apilar o desapilar datos de ella. De igual manera, hasta el momento no ha sido tenida en cuenta la posibilidad de que el procesador reciba pedidos de interrupción tanto por hardware como por software, por lo cual todas las instrucciones asociadas a interrupciones tampoco han sido implementadas. A

diferencia de la situación mencionada en el párrafo anterior, la situación aquí se agrava porque no sólo es necesario incluir los nuevos códigos de operación en la unidad de control, sino que además tanto el manejo de la pila como de las interrupciones involucra una lógica adicional que deberá ser planificada y diseñada detenidamente antes de proceder a implementar las instrucciones propiamente dichas.

- Como consecuencia de la metodología seleccionada para la descripción del diseño de la CPU (comportamental), no se encuentra específica ni individualmente definida una unidad aritmético-lógica (ALU) para realizar operaciones aritmético-lógicas ni un banco de registros integrado en el procesador para leer y escribir datos temporalmente en él a una mayor velocidad que si se viera obligado en cada operación a acceder a la memoria principal de la PC. No obstante, esto no significa que sus funcionalidades no estén ya implementadas en este diseño, considerando que actualmente hay definidas algunas operaciones de suma junto con algunos registros de propósito general que son accedidos directamente durante la ejecución de las instrucciones que así lo requieran. El principal inconveniente que aquí se presenta consiste en que técnicamente no se está respetando la arquitectura von Neumann, el pilar fundamental para describir los diseños de todas las computadoras actuales. En dicha arquitectura, la CPU debe poseer tres secciones claramente diferenciables: la unidad de control, la ALU y el banco de registros. En el código desarrollado para el procesador de este proyecto original podría argumentarse que se ha considerado la existencia de la unidad de control, encargada de decodificar el código de operación de la instrucción actual y, a partir de él, determinar las acciones a realizar para ejecutarla. Aun así, este diseño determina que la ejecución de cada instrucción debe llevarse a cabo en una única etapa y que la unidad de control deberá llevar a cabo por sí sola la totalidad de las tareas requeridas: buscar la siguiente instrucción, decodificarla e interpretarla, ejecutarla y almacenarla en memoria o en un registro según corresponda. Esto refleja la problemática mencionada anteriormente respecto a la descripción comportamental y la necesidad de estructurar el diseño y modularizar el código: a este ritmo, si se desean incorporar nuevas funcionalidades la unidad de control se complejizará hasta tal punto que se volverá imposible de estudiar, manejar y mejorar para cualquier programador, por lo cual se recomienda como primera acción asignar a la unidad de control sólo las funciones que le corresponde desempeñar y definir una ALU y un banco de registros separados para realizar el resto de las tareas, tal y como lo determina la arquitectura von Neumann.
- Teniendo en cuenta el objetivo inicialmente propuesto en la introducción de este informe, se puede apreciar que existen algunas funcionalidades planteadas en el diseño original que

resultan superficiales y complejizan innecesariamente el funcionamiento del procesador, por lo que podrían ser eliminadas sin mayores inconvenientes para simplificar el modelo y focalizarlo en las tareas que sí se esperan que el mismo pueda cumplir. Se debe recordar que en este proyecto se está realizando una simulación, por lo tanto, no necesariamente el diseño debe representar fielmente la CPU: alcanza con que se contemplen los aspectos indispensables para el estudio de la misma en función de las metas estipuladas al comenzar el proceso de diseño. Por ejemplo, aquí no sería necesario que el procesador posea la capacidad de ser reiniciado ni que aguarde una señal de reconocimiento por parte del usuario ya que, aunque estas prestaciones asemejen el comportamiento de la CPU al que tendría en el mundo real, no tienen ninguna relación con el objetivo determinado para este proyecto en particular.

- Para finalizar, se describirá en términos generales el ciclo de ejecución definido en este diseño para cada instrucción:
    - En primer lugar, se activa la señal "RequestPending" para indicar que existe una instrucción pendiente de ser atendida.
    - Luego, permanece ociosa hasta el próximo flanco descendente en la señal "READY_n". Una vez producido éste, desactiva la señal "RequestPending" y aguarda hasta el siguiente flanco descendente que ocurra en la señal "CLK".
    - Una vez producido el flanco descendente en el reloj interno del procesador, se recibe en la señal "Data" la información enviada desde el exterior correspondiente a la próxima instrucción a ejecutar (código de operación y operando) y se la almacena en la cola interna de la CPU (señal "InstQueue").
        - Recordar que, al no encontrarse implementado en este modelo la memoria principal de la PC, "exterior" es una expresión ambigua cuya representación varía en función de la intención del diseñador: puede ser la memoria principal ya efectivamente definida y descripta en el modelo o bien directamente el usuario final quien le esté transmitiendo directamente instrucciones individuales al procesador para que éste las ejecute.
        - La cola interna a la que se hizo referencia anteriormente consiste en una estructura auxiliar que utiliza temporalmente el procesador para almacenar temporalmente la instrucción recibida del usuario hasta el momento de ejecutarla. Aunque esta "cola" auxiliar integrada en la CPU no debería estar incluida en el diseño bajo ningún punto de vista ya que las únicas unidades de almacenamiento permitidas en el procesador son los registros, podría

justificarse su existencia en la necesidad de reemplazar la memoria principal ausente en este modelo por una estructura de almacenamiento que desempeñe una función similar. Cada dirección en la cola puede almacenar un byte de información.

- Los formatos adecuados para enviar una instrucción a esta CPU a fin de que esta última pueda ejecutarla son los siguientes:

			Código de operación
Byte 3 (bits 31-24)	Byte 2 (bits 23-16)	Byte 1 (bits 15-8)	Byte 0 (bits 7-0)

Figura 28. Formato de instrucción sin operandos para la CPU original. Adecuado para instrucciones de control (NOP).

		Operando	**Código de operación**
Byte 3 (bits 31-24)	Byte 2 (bits 23-16)	Byte 1 (bits 15-8)	Byte 0 (bits 7-0)

Figura 29. Formato de instrucción con un operando de 8 bits para la CPU original. Adecuado para instrucciones de transferencia de datos y aritméticas.

	Operando (byte 1)	**Operando (byte 0)**	**Código de operación**
Byte 3 (bits 31-24)	Byte 2 (bits 23-16)	Byte 1 (bits 15-8)	Byte 0 (bits 7-0)

Figura 30. Formato de instrucción con un operando de 16 bits para la CPU original. Adecuado para instrucciones de transferencia de datos y aritméticas.

Operando (byte 2)	**Operando (byte 1)**	**Operando (byte 0)**	**Código de operación**
Byte 3 (bits 31-24)	Byte 2 (bits 23-16)	Byte 1 (bits 15-8)	Byte 0 (bits 7-0)

Figura 31. Formato de instrucción con un operando de 32 bits para la CPU original (primera transmisión). Adecuado para instrucciones de transferencia de datos, aritméticas y de transferencia de control.

- ✓ Nota: Si el operando es de 32 bits, se requerirán dos transmisiones para enviar la totalidad de la instrucción (en el primer paquete se podrán transmitir únicamente los tres bytes menos significativos del operando). A continuación, se define el formato para realizar la segunda transmisión (sólo restaría para enviar el byte más significativo del operando):

			Operando (byte 3)
Byte 3 (bits 31-24)	Byte 2 (bits 23-16)	Byte 1 (bits 15-8)	Byte 0 (bits 7-0)

Figura 32. Formato de instrucción con un operando de 32 bits para la CPU original (segunda transmisión).

- ➢ En la próxima figura se representa la disposición de una instrucción con un operando de 32 bits en la cola interna de la CPU una vez se encuentre efectivamente almacenada en ella. Recordar que la ubicación de cada uno de los bytes de la instrucción (código de operación y operando) en las distintas direcciones de la cola dependerá del mecanismo de almacenamiento implementado e incorporado en la arquitectura de la computadora original. En este caso, dicho mecanismo, tal como se indicó en la tabla 24, se trata del formato Little Endian: los bytes se almacenan desde el menos significativo en la dirección más baja hacia el más significativo en la más alta.

Dirección	Contenido
0	Código de operación
1	Operando (byte 0)
2	Operando (byte 1)
3	Operando (byte 2)
4	Operando (byte 3)
...	...

Figura 33. Disposición de una instrucción con un operando de 32 bits en la cola interna de la CPU original.

- A continuación, se realiza la decodificación de la próxima instrucción a ejecutar: se obtiene de la cola de la CPU su código de operación (aunque sin desencolarlo ni borrarlo de la misma) y se llevan a cabo las acciones que se requieran para ejecutarlo según corresponda: lectura/escritura en registros, operaciones aritméticas, operaciones de entrada/salida (las cuales implicarán leer o escribir nuevamente sobre la señal "Data"), etc. Si es necesario, se realizará un segundo acceso a la cola para leer el operando asociado a la instrucción actual.
- En el caso excepcional de que el operando asociado a la instrucción sea de 32 bits, se deberá diferir la ejecución de la instrucción hasta el próximo ciclo de reloj, durante el cual se recibirá el byte restante del operando y se ejecutará efectivamente la instrucción demorada.
- Antes de proceder a la próxima sección del informe, se representará en la siguiente figura un ejemplo teórico sobre el progreso esperado a través del tiempo para la ejecución de algunas de las instrucciones que conforman el repertorio ya implementado del procesador para que el lector pueda ver aplicados los conceptos explicados en los incisos anteriores y así terminar de comprender el ciclo de ejecución de esta CPU y la variación en la cantidad de ciclos de reloj necesarios para ejecutar cada instrucción en función del tamaño de su operando:

Instrucción	Progreso de la ejecución de las instrucciones								
MOV EAX, 00000F00H									
INC EAX									
MOV [EBX], EAX									
MOV EBX, 01020304H									
IN AL, 0ABH									

Figura 34. Progreso de la ejecución de algunas instrucciones de prueba en la CPU original a través del tiempo.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones y operandos de las instrucciones de prueba ejecutadas, representa el transcurso de un ciclo de reloj.

3. TDA 1819

A partir de todas las consideraciones previamente mencionadas, se ha determinado que el diseño inicialmente seleccionado no era apto para los objetivos planteados para este proyecto. Por lo tanto, se ha optado por la alternativa de realizar un nuevo modelo absolutamente original que, no obstante, conservara algunos de los aspectos de la arquitectura inicial, como el tamaño del bus de datos, el formato de almacenamiento en memoria y la variabilidad en el formato y el tamaño de las instrucciones, pero con la drástica determinación de cambiar el tipo de la arquitectura de CISC a RISC. Como cabría esperarse, al mantener algunas características del diseño CISC original el resultado final no será un procesador RISC puro sino un "híbrido", aunque como se demostrará a continuación definitivamente presentará más atributos de una arquitectura RISC que de una CISC.

La razón fundamental para llevar a cabo esta migración en el tipo de arquitectura se encuentra basada en la motivación a futuro de implementar una arquitectura "multi-core" a partir de esta CPU, donde cada núcleo pueda ejecutar instrucciones en forma paralela con el resto de los núcleos del conjunto. Un primer paso orientado hacia este paralelismo podría ser la incorporación de una segmentación para el cauce del procesador, a través de la cual se divide el ciclo de ejecución de cada instrucción en etapas con funciones específicas e independientes de manera tal de poder ejecutar múltiples instrucciones al mismo tiempo, aunque cada una en

una etapa distinta. Idealmente, se podrían tener tantas instrucciones en ejecución como etapas en las cuales se divida el cauce. Sin embargo, existen diversas situaciones en las cuales dicho cauce se verá interrumpido debido a atascos que se presenten debido a conflictos por los recursos de hardware (memoria de datos, registro de propósito general), los datos (RAW, WAR, WAW) o la próxima instrucción a ejecutar (dependencia de control). Aunque esta técnica en términos estrictos no implique un paralelismo en la ejecución de las instrucciones (ninguna de las etapas podrá ser realizada simultáneamente), podrá comprobarse que su aplicación reducirá el tiempo de ejecución total del programa, por lo que siempre es recomendable que cualquier procesador posea incorporado este mecanismo.

Como se recordará, la ejecución de las instrucciones en el diseño original de la CPU era estrictamente secuencial, sin que existiera ningún tipo de división de la misma en etapas para implementar la segmentación en el cauce. Ahora bien, al intentar planificar para su arquitectura CISC la manera de subdividir el proceso de ejecución de cada instrucción en tareas más sencillas e independientes entre sí se determinó, en primer lugar, que la implementación más tradicional para la segmentación del cauce (cinco etapas: búsqueda, decodificación, ejecución, acceso a memoria y almacenamiento en registro) era imposible de aplicar en este caso debido a la inexistencia de una instrucción específica para acceder a la memoria de datos ya sea para leer o escribir un valor, lo cual significa que básicamente cualquier tipo de instrucción puede acceder a la misma para buscar un operando antes de proceder con su ejecución. Esta situación es incompatible con la segmentación tradicional, según la cual sólo puede accederse a la memoria de datos después de la etapa de ejecución. Además, la presencia de un número tan elevado de modos de direccionamiento, sumado a la complejidad que poseían algunos de ellos, volvía muy complicada la tarea de plantear un flujo de ejecución general para cualquier instrucción que contemplara todas las alternativas posibles. A su vez, esta adversidad dificultaba considerablemente la labor de diseñar una unidad de control para el nuevo procesador capaz de determinar todos los pasos a seguir para ejecutar una instrucción una vez decodificado e interpretado su código de operación. Finalmente, todo esto permite concluir que resulta sumamente arduo para este tipo de arquitectura llevar a cabo una división del ciclo en ejecución en etapas para segmentar el cauce de la CPU, tal como se había expuesto previamente en la sección introductoria de este informe.

Por su parte, la arquitectura RISC presenta en su ISA (instruction set architecture – repertorio de instrucciones) instrucciones particulares para acceder a la memoria de datos y modos de direccionamiento sencillos y limitados en número. La primera consecuencia de este diseño consiste en que aquí en ningún caso podría existir la posibilidad de que la CPU requiera

acceder a la memoria de datos para obtener un operando antes de proceder con la ejecución de la instrucción en cuestión. Asimismo, se reduce considerablemente el abanico de alternativas posibles de ejecución para una instrucción, por lo cual la tarea de predecir y determinar un flujo de ejecución general para todas las instrucciones se vuelve mucho más fácil y sencilla. Como cabría esperarse, el diseño de la unidad de control encargada de decodificar la próxima instrucción a ejecutar y dictaminar el resto de los pasos para finalizar su ejecución también será más asequible, ya que la mayor parte del trabajo recaerá sobre el ensamblador: el número de instrucciones del programa a traducir a código máquina será superior al requerido para una arquitectura CISC debido a que en esta última una única instrucción puede desempeñar la misma tarea que varias instrucciones RISC, con la desventaja ya previamente mencionada de que su ejecución será considerablemente más compleja y el mayor peso de la labor de llevarla a cabo deberá ser soportado directamente por la unidad de control, dificultando su diseño para el desarrollador del mismo. Por último, como ya se habrá podido adivinar, este tipo de arquitectura es plenamente compatible con la implementación tradicional de cinco etapas para la segmentación del cauce del procesador. De hecho, el simulador WinMIPS64 mencionado en la introducción de este informe representa una CPU cuya arquitectura es de tipo RISC y posee implementada exactamente esta misma metodología de segmentación para acelerar la ejecución de sus instrucciones. Por lo tanto, ya se posee un modelo completamente funcional y operativo como punto de partida para incluir esta técnica en el nuevo procesador a diseñar. Aunque dicho modelo no se encuentre descripto en ningún lenguaje de descripción de hardware, ofrece una perspectiva general, si bien abstracta, sobre el camino a seguir para aplicar efectivamente esta segmentación, sin necesidad de pensar en una división alternativa en etapas para la ejecución de las instrucciones ni en una manera para implementarla en la nueva CPU.

Teniendo en cuenta que el modelo original únicamente ofrecía una descripción para la CPU pero no para la PC de la cual debería formar parte, ni tampoco incluía un ensamblador para traducir las instrucciones escritas por el programador en el lenguaje Assembler en código máquina inteligible para el procesador, otro apartado cuya consideración se consideró prioritaria fue la inclusión tanto de una memoria principal para almacenar las instrucciones y los datos necesarios para la ejecución del programa como de un ensamblador que fuera capaz de leer un archivo creado por el usuario que contuviera un programa Assembler y convertirlo a código máquina. De esta manera podría ofrecerse un diseño tan similar como sea posible al representado en los simuladores MSX88 y WinMIPS64, los cuales presentan tanto la totalidad de los componentes de la PC (CPU, memoria principal y periféricos de E/S) como el software necesario para traducir código Assembler a código binario y así el usuario pueda programar en

un lenguaje más abstracto y cercano a su idioma natural que el código máquina puro. Así también se podría prescindir de la estructura interna incluida en el diseño original para suplantar la ausencia de la memoria principal, la cual funcionaba como una cola auxiliar integrada directamente en el procesador para almacenar temporalmente las instrucciones y los datos transmitidos directamente por el usuario en código máquina hasta que la CPU requiriera hacer uso de ellos. Como se recordará, la existencia de esta dicha estructura no respetaba los principios de la arquitectura von Neumann, según la cual las únicas unidades de almacenamiento permitidas en el procesador son los registros, y además no poseía ningún correlato con la disposición y el funcionamiento real de una CPU, por lo que se recomendaba eliminarla del diseño con la mayor prontitud posible.

Desafortunadamente, considerando que el objetivo planteado para este proyecto no posee inicialmente ninguna relación con las operaciones que puedan involucrar alguno de los periféricos de E/S de la PC (impresora, microconmutadores, leds, pantalla de la terminal, etc.), se ha determinado omitir por el momento la inclusión de estos últimos en el nuevo diseño por considerarse la tarea demasiado laboriosa y compleja aun sabiendo que la computadora permanecería incompleta si nos basamos en los principios de la arquitectura von Neumann, en la cual toda PC debe estar conformada tanto por la CPU y la memoria principal como así también por los dispositivos de E/S para poder interactuar con el usuario. Una situación similar ocurre con las interrupciones de hardware y software: al no haberse juzgado como indispensables para lograr el objetivo del proyecto, su implementación ha sido pospuesta en esta versión inicial del diseño de la PC. Por lo tanto, la modelización de estos atributos quedará como desafío para algún diseñador que se encuentre interesado en optimizar el modelo actual en alguno de estos apartados.

Uno de los atributos cuya implementación se consideró prioritaria fue la incorporación de la mayor cantidad posible de instrucciones del repertorio del procesador, siempre teniendo en cuenta que la migración de la arquitectura CISC a una RISC podría acarrear modificaciones considerables en el formato de dichas instrucciones debido a la limitación y la simplificación en los modos de direccionamiento y en el funcionamiento de las mismas hasta tal punto de tener que redefinirlas por completo desde el comienzo a partir de los principios de la arquitectura RISC. Afortunadamente ya se dispone del simulador WinMIPS64 con un repertorio de instrucciones plenamente definido, operativo y compatible con una CPU de tipo RISC, el cual obviamente fue utilizado como base para determinar el repertorio para el nuevo procesador. Partiendo de la premisa de que resultaría imposible obtener una implementación total del nuevo repertorio debido a su elevado número de instrucciones y la considerable complejidad que

presentan algunas de ellas, se decidió prestarles especial atención a aquellas instrucciones vinculadas a las operaciones de transferencia de datos y de control, aritmético-lógicas, control y de desplazamiento de bits, ya que éstas son las que el programador suele utilizar con mayor asiduidad durante la escritura de sus programas y al mismo tiempo resultan ser para el diseñador las más sencillas de incorporar al diseño dado que no requieren alterar drásticamente el diseño de la CPU ni modelar nuevos componentes sino simplemente realizar algunas modificaciones a aquéllos ya existentes para que puedan buscar, decodificar y ejecutar las nuevas instrucciones. En cambio, la incorporación de otro tipo de instrucciones relacionados con el manejo de pila y subrutinas, interrupciones y periféricos de E/S requeriría, además de los cambios previamente mencionados, no sólo diseñar nuevos componentes para representar y gestionar el comportamiento de los nuevos dispositivos sino también remodelar el diseño original para poder adaptarlo a ellos, por lo cual la consideración de estos tópicos se ha omitido por completo para esta versión inicial de la CPU. Aun así, se asume que la cantidad y variedad de las instrucciones implementadas hasta el momento resultan más que suficientes para que el usuario pueda desarrollar distintos programas con el nivel de complejidad necesario para comprobar con un alto grado de detalle y profundidad el comportamiento del procesador ante las diversas tareas que se le asignen a través de dichos programas.

Con respecto al diseño de los componentes internos de la nueva CPU, se optó por tomar como referencia la arquitectura von Neumann, según la cual el procesador debe presentar tres secciones principales: la unidad de control, la unidad aritmético-lógica (ALU) y el banco de registros, cada una de las cuales encargada de cumplir una función particular e independiente de las demás. En la sección anterior ya se había descripto la disposición interna planificada para la CPU original, la cual consistía en realidad de un único componente central que concentraba en su lógica interna todas las funcionalidades de las tres secciones del procesador previamente mencionadas: decodificar la próxima instrucción a ejecutar y determinar el flujo restante de su ciclo de ejecución (unidad de control); y, en caso de ser necesario, realizar la operación aritmético-lógica requerida para ejecutarla (ALU) y/o almacenar un valor final en uno de los registros de propósito general del procesador (banco de registros). También se encargaba de llevar a cabo la búsqueda de la próxima instrucción a ejecutar y de gestionar los accesos a la memoria principal (en este caso, la cola interna auxiliar del procesador) para realizar operaciones de lectura o escritura sobre ella. Como se recordará, se había mencionado como principal inconveniente de este diseño la posibilidad casi segura de que, cuando el diseñador decidiera comenzar a incorporar nuevas prestaciones al procesador (más instrucciones del repertorio, segmentación del cauce, memoria principal y periféricos de E/S, operaciones en

punto flotante, etc.), el modelo se volvería tan complejo que la implementación de dichas mejoras se volvería prácticamente inviable a menos que se dividiera y asignara cada una de las tareas de la CPU a un componente separado. Por lo tanto, al planificar el nuevo diseño se planteó inicialmente la labor de modelar tres componentes individuales para representar las tres secciones principales de la CPU, cada una con su propia tarea particular y exclusiva; así como también se consideraron las cinco etapas que constituyen el ciclo de ejecución de la implementación tradicional de la segmentación del cauce ya que éste será al fin y al cabo el que terminará rigiendo el comportamiento general de la CPU, concluyéndose así la necesidad de incorporar dos componentes adicionales, uno encargado exclusivamente de realizar la búsqueda de la siguiente instrucción y actualizar el Puntero de Instrucción (etapa de búsqueda) y otro para gestionar el acceso a la memoria principal desde la CPU para operaciones de lectura o escritura (etapa de acceso a memoria). Por su parte, las restantes etapas del ciclo de ejecución ya se encuentran asociadas y representadas cada una por una de las secciones principales de la CPU: decodificación (unidad de control), ejecución (ALU) y almacenamiento en registro (banco de registros), por lo que resulta innecesario diseñar componentes adicionales para modelar sus respectivos comportamientos. De esta manera, mientras que en el modelo original cualquier modificación podía tener repercusiones sobre el funcionamiento de la totalidad de la CPU, aquí resulta posible realizar alteraciones sobre uno de los componentes sin que los demás se vean afectados. Por ejemplo, si se desean implementar nuevas instrucciones del repertorio, en un principio sólo sería necesario modificar la unidad de control para que reconozca e interprete los nuevos códigos de operación, sin necesidad de alterar el resto de los componentes ni que su funcionalidad se vea comprometida en caso de que los cambios no hayan sido correctamente incorporados, lo cual sería imposible en el modelo original. Además, al dividir el trabajo total de la CPU en subtareas más pequeñas el diseño final se simplifica considerablemente, resultando el mismo más fácil de entender y modificar principalmente para un diseñador que no se encuentre familiarizado con él y desee realizarle cualquier tipo de optimizaciones.

Se entendió que la máquina de estados desempeña un rol esencial en el diseño de la PC en primer lugar por su capacidad para administrar el comportamiento global de la CPU gestionando la transición entre las distintas etapas posibles del ciclo de ejecución del procesador a través del tiempo en función de la etapa ejecutada anteriormente, el tipo de la instrucción actualmente en ejecución y, en caso de encontrarse activada la segmentación del cauce, la presencia de posibles atascos que difieran temporalmente la ejecución tanto de la instrucción actual como de las subsiguientes. La máquina se ha diseñado de manera tal que cada estado estaría representando una etapa distinta del ciclo de ejecución, aunque la acción a realizar en

cada uno de ellos podría variar según sea requerido por la instrucción ejecutada. La tarea de la máquina de estados se vuelve especialmente vital y compleja si el cauce se encuentra segmentado ya que la máquina de estados deberá ser capaz de gestionar simultáneamente las transiciones entre etapas de múltiples instrucciones ejecutándose en forma paralela, aunque, como se recordará, siempre lo harán en etapas diferentes debido a las obvias limitaciones en el hardware de la CPU que impiden que la ejecución de las mismas sea estrictamente paralela. Por otra parte, esta máquina representa para el usuario la interfaz más intuitiva y eficaz para analizar y comprender externamente el funcionamiento del procesador, manteniéndose detalladamente informado y actualizado a través de ella sobre la actividad del mismo en todo momento. Dicha información le permitirá identificar unívocamente la/s instrucciones del programa que están siendo ejecutadas en un instante dado y la etapa exacta del ciclo de ejecución en la cual se encuentra cada una de ellas. Sin esta interfaz, el único método del cual podría disponer el usuario para interiorizarse sobre el estado de la CPU sería estudiar las variaciones a través del tiempo en los valores contenidos en cada una de las unidades de almacenamiento de la PC (memoria principal y banco de registros de propósito general), así como también los cambios en las señales transmitidas entre las distintas etapas conforme progresa la ejecución del programa. Por lo tanto, se ha realizado e invertido todo el esfuerzo necesario para que en este diseño la máquina de estados refleje fielmente la actividad del procesador de una manera fácil de interpretar para el usuario, utilizando como modelo la interfaz gráfica provista por el simulador WinMIPS64 para representar en el tiempo el avance de la ejecución de las instrucciones a través de las etapas en las que se encuentra segmentado el cauce del procesador, las cuales resultan coincidir con aquéllas elegidas para segmentar el cauce de la nueva CPU, con algunas ligeras modificaciones que serán explicadas en la próximamente en esta misma sección.

En consonancia con las conclusiones ofrecidas en la sección anterior sobre las desventajas presentadas por el diseño original del procesador, se ha optado por simplificar el nuevo modelo tanto como fuera posible, eliminando por completo cualquier tipo de señales prescindibles cuya tarea no se encontrara plenamente vinculada con el objetivo principal de este proyecto. Por lo tanto, esta CPU no poseerá la posibilidad de reiniciarse a menos que se reinicie la totalidad de la simulación, además de que su flujo de ejecución también se ha modificado considerablemente respecto al original: ahora el usuario se limitará simplemente a impartir a la PC la orden de ejecutar el programa y se abstraerá del resto de la ejecución, encargándose la CPU por sí sola se ejecutar secuencialmente las instrucciones a medida que se vayan completando las anteriores sin esperar ninguna clase de señal de reconocimiento por parte del

usuario. De esta manera, se puede concentrar el diseño exclusivamente en la implementación de la segmentación del cauce, la cual se asume que será la parte más compleja y delicada del modelo, sin mencionar la ALU, la unidad de control y el banco de registros, e incluso la memoria principal y el ensamblador si se considera que el diseño final consistirá en una computadora que sea capaz de recibir programas escritos por el usuario/programador en Assembler, traducirlos a código máquina y ejecutarlos en su procesador para alcanzar el o los objetivos por el mismo. Si a todos estos componentes se les agregaran las funcionalidades del modelo original, se estaría dificultando innecesariamente una descripción ya de por sí sumamente compleja para el diseñador, quien agradecerá cualquier tipo de facilitación que se le ofrezca para desarrollar su tarea.

Como se recordará, se había mencionado como una de las principales desventajas del diseño original la metodología seleccionada para realizar la descripción del mismo, conocida como comportamental. Aunque la utilización de la misma en lugar de otra de las técnicas disponibles pudo haberle resultado en un principio más fácil y atractiva al diseñador por ser la estructura de su código la más semejante a la de los lenguajes de programación de alto nivel, precisamente por esta última razón el resultado obtenido será demasiado abstracto y alejado de la representación real de un circuito digital. Esto se debe a que, al estar esta metodología más orientada al software que al hardware, a pesar de que existe la posibilidad de que a través de ella se pueda describir correctamente el comportamiento esperado por parte de dicho circuito, el diseño terminará concentrando toda su lógica en un único componente extremadamente complejo encargado de ejecutar por sí sólo todas las tareas necesarias para cumplir los objetivos planteados para el modelo. A medida que el circuito a representar se complejice, aún más acentuadamente se presentará este fenómeno y, en el caso de este proyecto en particular, al tratarse el procesador precisamente del circuito digital más complicado posible de todos los que existen, conforme se le incorporen nuevas funcionalidades y mejoras eventualmente el modelo se volverá prácticamente ininteligible e inmanejable para cualquier diseñador, especialmente si nunca antes había trabajado sobre él. Por lo tanto, antes de alcanzar esta situación límite, se aprovechó la oportunidad que presentaba la determinación de replantear todo el diseño de la CPU desde el principio para proceder directamente a cambiar por completo la metodología utilizada para llevar a cabo su descripción: en lugar del método comportamental se recurrirá al estructural, el cual permite definir el circuito a partir de instancias de múltiples componentes, representando cada componente a una sección de hardware distinta dentro de la PC en general (CPU, memoria principal y periféricos de E/S cuando estos últimos se implementen en el diseño) y del procesador en particular (unidad de control, ALU, banco de registros, etc.). Se

observa que, al menos inicialmente, se deberá diseñar una jerarquía de tres niveles para obtener una descripción fiel de la computadora, ubicándose la PC en el nivel superior, sus tres partes fundamentales en el nivel intermedio y, finalmente, los componentes de la CPU en el nivel inferior del diseño. Mientras que en la metodología comportamental la totalidad de las actividades a realizar por todos los dispositivos previamente mencionados debería ser llevada a cabo por un único componente con la consecuencia de que su lógica interna sería extremadamente compleja; en la descripción estructural, al dividir el problema general en tareas menores entre distintos componentes independientes entre sí, cada uno responsable de ejecutar una de ellas, se obtendrá un diseño más fácil de comprender y de modificar para cualquier diseñador ya que, al ser su funcionalidad más sencilla, ninguno de estos componentes individuales podrá poseer una lógica tan compleja como la del componente central que resultaría de utilizar el método comportamental.

3.1. Arquitectura

A continuación, se describirán las características principales que definen la arquitectura de la nueva computadora. Como se explicó en la introducción de esta sección del informe (“3. TDA 1819”), se observarán notorias diferencias respecto al diseño de la PC original, las cuales incluyen tanto atributos que ya se encontraban implementados como funcionalidades adicionales que se han incorporado en esta versión.

3.1.1. Atributos generales

Se comenzará presentando una tabla con los atributos generales del nuevo diseño tal y como se hizo en el caso del procesador original. En ella se podrá observar la presencia de funcionalidades adicionales, como la implementación de la segmentación en el cauce, de la memoria principal y de registros de uso particular, resultando la incorporación de estos dos últimos componentes esencial para que la descripción de la PC sea lo más fiel posible a la estructura y el comportamiento de una computadora real. También se ha elevado considerablemente la cantidad de registros de uso general tanto enteros como de punto flotante y se han modificado algunos de los modos de direccionamiento para que las instrucciones para que ahora requieran un único ciclo de reloj para ejecutarse, tal y como debería ser para una arquitectura de tipo RISC. Sin embargo, ya sea para conservar algunos de los atributos del

diseño original que ya se encontraban implementados o bien por decisión personal del autor del informe por considerarlas más fáciles de incorporar se decidió incluir en el procesador características propias de una arquitectura de tipo CISC, como una interfaz de entrada/salida independiente de la memoria principal, formatos y longitudes variables para las instrucciones del repertorio y modos de direccionamiento adicionales, entre otras. Por lo tanto, tal y como se había mencionado en la introducción de esta sección del informe ("3. TDA 1819"), el resultado final no será una arquitectura RISC pura sino un híbrido, es decir, una combinación entre ambos tipos de arquitectura, aunque, como se apreciará a continuación, presentando un mayor número de características que la inclinan hacia el tipo RISC.

Tabla 35. Atributos generales del nuevo diseño de la computadora.

Tipo de arquitectura	RISC
Ciclos de ejecución	Cada instrucción requiere exactamente un único ciclo de reloj para ejecutarse
Interfaz de Entrada/Salida	E/S independiente (*) (***)
Formato y longitud de las instrucciones	Instrucciones de formato y longitud variables: dos, cuatro, cinco, seis u ocho bytes en función de su cantidad y tamaño de operandos (***)
Frecuencia del reloj	10 MHz
Bus de datos	32 bits
Bus de direcciones	16 bits
Bus de control	2 bits: uno para determinar el tipo de operación a realizar (lectura o escritura) y otro para establecer la sección de memoria a acceder (datos, instrucciones o pila)
Mínima unidad de memoria direccionable	1 byte
Tamaño total de memoria	64 kilobytes
Modos de direccionamiento	Implícito (*) (***)
	Inmediato
	Directo de Registro
	Directo de Memoria
	Indirecto con Desplazamiento Relativo al SP (*) (***)
Ejecución de operaciones aritmético-lógicas	1 ALU (Arithmetic Logic Unit o Unidad Aritmético-Lógica) para operandos enteros

	1 FPU (Floating-Point Unit o Unidad de Punto Flotante) para operandos en punto flotante
Registros de uso general	16 registros enteros: r0, r1, r2, r3, r4, r5, r6, r7, r8, r9, r10, r11, r12, r13, r14 y r15
	16 registros de punto flotante: f0, f1, f2, f3, f4, f5, f6, f7, f8, f9, f10, f11, f12, f13, f14 y f15
Registros de uso particular	IR (Instruction Register o Registro de Instrucción, 8 bits)
	IP (Instruction Pointer o Puntero de Instrucción, 16 bits)
	FLAGS (Banderas, 8 bits) (**) (***)
	FPFLAGS (Banderas para Punto Flotante, 8 bits) (**) (***)
	BP (Base Pointer o Puntero Base, 32 bits) (*) (***)
	SP (Stack Pointer o Puntero de Pila, 32 bits) (***)
	RA (Return Address o Dirección de Retorno, 32 bits) (*) (***)
Segmentación	Cinco etapas: búsqueda, decodificación, ejecución, acceso a memoria y almacenamiento en registro
Almacenamiento en memoria	Little Endian (***)

Observaciones:

- (*): Atributo sin implementar.
- (**): Atributo parcialmente implementado: se explicará con mayor detenimiento en una observación posterior y en la próxima tabla de este informe.
- (***): Atributo incompatible con una arquitectura RISC:
    - ❖ Interfaz de Entrada/Salida: RISC propone que tanto la memoria principal como los periféricos de E/S compartan el mismo espacio de direcciones y las mismas instrucciones. Sin embargo, aquí se optó por separar ambas secciones, por lo que existirá un espacio de direcciones distinto y aislado e instrucciones particulares para realizar interactuar con los periféricos de E/S que posea la PC, aunque éstas aún no se encuentren implementadas en el diseño actual del TDA 1819.
    - ❖ Formato y longitud de las instrucciones: RISC establece que todas las instrucciones deben poseer un formato único y fijo. No obstante, en este diseño se determinó que la longitud de cada instrucción pueda variar en función de la cantidad y el tamaño de sus operandos para evitar desperdiciar espacio en memoria con direcciones sin utilizar por reservar más lugar del estrictamente necesario para almacenar la instrucción en cuestión.

- Modos de direccionamiento: en la arquitectura RISC los modos de direccionamiento deben reducirse y limitarse en número y complejidad tanto como sea posible. Sin embargo, en este caso se incluyeron un par de modos adicionales que no serían imprescindibles para implementar todas las instrucciones del repertorio. Esto se debe a la decisión de incorporar un conjunto de instrucciones particulares para el manejo de la pila de la PC, también contradiciendo los principios del tipo RISC, según los cuales no debería existir ningún motivo para implementar instrucciones separadas para el acceso a la pila ya que el usuario tendría que ser capaz de utilizarla manipulando directamente el SP (Puntero de Pila) como si se tratara de uno de los registros de uso general de la CPU. De todas maneras, dichas instrucciones aún no se encuentran implementadas en su totalidad, por lo cual el programador no debería prestar atención a esta cuestión por el momento.
- Registros de uso particular:
  - Registros FLAGS y FPFLAGS: La arquitectura CISC presenta un registro de uso particular llamado FLAGS, cuyos bits representan banderas asociadas al estado actual del procesador, principalmente respecto a su última operación aritmético-lógica ejecutada y la posibilidad de atender o ignorar peticiones vinculadas a interrupciones de hardware enmascarables. Esto se debe a que el repertorio de un procesador CISC posee instrucciones explícitas para realizar saltos condicionales en función del valor de la bandera que se desee (por ejemplo: “jz” y “jnz” para la bandera “Z”, “js” y “jns” para la bandera “S”, etc.). A partir de esto, la idea sería que el usuario realice una operación aritmético-lógica (suma, resta, comparación, etc.) y, a partir del estado posterior a su ejecución y del objetivo buscado, determine el tipo de bandera y, en consecuencia, del salto condicional a utilizar. En cambio, el repertorio asociado a la arquitectura RISC cuenta con instrucciones (“beq”, “bne”, “beqz” y “bnez”) para realizar saltos condicionales que efectúan ambos pasos a la vez: llevan a cabo la operación aritmético-lógica y la comparación necesaria para determinar si se cumple o no la condición de salto establecida, sin necesidad de utilizar la mayoría de las banderas del registro FLAGS de la arquitectura CISC, por lo que suele descartarse la incorporación de dicho registro en los procesadores RISC. No obstante, en el caso de la nueva CPU se determinó que la presencia tanto de interrupciones de hardware enmascarables (aún no implementadas) como de

instrucciones de transferencia de control especiales (“bfpt” y “bfpf”) que se comportan de una manera similar a los saltos condicionales en CISC, imponía la necesidad de preservar la existencia del registro en el nuevo diseño con la misma funcionalidad que tendría en una arquitectura CISC. Además, al incorporarse adicionalmente la unidad de punto flotante o FPU junto con la unidad aritmético-lógica o ALU, se juzgó conveniente crear un nuevo registro de banderas llamado FPFLAGS con un contenido y comportamiento similar al registro FLAGS original, aunque, mientras FLAGS reflejará el estado de la última operación ejecutada en la ALU, FPFLAGS se asociará a las operaciones ejecutadas en la FPU. De esta manera, ambas unidades pueden trabajar en forma paralela libre e independientemente sin que ninguna tenga que preocuparse por la posibilidad de intentar actualizar simultáneamente su estado sobre el mismo registro. No obstante, la elección de la arquitectura RISC para esta computadora implica que la mayoría de las banderas de ambos registros quedarán inutilizadas dado que sus valores no presentan ningún interés para la ejecución de las instrucciones de salto condicionales. Por lo tanto, la implementación de los mismos en este diseño se encuentra parcialmente finalizada, por lo que no puede garantizarse su correcta actualización posterior a la ejecución de algunas de las operaciones aritmético-lógicas más complejas como la multiplicación y la división tanto en la ALU como en la FPU.

- Registros BP, SP y RA: En consonancia con la observación realizada anteriormente respecto a los modos de direccionamiento existentes en la arquitectura de la nueva computadora, en este diseño la presencia de instrucciones explícitas para apilar y desapilar valores en la pila implica que tanto el BP (Puntero Base) como el SP (Puntero de Pila) pertenecen al conjunto de registros de uso particular del procesador, es decir, no pueden ser accedidos ni modificador por el usuario bajo ningún concepto, mientras que normalmente en una arquitectura RISC esto no suele ser así, ya que, al no existir instrucciones particulares para el manejo de la pila el programador obligatoriamente debe manipular estos registros para poder leer o almacenar datos en ella. Algo similar ocurre con el registro RA (Dirección de Retorno): a diferencia de una arquitectura RISC tradicional, en este caso sí existe un

manejo implícito de la pila en la invocación a las subrutinas a través de la ejecución de instrucciones especiales del repertorio (“call” y “ret”) que se encargan automáticamente de apilar y desapilar la dirección de retorno según corresponda sin intervención del usuario, por lo cual éste no tendría ningún motivo para acceder al RA, por lo tanto este registro también se incluyó dentro del conjunto de registros de uso particular, mientras que en una RISC normal suele formar parte de los registros de uso general. De todas maneras, dichas instrucciones aún no se encuentran implementadas, ergo, el programador no debería prestar atención a esta cuestión por el momento.

- Almacenamiento en memoria: por último, mientras que los procesadores cuya arquitectura es de tipo RISC suelen almacenar los bytes en la memoria principal de la PC desde el más significativo en la dirección más baja hacia el menos significativo en la más alta (Big Endian), en este modelo se determinó e implementó el formato y orden inversos: los bytes se guardarán desde el menos significativo en la dirección más baja hacia el más significativo en la más alta (Little Endian). Esta decisión se debió simplemente a una preferencia personal del autor de este informe ya que esta última manera de almacenar la información le pareció más lógica y natural que la primera. Cabe aclarar que esta modificación actúa sobre un nivel de abstracción tan bajo que el usuario que escriba programas para la CPU en el lenguaje Assembler ni siquiera debería percatarse de la diferencia, por lo cual su manera de programar no tendría que sufrir ningún cambio a pesar de que el formato de almacenamiento en memoria elegido por el diseñador sea incompatible con el de los procesadores RISC tradicionales.
- Para más detalles, comparar con la tabla 1 presentada en la introducción de este informe.

Como se acaba de mencionar en una de las observaciones de la tabla anterior, en este diseño se encuentran incluidos dos registros de banderas, FLAGS y FPFLAGS, asociados a las operaciones aritmético-lógicas realizadas en la ALU y la FPU respectivamente. Ambos poseen las mismas banderas, pero su valor no será el mismo ya que representan el estado de dos unidades de ejecución distintas. A continuación, se especifican la disposición y ubicación de dichas banderas en los dos registros y el propósito de cada una de ellas:

Tabla 36. Banderas de los registros FLAGS y FPFLAGS del nuevo procesador.

	Bit 7	**Bit 6**	**Bit 5**	**Bit 4**	**Bit 3**	**Bit 2**	**Bit 1**	**Bit 0**
Bandera	F	I	Z	S	O	C	A	P

Observaciones:

- Bandera F (Punto Flotante): su valor se actualiza en el registro FPFLAGS sólo si en la FPU se ejecuta alguna de las instrucciones de comparación especiales incorporadas en el repertorio a tal efecto. Esta bandera es utilizada por ciertas instrucciones particulares de transferencia de control condicional (saltos condicionales) en función del resultado obtenido en la última comparación realizada en la FPU. Por su parte, en el registro FLAGS la bandera permanece inutilizada y su valor, irrelevante.
- Bandera I (Interrupción): se asocia al manejo de interrupciones de hardware enmascarables, es decir, aquellas solicitudes por parte de los dispositivos de hardware cuyas prioridades son lo suficientemente bajas como para que su atención pueda ser ignorada por la CPU si el usuario así lo deseara. Si el valor de la bandera es un uno, las peticiones serán atendidas; en cambio, si es un cero, serán ignoradas. Esta bandera sólo posee relevancia en el registro FLAGS, mientras que en el registro FPFLAGS permanece inutilizada.
- Bandera Z (Cero): ofrece información asociada al estado de la última operación aritmético-lógica ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). Si el valor de la bandera es un uno, el resultado obtenido fue igual a cero; en cambio, si es un cero, fue distinto de cero. En una arquitectura CISC esta bandera suele utilizarse para la evaluación de saltos condicionales, pero aquí, al tratarse de una arquitectura RISC, su utilidad es prácticamente nula, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento no está plenamente garantizado para todas las operaciones aritmético-lógicas ejecutadas tanto en la ALU como en la FPU.
- Bandera S (Signo): ofrece información asociada al estado de la última operación aritmético-lógica ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). El valor de esta bandera coincide exactamente con el del bit más significativo del resultado obtenido. De esta manera, si se interpretara a dicho resultado en términos del sistema de representación conocido como complemento a dos, podría decirse que, si el valor de la bandera es un uno, el resultado de la última operación aritmético-lógica fue negativo; en

cambio, si es un cero, fue un valor natural. En una arquitectura CISC esta bandera suele utilizarse para la evaluación de saltos condicionales, pero aquí, al tratarse de una arquitectura RISC, su utilidad es prácticamente nula, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento no está plenamente garantizado para todas las operaciones aritmético-lógicas ejecutadas tanto en la ALU como en la FPU.

- Bandera O (Overflow o Desbordamiento): ofrece información asociada al estado de la última operación aritmética ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). Más precisamente, indica si durante la operación se produjo un desbordamiento aritmético, es decir, si el número de bits utilizados para realizar dicha operación (ancho de la ALU) no alcanza para representar el resultado en complemento a dos. Si el valor de la bandera es un uno, el desbordamiento ocurrió; en cambio, si es un cero, no se produjo. Esta bandera sólo debe ser tenida en cuenta si los operandos de la operación eran valores con signo, en caso contrario no posee ninguna utilidad. En una arquitectura CISC esta bandera suele utilizarse para la evaluación de saltos condicionales, pero aquí, al tratarse de una arquitectura RISC, su utilidad es prácticamente nula, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento no está plenamente garantizado para todas las operaciones aritméticas ejecutadas tanto en la ALU como en la FPU.
- Bandera C (Carry o Acarreo): ofrece información asociada al estado de la última operación aritmética ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). Más precisamente, indica si durante la operación se produjo un acarreo aritmético desde el bit más significativo del resultado (sólo si la operación realizada se hubiera tratado de una suma) o bien si existió un préstamo aritmético hacia el bit más significativo del primero de los dos operandos (sólo si la operación realizada se hubiera tratado de una resta). Si el valor de la bandera es un uno, el acarreo o préstamo ocurrió; en cambio, si es un cero, no se produjo. Esta bandera permite llevar a cabo sumas o restas entre números que requieren una mayor cantidad de bits para ser representados que el ancho de la ALU o la FPU, acarreando o sumando un dígito binario de una suma o resta parcial al bit menos significativo de una palabra o conjunto de bits más significativos. Además, en una arquitectura CISC esta bandera suele emplearse para la evaluación de saltos condicionales, pero aquí, al tratarse de una arquitectura RISC, su utilidad se reduce considerablemente, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento

no está plenamente garantizado para todas las operaciones aritméticas ejecutadas tanto en la ALU como en la FPU.

- Bandera A (Acarreo Auxiliar): ofrece información asociada al estado de la última operación aritmética ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). Más precisamente, indica si durante la operación se produjo un acarreo aritmético desde el cuarto bit menos significativo del resultado (sólo si la operación realizada se hubiera tratado de una suma) o bien si existió un préstamo aritmético hacia el cuarto bit menos significativo del primero de los dos operandos (sólo si la operación realizada se hubiera tratado de una resta). Si el valor de la bandera es un uno, el acarreo o préstamo ocurrió; en cambio, si es un cero, no se produjo. Esta bandera suele utilizarse principalmente a fin de ofrecer apoyo para operaciones aritméticas cuyos operandos se encuentran expresados en el sistema de representación BCD (Binary-Coded Decimal o Decimal Codificado en Binario), en el cual cada dígito decimal es codificado con una secuencia de cuatro bits. No obstante, al considerarse altamente improbable que el usuario realice operaciones aritméticas en dicho sistema de representación, esta bandera no se juzgó como imprescindible para este diseño, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento no está plenamente garantizado para todas las operaciones aritméticas ejecutadas tanto en la ALU como en la FPU.
- Bandera P (Paridad): ofrece información asociada al estado de la última operación aritmética ejecutada en la ALU (registro FLAGS) o la FPU (registro FPFLAGS). Más precisamente indica si, en la representación binaria del resultado obtenido (en el sistema binario sin signo o complemento a dos según corresponda), el número total de bits cuyo valor es igual a uno es par o impar. Si el valor de la bandera es un uno, el número de bits es par; en cambio, si es un cero, es impar. En una arquitectura CISC esta bandera suele utilizarse para la evaluación de saltos condicionales, pero aquí, al tratarse de una arquitectura RISC, su utilidad es prácticamente nula, por lo que su implementación se encuentra incompleta y su correcto comportamiento y funcionamiento no está plenamente garantizado para todas las operaciones aritméticas ejecutadas tanto en la ALU como en la FPU.

3.1.2. Directivas y repertorio de instrucciones

A continuación, se incluye una tabla que contiene las directivas reconocidas por el ensamblador diseñado e incorporado a la PC con el objetivo de traducir las instrucciones escritas por el usuario en un lenguaje de programación de bajo nivel (Assembler) a un código máquina que pueda resultar inteligible para la unidad de control de la nueva CPU, la cual a su vez se encargará de coordinar su posterior ejecución conforme logre interpretarlas. Dichas directivas deberán ser especialmente tenidas en cuenta por cualquier individuo que desee escribir programas en el lenguaje Assembler para ser ejecutados por el procesador ya que a través de las mismas será capaz de definir las distintas secciones del programa (datos, subrutinas y código del programa principal), así como también los diversos tipos de datos que pueden adoptar las variables a declarar. Se puede observar que la elección de las directivas por parte del diseñador se encuentra basada en aquéllas ya ofrecidas en el simulador WinMIPS64 para su propio ensamblador.

Tabla 37. Directivas válidas para el ensamblador de la nueva computadora.

Directiva	**Comentario**
.data	Comienzo de la sección de datos
.subr	Comienzo de la sección de subrutinas (*)
.text	Comienzo de la sección de código del programa principal
.code	Comienzo de la sección de código del programa principal (igual que .text)
.ascii *s1, s2, ...*	Almacena cadena/s ASCII
.asciiz *s1, s2, ...*	Almacena cadena/s ASCII terminada/s en cero
.byte *n1, n2, ...*	Almacena número/s de 8 bits con signo
.ubyte *n1, n2, ...*	Almacena número/s de 8 bits sin signo
.hword *n1, n2, ...*	Almacena número/s de 16 bits con signo
.uhword *n1, n2, ...*	Almacena número/s de 16 bits sin signo
.word *n1, n2, ...*	Almacena número/s de 32 bits con signo
.uword *n1, n2, ...*	Almacena número/s de 32 bits sin signo
.float *n1, n2, ...*	Almacena número/s en punto flotante (32 bits)
0x*n*	Identifica un número representado en el sistema hexadecimal
0b*n*	Identifica un número representado en el sistema binario

Observaciones:

- (*): Directiva sin implementar.
- *s1, s2, ...*: Denota una sucesión de cadenas de caracteres ASCII como “hola”, “bienvenido”, “lunes”, etc.
- *n1, n2, ...*: Denota una sucesión de números enteros o en punto flotante según corresponda como “1”, “25”, “-5.6”, etc.
- *n*: Denota un número representado en el sistema hexadecimal o binario según corresponda como “FD”, “1A”, “00101011”, “10011100”, etc.

Otro de los pilares fundamentales que el usuario debería considerar, estudiar y aprender si posee la intención de escribir programas en el lenguaje Assembler que resulten compatibles con la nueva CPU es el repertorio de instrucciones que pueden ser ejecutadas por la misma. En la siguiente tabla se incluyen tanto las instrucciones que ya se encuentran implementadas como las que aún no lo están, en caso de que el lector tenga el interés de incorporarlas por sus propios medios al modelo actual o al menos desee conocer en términos generales cuál sería la manera que el diseñador tiene pensada para añadir al procesador las funcionalidades pendientes. Como podrá deducirse rápidamente, los repertorios del simulador WinMIPS64 y en menor medida del MSX88 han influenciado de algún modo sobre la forma y la disposición elegidas para denominar todas las nuevas instrucciones.

Tabla 38. Repertorio de instrucciones de la nueva CPU.

Tipo de instrucción	Instrucción	Comentario
Transferencia de Datos	lb r_d, $Inm(r_i)$	Copia en r_d un byte (8 bits) desde la dirección ($Inm + r_i$)
	sb r_f, $Inm(r_i)$	Guarda los 8 bits menos significativos de r_f en la dirección ($Inm + r_i$)
	lh r_d, $Inm(r_i)$	Copia en r_d un half-word (16 bits) desde la dirección ($Inm + r_i$)
	sh r_f, $Inm(r_i)$	Guarda los 16 bits menos significativos de r_f a partir de la dirección ($Inm + r_i$)

	lw r_d, $Inm(r_i)$	Copia en r_d un word (32 bits) desde la dirección ($Inm + r_i$)
	sw r_f, $Inm(r_i)$	Guarda los 32 bits de r_f a partir de la dirección ($Inm + r_i$)
	lf f_d, $Inm(r_i)$	Copia en r_d un valor en punto flotante (32 bits) desde la dirección ($Inm + r_i$)
	sf f_f, $Inm(r_i)$	Guarda los 32 bits de f_f a partir de la dirección ($Inm + r_i$)
	mff f_d, f_f	Copia el valor del registro f_f (32 bits) al registro f_d
	mfr f_d, r_f	Copia los 32 bits del registro r_f entero al registro f_d de punto flotante
	mrf r_d, f_f	Copia los 32 bits del registro f_f de punto flotante al registro r_d entero
	tf f_d, f_f	Convierte a punto flotante el valor entero copiado al registro f_f (32 bits), dejándolo en f_d
	ti f_d, f_f	Convierte a entero el valor en punto flotante contenido en f_f (32 bits), dejándolo en f_d
Aritmética	dadd r_d, r_f, r_g	Suma r_f con r_g, dejando el resultado en r_d (valores con signo)
	daddi r_d, r_f, N	Suma r_f con el valor inmediato N, dejando el resultado en r_d (valores con signo)
	daddu r_d, r_f, r_g	Suma r_f con r_g, dejando el resultado en r_d (valores sin signo)
	daddui r_d, r_f, N	Suma r_f con el valor inmediato N, dejando el resultado en r_d (valores sin signo)
	addf f_d, f_f, f_g	Suma f_f con f_g, dejando el resultado en f_d (en punto flotante)
	dsub r_d, r_f, r_g	Resta r_g a r_f, dejando el resultado en r_d (valores con signo)
	dsubu r_d, r_f, r_g	Resta r_g a r_f, dejando el resultado en r_d (valores sin signo)

	subf f_d, f_f, f_g	Resta f_g a f_f, dejando el resultado en f_d (en punto flotante)
	dmul r_d, r_f, r_g	Mutiplica r_f con r_g, dejando el resultado en r_d (valores con signo)
	dmulu r_d, r_f, r_g	Mutiplica r_f con r_g, dejando el resultado en r_d (valores sin signo)
	mulf f_d, f_f, f_g	Multiplica f_f con f_g, dejando el resultado en f_d (en punto flotante)
	ddiv r_d, r_f, r_g	Divide r_f por r_g, dejando el resultado en r_d (valores con signo)
	ddivu r_d, r_f, r_g	Divide r_f por r_g, dejando el resultado en r_d (valores sin signo)
	divf f_d, f_f, f_g	Divide f_f por f_g, dejando el resultado en f_d (en punto flotante)
	slt r_d, r_f, r_g	Compara r_f con r_g, dejando r_d =1 si r_f es menor que r_g (valores con signo)
	slti r_d, r_f, N	Compara r_f con el valor inmediato N, dejando r_d =1 si r_f es menor que N (valores con signo)
	ltf f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es menor que f_g (en punto flotante)
	lef f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es menor o igual que f_g (en punto flotante)
	eqf f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es igual que f_g (en punto flotante)
	negr r_d, r_f	Calcula el opuesto de r_f, dejando el resultado en r_d (valores con signo)
Lógica	andr r_d, r_f, r_g	Realiza un AND entre r_f y r_g (bit a bit), dejando el resultado en r_d
	andi r_d, r_f, N	Realiza un AND entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d
	orr r_d, r_f, r_g	Realiza un OR entre r_f y r_g (bit a bit), dejando el resultado en r_d
	ori r_d, r_f, N	Realiza un OR entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d

	xorr r_d, r_f, r_g	Realiza un XOR entre r_f y r_g (bit a bit), dejando el resultado en r_d
	xori r_d, r_f, N	Realiza un XOR entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d
	notr r_d, r_f	Realiza un NOT sobre r_f (bit a bit), dejando el resultado en r_d
Desplazamiento de Bits	dsl r_d, r_f, r_N	Desplaza a izquierda r_N veces los bits del registro r_f, dejando el resultado en r_d
	dsli r_d, r_f, N	Desplaza a izquierda N veces los bits del registro r_f, dejando el resultado en r_d
	dsr r_d, r_f, r_N	Desplaza a derecha r_N veces los bits del registro r_f, dejando el resultado en r_d
	dsri r_d, r_f, N	Desplaza a derecha N veces los bits del registro r_f, dejando el resultado en r_d
	dsls r_d, r_f, r_N	Igual que la instrucción **dsl** pero mantiene el signo del valor desplazado
	dslsi r_d, r_f, N	Igual que la instrucción **dsli** pero mantiene el signo del valor desplazado
	dsrs r_d, r_f, r_N	Igual que la instrucción **dsr** pero mantiene el signo del valor desplazado
	dsrsi r_d, r_f, N	Igual que la instrucción **dsri** pero mantiene el signo del valor desplazado
Transferencia de Control	jmp $offN_I$	Salta incondicionalmente a la instrucción etiquetada $offN_I$
	beq r_f, r_g, $offN_I$	Si r_f es igual a r_g, salta a la instrucción etiquetada $offN_I$
	bne r_f, r_g, $offN_I$	Si r_f no es igual a r_g, salta a la instrucción etiquetada $offN_I$
	beqz r_f, $offN_I$	Si r_f es igual a 0, salta a la instrucción etiquetada $offN_I$
	bnez r_f, $offN_I$	Si r_f no es igual a 0, salta a la instrucción etiquetada $offN_I$
	bfpt $offN_I$	Salta a la instrucción etiquetada $offN_I$ si la bandera F=1

	bfpf $offN_I$	Salta a la instrucción etiquetada $offN_I$ si la bandera F=0
Subrutinas	call $offN_S$	Llama a la subrutina cuyo inicio sea la etiqueta $offN_S$ (*)
	ret	Retorna de la subrutina (*)
Manejo de Interrupciones	int N_I	Salva las banderas y ejecuta la interrupción por software N_I (*)
	iret	Retorna de la interrupción y restablece las banderas (*)
	cli	Inhabilita interrupciones enmascarables (*)
	sti	Habilita interrupciones enmascarables (*)
Manejo de la Pila	pushb r_f	Apila los 8 bits menos significativos de r_f (*)
	popb r_d	Desapila un byte (8 bits) y lo carga en r_d (*)
	pushh r_f	Apila los 16 bits menos significativos de r_f
	poph r_d	Desapila un half-word (16 bits) y lo carga en r_d
	pushw r_f	Apila los 32 bits de r_f (*)
	popw r_d	Desapila un word (32 bits) y lo carga en r_d (*)
	pushfl	Apila los 8 bits del registro FLAGS (*)
	popfl	Desapila un byte (8 bits) y lo carga en el registro FLAGS (*)
	pushfp	Apila los 8 bits del registro FLAGS (punto flotante) (*)
	popfp	Desapila un byte (8 bits) y lo carga en el registro FLAGS (punto flotante) (*)
	pushra	Apila los 32 bits del registro RA (*)
	popra	Desapila un word (32 bits) y lo carga en el registro RA (*)
Entrada/Salida	inb r_d, p_f	Carga el valor del puerto p_f en los 8 bits menos significativos de r_d (*)
	outb p_d, r_f	Carga los 8 bits menos significativos de r_f en el puerto p_d (*)
	inh r_d, p_f	Carga el valor del puerto p_f en los 16 bits menos significativos de r_d (*)

	outh p_d, r_f	Carga los 16 bits menos significativos de r_f en el puerto p_d (*)
	inw r_d, p_f	Carga el valor del puerto p_f en los 32 bits de r_d (*)
	outw p_d, r_f	Carga los 32 bits de r_f en el puerto p_d (*)
Control	nop	Operación nula (no realiza ninguna acción)
	halt	Detiene la ejecución del procesador

Observaciones:

- (*): Instrucción sin implementar.

CORREGIR LAS DEMÁS TABLAS EN LAS QUE SE HAGA REFERENCIA A INSTRUCCIONES DE E/S.

3.1.3. Códigos de operación e identificadores

Ahora se procederá a tratar asuntos que describen la lógica del procesador a un nivel de abstracción mucho más inferior que aquél en el cual se ubica el lenguaje Assembler y que, por lo tanto, el usuario que esté simplemente interesado en escribir programas a través de dicho lenguaje no tiene la obligación de conocerlos. No obstante, permiten comprender algunas cuestiones que pueden resultar sumamente útiles para comprender el funcionamiento interno y el progreso de la CPU a un nivel más detallado que la información ofrecida por la máquina de estados durante la ejecución de los distintos programas que se propongan para las simulaciones que se lleven a cabo. Por lo tanto, la lectura de los siguientes tópicos, aunque optativa, no deja de ser recomendable.

En primer lugar, se presentarán los códigos de operación asignados por el diseñador para cada una de las instrucciones que constituyen el repertorio del procesador. Cada instrucción posee un código único, el cual resulta fundamental para que la unidad de control de la CPU pueda interpretar e identificar a partir de él la denominación de la próxima instrucción a ejecutar y determinar a continuación los pasos a seguir para poder llevarla a cabo. Asimismo, el ensamblador deberá tener en cuenta estos identificadores para asociar cada instrucción a traducir al lenguaje máquina con su respectivo código de operación antes de almacenar la misma en la memoria principal de la PC para su posterior interpretación y ejecución.

Tabla 39. Códigos de operación de las instrucciones para la nueva CPU.

Tipo de instrucción	Instrucción	Código de operación
Transferencia de Datos	lb r_d, $Inm(r_i)$	00000100
	sb r_f, $Inm(r_i)$	00000101
	lh r_d, $Inm(r_i)$	00000110
	sh r_f, $Inm(r_i)$	00000111
	lw r_d, $Inm(r_i)$	00001000
	sw r_f, $Inm(r_i)$	00001001
	lf f_d, $Inm(r_i)$	00001010
	sf f_f, $Inm(r_i)$	00001011
	mff f_d, f_f	00001100
	mfr f_d, r_f	00001101
	mrf r_d, f_f	00001110
	tf f_d, f_f	00001111
	ti f_d, f_f	00010000
Aritmética	dadd r_d, r_f, r_g	00011000
	daddi r_d, r_f, N	00011001
	daddu r_d, r_f, r_g	00011010
	daddui r_d, r_f, N	00011011
	addf f_d, f_f, f_g	00011100
	dsub r_d, r_f, r_g	00011101
	dsubu r_d, r_f, r_g	00011110
	subf f_d, f_f, f_g	00011111
	dmul r_d, r_f, r_g	00100000
	dmulu r_d, r_f, r_g	00100001
	mulf f_d, f_f, f_g	00100010
	ddiv r_d, r_f, r_g	00100011
	ddivu r_d, r_f, r_g	00100100
	divf f_d, f_f, f_g	00100101
	slt r_d, r_f, r_g	00100110
	slti r_d, r_f, N	00100111
	ltf f_f, f_g	00101000
	lef f_f, f_g	00101001

	eqf f_f, f_g	00101010
	negr r_d, r_f	00101011
Lógica	andr r_d, r_f, r_g	00110000
	andi r_d, r_f, N	00110001
	orr r_d, r_f, r_g	00110010
	ori r_d, r_f, N	00110011
	xorr r_d, r_f, r_g	00110100
	xori r_d, r_f, N	00110101
	notr r_d, r_f	00110110
Desplazamiento de Bits	dsl r_d, r_f, r_N	01000000
	dsli r_d, r_f, N	01000001
	dsr r_d, r_f, r_N	01000010
	dsri r_d, r_f, N	01000011
	dsls r_d, r_f, r_N	01000100
	dslsi r_d, r_f, N	01000101
	dsrs r_d, r_f, r_N	01000110
	dsrsi r_d, r_f, N	01000111
Transferencia de Control	jmp $offN_I$	01001000
	beq r_f, r_g, $offN_I$	01001001
	bne r_f, r_g, $offN_I$	01001010
	beqz r_f, $offN_I$	01001011
	bnez r_f, $offN_I$	01001100
	bfpt $offN_I$	01001101
	bfpf $offN_I$	01001110
Subrutinas	call $offN_S$	01010000
	ret	01010001
Manejo de Interrupciones	int N_I	01010100
	iret	01010101
	cli	01010110
	sti	01010111
Manejo de la Pila	pushb r_f	01100000
	popb r_d	01100001
	pushh r_f	01100010
	poph r_d	01100011

	pushw r_f	01100100
	popw r_d	01100101
	pushfl	01100110
	popfl	01100111
	pushfp	01101000
	popfp	01101001
	pushra	01101010
	popra	01101011
Entrada/Salida	inb r_d, $offN_P$	01110000
	outb r_f, $offN_P$	01110001
	inh r_d, $offN_P$	01110010
	outh r_f, $offN_P$	01110011
	inw r_d, $offN_P$	01110100
	outw r_f, $offN_P$	01110101
Control	nop	10000000
	halt	10000001

Observaciones:

- Cada código de operación ocupa exactamente un byte u ocho bits.
- Los códigos de operación se encuentran representados en el sistema de numeración binario.

El diseñador ha determinado que cada uno de todos los registros del procesador, tanto de uso general como de propósito particular, debe poseer asociado un identificador único. Estos identificadores deberán ser especialmente tenidos en cuenta por el ensamblador ya que los utilizará para representar y traducir en código máquina los registros de uso general que pueden ser utilizados por el usuario en cualquiera de las instrucciones del repertorio. Asimismo, cumplirán una función similar a la de los códigos de operación para las instrucciones: ofrecer a todos los componentes de la CPU que lo requieran (unidad de búsqueda, unidad de control, ALU, unidad de almacenamiento en registro) una manera de interpretar unívocamente el registro al cual se debe acceder ya sea para realizar una operación de lectura o escritura sobre él sin cometer ningún tipo de errores.

En particular, se han asignado los siguientes identificadores para los registros del nuevo procesador:

Tabla 40. Identificadores asignados para todos los registros de la nueva CPU.

Tipo de registro	Denominación	Identificador
Uso general	r0	0
	r1	1
	r2	2
	r3	3
	r4	4
	r5	5
	r6	6
	r7	7
	r8	8
	r9	9
	r10	10
	r11	11
	r12	12
	r13	13
	r14	14
	r15	15
	f0	16
	f1	17
	f2	18
	f3	19
	f4	20
	f5	21
	f6	22
	f7	23
	f8	24
	f9	25
	f10	26
	f11	27
	f12	28
	f13	29

	f14	30
	f15	31
Uso particular	IR	32
	IP	33
	FLAGS	34
	FPFLAGS	35
	BP	36
	SP	37
	RA	38

3.1.4. Formatos de instrucciones

Por otra parte, a continuación, se muestran los formatos elegidos para representar las instrucciones del repertorio de manera tal de poder ser almacenadas por el ensamblador en la memoria principal de la PC y posteriormente interpretadas y ejecutadas por la CPU. Todas las instrucciones se encuentran conformadas por dos campos cuya presencia es obligatoria: una cabecera, cuyo propósito se explicará próximamente, y uno de los códigos de operación incluidos en la tabla anterior el cual, obviamente, variará en función de la instrucción. Los siguientes campos del formato, si los hubiere, dependerán del tamaño y la cantidad de operandos que posea la instrucción.

Aunque se admite que la opción seleccionada no es la más eficiente dado que desperdicia espacio en memoria (en varios de los campos quedan algunos bits sin utilizar), se justifica la elección en la priorización de una mayor agilidad y facilidad en el proceso de envío de los campos de las instrucciones traducidas por el ensamblador para guardarlos en la memoria principal. Esto se fundamenta en uno de los atributos definidos en la tabla 35 para la arquitectura de esta computadora: la mínima unidad de memoria direccionable, la cual, como se recordará, consiste en un byte. Por lo tanto, si se pretendía asignar un formato para cada uno de los campos de las instrucciones utilizando como base una unidad de información más pequeña que el byte como, por ejemplo, el bit, el ensamblador presentaba algunos inconvenientes de incompatibilidad para transmitir el campo traducido en código binario a fin de ser almacenado en una o varias direcciones de la memoria principal según correspondiera ya que, en un principio, existía la posibilidad de que no encajara correctamente en las mismas y dejara un hueco vacío que no se completaría por el siguiente campo. De este modo, el formato de la

instrucción terminaría siendo incorrecto y la unidad de control tendría serios problemas para interpretar el código de operación y/o recuperar los operandos necesarios para ejecutar dicha instrucción. Además, si el usuario estuviera interesado en analizar el contenido de la memoria principal durante las distintas simulaciones que se realicen, le resultará mucho más sencilla la tarea de identificar cada uno de los campos de las instrucciones del programa si conoce al menos que jamás podrán coexistir dos campos diferentes en una misma dirección, ahorrándose así el esfuerzo de llevar a cabo un estudio pormenorizado bit a bit para diferenciar un campo del otro.

Tabla 41. Formatos de las instrucciones para la nueva CPU.

Instrucción	Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0
lb **lh** **lw**			ri	Inm (byte 1)	Inm (byte 0)	rd	Código de operación	Cabecera
sb **sh** **sw**			ri	Inm (byte 1)	Inm (byte 0)	rf	Código de operación	Cabecera
lf			ri	Inm (byte 1)	Inm (byte 0)	fd	Código de operación	Cabecera
sf			ri	Inm (byte 1)	Inm (byte 0)	ff	Código de operación	Cabecera
mff **tf** **ti**					ff	fd	Código de operación	Cabecera
mfr					rf	fd	Código de operación	Cabecera
mrf					ff	rd	Código de operación	Cabecera
dadd **daddu** **dsub** **dsubu** **dmul**				rg	rf	rd	Código de operación	Cabecera

dmulu **ddiv** **ddivu** **slt** **andr** **orr** **xorr**								
daddi **daddui** **slti** **andi** **ori** **xori** **dsli** **dsri** **dslsi** **dsrsi**	N (byte 3)	N (byte 2)	N (byte 1)	N (byte 0)	rf	rd	Código de operación	Cabecera
addf **subf** **mulf** **divf**				fg	ff	fd	Código de operación	Cabecera
ltf **lef** **eqf**					fg	ff	Código de operación	Cabecera
negr **notr**					rf	rd	Código de operación	Cabecera
dsl **dsr** **dsls** **dsrs**				rN	rf	rd	Código de operación	Cabecera
jmp **bfpt** **bfpf**					offNI (byte 1)	offNI (byte 0)	Código de operación	Cabecera

beq **bne**			offNI (byte 1)	offNI (byte 0)	rg	rf	Código de operación	Cabecera
beqz **bnez**				offNI (byte 1)	offNI (byte 0)	rf	Código de operación	Cabecera
call (*)					offNS (byte 1)	offNS (byte 0)	Código de operación	Cabecera
ret **iret** **cli** **sti** **pushfl** **popfl** **pushfp** **popfp** **pushra** **popra** **nop** **halt**							Código de operación	Cabecera
int (*)						NI	Código de operación	Cabecera
pushb (*) **pushh (*)** **pushw (*)**						rf	Código de operación	Cabecera
popb (*) **poph (*)** **popw (*)**						rd	Código de operación	Cabecera
inb (*) **inh (*)** **inw (*)**				offNP (byte 1)	offNP (byte 0)	rd	Código de operación	Cabecera
outb (*) **outh (*)** **outw (*)**				offNP (byte 1)	offNP (byte 0)	rf	Código de operación	Cabecera

Observaciones:

- (*): Formato sin implementar.
- Cabecera: byte inicial que contiene el tamaño en bytes de la instrucción (incluyendo la cabecera) para ayudar a la unidad de búsqueda de la CPU a determinar el incremento que deberá tener el registro IP para que pueda apuntar a la próxima instrucción a ejecutar.
- Código de operación: identificador único para cada instrucción a fin de que la unidad de control pueda interpretarlo y determinar los próximos pasos a seguir para completar la ejecución de dicha instrucción.
- rd: identificador del registro destino de la instrucción. Puede variar entre 0 (r0) y 15 (r15).
- rf: identificador del primer registro fuente de la instrucción. Puede variar entre 0 (r0) y 15 (r15).
- rg: identificador del segundo registro fuente de la instrucción. Puede variar entre 0 (r0) y 15 (r15).
- Inm: dirección base para determinar la dirección de memoria de datos a acceder para leer o escribir un valor en ella. Puede variar entre 4096 (1000_{16}) y 8191 ($1FFF_{16}$).
- ri: identificador del registro índice de la instrucción que contiene el desplazamiento a partir de la dirección base "Inm" para determinar la dirección de memoria de datos a acceder. Puede variar entre 0 (r0) y 15 (r15).
- fd: identificador del registro destino de la instrucción (punto flotante). Puede variar entre 16 (f0) y 31 (f15).
- ff: identificador del primer registro fuente de la instrucción (punto flotante). Puede variar entre 16 (f0) y 31 (f15).
- fg: identificador del segundo registro fuente de la instrucción (punto flotante). Puede variar entre 16 (f0) y 31 (f15).
- N: valor inmediato del tercer operando de la instrucción (segundo operando fuente). Puede variar entre 0 y 4.294.967.295 (sin signo) o bien –2.147.483.648 y 2.147.483.647 (con signo) según el tipo de instrucción.
- offNI: dirección de salto de la instrucción dentro de la memoria de instrucciones. Puede variar entre 8192 (2000_{16}) y 12287 ($2FFF_{16}$).
- offNS: dirección de inicio de la subrutina invocada por la instrucción. Puede variar entre 12288 (3000_{16}) y 28671 ($6FFF_{16}$).
- offNP: dirección de memoria asociada al puerto de E/S a acceder para leer o escribir un valor.

- NI: número de la instrucción por software a ejecutar. Puede variar entre 0 y 255.

3.1.5. Estructura de la memoria principal

Si el usuario estuviera interesado en estudiar el contenido de la memoria principal en tiempo de ejecución durante las pruebas que realice, también debería conocer la estructura de la misma, en particular las distintas secciones que la conforman, la naturaleza de la información que almacenan, su ubicación y sus delimitaciones (dirección inicial y final). De este modo, podrá determinar rápidamente qué dirección de memoria debería tomar como punto de referencia para localizar la información deseada en función de su tipo (dato, instrucción, subrutina, etc.). La estructura fue diseñada a partir del modelo ofrecido por el simulador MSX88 para la memoria principal de su propia computadora, aunque, como se observará, la amplia mayoría de las secciones aún no se encuentran implementadas en este diseño.

Para facilitar la visualización de las distintas secciones de la memoria principal, se optó por representar en primer lugar, en la figura 42.1, un panorama general en el cual se contemplen únicamente los tipos de memoria: RAM, ROM y Entrada/Salida. Este último se define en forma separada de los otros porque, tal como fue reflejado en la tabla 35, uno de los atributos definidos para la arquitectura de esta computadora es una interfaz de entrada/salida independiente. Esta característica adquiere una especial importancia aquí dado que implica la coexistencia de dos espacios de direcciones de memoria aislados y separados entre sí: uno especial para la lectura/escritura de los periféricos de entrada/salida y otro para el resto de las operaciones.

Luego, se procede a ofrecer en la figura 42.2 una representación pormenorizada de las secciones que conforman la memoria RAM de la PC, dado que aquí se provocarán la mayor parte de los cambios cuyo estudio podría resultarle de interés al lector. No obstante, aunque se tratará este tema en detalle la sección “3.3. Programación”, puede adelantarse aquí que, aunque el simulador MSX88 y su CPU descripta, la Intel 8088, fueron utilizadas como plantilla para desarrollar la estructura de la memoria de la nueva computadora, el mecanismo para la programación del procesador a través del lenguaje Assembler fue diseñado principalmente a partir del WinMIPS64 y su propia CPU, el MIPS64. Uno de los motivos para esta elección radica en que, precisamente, al utilizarse directivas genéricas para indicarle al ensamblador el comienzo de cada una de las secciones del programa Assembler para su almacenamiento en la sección correspondiente de la memoria principal (ver tabla 37), permite al programador en este lenguaje abstraerse de estas cuestiones y escribir su código en forma independiente de la

dirección efectiva de memoria en la cual empiece cada sección. Si en algún momento optara por variarse la estructura de la memoria del TDA 1819, esto no debería alterar en absoluto el comportamiento de los programas escritos previamente a dicho cambio. Se recuerda que, en la programación para el MSX88, el comienzo de cada sección del programa debe establecerse mediante una directiva que incluya la dirección inicial exacta de memoria que el programador hubiera definido para ese programa en particular. En cambio, aquí el ensamblador almacenaría cada sección a partir de la primera dirección definida en la estructura para la memoria, sea cual fuere. Podría presentarse como desventaja que aquí el programador posee una menor libertad para escribir sus programas, dado que tanto la ubicación como el tamaño de las distintas secciones ya se encuentran predefinidas y fijas, por lo que en todo momento se verificará que bajo ninguna circunstancia una instrucción pueda acceder a una sección inválida según su tarea para la cual fue concebida. Por ejemplo, una instrucción de transferencia de datos solo podrá acceder a la memoria de datos, una de acceso a la pila, solo

Tabla 42.1. Estructura de la memoria principal diseñada para la nueva computadora.

Tipo de memoria	**Sección de memoria**	**Dirección inicial**	**Dirección final**	**Tamaño total**
Entrada/Salida (*)		**0000**	**00FF**	**256 bytes**
RAM	Interrupciones (*)	0000	03FF	1 kilobyte
	Datos	0400	13FF	4 kilobytes
	Instrucciones	1400	33FF	8 kilobytes
	Subrutinas (*)	3400	63FF	12 kilobytes
	Pila (*)	6400	7FFF	7 kilobytes
ROM (*)		8000	FFFF	32 kilobytes
RAM + ROM		**0000**	**FFFF**	**64 kilobytes**

Sección de memoria	Tipos de instrucciones permitidos
Entrada/Salida (*)	Manejo de Interrupciones (“int” e “iret”, revisar)
	Entrada/Salida
Interrupciones (*)	Manejo de Interrupciones (“int”, revisar)
Datos	Transferencia de Datos
Instrucciones	Transferencia de Control (solo dentro de esta sección, aclarar observación)
	Interrupciones (“iret”, revisar)
	Subrutinas (“call” y “ret”, pueden ser subrutinas anidadas)
Subrutinas (*)	Transferencia de Control (solo dentro de esta sección, aclarar observación)
	Subrutinas (solo “call”)
	Manejo de Interrupciones (“iret”, revisar)
Pila (*)	Manejo de la Pila

Tipo de instrucción	Instrucción	Sección de memoria accesible
Transferencia de Datos	lb r_d, $Inm(r_i)$	Datos
	sb r_f, $Inm(r_i)$	Datos
	lh r_d, $Inm(r_i)$	Datos
	sh r_f, $Inm(r_i)$	Datos
	lw r_d, $Inm(r_i)$	Datos
	sw r_f, $Inm(r_i)$	Datos
	lf f_d, $Inm(r_i)$	Datos
	sf f_f, $Inm(r_i)$	Datos
	mff f_d, f_f	
	mfr f_d, r_f	
	mrf r_d, f_f	
	tf f_d, f_f	
	ti f_d, f_f	
Aritmética	dadd r_d, r_f, r_g	
	daddi r_d, r_f, N	

	daddu r_d, r_f, r_g	
	daddui r_d, r_f, N	
	addf f_d, f_f, f_g	
	dsub r_d, r_f, r_g	
	dsubu r_d, r_f, r_g	
	subf f_d, f_f, f_g	
	dmul r_d, r_f, r_g	
	dmulu r_d, r_f, r_g	
	mulf f_d, f_f, f_g	
	ddiv r_d, r_f, r_g	
	ddivu r_d, r_f, r_g	
	divf f_d, f_f, f_g	
	slt r_d, r_f, r_g	
	slti r_d, r_f, N	
	ltf f_f, f_g	
	lef f_f, f_g	
	eqf f_f, f_g	
	negr r_d, r_f	
Lógica	andr r_d, r_f, r_g	
	andi r_d, r_f, N	
	orr r_d, r_f, r_g	
	ori r_d, r_f, N	
	xorr r_d, r_f, r_g	
	xori r_d, r_f, N	
	notr r_d, r_f	
Desplazamiento de Bits	dsl r_d, r_f, r_N	
	dsli r_d, r_f, N	
	dsr r_d, r_f, r_N	
	dsri r_d, r_f, N	
	dsls r_d, r_f, r_N	
	dslsi r_d, r_f, N	
	dsrs r_d, r_f, r_N	
	dsrsi r_d, r_f, N	
	jmp $offN_I$	Instrucciones

Transferencia de Control		Subrutinas
	beq r_f, r_g, $offN_I$	Instrucciones
		Subrutinas
	bne r_f, r_g, $offN_I$	Instrucciones
		Subrutinas
	beqz r_f, $offN_I$	Instrucciones
		Subrutinas
	bnez r_f, $offN_I$	Instrucciones
		Subrutinas
	bfpt $offN_I$	Instrucciones
		Subrutinas
	bfpf $offN_I$	Instrucciones
		Subrutinas
Subrutinas	call $offN_S$	Instrucciones
		Subrutinas
	ret	Instrucciones
		Subrutinas
Manejo de Interrupciones	int N_I	Interrupciones
		ROM
	iret	Instrucciones
		Subrutinas
	cli	
	sti	
Manejo de la Pila	pushb r_f	Pila
	popb r_d	Pila
	pushh r_f	Pila
	poph r_d	Pila
	pushw r_f	Pila
	popw r_d	Pila
	pushfl	Pila
	popfl	Pila
	pushfp	Pila
	popfp	Pila
	pushra	Pila

	popra	Pila
Entrada/Salida	inb r_d, $offN_P$	Entrada/Salida
	outb r_f, $offN_P$	Entrada/Salida
	inh r_d, $offN_P$	Entrada/Salida
	outh r_f, $offN_P$	Entrada/Salida
	inw r_d, $offN_P$	Entrada/Salida
	outw r_f, $offN_P$	Entrada/Salida
Control	nop	
	halt	

¿AGREGAR TAMBIÉN REGISTROS DE USO GENERAL Y/O PARTICULAR ACCESIBLES POR LAS INSTRUCCIONES? ¿DIVIDIR ENTRE SOLO LECTURA, SOLO ESCRITURA Y LECTURA/ESCRITURA?

Tipo de instrucción	**Instrucción**	**Registros accesibles**
Transferencia de Datos	lb r_d, $Inm(r_i)$	Uso general: enteros
	sb r_f, $Inm(r_i)$	Uso general: enteros
	lh r_d, $Inm(r_i)$	Uso general: enteros
	sh r_f, $Inm(r_i)$	Uso general: enteros
	lw r_d, $Inm(r_i)$	Uso general: enteros
	sw r_f, $Inm(r_i)$	Uso general: enteros
	lf f_d, $Inm(r_i)$	Uso general: enteros
	sf f_f, $Inm(r_i)$	Uso general: enteros
	mff f_d, f_f	Uso general: punto flotante
	mfr f_d, r_f	Uso general: enteros
		Uso general: punto flotante
	mrf r_d, f_f	Uso general: enteros
		Uso general: punto flotante
	tf f_d, f_f	Uso general: punto flotante
		Uso particular: FPFLAGS
	ti f_d, f_f	Uso general: punto flotante
		Uso particular: FPFLAGS
Aritmética	dadd r_d, r_f, r_g	Uso general: enteros

		Uso particular: FLAGS
	daddi r_d, r_f, N	Uso general: enteros
		Uso particular: FLAGS
	daddu r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	daddui r_d, r_f, N	Uso general: enteros
		Uso particular: FLAGS
	addf f_d, f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	dsub r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	dsubu r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	subf f_d, f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	dmul r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	dmulu r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	mulf f_d, f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	ddiv r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	ddivu r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	divf f_d, f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	slt r_d, r_f, r_g	Uso general: enteros
		Uso particular: FLAGS
	slti r_d, r_f, N	Uso general: enteros
		Uso particular: FLAGS
	ltf f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	lef f_f, f_g	Uso general: punto flotante

		Uso particular: FPFLAGS
	eqf f_f, f_g	Uso general: punto flotante
		Uso particular: FPFLAGS
	negr r_d, r_f	Uso general: enteros
		Uso particular: FLAGS
Lógica	andr r_d, r_f, r_g	Uso general: enteros
	andi r_d, r_f, N	Uso general: enteros
	orr r_d, r_f, r_g	Uso general: enteros
	ori r_d, r_f, N	Uso general: enteros
	xorr r_d, r_f, r_g	Uso general: enteros
	xori r_d, r_f, N	Uso general: enteros
	notr r_d, r_f	Uso general: enteros
Desplazamiento de Bits	dsl r_d, r_f, r_N	Uso general: enteros
	dsli r_d, r_f, N	Uso general: enteros
	dsr r_d, r_f, r_N	Uso general: enteros
	dsri r_d, r_f, N	Uso general: enteros
	dsls r_d, r_f, r_N	Uso general: enteros
	dslsi r_d, r_f, N	Uso general: enteros
	dsrs r_d, r_f, r_N	Uso general: enteros
	dsrsi r_d, r_f, N	Uso general: enteros
Transferencia de Control	jmp $offN_I$	
	beq r_f, r_g, $offN_I$	Uso general: enteros
	bne r_f, r_g, $offN_I$	Uso general: enteros
	beqz r_f, $offN_I$	Uso general: enteros

	bnez r_f, $offN_I$	Uso general: enteros
	bfpt $offN_I$	
	bfpf $offN_I$	
Subrutinas	call $offN_S$	
	ret	
Manejo de Interrupciones	int N_I	
	iret	
	cli	
	sti	
Manejo de la Pila	pushb r_f	Uso general: enteros
	popb r_d	Uso general: enteros
	pushh r_f	Uso general: enteros
	poph r_d	Uso general: enteros
	pushw r_f	Uso general: enteros
	popw r_d	Uso general: enteros
	pushfl	
	popfl	
	pushfp	
	popfp	
	pushra	
	popra	
Entrada/Salida	inb r_d, $offN_P$	Uso general: enteros
	outb r_f, $offN_P$	Uso general: enteros
	inh r_d, $offN_P$	Uso general: enteros
	outh r_f, $offN_P$	Uso general: enteros
	inw r_d, $offN_P$	Uso general: enteros

	outw r_f, $offN_P$	Uso general: enteros
Control	nop	
	halt	

Tipo de instrucción	Instrucción	Comentario
Transferencia de Datos	lb r_d, $Inm(r_i)$	Copia en r_d un byte (8 bits) desde la dirección ($Inm + r_i$)
	sb r_f, $Inm(r_i)$	Guarda los 8 bits menos significativos de r_f en la dirección ($Inm + r_i$)
	lh r_d, $Inm(r_i)$	Copia en r_d un half-word (16 bits) desde la dirección ($Inm + r_i$)
	sh r_f, $Inm(r_i)$	Guarda los 16 bits menos significativos de r_f a partir de la dirección ($Inm + r_i$)
	lw r_d, $Inm(r_i)$	Copia en r_d un word (32 bits) desde la dirección ($Inm + r_i$)
	sw r_f, $Inm(r_i)$	Guarda los 32 bits de r_f a partir de la dirección ($Inm + r_i$)
	lf f_d, $Inm(r_i)$	Copia en r_d un valor en punto flotante (32 bits) desde la dirección ($Inm + r_i$)
	sf f_f, $Inm(r_i)$	Guarda los 32 bits de f_f a partir de la dirección ($Inm + r_i$)
	mff f_d, f_f	Copia el valor del registro f_f (32 bits) al registro f_d
	mfr f_d, r_f	Copia los 32 bits del registro r_f entero al registro f_d de punto flotante
	mrf r_d, f_f	Copia los 32 bits del registro f_f de punto flotante al registro r_d entero
	tf f_d, f_f	Convierte a punto flotante el valor entero copiado al registro f_f (32 bits), dejándolo en f_d
	ti f_d, f_f	Convierte a entero el valor en punto flotante contenido en f_f (32 bits), dejándolo en f_d

Aritmética	dadd r_d, r_f, r_g	Suma r_f con r_g, dejando el resultado en r_d (valores con signo)
	daddi r_d, r_f, N	Suma r_f con el valor inmediato N, dejando el resultado en r_d (valores con signo)
	daddu r_d, r_f, r_g	Suma r_f con r_g, dejando el resultado en r_d (valores sin signo)
	daddui r_d, r_f, N	Suma r_f con el valor inmediato N, dejando el resultado en r_d (valores sin signo)
	addf f_d, f_f, f_g	Suma f_f con f_g, dejando el resultado en f_d (en punto flotante)
	dsub r_d, r_f, r_g	Resta r_g a r_f, dejando el resultado en r_d (valores con signo)
	dsubu r_d, r_f, r_g	Resta r_g a r_f, dejando el resultado en r_d (valores sin signo)
	subf f_d, f_f, f_g	Resta f_g a f_f, dejando el resultado en f_d (en punto flotante)
	dmul r_d, r_f, r_g	Mutiplica r_f con r_g, dejando el resultado en r_d (valores con signo)
	dmulu r_d, r_f, r_g	Mutiplica r_f con r_g, dejando el resultado en r_d (valores sin signo)
	mulf f_d, f_f, f_g	Multiplica f_f con f_g, dejando el resultado en f_d (en punto flotante)
	ddiv r_d, r_f, r_g	Divide r_f por r_g, dejando el resultado en r_d (valores con signo)
	ddivu r_d, r_f, r_g	Divide r_f por r_g, dejando el resultado en r_d (valores sin signo)
	divf f_d, f_f, f_g	Divide f_f por f_g, dejando el resultado en f_d (en punto flotante)
	slt r_d, r_f, r_g	Compara r_f con r_g, dejando r_d =1 si r_f es menor que r_g (valores con signo)
	slti r_d, r_f, N	Compara r_f con el valor inmediato N, dejando r_d =1 si r_f es menor que N (valores con signo)
	ltf f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es menor que f_g (en punto flotante)

	lef f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es menor o igual que f_g (en punto flotante)
	eqf f_f, f_g	Compara f_f con f_g, dejando la bandera F=1 si f_f es igual que f_g (en punto flotante)
	negr r_d, r_f	Calcula el opuesto de r_f, dejando el resultado en r_d (valores con signo)
Lógica	andr r_d, r_f, r_g	Realiza un AND entre r_f y r_g (bit a bit), dejando el resultado en r_d
	andi r_d, r_f, N	Realiza un AND entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d
	orr r_d, r_f, r_g	Realiza un OR entre r_f y r_g (bit a bit), dejando el resultado en r_d
	ori r_d, r_f, N	Realiza un OR entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d
	xorr r_d, r_f, r_g	Realiza un XOR entre r_f y r_g (bit a bit), dejando el resultado en r_d
	xori r_d, r_f, N	Realiza un XOR entre r_f y el valor inmediato N (bit a bit), dejando el resultado en r_d
	notr r_d, r_f	Realiza un NOT sobre r_f (bit a bit), dejando el resultado en r_d
Desplazamiento de Bits	dsl r_d, r_f, r_N	Desplaza a izquierda r_N veces los bits del registro r_f, dejando el resultado en r_d
	dsli r_d, r_f, N	Desplaza a izquierda N veces los bits del registro r_f, dejando el resultado en r_d
	dsr r_d, r_f, r_N	Desplaza a derecha r_N veces los bits del registro r_f, dejando el resultado en r_d
	dsri r_d, r_f, N	Desplaza a derecha N veces los bits del registro r_f, dejando el resultado en r_d
	dsls r_d, r_f, r_N	Igual que la instrucción **dsl** pero mantiene el signo del valor desplazado
	dslsi r_d, r_f, N	Igual que la instrucción **dsli** pero mantiene el signo del valor desplazado
	dsrs r_d, r_f, r_N	Igual que la instrucción **dsr** pero mantiene el signo del valor desplazado

	dsrsi r_d, r_f, N	Igual que la instrucción **dsri** pero mantiene el signo del valor desplazado
Transferencia de Control	jmp $offN_I$	Salta incondicionalmente a la instrucción etiquetada $offN_I$
	beq r_f, r_g, $offN_I$	Si r_f es igual a r_g, salta a la instrucción etiquetada $offN_I$
	bne r_f, r_g, $offN_I$	Si r_f no es igual a r_g, salta a la instrucción etiquetada $offN_I$
	beqz r_f, $offN_I$	Si r_f es igual a 0, salta a la instrucción etiquetada $offN_I$
	bnez r_f, $offN_I$	Si r_f no es igual a 0, salta a la instrucción etiquetada $offN_I$
	bfpt $offN_I$	Salta a la instrucción etiquetada $offN_I$ si la bandera F=1
	bfpf $offN_I$	Salta a la instrucción etiquetada $offN_I$ si la bandera F=0
Subrutinas	call $offN_S$	Llama a la subrutina cuyo inicio sea la etiqueta $offN_S$ (*)
	ret	Retorna de la subrutina (*)
Manejo de Interrupciones	int N_I	Salva las banderas y ejecuta la interrupción por software N_I (*)
	iret	Retorna de la interrupción y restablece las banderas (*)
	cli	Inhabilita interrupciones enmascarables (*)
	sti	Habilita interrupciones enmascarables (*)
Manejo de la Pila	pushb r_f	Apila los 8 bits menos significativos de r_f (*)
	popb r_d	Desapila un byte (8 bits) y lo carga en r_d (*)
	pushh r_f	Apila los 16 bits menos significativos de r_f (*)
	poph r_d	Desapila un half-word (16 bits) y lo carga en r_d (*)
	pushw r_f	Apila los 32 bits de r_f (*)
	popw r_d	Desapila un word (32 bits) y lo carga en r_d (*)
	pushfl	Apila los 8 bits del registro FLAGS (*)

	popfl	Desapila un byte (8 bits) y lo carga en el registro FLAGS (*)
	pushfp	Apila los 8 bits del registro FLAGS (punto flotante) (*)
	popfp	Desapila un byte (8 bits) y lo carga en el registro FLAGS (punto flotante) (*)
	pushra	Apila los 32 bits del registro RA (*)
	popra	Desapila un word (32 bits) y lo carga en el registro RA (*)
Entrada/Salida	inb r_d, p_f	Carga el valor del puerto p_f en los 8 bits menos significativos de r_d (*)
	outb p_d, r_f	Carga los 8 bits menos significativos de r_f en el puerto p_d (*)
	inh r_d, p_f	Carga el valor del puerto p_f en los 16 bits menos significativos de r_d (*)
	outh p_d, r_f	Carga los 16 bits menos significativos de r_f en el puerto p_d (*)
	inw r_d, p_f	Carga el valor del puerto p_f en los 32 bits de r_d (*)
	outw p_d, r_f	Carga los 32 bits de r_f en el puerto p_d (*)
Control	nop	Operación nula (no realiza ninguna acción)
	halt	Detiene la ejecución del procesador

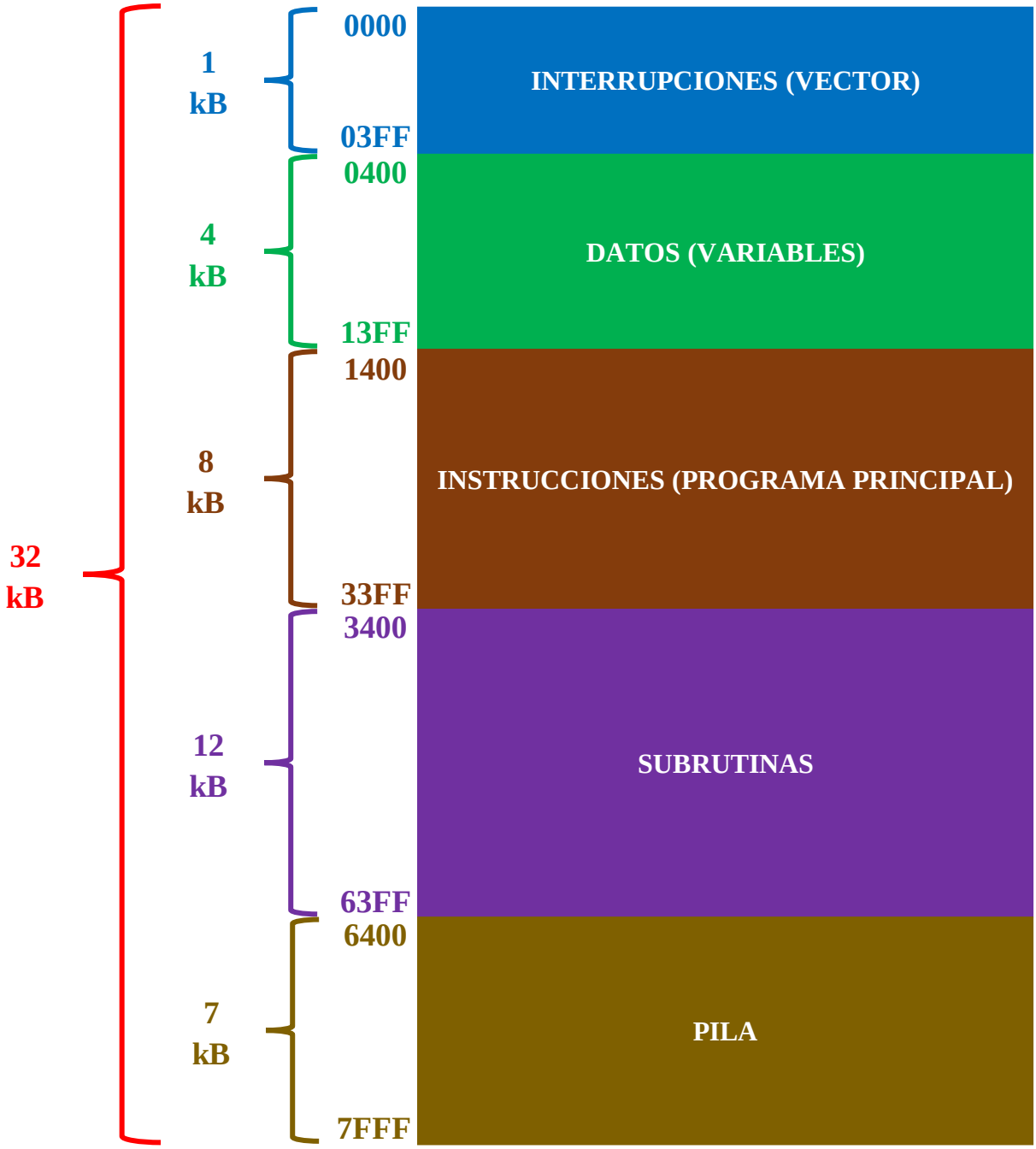
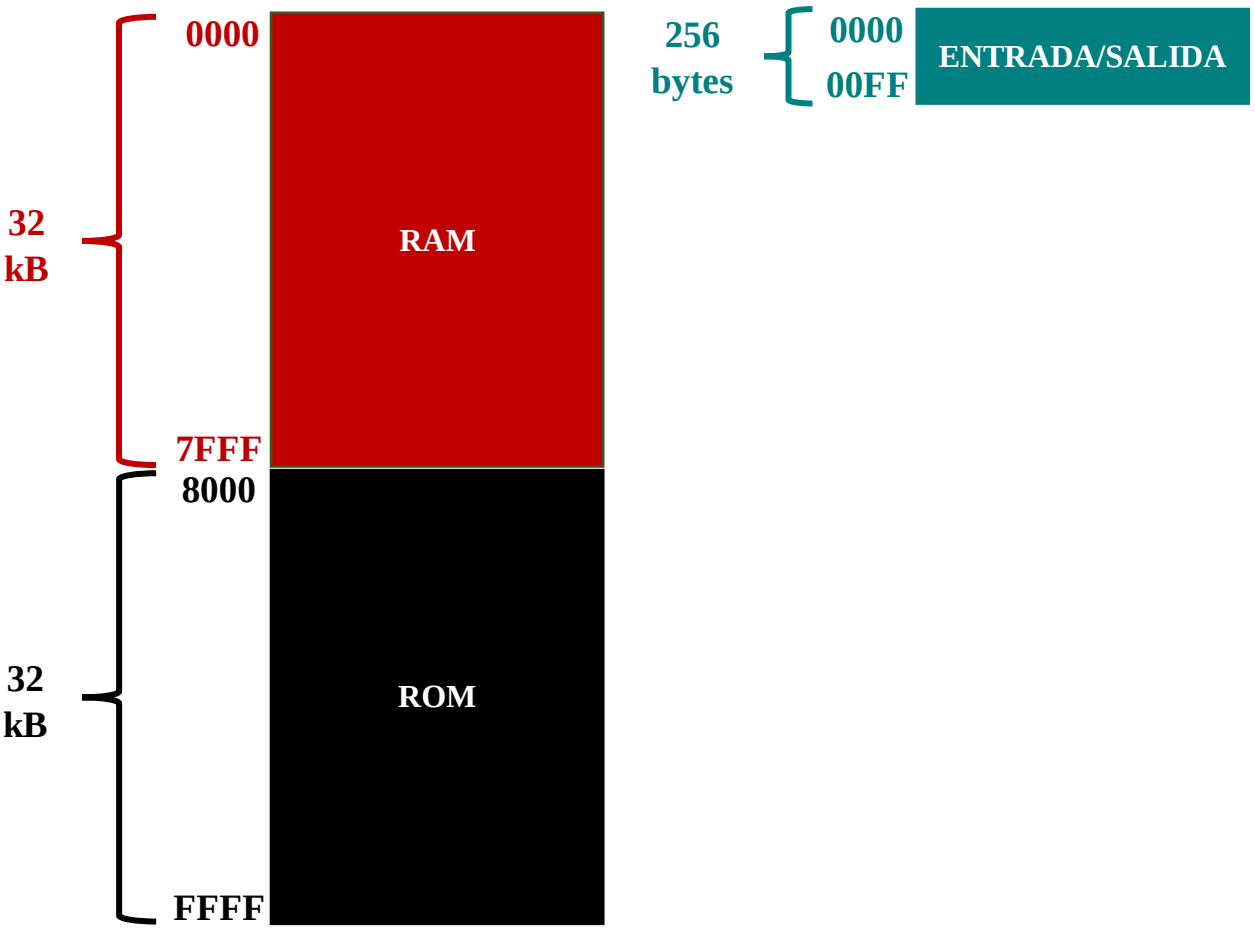


Figura 42.2. Estructura de la memoria principal diseñada para la nueva computadora.


0000
RAM
7FFF
8000
ROM
FFFF
32
kB
32
kB
256
bytes
0000
00FF
ENTRADA/SALIDA

Observaciones:

- (*): Sección de memoria sin implementar.
- Las direcciones de memoria iniciales y finales se encuentran representadas en el sistema de numeración hexadecimal.
- RAM: Random Access Memory (Memoria de Acceso Aleatorio).
- ROM: Read Only Memory (Memoria de Solo Lectura).
- ACLARAR QUE, AL SER E/S INDEPENDIENTE, E/S TIENE MEMORIA AISLADA CON SU PROPIO ESPACIO DE DIRECCIONES.
- CORREGIR EN EL PROYECTO VHDL Y LOS PROGRAMAS ASSEMBLER COMIENZO DE MEMORIA DE DATOS E INSTRUCCIONES.
- ACLARAR QUE SON SECCIONES FIJAS Y DELIMITADAS: NUNCA SE PODRÁ ESCRIBIR UN DATO EN UNA SECCIÓN QUE NO SEA LA MEMORIA DE DATOS, NO SE PODRÁ ACCEDER A LA MEMORIA ROM, ETC.
- DIRECCIONES IRRELEVANTES PARA EL PROGRAMADOR ASSEMBLER.
- HACER GRÁFICOS POR SECCIONES (RAM, ROM Y E/S; ZOOM EN RAM).
- VERIFICAR OPERACIONES ARITMÉTICO-LÓGICAS EN PUNTO FLOTANTE O SOLO ARITMÉTICAS.

3.1.5.1. Caso de prueba

Una vez asimilada toda la información anterior el lector ya debería encontrarse en condiciones de determinar a partir de cualquier programa escrito en el lenguaje Assembler la disposición de cada uno de sus datos e instrucciones en la memoria principal, desde la/s direcciones que ocuparían hasta el orden particular en el cual se guardarían cada uno de sus campos en ellas teniendo en cuenta principalmente las distintas secciones de la estructura de la memoria, el formato de las instrucciones y el mecanismo de almacenamiento utilizado por la computadora (little endian).

Se propone en esta subsección del informe un caso de prueba para su observación, análisis y estudio, conformado por el siguiente programa:

```
        .data
num1:   .word   –20
num2:   .hword  80
        .code
        lw r1, num1(r0)
        daddi r3, r2, –5
        addf f3, f1, f2
        halt
```

Figura 43. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

Como se habrá podido observar, la sintaxis definida para la programación de código en el lenguaje Assembler que resulte compatible con este nuevo procesador es muy similar a la utilizada por el simulador WinMIPS64 para su propia CPU, con algunas pequeñas modificaciones en la denominación de algunas instrucciones y tipos de datos. Se profundizará más en este apartado en la subsección "3.3. Programación Assembler" de este informe.

Ahora mismo se prestará una especial importancia a explicar paso a paso el proceso deductivo realizado para determinar la disposición del programa de la figura anterior en la memoria principal:

1. Sección de datos (directiva .data):
    a. Dirección inicial: 1000_{16}.
    b. Primer dato (num1):
        i. Se trata de un número de 32 bits con signo (directiva .word).
        ii. Su dirección inicial coincidirá con la dirección inicial de la sección de datos, es decir, la número 1000_{16}.
        iii. Su longitud es de 32 bits, es decir, 4 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, este dato ocupará 4 direcciones: desde la 1000_{16} hasta la 1003_{16} inclusive.
        iv. La computadora utiliza el sistema de numeración conocido como complemento a dos para representar valores con signo (para números sin signo usa el sistema binario sin signo). Por lo tanto, para determinar la representación correcta del valor a almacenar deberá

realizarse la conversión a dicho sistema, siempre recordando que la longitud de la variable es de 32 bits. El resultado de dicha conversión tendría que ser el siguiente: -20_{10} = $11111111111111111111111111101100_{Ca2}$.

c. Segundo dato (num2):
    i. Se trata de un número de 16 bits con signo (directiva .hword).
    ii. Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del dato anterior, es decir, la número 1004_{16}.
    iii. Su longitud es de 16 bits, es decir, 2 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, este dato ocupará 2 direcciones: desde la 1004_{16} hasta la 1005_{16} inclusive.
    iv. La computadora utiliza el sistema de numeración conocido como complemento a dos para representar valores con signo (para números sin signo usa el sistema binario sin signo). Por lo tanto, para determinar la representación correcta del valor a almacenar deberá realizarse la conversión a dicho sistema, siempre recordando que la longitud de la variable es de 16 bits. El resultado de dicha conversión tendría que ser el siguiente: 80_{10} = 0000000001010000_{Ca2}.

2. Sección de código del programa principal (directiva .code):
    a. Dirección inicial: 2000_{16}.
    b. Primera instrucción (lw):
        i. La dirección inicial de esta instrucción coincidirá con la dirección inicial de la sección de código del programa principal, es decir, la número 2000_{16}.
        ii. Cabecera:
            - Su dirección inicial coincidirá con la dirección inicial de esta instrucción, es decir, la número 2000_{16}.
            - Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2000_{16}.
            - Tamaño de la instrucción:
                ❖ Cabecera: 1 byte.
                ❖ Código de operación: 1 byte.

- ❖ Registro destino (rd): 1 byte.
- ❖ Dirección base de memoria de datos (Inm): 2 bytes.
- ❖ Registro índice (ri): 1 byte.
- ❖ Total: 6 bytes.

- Como se recordará, el campo cabecera contiene el tamaño total en bytes de la instrucción almacenada (incluyendo dicha cabecera). Por lo tanto, el valor almacenado será la representación del total obtenido en el inciso anterior en el sistema de numeración binario sin signo (al ser siempre el tamaño de la instrucción un valor positivo, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $6_{10} = 00000110_2$.

iii. Código de operación:

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2001_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2001_{16}.
- El código de operación a almacenar para esta instrucción se obtiene de la tabla 39 de este informe: 00001000.

iv. Registro destino (rd):

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2002_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2002_{16}.
- Al tratarse r1 del registro destino de esta instrucción, se debe calcular la representación de su identificador de registro, 1, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora

utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $1_{10} = 00000001_2$.

v. Dirección base de memoria de datos (Inm):
    - Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2003_{16}.
    - Su longitud es de 16 bits, es decir, 2 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, este dato ocupará 2 direcciones: desde la 2003_{16} hasta la 2004_{16} inclusive.
    - Al hacerse referencia en este campo a la etiqueta o nombre del primer dato del programa (num1), el valor a almacenar será la dirección inicial de dicho dato en memoria, 1000_{16}, representada en el sistema de numeración binario sin signo (al ser siempre la dirección de memoria un valor natural, la computadora utiliza este sistema para representarla). Una vez realizada la conversión, sin olvidar que este campo ocupa 16 bits, puede obtenerse el siguiente resultado: $1000_{16} = 0001000000000000_2$.

vi. Registro índice (ri):
    - Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2005_{16}.
    - Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2005_{16}.
    - Al tratarse r0 del registro índice de esta instrucción, se debe calcular la representación de su identificador de registro, 0, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora utiliza este sistema para representarlo). Una vez realizada la

conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $0_{10} = 00000000_2$.

vii. La longitud total de esta instrucción es de 6 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, dicha instrucción ocupará 6 direcciones: desde la 2000_{16} hasta la 2005_{16} inclusive.

c. Segunda instrucción (daddi):

i. La dirección inicial de esta instrucción será aquélla ubicada inmediatamente a continuación de la dirección final de la instrucción anterior, es decir, la número 2006_{16}.

ii. Cabecera:

- Su dirección inicial coincidirá con la dirección inicial de esta instrucción, es decir, la número 2006_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2006_{16}.
- Tamaño de la instrucción:
  - ❖ Cabecera: 1 byte.
  - ❖ Código de operación: 1 byte.
  - ❖ Registro destino (rd): 1 byte.
  - ❖ Primer registro fuente (rf): 1 byte.
  - ❖ Segundo operando fuente (N): 4 bytes.
  - ❖ Total: 8 bytes.
- Como se recordará, el campo cabecera contiene el tamaño total en bytes de la instrucción almacenada (incluyendo dicha cabecera). Por lo tanto, el valor almacenado será la representación del total obtenido en el inciso anterior en el sistema de numeración binario sin signo (al ser siempre el tamaño de la instrucción un valor positivo, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $8_{10} = 00001000_2$.

iii. Código de operación:

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2007_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2007_{16}.
- El código de operación a almacenar para esta instrucción se obtiene de la tabla 39 de este informe: 00011001.

iv. Registro destino (rd):

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2008_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2008_{16}.
- Al tratarse r3 del registro destino de esta instrucción, se debe calcular la representación de su identificador de registro, 3, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $3_{10} = 00000011_2$.

v. Primer registro fuente (rf):

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2009_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2009_{16}.
- Al tratarse r2 del primer registro fuente de esta instrucción, se debe calcular la representación de su identificador de registro, 2, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la

computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $2_{10} = 00000010_2$.

vi. Segundo operando fuente (N):

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número $200A_{16}$.
- Su longitud es de 32 bits, es decir, 4 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 4 direcciones: desde la $200A_{16}$ hasta la $200D_{16}$ inclusive.
- Al tratarse –5 del valor inmediato del segundo operando fuente de esta instrucción, se debe calcular la representación del mismo en el sistema de numeración conocido como complemento a dos (al existir posibilidades de que el valor inmediato sea negativo, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 32 bits, puede obtenerse el siguiente resultado: $-5_{10} = 11111111111111111111111111111011_{Ca2}$.

vii. La longitud total de esta instrucción es de 8 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, dicha instrucción ocupará 8 direcciones: desde la 2006_{16} hasta la $200D_{16}$ inclusive.

d. Tercera instrucción (addf):

i. La dirección inicial de esta instrucción será aquélla ubicada inmediatamente a continuación de la dirección final de la instrucción anterior, es decir, la número $200E_{16}$.

ii. Cabecera:

- Su dirección inicial coincidirá con la dirección inicial de esta instrucción, es decir, la número $200E_{16}$.

- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número $200E_{16}$.
- Tamaño de la instrucción:
  - ❖ Cabecera: 1 byte.
  - ❖ Código de operación: 1 byte.
  - ❖ Registro destino (fd): 1 byte.
  - ❖ Primer registro fuente (ff): 1 byte.
  - ❖ Segundo registro fuente (fg): 1 byte.
  - ❖ Total: 5 bytes.
- Como se recordará, el campo cabecera contiene el tamaño total en bytes de la instrucción almacenada (incluyendo dicha cabecera). Por lo tanto, el valor almacenado será la representación del total obtenido en el inciso anterior en el sistema de numeración binario sin signo (al ser siempre el tamaño de la instrucción un valor positivo, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $5_{10} = 00000101_2$.

iii. Código de operación:

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número $200F_{16}$.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número $200F_{16}$.
- El código de operación a almacenar para esta instrucción se obtiene de la tabla 39 de este informe: 00011100.

iv. Registro destino (fd):

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2010_{16}.

- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2010_{16}.
- Al tratarse f3 del registro destino de esta instrucción, se debe calcular la representación de su identificador de registro, 19, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $19_{10} = 00010011_2$.

v. Primer registro fuente (ff):
   - Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2011_{16}.
   - Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2011_{16}.
   - Al tratarse f1 del primer registro fuente de esta instrucción, se debe calcular la representación de su identificador de registro, 17, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $17_{10} = 00010001_2$.

vi. Segundo registro fuente (fg):
   - Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2012_{16}.
   - Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2012_{16}.

- Al tratarse f2 del segundo registro fuente de esta instrucción, se debe calcular la representación de su identificador de registro, 18, en el sistema de numeración binario sin signo (al ser siempre el identificador de registro un valor natural, la computadora utiliza este sistema para representarlo). Una vez realizada la conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $18_{10} = 00010010_2$.

vii. La longitud total de esta instrucción es de 5 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, dicha instrucción ocupará 5 direcciones: desde la $200E_{16}$ hasta la 2012_{16} inclusive.

e. Cuarta y última instrucción (halt):

i. La dirección inicial de esta instrucción será aquélla ubicada inmediatamente a continuación de la dirección final de la instrucción anterior, es decir, la número 2013_{16}.

ii. Cabecera:

- Su dirección inicial coincidirá con la dirección inicial de esta instrucción, es decir, la número 2013_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2013_{16}.
- Tamaño de la instrucción:
  - ❖ Cabecera: 1 byte.
  - ❖ Código de operación: 1 byte.
  - ❖ Total: 2 bytes.
- Como se recordará, el campo cabecera contiene el tamaño total en bytes de la instrucción almacenada (incluyendo dicha cabecera). Por lo tanto, el valor almacenado será la representación del total obtenido en el inciso anterior en el sistema de numeración binario sin signo (al ser siempre el tamaño de la instrucción un valor positivo, la computadora utiliza este sistema para representarlo). Una vez realizada la

conversión, sin olvidar que este campo ocupa 8 bits, puede obtenerse el siguiente resultado: $2_{10} = 00000010_2$.

iii. Código de operación:

- Su dirección inicial será aquélla ubicada inmediatamente a continuación de la dirección final del campo anterior, es decir, la número 2014_{16}.
- Su longitud es de 8 bits, es decir, 1 byte. Al contener cada dirección de memoria exactamente 1 byte de información, este campo ocupará 1 dirección: la número 2014_{16}.
- El código de operación a almacenar para esta instrucción se obtiene de la tabla 39 de este informe: 10000001.

iv. La longitud total de esta instrucción es de 2 bytes. Al contener cada dirección de memoria exactamente 1 byte de información, dicha instrucción ocupará 2 direcciones: desde la 2013_{16} hasta la 2014_{16} inclusive.

Tabla 44. Disposición de los datos e instrucciones del programa de prueba de la figura anterior en la memoria principal de la nueva computadora.

Dirección de memoria	Contenido	Campo del dato o instrucción
1000	11101100	n1 (byte 0)
1001	11111111	n1 (byte 1)
1002	11111111	n1 (byte 2)
1003	11111111	n1 (byte 3)
1004	01010000	n1 (byte 0)
1005	00000000	n1 (byte 1)

1006	...	...
1007	...	...
...	...	...
2000	00000110	Cabecera
2001	00001000	Código de operación
2002	00000001	rd
2003	00000000	Inm (byte 0)
2004	00010000	Inm (byte 1)
2005	00000000	ri
2006	00001000	Cabecera
2007	00011001	Código de operación
2008	00000011	rd
2009	00000010	rf
200A	11111011	N (byte 0)
200B	11111111	N (byte 1)
200C	11111111	N (byte 2)
200D	11111111	N (byte 3)
200E	00000101	Cabecera
200F	00011100	Código de operación
2010	00000011	fd
2011	00000001	ff
2012	00000010	fg
2013	00000010	Cabecera
2014	10000001	Código de operación
2015	...	...
2016	...	...
...	...	...

Observaciones:

- Las direcciones de memoria se encuentran representadas en el sistema de numeración hexadecimal.

3.1.6. Ciclo de ejecución: segmentación del cauce

El ciclo de ejecución propuesto para las instrucciones en este procesador se encuentra basado en el tipo de arquitectura elegido para la nueva computadora (RISC) y la compatibilidad con una posible implementación de la segmentación en el cauce sin que existan inconvenientes. Para ello, se determinó, en primer lugar, en consonancia con la arquitectura seleccionada, que el flujo de ejecución debía ser fijo para cualquier instrucción sin importar su tipo ni su denominación particular (recordar que en el diseño original podía variar en función del tamaño del operando) ya que, en caso contrario, sería imposible realizar la división del ciclo de ejecución en etapas para segmentar el cauce si las mismas pudieran ejecutarse o no según la instrucción en cuestión.

Las etapas en las cuales fue fraccionado el ciclo de ejecución con el objetivo de implementar la segmentación en el cauce del procesador coinciden exactamente con la división ofrecida por el simulador WinMIPS64, siendo ésta también la más comúnmente mencionada al momento de introducir esta técnica en las explicaciones teóricas sobre este tema. No debe olvidarse que, incluso si ya se encuentra plenamente incorporada y operativa la segmentación en la CPU, las etapas siempre se ejecutarán de manera secuencial dentro de una misma instrucción. A continuación, se describe en términos generales cada una de ellas:

Tabla 45. Etapas del ciclo de ejecución de las instrucciones en el nuevo procesador.

Etapa	Tareas a ejecutar (en orden secuencial)
1. Búsqueda	Acceder al registro IP y obtener a partir de él la dirección inicial en memoria principal de la próxima instrucción a ejecutar.
	Acceder a la dirección de memoria adquirida en el paso anterior y obtener a partir de ella y las contiguas todos los campos de la próxima instrucción.
	Acceder al registro IR y actualizarlo con el código de operación de la nueva instrucción obtenida en el paso anterior.

	Transmitir todos los campos de la nueva instrucción, excepto la cabecera y el código de operación, a la unidad de control de la CPU para que ésta pueda continuar con la siguiente etapa del ciclo de ejecución.
	Incrementar el registro IP a partir del valor contenido en el campo cabecera de la instrucción adquirida en el paso anterior para que apunte a la próxima instrucción a ejecutar.
2. Decodificación	Acceder al registro IR y obtener a partir de él el código de operación de la nueva instrucción a ejecutar, actualizado en la etapa anterior del ciclo de ejecución.
	Decodificar el código de operación obtenido en el paso anterior e interpretar a partir de él la denominación de la nueva instrucción a ejecutar.
	Si se identifica que el código interpretado en el paso anterior corresponde al de una instrucción de transferencia de control incondicional, actualizar el registro IP con la dirección de salto indicada en dicha instrucción.
	En caso contrario, si es necesario, acceder a los registros de uso general de la CPU asociados con los identificadores recibidos de la etapa anterior del ciclo de ejecución y obtener a partir de ellos los operandos de la nueva instrucción.
	Enviar los operandos obtenidos en el paso anterior a las unidades de la CPU encargadas de ejecutar las etapas restantes del ciclo de ejecución junto con la información necesaria para indicarle a cada una de ellas, en caso de corresponder, la acción a realizar cuando le llegue el turno de ejecutar su etapa asignada del ciclo de ejecución de la nueva instrucción.
3.1. Ejecución	Determinar a partir de la información recibida de la etapa de decodificación, en caso de corresponder, la operación aritmético-lógica requerida por la nueva instrucción para ser ejecutada en esta etapa del ciclo de ejecución.
	Ejecutar la operación aritmético-lógica determinada en el paso anterior.

	Acceder al registro FLAGS y actualizar las banderas que deban ser modificadas como consecuencia de la ejecución de la operación aritmético-lógica en el paso anterior.
	En caso de tratarse la nueva instrucción de un salto condicional, acceder nuevamente al registro FLAGS y obtener a partir de él el valor actual de cada una de sus banderas.
	A partir de las banderas obtenidas en el paso anterior, analizar aquélla que corresponda para determinar si el salto debe ser tomado o no.
	Si la respuesta determinada en el paso anterior fue afirmativa, actualizar el registro IP con la dirección de salto indicada en la nueva instrucción.
	Enviar el resultado de la operación aritmético-lógica obtenido en esta etapa a las unidades de la CPU encargadas de ejecutar las etapas restantes del ciclo de ejecución para que puedan utilizarlo en caso de requerirlo para llevar a cabo sus respectivas tareas.
3.2. Ejecución en punto flotante	Determinar a partir de la información recibida de la etapa de decodificación, en caso de corresponder, la operación aritmético-lógica en punto flotante requerida por la nueva instrucción para ser ejecutada en esta etapa del ciclo de ejecución.
	Ejecutar la operación aritmético-lógica en punto flotante determinada en el paso anterior.
	Acceder al registro FPFLAGS y actualizar las banderas que deban ser modificadas como consecuencia de la ejecución de la operación aritmético-lógica en punto flotante en el paso anterior.
	En caso de tratarse la nueva instrucción de una comparación, acceder nuevamente al registro FPFLAGS y obtener a partir de él el valor actual de cada una de sus banderas.
	A partir de las banderas obtenidas en el paso anterior, analizar aquélla que corresponda para determinar el valor con el cual deberá actualizarse la bandera F.
	Acceder una vez más al registro FPFLAGS y actualizar la bandera F con el valor determinado en el paso anterior.

	Enviar el resultado de la operación aritmético-lógica en punto flotante obtenido en esta etapa a las unidades de la CPU encargadas de ejecutar las etapas restantes del ciclo de ejecución para que puedan utilizarlo en caso de requerirlo para llevar a cabo sus respectivas tareas.
4. Acceso a memoria	Determinar a partir de la información recibida de la etapa de decodificación, en caso de corresponder, el tipo de acceso a memoria de la computadora requerido por la nueva instrucción para ser llevado a cabo en esta etapa del ciclo de ejecución.
	Acceder a la dirección de memoria de datos o el puerto del periférico de E/S determinado en el paso anterior para realizar sobre él una operación de lectura o escritura según corresponda.
	En caso de haberse tratado de una operación de lectura, enviar el valor obtenido a la unidad de almacenamiento en registro para que pueda utilizarlo en caso de requerirlo para ejecutar la última etapa del ciclo de ejecución de la nueva instrucción.
5. Almacenamiento en registro	Determinar a partir de la información recibida de la etapa de decodificación, en caso de corresponder, el tipo de acceso a registro de uso general de la CPU requerido por la nueva instrucción para ser ejecutada en esta etapa del ciclo de ejecución.
	Acceder al registro de uso general determinado en el paso anterior y actualizarlo con el valor que corresponda según lo indicado en la instrucción actual. Dicho valor habrá sido recibido de alguna de las etapas anteriores del ciclo de ejecución: decodificación, ejecución/ejecución en punto flotante o acceso a memoria en función de la instrucción ejecutada.

Observaciones:

- Las etapas 3.1. (ejecución) y 3.2. (ejecución en punto flotante) son mutuamente excluyentes, es decir, no pueden pertenecer simultáneamente al ciclo de ejecución de una misma instrucción. Esto se debe a que, como su nombre lo indica, la etapa de ejecución en punto flotante está orientada únicamente a instrucciones aritméticas o de transferencia de datos que involucran la realización de operaciones aritméticas en punto flotante en la FPU (unidad

de punto flotante) de la nueva CPU; mientras que la etapa de ejecución forma parte del ciclo de ejecución de la totalidad del resto de las instrucciones del repertorio del procesador, incluyendo a aquéllas que no requieren realizar ningún tipo de acción en esta etapa.

- Para realizar la descripción ofrecida en la tabla anterior se consideró un escenario de ejecución considerablemente más simplificado con respecto al que se presentaría en una simulación real de este procesador. Más concretamente, se omitió la presencia de un fenómeno adverso asociado a la implementación de la segmentación en el cauce: posibles atascos en la ejecución de las instrucciones debido a diversos conflictos ya sea por el acceso y utilización de los limitados recursos de hardware de la computadora, el orden de lectura y escritura de algún dato contenido en uno de los registros de uso general de la CPU o la determinación de la próxima instrucción a ejecutar debido a la existencia de una instrucción de transferencia de control que podría alterar el normal flujo de ejecución del programa. Cada uno de estos tipos de atascos debe ser detectado con anticipación para detener temporalmente la ejecución del procesador durante uno o varios ciclos de reloj hasta que el conflicto sea resuelto y de esta manera prevenir que el programa en ejecución ofrezca un comportamiento y/o resultados incorrectos debido al inadecuado manejo de estos conflictos. Obviamente, dicha labor de detección tiene que llevarse a cabo durante el ciclo de ejecución de cada una de las instrucciones del programa, lo cual involucrará ciertas modificaciones en algunas etapas de la segmentación para que, además de todas las tareas más conocidas y tradicionales mencionadas en la tabla, posean la capacidad de conocer algunos datos puntuales sobre el estado actual de la ejecución del programa (datos pendientes de escritura, futuros accesos a memoria de datos, instrucciones de transferencia de control actualmente en ejecución), junto con la lógica necesaria para determinar si puede producirse un atasco y, de ser así, tomar todas las medidas necesarias para gestionarlo. Los detalles más técnicos sobre las metodologías seleccionadas para la incorporación de estas tareas no serán ofrecidos en este informe por considerarse demasiado complejos e intrincados y ultimadamente irrelevantes para el usuario interesado simplemente en programar la CPU desde un nivel de abstracción más elevado como lo es el del lenguaje Assembler. Debería bastar con informar que la mayoría de estas labores de detección son realizadas por componentes de la CPU completamente ajenos a las unidades encargadas de ejecutar las tareas más conocidas de cada una de las etapas del ciclo de ejecución, enunciadas en la tabla anterior. No obstante, huelga decir que, si el usuario continúa ávido de interiorizarse aún más sobre los pormenores del funcionamiento de estos “detectores”, es libre de estudiar

directamente el código utilizado por el diseñador para la descripción de la computadora y obtener por sí mismo sus propias conclusiones.

A continuación, se definen las unidades de la CPU encargadas de ejecutar cada una de las etapas del ciclo de ejecución mencionadas y descriptas en la tabla precedente a ésta. Como se detalló en una de las observaciones anteriores, aquí se está considerando un escenario simplificado en el cual no intervienen ninguno de los distintos tipos de atascos asociados a la implementación de la segmentación en el cauce debido a potenciales conflictos respecto a recursos de hardware, dependencias de datos o transferencias de control. Por lo tanto, sólo se tienen en cuenta las tareas enunciadas en la tabla anterior omitiendo cualquier labor vinculada a los mecanismos de detección de atascos utilizados en la mayoría de las etapas, los cuales son gestionados y ejecutados en su mayor parte por unidades del procesador independientes y separadas de aquéllas definidas en la próxima tabla.

Tabla 46. Unidad encargada de ejecutar cada etapa del ciclo de ejecución de las instrucciones en el nuevo procesador.

Etapa	**Unidad de la CPU encargada de su ejecución**
1. Búsqueda	Unidad de búsqueda
2. Decodificación	Unidad de control
3.1. Ejecución	Unidad aritmético-lógica (ALU)
3.2. Ejecución en punto flotante	Unidad de punto flotante (FPU)
4. Acceso a memoria	Unidad de acceso a memoria
5. Almacenamiento en registro	Unidad de almacenamiento en registro

Aunque ya se hizo referencia a este apartado durante la definición de las tareas a ejecutar en las distintas etapas que constituyen el ciclo de ejecución de las instrucciones en el nuevo procesador (ver tabla 45), sería conveniente incluir una tabla adicional en la cual se enuncien sintéticamente los dispositivos de almacenamiento que pueden ser accedidos en cada una de dichas etapas, diferenciando el tipo de operación con el cual podría encontrarse asociado cada posible acceso: lectura y/o escritura de información (ya sea datos o instrucciones, la distinción entre ambos tipos de información resulta irrelevante para el nivel de este caso de estudio) según

corresponda. Esto será útil más adelante cuando se necesite detallar una importante consideración que se debió tener en cuenta para describir el diseño de la arquitectura para la nueva computadora.

Tabla 47. Dispositivos de almacenamiento que pueden ser accedidos en cada etapa del ciclo de ejecución de las instrucciones en la nueva computadora.

Dispositivo de almacenamiento		**Etapa**	**Tipo de acceso**
Banco de registros de uso general		2. Decodificación	Lectura
		5. Almacenamiento en registro	Escritura
Registros de uso particular	Registro IR	1. Búsqueda	Escritura
		2. Decodificación	Lectura
	Registro IP	1. Búsqueda	Lectura/Escritura
		2. Decodificación	Escritura
		3.1. Ejecución	Escritura
	Registro FLAGS	3.1. Ejecución	Lectura/Escritura
	Registro FPFLAGS	2. Decodificación	Lectura
		3.2. Ejecución en punto flotante	Lectura/Escritura
Memoria principal	Sección de datos	4. Acceso a memoria	Lectura/Escritura
	Sección de instrucciones	1. Búsqueda	Lectura

Ahora bien, como podrá recordarse, en la primera tabla de este informe se mencionó como uno de los principios fundamentales de la arquitectura RISC la imperiosa necesidad de que cada instrucción requiera exactamente un único ciclo de reloj para ejecutarse, sin importar su denominación o complejidad exigida para ser llevada a cabo. Asimismo, el lector se acordará que, tal y como se refleja en la tabla 35, el procesador de este nuevo diseño, al haberse inclinado el diseñador por una arquitectura para él más cercana al tipo RISC, se mantuvo fiel a dicho principio. No obstante, se debe considerar que, al implementar la segmentación en el cauce, cada instrucción ya no podrá ser tenida en cuenta como una unidad compacta e inseparable sino que ahora cada una de las etapas en las cuales fue dividido su ciclo de ejecución conformará su propia entidad independiente, por lo cual el principio original de la arquitectura RISC en lugar de aplicarse sobre cada instrucción deberá destinarse a cada etapa particular del ciclo, es decir,

a partir de este momento serán estas últimas las que deberán requerir, cada una, exactamente un único ciclo de reloj para ser ejecutada. La metodología que suele utilizarse a fin de cumplir esta regla es determinar el tiempo de ejecución necesario para ejecutar cada etapa y optar por el más lento de todos para que sea el nuevo ciclo de reloj, de manera tal de garantizar que todas las etapas puedan completar su ejecución durante su espacio de tiempo asignado (algunas de las etapas más veloces permanecerán ociosas durante un pequeño intervalo de tiempo hasta que finalice el período de reloj).

Una particularidad a tener en cuenta sería la situación de la etapa 3.2. del ciclo de ejecución de las instrucciones, denominada "ejecución en punto flotante". En el párrafo anterior se definió uno de los principios fundamentales de la arquitectura RISC, el cual determina que cada instrucción o, en caso de implementarse la segmentación en el cauce, cada etapa del ciclo de ejecución debe requerir exactamente un único ciclo de reloj para ser ejecutada. Sin embargo, al diseñarse el ciclo de ejecución para el nuevo procesador, se determinó que la etapa 3.2., al encontrarse destinada a la ejecución de operaciones aritméticas en punto flotante en la FPU (unidad de punto flotante), y al considerarse ésta como una labor considerablemente más compleja y, por lo tanto, más lenta que cualquiera de las demás tareas del ciclo de ejecución de una instrucción, necesitará cuatro ciclos de reloj para completarse en lugar de uno como el resto de las etapas, procurando así que la descripción de la CPU sea tan cercana a la realidad como resulte posible.

Una alternativa para evitar esta anomalía podría haber sido utilizar el tiempo de ejecución de la etapa 3.2. como el nuevo ciclo de reloj para la ejecución del cauce segmentado del procesador. Sin embargo, una importante desventaja presentada como consecuencia de esta elección sería que, al existir una diferencia tan elevada respecto a los tiempos de ejecución del resto de las etapas, estas últimas permanecerían ociosas en cada período durante un intervalo de tiempo demasiado considerable como para ser ignorado, repercutiendo así muy negativamente sobre el rendimiento general del procesador.

En cambio, se optó por subdividir esta etapa en cuatro etapas distintas, las cuales, tanto ellas como la FPU encargada de ejecutarlas, están definidas y configuradas de manera tal de permitirle al procesador seguir cumpliendo con los principios fundamentales tanto de la arquitectura RISC como de la segmentación del cauce: cada una de estas cuatro etapas requerirá exactamente un único ciclo de reloj para ser ejecutada y, además, múltiples instrucciones podrán ejecutarse simultáneamente en un mismo ciclo de reloj, siempre y cuando se trate, obviamente, de etapas distintas del ciclo de ejecución (recordar que no se debe confundir

segmentación con paralelismo: en esta CPU jamás podrá ejecutarse al mismo tiempo una misma etapa para múltiples instrucciones bajo ninguna circunstancia).

A continuación, se exhibirán por separado los dos posibles ciclos de ejecución que pueden presentar las instrucciones del repertorio del nuevo procesador en función de su necesidad de utilizar la FPU para realizar operaciones aritméticas en punto flotante.

Tabla 48. Ciclo de ejecución de las instrucciones que no requieren realizar operaciones aritméticas en punto flotante en la FPU del nuevo procesador.

Etapa	Denominación
1.	Búsqueda
2.	Decodificación
3.1.	Ejecución
4.	Acceso a memoria
5.	Almacenamiento en registro

Tabla 49. Ciclo de ejecución de las instrucciones que requieren realizar operaciones aritméticas en punto flotante en la FPU del nuevo procesador.

Etapa	Denominación
1.	Búsqueda
2.	Decodificación
3.2.1.	Ejecución en punto flotante (Etapa 1)
3.2.2.	Ejecución en punto flotante (Etapa 2)
3.2.3.	Ejecución en punto flotante (Etapa 3)
3.2.4.	Ejecución en punto flotante (Etapa 4)

4.	Acceso a memoria
5.	Almacenamiento en registro

Observaciones:

- Recordar que las etapas 3.2.1., 3.2.2., 3.2.3. y 3.2.4. surgen simplemente de la subdivisión de la etapa 3.2. (ejecución en punto flotante) en cuatro etapas independientes a fin de preservar el cumplimiento por parte de la CPU de los principios fundamentales tanto de la arquitectura RISC como de la segmentación del cauce.
- Huelga decir que una misma instrucción del repertorio jamás podrá exhibir un tipo de ciclo durante una de sus ejecuciones y el otro tipo en una instancia distinta, ya que en todas las ocasiones su función y sus tareas permanecerán inalterables.
- Como habrá podido adivinarse, al encontrarse su etapa de ejecución subdividida en cuatro etapas distintas, cada una con un tiempo de ejecución igual a un ciclo de reloj, el ciclo de ejecución de una instrucción que necesite utilizar la FPU del procesador para realizar una operación aritmética en punto flotante será considerablemente más lento, extenso y complejo que el de una que no requiera hacer uso de ella. Más precisamente, la diferencia entre los tiempos de ejecución totales de ambos ciclos de ejecución será exactamente igual a tres ciclos de reloj (ocho vs. cinco ciclos de reloj).
- Se mencionó anteriormente que, a diferencia del resto de las etapas en las cuales fue segmentado el cauce del nuevo procesador para conformar el ciclo de ejecución de las instrucciones de su repertorio (ver tabla 45), la etapa 3.2. (ejecución en punto flotante) requerirá cuatro ciclos de reloj para completarse. Por lo tanto, sería conveniente aclarar a continuación la distribución exacta de todas las tareas a realizarse en esta etapa entre sus cuatro distintas subetapas (3.2.1., 3.2.2., 3.2.3. y 3.2.4.), cada una de las cuales se ejecutará a su vez en un ciclo de reloj diferente de dicha etapa a la que pertenecen, completando así los cuatro ciclos de reloj que esta última necesita para efectuarse en su totalidad. Esta aclaración resultará extremadamente útil para explicar en la subsección “3.1.8. Riesgos de la segmentación y atascos en el cauce” de este informe la presencia y la resolución de los atascos en el cauce de la CPU debido a dependencias de datos de tipo RAW (Read After Write, es decir, Lectura Después de Escritura).

Tabla 50. Distribución de las tareas de la etapa “3.2. Ejecución en punto flotante” del ciclo de ejecución de las instrucciones en el nuevo procesador entre sus cuatro subetapas distintas.

Subetapa	Tareas a ejecutar (en orden secuencial)
3.2.1. Ejecución en punto flotante (Etapa 1)	Determinar a partir de la información recibida de la etapa de decodificación, en caso de corresponder, la operación aritmético-lógica en punto flotante requerida por la nueva instrucción para ser ejecutada en esta etapa del ciclo de ejecución.
3.2.2. Ejecución en punto flotante (Etapa 2)	Ejecutar la operación aritmético-lógica en punto flotante determinada en el paso anterior.
3.2.3. Ejecución en punto flotante (Etapa 3)	
3.2.4. Ejecución en punto flotante (Etapa 4)	Acceder al registro FPFLAGS y actualizar las banderas que deban ser modificadas como consecuencia de la ejecución de la operación aritmético-lógica en punto flotante en el paso anterior.
	En caso de tratarse la nueva instrucción de una comparación, acceder nuevamente al registro FPFLAGS y obtener a partir de él el valor actual de cada una de sus banderas.
	A partir de las banderas obtenidas en el paso anterior, analizar aquélla que corresponda para determinar el valor con el cual deberá actualizarse la bandera F.
	Acceder una vez más al registro FPFLAGS y actualizar la bandera F con el valor determinado en el paso anterior.
	Enviar el resultado de la operación aritmético-lógica en punto flotante obtenido en esta etapa a las unidades de la CPU encargadas de ejecutar las etapas restantes del ciclo de ejecución para que puedan utilizarlo en caso de requerirlo para llevar a cabo sus respectivas tareas.

A fin de poder apreciar más claramente las ventajas de la implementación de la segmentación en el cauce del procesador como una alternativa recomendable para la ejecución de las instrucciones de los programas, se representarán en las siguientes figuras distintos casos de prueba.

Cada escenario estará conformado por la ejecución de una serie distinta de instrucciones, cada una de las cuales a su vez puede o no requerir utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante según sea el caso (recordar que, tal como se exhibió en las tablas 48 y 49, el uso o no de la FPU implicará notorias diferencias respecto a la estructura y la duración del ciclo de ejecución de la instrucción en cuestión).

Más precisamente, se presentarán en total cuatro casos de prueba para ejecutar dos series distintas de instrucciones. Cada uno de los dos primeros escenarios estará destinado a la ejecución de una serie diferente, con la particularidad de que dicha ejecución será llevada a cabo de manera secuencial. Es decir, en esta modalidad de ejecución, aunque se mantenga la división del ciclo de ejecución de las instrucciones en etapas independientes, separadas y compactas como en una ejecución segmentada, cada instrucción deberá esperar a que la anterior finalice la ejecución de todas sus etapas para comenzar a ejecutarse: nunca podrán encontrarse múltiples instrucciones en ejecución al mismo tiempo.

Por su parte, en los últimos dos escenarios de prueba se representarán las ejecuciones de las mismas dos series de instrucciones que en los primeros dos casos, pero ahora la ejecución se realizará en forma segmentada. En otras palabras, en esta modalidad de ejecución, además de dividir el ciclo de ejecución de las instrucciones en etapas independientes, separadas y compactas, cada instrucción no deberá esperar a que la anterior finalice la ejecución de todas sus etapas para comenzar a ejecutarse: podrán existir múltiples instrucciones ejecutándose simultáneamente siempre y cuando se encuentren en etapas distintas del ciclo de ejecución.

De este modo, el lector observará una notoria mejora en el rendimiento del procesador como consecuencia de la implementación de la segmentación en su cauce: a pesar de que cada instrucción individual demora el mismo tiempo para ejecutarse que en un programa secuencial, al habilitarse en la ejecución segmentada la posibilidad de ejecutar múltiples instrucciones simultáneamente, el tiempo de ejecución total del programa se verá drásticamente reducido ya que requerirá un menor número de ciclos de reloj para finalizar su ejecución. Existen varias métricas específicamente diseñadas para evaluar y medir cuantitativamente la mejora en el rendimiento del procesador segmentado respecto al secuencial. No obstante, para esta situación en particular no se consideró relevante el análisis y la utilización de su expresión algebraica por

tratarse ésta por sí misma de una temática de estudio absolutamente diferente y compleja que no pertenece estrictamente al área de interés de este informe.

Tabla 51. Colores utilizados para representar cada una de las etapas del ciclo de ejecución de las instrucciones de los casos de prueba en las próximas figuras.

Etapa	**Color**
1. Búsqueda	Rojo
2. Decodificación	Violeta
3.1. Ejecución	Verde
3.2. Ejecución en punto flotante	Marrón
4. Acceso a memoria	Azul
5. Almacenamiento en registro	Rosa

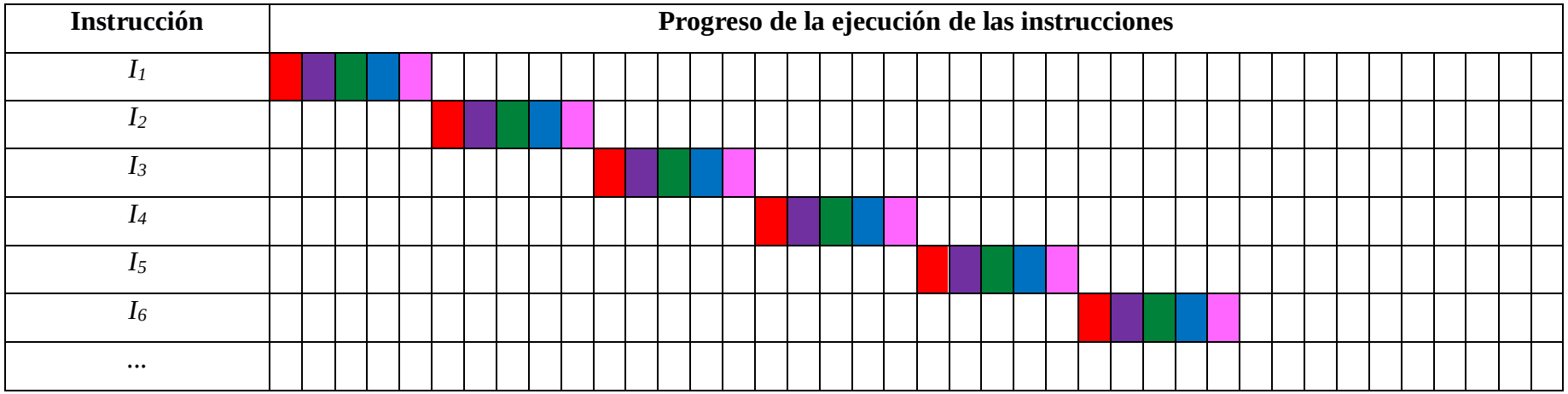


Figura 52. Progreso de la ejecución secuencial de una serie de instrucciones en la nueva CPU a través del tiempo que no requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.

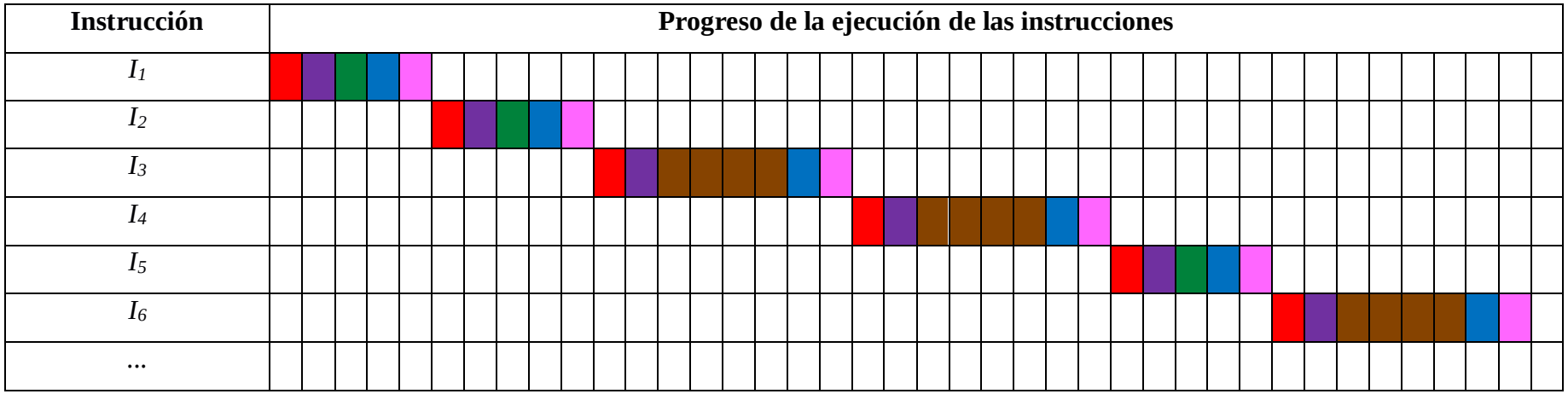


Figura 53. Progreso de la ejecución secuencial de una serie de instrucciones en la nueva CPU a través del tiempo, algunas de las cuales requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Las instrucciones I_1, I_2 e I_5 no requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante. Por ello, su ciclo de ejecución es más corto y rápido que el del resto de las instrucciones de la serie.
- Las instrucciones I_3, I_4 e I_6 requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante. Por ello, su ciclo de ejecución es más largo y lento que el del resto de las instrucciones de la serie.

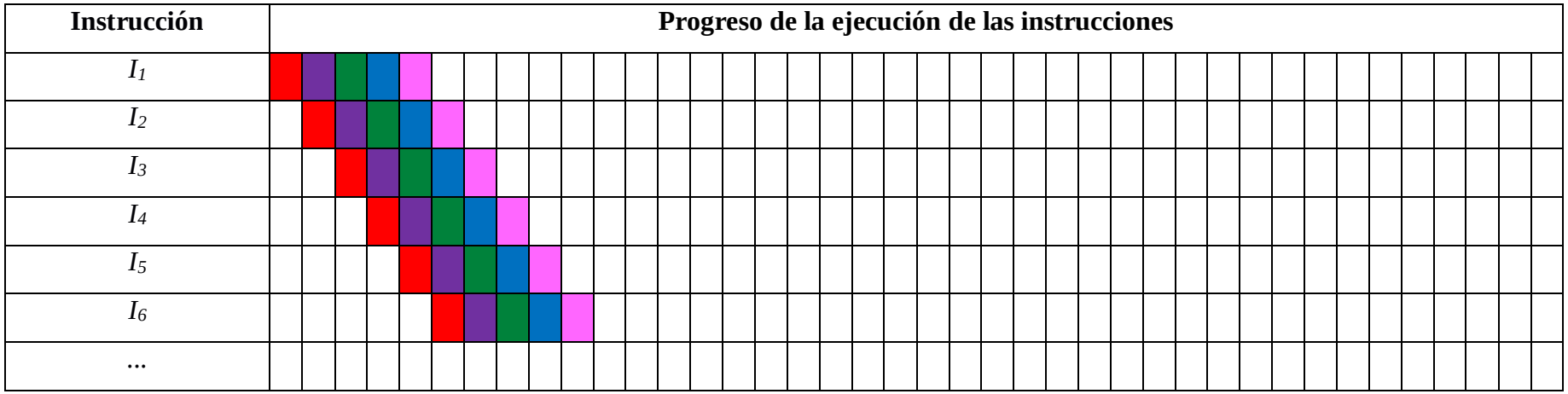


Figura 54. Progreso de la ejecución segmentada de una serie de instrucciones en la nueva CPU a través del tiempo que no requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.

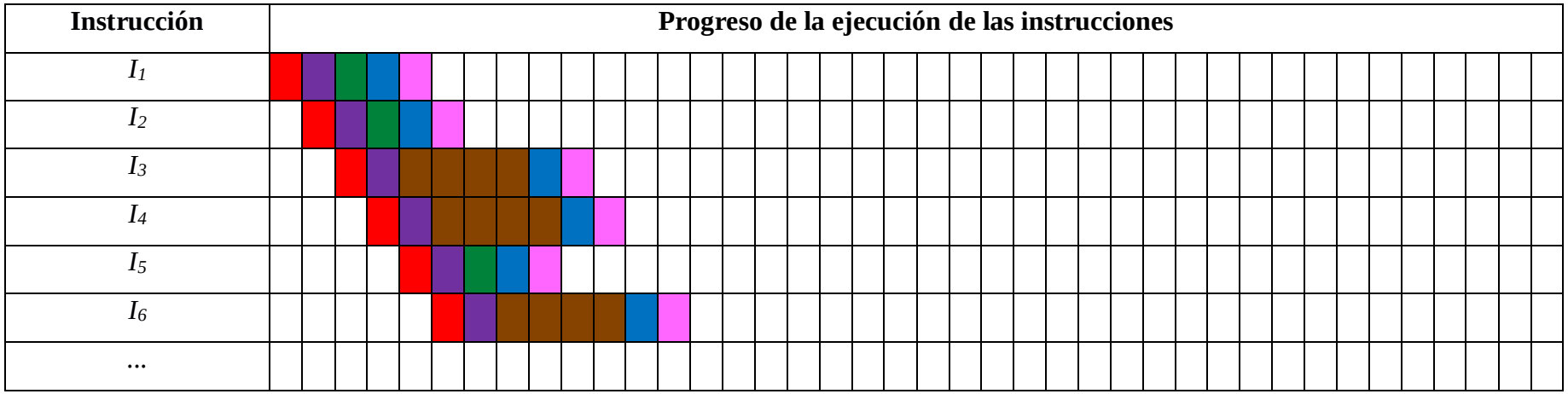


Figura 55. Progreso de la ejecución segmentada de una serie de instrucciones en la nueva CPU a través del tiempo, algunas de las cuales requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Las instrucciones I_1, I_2 e I_5 no requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante. Por ello, su ciclo de ejecución es más corto y rápido que el del resto de las instrucciones de la serie.
- Las instrucciones I_3, I_4 e I_6 requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante. Por ello, su ciclo de ejecución es más largo y lento que el del resto de las instrucciones de la serie.

En la próxima tabla se definirán específicamente las instrucciones del repertorio del nuevo procesador que requieren realizar operaciones aritméticas en punto flotante en la FPU de la CPU y que, por lo tanto, presentarán su etapa de ejecución subdividida en cuatro etapas distintas, cada una de ellas con un tiempo de ejecución igual a un ciclo de reloj, tal como lo indican las figuras 53 y 55, siendo el tiempo de ejecución total de esta etapa igual a cuatro ciclos de reloj. Asimismo, se incluyen el resto de las instrucciones que no necesitan realizar ningún tipo de operación aritmético-lógica o, en caso de requerirlo, sólo intervendrían en ella operandos enteros, por lo que únicamente tendrían que utilizar la ALU del procesador. En consecuencia, tal como lo muestran las figuras 52 y 54, su etapa de ejecución no presentaría subdivisión alguna y el tiempo de ejecución de la misma sería igual a un solo ciclo de reloj.

Tabla 56. Tipo de etapa de ejecución perteneciente al ciclo de ejecución de cada instrucción del repertorio del nuevo procesador.

Tipo de instrucción	Instrucción	Etapa de ejecución
Transferencia de Datos	lb r_d, $Inm(r_i)$	3.1. Ejecución
	sb r_f, $Inm(r_i)$	3.1. Ejecución
	lh r_d, $Inm(r_i)$	3.1. Ejecución
	sh r_f, $Inm(r_i)$	3.1. Ejecución
	lw r_d, $Inm(r_i)$	3.1. Ejecución
	sw r_f, $Inm(r_i)$	3.1. Ejecución
	lf f_d, $Inm(r_i)$	3.1. Ejecución
	sf f_f, $Inm(r_i)$	3.1. Ejecución
	mff f_d, f_f	3.1. Ejecución
	mfr f_d, r_f	3.1. Ejecución
	mrf r_d, f_f	3.1. Ejecución
	tf f_d, f_f	3.2. Ejecución en punto flotante
	ti f_d, f_f	3.2. Ejecución en punto flotante
Aritmética	dadd r_d, r_f, r_g	3.1. Ejecución
	daddi r_d, r_f, N	3.1. Ejecución
	daddu r_d, r_f, r_g	3.1. Ejecución
	daddui r_d, r_f, N	3.1. Ejecución
	addf f_d, f_f, f_g	3.2. Ejecución en punto flotante

	dsub r_d, r_f, r_g	3.1. Ejecución
	dsubu r_d, r_f, r_g	3.1. Ejecución
	subf f_d, f_f, f_g	3.2. Ejecución en punto flotante
	dmul r_d, r_f, r_g	3.1. Ejecución
	dmulu r_d, r_f, r_g	3.1. Ejecución
	mulf f_d, f_f, f_g	3.2. Ejecución en punto flotante
	ddiv r_d, r_f, r_g	3.1. Ejecución
	ddivu r_d, r_f, r_g	3.1. Ejecución
	divf f_d, f_f, f_g	3.2. Ejecución en punto flotante
	slt r_d, r_f, r_g	3.1. Ejecución
	slti r_d, r_f, N	3.1. Ejecución
	ltf f_f, f_g	3.2. Ejecución en punto flotante
	lef f_f, f_g	3.2. Ejecución en punto flotante
	eqf f_f, f_g	3.2. Ejecución en punto flotante
	negr r_d, r_f	3.1. Ejecución
Lógica	andr r_d, r_f, r_g	3.1. Ejecución
	andi r_d, r_f, N	3.1. Ejecución
	orr r_d, r_f, r_g	3.1. Ejecución
	ori r_d, r_f, N	3.1. Ejecución
	xorr r_d, r_f, r_g	3.1. Ejecución
	xori r_d, r_f, N	3.1. Ejecución
	notr r_d, r_f	3.1. Ejecución
Desplazamiento de Bits	dsl r_d, r_f, r_N	3.1. Ejecución
	dsli r_d, r_f, N	3.1. Ejecución
	dsr r_d, r_f, r_N	3.1. Ejecución
	dsri r_d, r_f, N	3.1. Ejecución
	dsls r_d, r_f, r_N	3.1. Ejecución
	dslsi r_d, r_f, N	3.1. Ejecución
	dsrs r_d, r_f, r_N	3.1. Ejecución
	dsrsi r_d, r_f, N	3.1. Ejecución
Transferencia de Control	jmp $offN_I$	3.1. Ejecución
	beq r_f, r_g, $offN_I$	3.1. Ejecución
	bne r_f, r_g, $offN_I$	3.1. Ejecución

	beqz r_f, $offN_I$	3.1. Ejecución
	bnez r_f, $offN_I$	3.1. Ejecución
	bfpt $offN_I$	3.1. Ejecución
	bfpf $offN_I$	3.1. Ejecución
Subrutinas	call $offN_S$	3.1. Ejecución
	ret	3.1. Ejecución
Manejo de Interrupciones	int N_I	3.1. Ejecución
	iret	3.1. Ejecución
	cli	3.1. Ejecución
	sti	3.1. Ejecución
Manejo de la Pila	pushb r_f	3.1. Ejecución
	popb r_d	3.1. Ejecución
	pushh r_f	3.1. Ejecución
	poph r_d	3.1. Ejecución
	pushw r_f	3.1. Ejecución
	popw r_d	3.1. Ejecución
	pushfl	3.1. Ejecución
	popfl	3.1. Ejecución
	pushfp	3.1. Ejecución
	popfp	3.1. Ejecución
	pushra	3.1. Ejecución
	popra	3.1. Ejecución
Entrada/Salida	inb r_d, $offN_P$	3.1. Ejecución
	outb r_f, $offN_P$	3.1. Ejecución
	inh r_d, $offN_P$	3.1. Ejecución
	outh r_f, $offN_P$	3.1. Ejecución
	inw r_d, $offN_P$	3.1. Ejecución
	outw r_f, $offN_P$	3.1. Ejecución
Control	nop	3.1. Ejecución
	halt	3.1. Ejecución

3.1.7. Arquitecturas von Neumann y Harvard

Ahora supóngase que se ejecuta un programa de prueba conformado por instrucciones que en ningún caso requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante. Por lo tanto, todas presentarán un ciclo de ejecución conformado por cinco etapas, cada una con una duración de un ciclo de reloj cada una: la etapa de ejecución no presenta a su vez ninguna clase de subdivisión en etapas, por lo cual sólo necesita un ciclo de reloj para ejecutarse.

A continuación, se describirá la ejecución de dicho programa en forma segmentada a través del tiempo, es decir, aprovechando la división del ciclo de ejecución en etapas separadas e independientes originada a partir de la implementación de la segmentación en el cauce del procesador de manera tal de poder ejecutar múltiples instrucciones simultáneamente, siempre tratándose de distintas etapas del ciclo de ejecución de cada una de ellas. Básicamente, se trata del mismo caso de prueba que aquél representado en la figura 54, pero se vuelve a presentar aquí para que el lector pueda concentrar su atención exclusivamente en él y no en los demás escenarios exhibidos junto a dicha figura en la subsección anterior del informe (“3.1.6. Ciclo de ejecución: segmentación del cauce”).

Instrucción	**Progreso de la ejecución de las instrucciones**										
I_1	F	D	X	M	W						
I_2		F	D	X	M	W					
I_3			F	D	X	M	W				
I_4				F	D	X	M	W			
I_5					F	D	X	M	W		
I_6						F	D	X	M	W	
...											

Figura 57. Progreso de la ejecución segmentada de una serie de instrucciones en la nueva CPU a través del tiempo que no requieren utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.

Tabla 58. Referencias para poder interpretar los ciclos de ejecución de las instrucciones representados en la figura anterior.

Etapa	Letra	Color
1. Búsqueda	F (Fetch)	Rojo
2. Decodificación	D (Decode)	Violeta
3.1. Ejecución	X (Execute)	Verde
4. Acceso a memoria	M (Memory Access)	Azul
5. Almacenamiento en registro	W (Writeback)	Rosa

Aquí es donde desempeña un papel crucial la información proporcionada por la tabla 47, en particular aquélla referida a los posibles accesos a la memoria principal de la computadora ya que, como se recordará, originalmente se había propuesto la arquitectura von Neumann como plantilla para describir el diseño de la arquitectura de la nueva computadora. A continuación, se ofrece un diagrama de la arquitectura von Neumann en el cual se incluyen los componentes principales que plantea para el diseño de la computadora junto con el sistema de interconexión dispuesto para transmitir datos entre los mismos (para este diseño de arquitectura no existe distinción alguna entre datos e instrucciones a este nivel de abstracción) a través de canales de comunicación conocidos como buses.

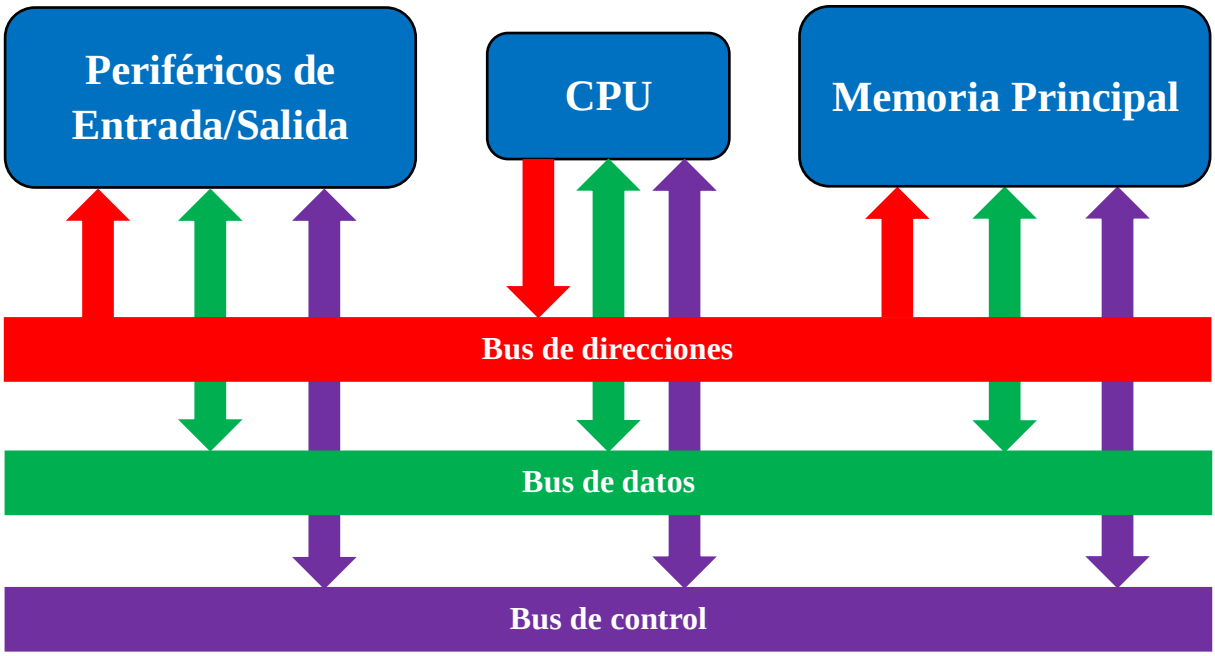


Figura 59. Diagrama de la arquitectura von Neumann.

Como se observará, en la arquitectura von Neumann la memoria principal de la computadora se considera como un único dispositivo de almacenamiento compacto sin distinción alguna entre sus distintas secciones. Por consiguiente, tanto la sección de datos de la misma como la sección de instrucciones tendrán en común el mismo sistema compartido de buses de direcciones, datos y control para poder comunicarse con la CPU para transmitir o recibir información según corresponda.

Si se recuerda la información brindada en la tabla 47, existen dos etapas distintas del ciclo de ejecución de las instrucciones que podrían requerir acceso a la memoria principal de la computadora para realizar operaciones de lectura o escritura: la etapa de búsqueda (memoria de instrucciones) y la de acceso a memoria (memoria de datos). Sin embargo, al establecer la arquitectura von Neumann que las secciones de datos y de instrucciones de la memoria principal deben compartir el mismo sistema de buses, la lógica consecuencia es la imposibilidad de que las dos etapas previamente mencionadas puedan ejecutarse simultáneamente en un mismo ciclo de reloj ya que una de ellas deberá esperar a que la otra libere la utilización del sistema de buses compartido, sin importar que ambas etapas estén intentando acceder a dos secciones distintas e independientes de memoria.

La siguiente figura representa la ejecución de la misma secuencia de instrucciones que en el caso de la figura 57, pero ahora la CPU encargada de ejecutarlas formará parte de una

arquitectura von Neumann, por lo cual la restricción establecida en el párrafo anterior no podrá ser ignorada.

Instrucción	**Progreso de la ejecución de las instrucciones**													
I_1	F	D	X	M	W									
I_2		F	D	X	M	W								
I_3			F	D	X	M	W							
I_4							F	D	X	M	W			
I_5								F	D	X	M	W		
I_6									F	D	X	M	W	
...														

Figura 60. Progreso de la ejecución segmentada de una serie de instrucciones idéntica a aquélla ejecutada en la figura 57, ahora en la CPU de una computadora cuya arquitectura sea von Neumann.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Las celdas tachadas representan la suspensión temporal de la ejecución de la etapa de búsqueda de la instrucción I_4 debido a que el sistema de buses compartido que debería utilizar para acceder a la memoria de instrucciones de la computadora ya se encuentra actualmente reservado por la etapa de acceso a memoria de las instrucciones I_1, I_2 e I_3 respectivamente para poder acceder a la memoria de datos (independientemente de si la instrucción en cuestión efectivamente requiera o no realizar el acceso a memoria de datos para leer o escribir en ella). Únicamente cuando no queden por ejecutar más etapas de acceso a memoria por parte de instrucciones anteriores podrá la instrucción I_4 comenzar la ejecución de su etapa de búsqueda.

Tabla 61. Referencias para poder interpretar los ciclos de ejecución de las instrucciones representados en la figura anterior.

Etapa	Letra	Color
1. Búsqueda	F (Fetch)	Rojo
2. Decodificación	D (Decode)	Violeta
3.1. Ejecución	X (Execute)	Verde
4. Acceso a memoria	M (Memory Access)	Azul
5. Almacenamiento en registro	W (Writeback)	Rosa

A partir de la situación representada en la figura anterior, claramente puede determinarse que la arquitectura von Neumann por sí sola, debido a la limitación en su sistema de buses previamente descripta, no puede ofrecer el soporte necesario para ejecutar las instrucciones en forma segmentada tal y como se presentó en la figura 57, ya que eventualmente siempre el cauce del procesador se verá temporalmente suspendido hasta que todas las etapas de acceso a memoria pendientes de ser ejecutadas se completen. Además, como se podrá adivinar, esta situación ocurrirá muy probablemente en reiteradas ocasiones durante la ejecución del programa, por lo cual el rendimiento final del procesador se verá seriamente afectado y será muy inferior al esperado cuando se decidió implementar originalmente la segmentación en el cauce.

La solución a este problema se encuentra en incorporar una ligera modificación al diseño de la arquitectura originalmente planteado, utilizando como plantilla la arquitectura Harvard, la cual consiste básicamente en una versión moderna de la arquitectura von Neumann con algunas pequeñas alteraciones orientadas precisamente a resolver el inconveniente descripto anteriormente.

Ahora se mostrará un diagrama de la arquitectura Harvard de una manera similar a la representación de la arquitectura von Neumann ofrecida en la figura 59 para que el lector pueda comparar ambas versiones y así comprender el motivo por el cual esta nueva versión sí es capaz de ofrecer el soporte necesario para que la ejecución segmentada del cauce del procesador sea idéntica a la inicialmente esperada (figura 57): múltiples instrucciones pueden ejecutarse simultáneamente en cualquiera de sus etapas, siempre y cuando estas últimas no coincidan entre sí.

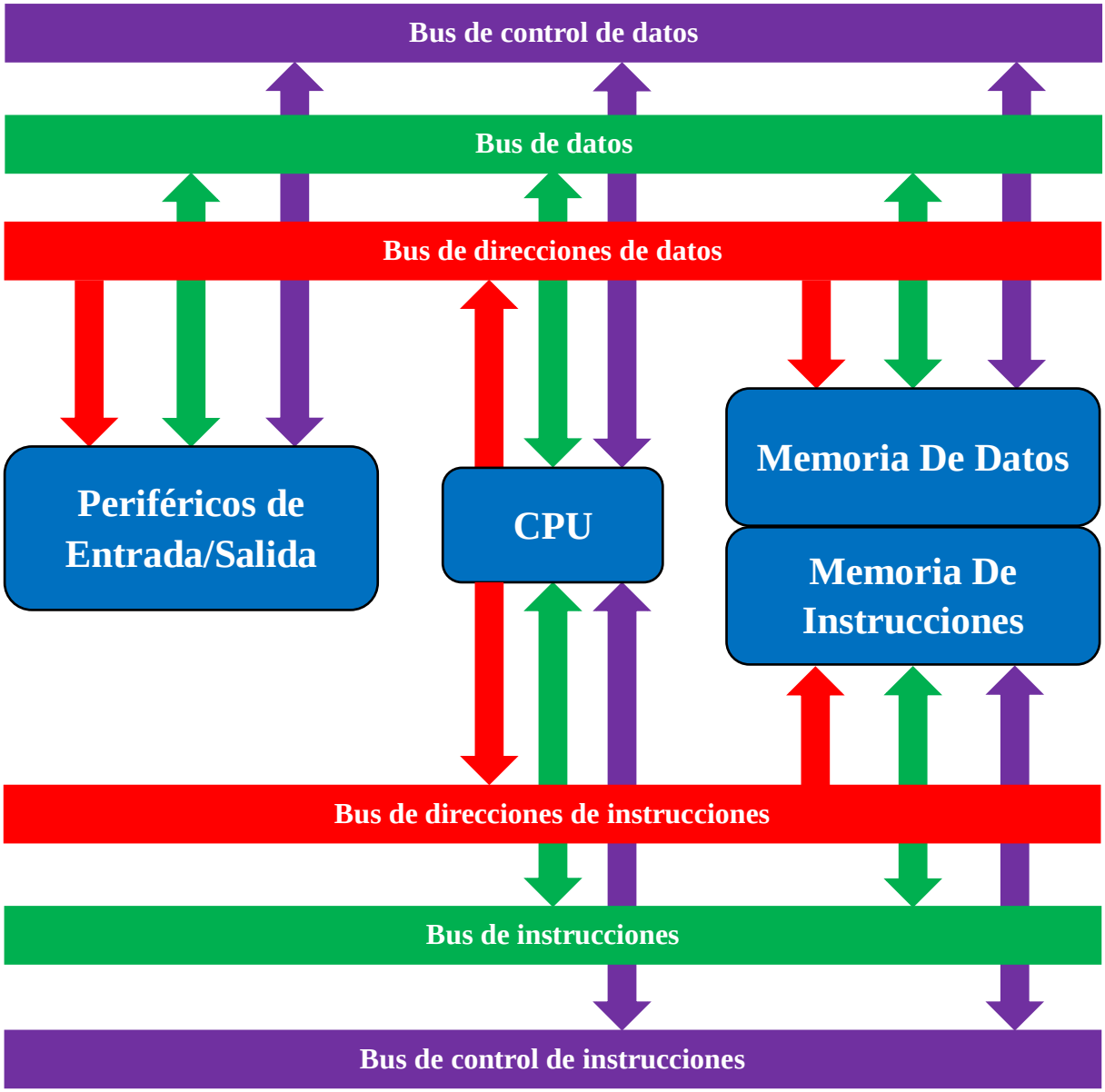


Figura 62. Diagrama de la arquitectura Harvard.

La figura anterior refleja claramente que la mayor diferencia entre las arquitecturas von Neumann y Harvard consiste en la división propuesta por esta última para la memoria principal en dos secciones independientes: una para almacenar datos y otra para preservar instrucciones. Al plantear esta distinción, también determina la necesidad de implementar un sistema de buses de direcciones, control y datos (o instrucciones según corresponda, para este diseño de arquitectura sí existe una diferenciación apreciable entre datos e instrucciones a este nivel de abstracción) separado para cada sección a fin de que la CPU sea capaz de acceder a ambas simultáneamente para realizar operaciones de lectura o escritura.

De este modo puede lograr evitarse la restricción impuesta por la arquitectura von Neumann respecto al acceso simultáneo por parte de la CPU a la memoria de datos y de instrucciones de la computadora. Por lo tanto, naturalmente se arribó a la conclusión de que la alternativa más recomendable y por la cual se optó para describir el diseño de la nueva PC era la arquitectura Harvard; cuya implementación implicó que ambas secciones de memoria, al poseer cada una un sistema de buses independiente asociado a ella, podrán ser accedidas al mismo tiempo por la CPU sin inconvenientes y libremente, sin que ninguna operación a una de las secciones deba esperar a que otra operación actualmente siendo llevada a cabo en la otra sección sea completada para poder comenzar a realizarse. Así, las etapas de búsqueda y de acceso a memoria del ciclo de ejecución del cauce segmentado podrán ser ejecutadas al mismo tiempo sin inconvenientes, por lo que la ejecución del cauce será idéntica a la esperada inicialmente, representada en la figura 57.

Por su parte, se recordará que durante la etapa de acceso a memoria la instrucción puede requerir acceder a la memoria de datos o bien a un puerto de memoria asociado a alguno de los periféricos de entrada/salida de la computadora. De esta manera, puede concluirse que los accesos a ambos dispositivos de almacenamiento son mutuamente excluyentes: al poder ser realizados únicamente en la misma etapa del ciclo de ejecución, y teniendo en cuenta que en una ejecución segmentada del cauce del procesador jamás podrán ejecutarse múltiples instrucciones simultáneamente en la misma etapa del ciclo, bajo ninguna circunstancia la CPU podrá intentar acceder al mismo tiempo a la memoria de datos y a algún periférico de entrada/salida para leer o escribir datos. Por lo tanto, en el diseño de la arquitectura Harvard se ha determinado que tanto la memoria de datos como los periféricos de entrada/salida pueden compartir el mismo sistema de buses de interconexión sin ningún tipo de conflictos por el uso de dicho sistema que puedan demorar temporalmente la ejecución de las instrucciones del programa.

3.1.8. Riesgos de la segmentación y atascos en el cauce

El último concepto teórico que deberá ser presentado en esta subsección del informe (“3.1. Arquitectura”) antes de proceder a realizar algunas demostraciones con programas de prueba escritos para ser ejecutados por el nuevo procesador está vinculado a un tópico que ya fue mencionado previamente en reiteradas ocasiones, pero sin profundizar demasiado sobre el tema hasta este momento.

Si el lector ya se encuentra familiarizado con algunos conceptos básicos del área de arquitectura de computadoras, entonces conocerá que la ejecución segmentada de un programa en el procesador implicará que existirán múltiples instrucciones ejecutándose en forma simultánea en el mismo, aunque siempre en etapas diferentes. La primera conclusión que se puede extraer de esta definición es que el número de instrucciones en ejecución en cualquier instante dado se encontrará siempre limitado por la cantidad de etapas en las cuales se haya optado por dividir el ciclo de ejecución de las instrucciones. Aquí se plantea la diferencia más notoria entre segmentación y paralelismo: en este último sí pueden ejecutarse múltiples instrucciones simultáneamente en la misma etapa, por lo cual no existiría en un principio restricción alguna para el número de instrucciones en ejecución en un instante dado (obviamente, en la práctica siempre se presentará alguna limitación vinculada principalmente a los recursos de los cuales dispone la computadora para ejecutar varias instrucciones al mismo tiempo).

Por otra parte, la ejecución simultánea de múltiples instrucciones en el procesador, aunque repercutirá favorablemente sobre el rendimiento del mismo ya que el tiempo de ejecución del programa debería disminuir considerablemente, también provocará posibles conflictos entre las distintas instrucciones que se encuentren en ejecución en un instante dado por distintos motivos que pueden estar vinculados tanto a los recursos de hardware de la computadora como a la lógica de ejecución (software) del programa ejecutado. Estos conflictos, si no son atendidos y resueltos de alguna manera, probablemente desencadenen un comportamiento no deseado por parte del programa que, a su vez, provocará que sus resultados finales sean incorrectos. Por lo tanto, es fundamental aprender los tipos de conflictos o riesgos conocidos que pueden presentarse durante la ejecución segmentada del procesador ya que su detección y resolución será tan importante como lo es la implementación de la propia segmentación para que esta última funcione correctamente.

En cualquiera de los casos que se describirán a continuación, las demoras que se vean obligadas a presentar en su ejecución las instrucciones afectadas por conflictos con instrucciones anteriores del programa, ya sea a raíz de la utilización de un recurso de hardware, una dependencia de datos o el resultado de la evaluación de una condición de salto, se conocen como atascos en el cauce del procesador. Cada atasco representará un retardo de un ciclo de reloj tanto en la instrucción conflictiva como en todas las que se encuentran a continuación de ella en la ejecución del programa:

- Riesgos estructurales: están vinculados a conflictos por los recursos de hardware de la computadora. Como éstos son limitados en número y capacidad, únicamente podrán ser

accedidos por un número reducido de instrucciones al mismo tiempo. No obstante, sólo en casos muy especiales se presentarán este tipo de riesgos, ya que, al encontrarse cada etapa del ciclo de ejecución de las instrucciones asociada a un recurso distinto de la PC (ver tabla 46 del informe), para que existan conflictos estructurales debería suceder que no se cumpliera la secuencia de ejecución natural del programa, es decir, que no todas las instrucciones terminen en el orden en el cual comenzaron a ejecutarse. Ahora bien, para determinar los casos en los cuales podría no mantenerse el orden de ejecución natural del programa, se debería tomar como punto de partida la división en etapas del ciclo de ejecución optada para implementar la segmentación del cauce en este procesador en particular (ver tablas 48 y 49 del informe). Entonces, puede plantearse la situación de una instrucción actualmente en ejecución que necesita realizar una operación aritmética en punto flotante en la FPU de la CPU. A continuación de ella, el programa presenta una serie de instrucciones que no requieren llevar a cabo ningún tipo de operación aritmética en punto flotante. Por lo tanto, cabría esperar que, si no existiera inconveniente alguno, la primera instrucción, al encontrarse dividida en más etapas y, por lo tanto, requerir un mayor número de ciclos de reloj para ejecutarse que el resto de las instrucciones del programa, finalice con posterioridad a algunas de las instrucciones que no hagan ningún uso de la FPU de la CPU y estén ubicadas en sus proximidades más inmediatas. Éste sería claramente un caso en el cual no se cumple el orden natural del programa: la primera instrucción se estaría completando después de que terminen de ejecutarse una o varias instrucciones ubicadas a continuación de ella y cuyas ejecuciones comenzarían con posterioridad al inicio de la suya propia. Por lo tanto, aquí existiría un alto riesgo de que surjan conflictos entre la primera instrucción y alguna de las demás instrucciones del programa ya que en algún momento ambas podrían intentar acceder simultáneamente a alguno de los recursos de hardware de la computadora, por lo cual una de las dos debería esperar a que la otra complete su acceso y utilización del recurso en conflicto para proceder a realizar su propia operación sobre él. Después de llevar a cabo numerosas pruebas, se ha determinado que los recursos de hardware de esta computadora en particular que pueden ser objeto de conflictos de acceso simultáneo por parte de múltiples instrucciones son los siguientes:

- Unidad de acceso a memoria / Memoria de datos: éstas son respectivamente la unidad de la CPU encargada de gestionar la operación de acceso y el recurso de hardware de la computadora en conflicto. Como se recordará, estos dos dispositivos se encuentran asociados a la etapa de acceso a memoria del ciclo de ejecución de las instrucciones: si dos de estas últimas pretenden acceder a la memoria de datos

simultáneamente, entonces existirá un conflicto entre ambas: una de ellas se verá obligada a detener su ejecución hasta que la otra haya completado su acceso y utilización del recurso en conflicto (memoria de datos). En la figura 65 se presenta un ejemplo del progreso de la ejecución de un programa a través del tiempo cuando existe este tipo de atascos en el cauce del procesador.

- ❖ Unidad de almacenamiento en registro / Banco de registros de uso general: éstas son respectivamente la unidad de la CPU encargada de gestionar la operación de acceso y el recurso de hardware de la computadora en conflicto. Como se recordará, estos dos dispositivos se encuentran asociados a la etapa de almacenamiento en registro del ciclo de ejecución de las instrucciones: si dos de estas últimas pretenden acceder al banco de registros de uso general simultáneamente para realizar operaciones de escritura sobre él, entonces existirá un conflicto entre ambas: una de ellas se verá obligada a detener su ejecución hasta que la otra haya completado su acceso y utilización del recurso en conflicto (banco de registros de uso general). En la figura 66 se presenta un ejemplo del progreso de la ejecución de un programa a través del tiempo cuando existe este tipo de atascos en el cauce del procesador.

- Riesgos por dependencias de datos: otra de las consecuencias no deseadas de la implementación de la segmentación en el cauce del procesador es una posible alteración en el orden de acceso a los registros de la CPU, ya sea de uso general o particular, con el objetivo de realizar operaciones de lectura o bien escritura sobre él, respecto a la secuencia de acceso que se desarrollaría si el programa se ejecutara en forma secuencial. No se debe confundir esta situación con el orden de ejecución natural del programa: en este caso incluso si las instrucciones finalizaran en el mismo orden en el cual comenzaron a ejecutarse, podría ocurrir el riesgo de que una instrucción intentara acceder a un registro antes de que una instrucción anterior a ella haya podido llevar a cabo su respectivo acceso a él para realizar la operación que necesite. Si en ambas instrucciones la operación en cuestión se tratara de una lectura, al no encontrarse involucrada una actualización del registro, entonces indudablemente no existiría bajo ningún concepto la posibilidad de un comportamiento inesperado y/o una respuesta incorrecta del programa por inconsistencias en el estado del valor de dicho registro. Sin embargo, si alguna de las operaciones consistiera en una escritura, podrían presentarse algunos problemas debido a varias circunstancias distintas, las cuales se discutirán a continuación (tener en cuenta en todo momento la división en etapas del ciclo de ejecución de las instrucciones propuesta en las tablas 48 y 49 del informe):

- Lectura Después de Escritura (Read After Write – RAW): en este caso, el conflicto se debe a la existencia de una instrucción en el programa que está intentando acceder a un registro del procesador para leer un dato que una instrucción anterior a ella todavía no ha podido almacenar en él. Por lo tanto, el dato que estaría leyendo en el momento actual no sería el correcto ya que no coincidiría con aquél que ya debería haber sido escrito por la instrucción anterior pero aún no lo fue, lo cual a su vez provocaría un comportamiento inesperado por parte del programa ejecutado. En consecuencia, la única solución posible para este riesgo es establecer una demora en la ejecución de la instrucción afectada hasta que la instrucción anterior alcance y finalice la etapa en la cual accederá al registro conflictivo para escribir en él el dato que corresponda. Con respecto a la identificación particular de este registro, se detectaron dos situaciones particulares:
  - Registro de uso general: ésta sería la situación más común, en la cual una instrucción se encuentra en la segunda etapa de su ciclo de ejecución (decodificación) intentando acceder al banco de registros de uso general del procesador para leer un registro que todavía posee pendiente una operación de escritura por parte de una instrucción anterior. Por lo tanto, deberá demorarse la ejecución tanto de la instrucción que pretende realizar la lectura como de todas las que le siguen en el programa hasta que la instrucción anterior haya completado su escritura pendiente, la cual, como se recordará, debería llevarse a cabo en la quinta y última etapa de su ciclo de ejecución (almacenamiento en registro). En la figura 67 se presenta un ejemplo del progreso de la ejecución de un programa a través del tiempo cuando existe este tipo de atascos en el cauce del procesador.
  - Registro de uso particular: aquí básicamente se presenta el mismo fenómeno que en el caso anterior, con la peculiaridad de que el registro en conflicto no pertenece al banco de registros de uso general del procesador, sino que se trata de un registro de uso particular: el registro de banderas FPFLAGS, asociado al estado de las operaciones aritméticas en punto flotante realizadas en la FPU. Más precisamente, la bandera conflictiva sería la de Punto Flotante (bandera F), ya que éste sería un caso excepcional, pero no por ello menos importante, en el cual una instrucción de transferencia de control condicional necesitaría obtener en su segunda etapa (decodificación) el estado actual de la bandera F para evaluar una condición y determinar así si

el salto planteado debe ser tomado o no. Ahora bien, si se recuerda el repertorio de instrucciones del procesador, las únicas instrucciones que podrían tener esta función son exactamente dos en particular: “bfpt” y “bfpf”. Por su parte, todas las instrucciones del repertorio que deban realizar operaciones aritméticas en punto flotante en la FPU tendrán que actualizar el registro FPFLAGS una vez completada su respectiva operación. Esta actualización puede o no incluir a la bandera F en función de la tarea de la instrucción ejecutada. Sin embargo, como la dependencia de datos se establece sobre la totalidad del registro FPFLAGS sin importar la bandera en particular que deba ser leída, si existiera una actualización pendiente de dicho registro por cualquier instrucción anterior, sin importar su naturaleza o las banderas específicas a sobrescribir, entonces la instrucción de transferencia de control condicional tendría que detener temporalmente su ejecución hasta que la operación de actualización del registro de banderas se haya completado. Asimismo, si se tiene en cuenta la división en etapas del ciclo de ejecución propuesta para segmentar el cauce de este procesador, esta sobrescritura del registro FPFLAGS se llevaría a cabo en el cuarto ciclo de la tercera etapa (ejecución en punto flotante), por lo cual, si se lo compara con la situación anterior, en la cual el registro de uso general en conflicto se actualizaba recién en la última etapa del ciclo de ejecución de la instrucción (almacenamiento en registro), aquí el número de ciclos demorados sería inferior, siempre y cuando las instrucciones en cuestión realicen alguna operación aritmética en punto flotante en la FPU y su ciclo de ejecución requiera una cantidad de ciclos mayor para completarse que una instrucción que no involucre de ningún modo el uso de la FPU. En las figuras 68 y 69 se presentan ejemplos del progreso de la ejecución de dos programas distintos a través del tiempo cuando existe este tipo de atascos en el cauce del procesador. Aunque en ambos casos tanto la naturaleza de la dependencia de datos de tipo RAW como la de su posterior resolución son exactamente idénticas, existe una diferencia con respecto al desenlace de los programas una vez reanudada la ejecución de las instrucciones demoradas. Las causas de esta divergencia serán explicadas exhaustivamente en las observaciones de ambas figuras.

- Escritura Después de Lectura (Write After Read – WAR): este tipo de conflicto se presentaría únicamente en caso de que una instrucción del programa pretendiera actualizar uno de los registros de uso general del procesador que aún mantuviera una operación de lectura pendiente por parte de una instrucción anterior. Si existiera esta situación, entonces dicha instrucción anterior al realizar su lectura no estaría obteniendo el dato esperado debido a que el mismo ya habría sido sobrescrito previa e incorrectamente por la instrucción posterior, desencadenando así una respuesta inadecuada por parte del programa en ejecución. Así como para los riesgos ya descriptos anteriormente, la solución más adecuada para resolver este conflicto sería una demora en la ejecución de la instrucción que pretende almacenar un nuevo valor en el registro conflictivo hasta que la instrucción anterior haya logrado completar su operación de lectura sobre él. De este modo, todas las instrucciones realizan sus accesos a los registros respetando su posición dentro del programa, tal y como lo harían si éste se ejecutara de manera secuencial, garantizándose así la consistencia del estado de los datos de los registros en cualquier instante de la ejecución del programa. Por otra parte, puede determinarse fácilmente a partir del estudio de la división en etapas del ciclo de ejecución propuesta para segmentar el cauce de este procesador que las etapas específicas en las cuales se presentaría el conflicto serían la quinta y última (almacenamiento en registro) de la instrucción que pretendería realizar la escritura y la segunda (decodificación) de la instrucción anterior que necesitaría llevar a cabo la lectura. Por lo tanto, al encontrarse ambas etapas tan alejadas en el tiempo (la segunda etapa del ciclo de ejecución de una instrucción se llevará a cabo muy anteriormente a la realización de la quinta y última etapa del ciclo de una instrucción posterior en el programa), podrá arribarse a la conclusión de que esta clase de riesgos es la más inusual de todas, incluyendo todos los tipos conocidos y aquí presentados (estructurales, datos, control y fin), ya que únicamente podrían existir conflictos de este tipo en caso de que se alterara el orden de ejecución de las instrucciones en el programa. No se está haciendo referencia aquí al orden de ejecución natural del programa, sino a un reordenamiento y reubicación de las instrucciones llevados a cabo deliberadamente por el ensamblador. Un posible objetivo buscado con este reposicionamiento podría ser mejorar el rendimiento del procesador a través de la implementación de una de las técnicas más conocidas para resolver riesgos de control, conocida como salto retardado. En ella, esencialmente, el ensamblador comienza seleccionando algunas instrucciones del programa que no

posean relación o dependencia alguna con la condición de salto evaluada en la instrucción de transferencia de control que provoca el riesgo de control a resolver. Luego, en función del tipo de instrucción, es decir, si el salto es condicional o incondicional, el ensamblador determina la cantidad total de instrucciones que podrían incorporarse inmediatamente a continuación de la instrucción de salto conflictiva de manera tal de aprovechar óptimamente el tiempo de ejecución del programa, procurando que no permanezca ningún ciclo de reloj durante el cual el procesador se encuentre ocioso entre la detección e identificación de la instrucción de transferencia de control en la segunda etapa de su ciclo de ejecución (decodificación) y la ejecución de la siguiente instrucción dentro del flujo del programa, la cual obviamente variará en función del cumplimiento o incumplimiento de la condición de salto evaluada. A partir de esta determinación del ensamblador se generan los huecos de retardo o delay slots, es decir, los espacios disponibles en el cauce del procesador entre la instrucción de salto conflictiva y la instrucción a ejecutar a continuación de ella para que el ensamblador ubique en ellos cualquiera de las instrucciones susceptibles de ser reordenadas que ya habían sido previamente seleccionadas. Cada una de estas instrucciones será introducida en un hueco distinto hasta completar, en caso de ser posible, todos los huecos de retardo disponibles, de manera tal que el procesador se mantenga la mayor cantidad de tiempo posible realizando trabajo útil y sin comprometer en ningún momento la correcta ejecución del programa ya que, como se recordará, para que estas instrucciones fueran elegidas por el ensamblador para su reordenamiento debían ser independientes de la condición evaluada. No obstante, al no poseer este procesador soporte para esta técnica de solución de riesgos de control, a pesar de las numerosas pruebas realizadas no logró detectarse en ninguna situación la presencia de un conflicto de tipo WAR, por lo que se decidió directamente omitir la implementación de este último para esta versión del proyecto. Aun así, en la figura 70 se presenta un ejemplo del progreso de la ejecución de un programa a través del tiempo sin aplicar la técnica de salto retardado y, en la figura 71, otro ejemplo del progreso de la ejecución del mismo programa a través del tiempo pero ahora con sus instrucciones efectivamente reordenadas por el ensamblador de manera tal de evitar la presencia de huecos de retardo vacíos, inutilizados y desaprovechados a continuación de las instrucciones de transferencia de control, pero con la consecuencia del surgimiento de atascos de tipo WAR en el cauce del procesador.

- Escritura Después de Escritura (Write After Write – WAW): la situación representada en este tipo de riesgo involucra, de una forma similar a los casos previos, dos instrucciones actualmente en ejecución en un programa: una instrucción posterior cuya intención es realizar una actualización del valor de un registro de uso general del procesador, y una anterior que todavía mantiene pendiente una solicitud de escritura sobre el mismo registro. Por lo tanto, si la instrucción posterior llevara a cabo la actualización que en el presente momento está pretendiendo efectuar, entonces en algún instante posterior la instrucción anterior concretaría eventualmente su operación de escritura pendiente, por lo cual el valor final del registro durante el resto de la ejecución del programa sería el correspondiente a esta última operación. Esta situación tendrá como consecuencia un estado incorrecto en el registro involucrado ya que, en una ejecución secuencial del programa, el valor actual de un registro siempre se encontrará determinado única y exclusivamente por la última instrucción que, siguiendo el orden de ejecución natural (todas las instrucciones finalizan en el mismo orden en el cual comenzaron a ejecutarse), haya realizado una actualización sobre él. En esta última sentencia se encuentra la clave de la cuestión: como se recordará, en la ejecución segmentada de un programa, existe la posibilidad de que no todos los ciclos de ejecución de las instrucciones coincidan entre sí, ya que algunos, más largos y lentos y divididos en un mayor número de etapas, requieren realizar operaciones aritméticas en punto flotante en la FPU del procesador y otros, más cortos y rápidos, no precisan de ninguna manera utilizar la FPU para sus operaciones. De esta manera, un programa segmentado podría no seguir su orden de ejecución natural. Supóngase la situación de un programa actualmente en ejecución que en un determinado momento presenta una serie de dos instrucciones: una de ellas con un ciclo más largo de ejecución, que requiere realizar una operación aritmética en punto flotante en la FPU, e inmediatamente a continuación de ella otra instrucción con un ciclo corto de ejecución, que no necesita utilizar la FPU para llevar a cabo ninguna tarea. Entonces, asumiendo que no se encuentra involucrado ningún otro tipo de riesgo de segmentación, la primera instrucción, a pesar de haber comenzado su ejecución antes, finalizará después de haberse completado la ejecución de la segunda instrucción debido a la diferencia en el número de etapas en ambos ciclos y, en consecuencia, en el tiempo de ejecución total de ambas instrucciones. Ésta sería una clásica situación en la cual podría ocurrir este conflicto: si ambas instrucciones

pretendieran realizar una operación de escritura sobre el mismo registro de uso general del procesador, entonces, si no se llevara a cabo ninguna acción preventiva, la segunda instrucción actualizaría el registro antes que la primera, provocando el error en el estado del registro ya mencionado anteriormente. Por lo tanto, para impedir este riesgo, esta segunda instrucción, aunque ya se encuentre lista para realizar su escritura, se verá obligada a detener temporalmente su ejecución y la del todo el cauce del procesador hasta que la primera instrucción se haya completado. Esto se debe a que, como se recordará, es recién en la última etapa del ciclo de ejecución (almacenamiento en registro) cuando las instrucciones llevan a cabo, si correspondiera, la actualización del registro de uso general del procesador que deban realizar. Finalmente, podrá observarse que éste es el único riesgo por dependencia de datos en el cual las dos instrucciones en conflicto presentan sus problemas en la misma etapa (almacenamiento en registro), ya que únicamente aquí ambas estarían intentando realizar la misma operación sobre el registro conflictivo: una escritura de datos. En las figuras 72 y 73 se presentan ejemplos del progreso de la ejecución de dos programas distintos a través del tiempo cuando existe este tipo de atascos en el cauce del procesador. Se ha optado por incluir dos casos diferentes ya que a partir de ellos se comprenderán algunas decisiones que el autor de este informe ha tomado respecto a la duración del retraso presentado por las instrucciones afectadas por esta dependencia de datos, ya que no todas se encontrarán demoradas durante el mismo intervalo de tiempo. Ésta es una característica exclusiva de esta clase de riesgos que lo diferencia de cualquiera de los demás atascos asociados a la implementación de la segmentación en el cauce de la CPU mencionados en esta sección. Por ello, se ha desarrollado una explicación exhaustiva y pormenorizada con respecto a este fenómeno que podrá encontrarse en las observaciones de ambas figuras.

- Riesgos de control: como sin duda se sabrá, al ejecutarse una instrucción de transferencia de control, el resultado de la evaluación de la condición de salto determinará si se debe modificar el flujo de ejecución natural del programa, es decir, si será necesario realizar alguna modificación adicional en el registro de uso particular del procesador conocido como Puntero de Instrucción (IP) para que, en lugar de apuntar a la instrucción cuya ubicación física se encuentra inmediatamente después de la instrucción de transferencia de control, contenga la dirección de salto a partir de la cual se almacena la nueva instrucción que deberá ejecutarse a continuación de la instrucción de salto en reemplazo de aquélla que estaba siendo previamente apuntada por el IP. Bajo ningún concepto el salto se encuentra obligado

a llevarse a cabo hacia adelante, es decir, la nueva instrucción a ejecutar puede localizarse antes o después de la instrucción de transferencia de control que originó el salto. Ahora bien, si no se cumpliera la condición evaluada en el salto, entonces no existiría ningún inconveniente ya que se estaría preservando el flujo de ejecución natural del programa, por lo que el procesador procedería con la ejecución de la instrucción ubicada a continuación de la instrucción de salto como en cualquier otra situación normal. Sin embargo, si la condición de salto llegara a cumplirse, entonces sería incorrecto continuar con la ejecución de la instrucción localizada a continuación de la instrucción de transferencia de control ya que ésta no pertenece más al flujo de ejecución del programa, encontrándose sustituida por la nueva instrucción contenida a partir de la dirección de salto que deberá ser actualizada en el IP para que el procesador pueda ejecutarla. Por su parte, al ejecutarse ahora el programa en forma segmentada en lugar de secuencial, se posee la certeza de que, en caso de alterarse el flujo de ejecución del programa debido al cumplimiento de la condición de salto planteada en una instrucción de transferencia de control, no se dispondrá del tiempo suficiente para evitar que la instrucción originalmente establecida para ejecutarse a continuación de la instrucción de salto comience a ser ejecutada antes de haber logrado determinar la nueva instrucción a ejecutar en su reemplazo de acuerdo con el flamante flujo de ejecución definido para el programa. En función del tipo de instrucción de control involucrado en esta situación, se pueden distinguir dos casos conocidos:

- Salto incondicional: las instrucciones que presentan este tipo de salto básicamente no poseen condición de salto alguna o, en otras palabras, tienen una condición de salto que siempre se cumple. Por lo tanto, aquí en todas las ocasiones que se ejecuten esta clase de instrucciones se alterará el flujo de ejecución del programa para que la próxima instrucción a ejecutar a continuación del salto sea aquélla almacenada a partir de la dirección destino de dicho salto. Al no requerirse la intervención de la ALU del procesador para realizar alguna operación aritmética a fin de evaluar la condición de salto de la instrucción, el cambio en el flujo de ejecución del programa puede ser llevado a cabo directamente por la unidad de control de la CPU sin ningún inconveniente. Esta acción se llevaría a cabo en la segunda etapa del ciclo de ejecución de la instrucción de transferencia de control (decodificación), seguidamente de que la unidad de control haya logrado decodificar tanto la identificación de dicha instrucción como la naturaleza incondicional de su salto asociado. Por lo tanto, la instrucción ubicada inmediatamente a continuación de la instrucción de transferencia de control incondicional, que originalmente formaba

parte del flujo de ejecución natural del programa y cuya ejecución ya había comenzado a efectuarse sería prontamente cancelada, procediendo en su lugar a iniciar la ejecución de la nueva instrucción apuntada por el IP como consecuencia del salto acontecido. Cabe aclarar que en este escenario no se provocarían atascos propiamente dichos en el cauce del procesador ya que simplemente se estaría cancelando prematuramente la ejecución de una instrucción con el consecuente desaprovechamiento de los ciclos de reloj durante los cuales dicha instrucción llevaría a cabo las etapas canceladas de su ciclo de ejecución. En otras palabras, bajo ningún concepto esta cancelación debería desencadenar demora alguna en las instrucciones ejecutadas inmediatamente a continuación de la instrucción de transferencia de control y la instrucción cancelada en el flujo del programa. No obstante, aunque los ciclos de reloj desaprovechados no sean utilizados por dicha instrucción cancelada, sí son empleados por el resto de las instrucciones del programa tanto anteriores como posteriores a ella, incluyendo la misma instrucción de transferencia de control, para completar todas las etapas de sus ciclos de ejecución. En consecuencia, asumiendo siempre que el programa está siendo ejecutado en forma segmentada, al permitirse en este escenario la coexistencia de múltiples instrucciones ejecutándose simultáneamente, el tiempo de ejecución total del programa sólo se vería incrementado en un ciclo de reloj. Este retraso añadido se estaría debiendo, obviamente, a la cancelación temprana de la instrucción inmediatamente posterior a la instrucción de transferencia de control a causa de la alteración en el flujo de ejecución del programa. Es decir, se iniciaría la ejecución de una instrucción adicional innecesariamente, la cual al fin y al cabo será interrumpida precozmente y prontamente dejará de ser considerada por completo para el resto de la ejecución del programa. Como se recordará, en cualquier ejecución segmentada de un programa cada nueva instrucción ejecutada, a excepción de la primera de todas, incrementa el tiempo de ejecución total del programa en un ciclo de reloj, siempre y cuando su ciclo de ejecución sea idéntico al de su instrucción previamente ejecutada (ver figuras 54 y 55). Entonces, si de alguna manera se hubiera previsto la alteración en el flujo de ejecución del programa, la instrucción cancelada jamás habría comenzado a ejecutarse, evitándose así el innecesario ciclo de reloj adicional que su ejecución frustrada e incompleta termina inevitablemente agregando al tiempo de ejecución total del programa. En la

figura 74 se presenta un ejemplo del progreso de la ejecución de un programa a través del tiempo cuando existe este tipo de riesgos en el cauce del procesador.

- Salto condicional: aquí la situación es ligeramente más complicada que en el caso anterior ya que ahora sí existe una condición de salto asociada a la instrucción de transferencia de control que deberá ser evaluada a fin de determinar si efectivamente debe alterarse el flujo de ejecución natural del programa. En consecuencia, si se compara este escenario con el previo, inevitablemente tendrá que demorarse la resolución de la evaluación, ya que la unidad de control no posee las prestaciones requeridas para llevar a cabo la operación aritmética que sí es capaz de realizar la ALU y que resultará imprescindible para determinar el cumplimiento o incumplimiento de la condición de salto. En consecuencia, esta evaluación, como ya se habrá adivinado, será llevada a cabo en la tercera etapa del ciclo de ejecución de la instrucción de transferencia de control (ejecución), en lugar de la segunda (decodificación) como ocurría en el caso de los saltos incondicionales. La primera conclusión que puede obtenerse de este retraso adicional indeseado es que, en cualquier caso, tanto si se modifica el flujo de ejecución del programa como si se mantiene inalterable, la instrucción ubicada a continuación de la instrucción de transferencia de control, que forma parte del flujo original y ya había comenzado con su ejecución antes de que la unidad de control lograra identificar la instrucción de salto, deberá demorar su ejecución en su propia etapa de decodificación hasta que la ALU del procesador haya realizado la operación aritmética necesaria para evaluar la condición de salto y finalizado la evaluación efectiva de dicha condición. Por último, si la ALU determina que la condición no se cumple, entonces la instrucción que se encontraba demorada reanudará su ejecución sin ningún otro tipo de inconvenientes. No obstante, en caso contrario, la ejecución de la instrucción temporalmente detenida deberá ser cancelada definitivamente, procediendo en cambio a comenzar a ejecutar la nueva instrucción que ocupará su lugar, ubicada a partir de la dirección de salto indicada en la instrucción de transferencia de control que está modificando el flujo de ejecución natural del programa. En las figuras 75 y 76 se exhiben respectivamente un ejemplo del progreso de la ejecución de un programa a través del tiempo para cada una de las dos alternativas previamente explicadas que pueden presentarse cuando existe este tipo de atascos en el cauce del procesador.

- Riesgos de fin del programa: a través de la ejecución de los distintos programas de prueba diseñados para comprobar el funcionamiento de la computadora se detectó una situación extremadamente particular y anómala que no había sido tenida en cuenta anteriormente ni tampoco pudo ser encuadrada dentro de ninguno de los riesgos previamente explicados. El escenario planteado a modo de ejemplo es el siguiente: se dispone de un programa que actualmente se encuentra en las instancias finales de su ejecución. El procesador comienza a ejecutar sus últimas dos instrucciones, las cuales consisten en una que realiza una operación aritmética en punto flotante en la FPU de la CPU, y otra que simplemente se trata de la instrucción "halt", encargada de detener definitivamente la ejecución del procesador, finalizando el programa ejecutado. Ahora bien, como se recordará, las instrucciones que deben llevar a cabo operaciones aritméticas en punto flotante en la FPU poseen un ciclo de ejecución más largo y lento que aquellas encargadas de realizar cualquier otra tarea distinta (ver tablas 48 y 49). Por su parte, la orden para detener la ejecución de la CPU es emitida por la unidad de control del procesador durante la segunda etapa del ciclo de ejecución (decodificación) de la instrucción "halt", la cual a su vez presentará un ciclo de ejecución corto (ver tabla 56), como el de cualquier instrucción cuya ejecución no involucra la utilización de la FPU. Esto significa que, en condiciones normales, si todas las instrucciones del repertorio soportado por el procesador compartieran exactamente la misma estructura para su ciclo de ejecución, sólo quedarían pendientes por ejecutarse una etapa de ejecución de un ciclo único (la etapa de ejecución de la propia instrucción "halt"), dos etapas de acceso a memoria y tres etapas de almacenamiento en registro (ver figura 77). Precisamente el procesador de este proyecto fue diseñado para, una vez decodificada e identificada la instrucción "halt", detener su ejecución a partir del escenario descripto en la sentencia anterior, desactivando una a una las unidades de la CPU encargadas de ejecutar las distintas etapas del ciclo de ejecución de las instrucciones (ver tabla 46) conforme completen todas sus respectivas etapas pendientes. Sin embargo, la presencia de una instrucción que realice una operación aritmética en punto flotante en la FPU, al ser su ciclo de ejecución más largo que el del resto de las instrucciones del repertorio del procesador, representaría una anomalía en este escenario ya que en él sólo se había considerado un único tipo de ciclo de ejecución: aquél conformado por cinco etapas de un ciclo de reloj cada una, asociado a las instrucciones que no requieren utilizar la FPU para efectuar sus operaciones. Esta alteración en el contexto de ejecución podría impedir la correcta finalización del programa ya que una instrucción que incluya una operación en punto flotante no sólo tardará más tiempo en completar su ejecución que el resto de las instrucciones, sino que además podría provocar

cualquiera de los otros tipos de riesgos de la segmentación del cauce ya mencionados que demorarían la ejecución de las instrucciones ubicadas a continuación de ella en el programa. De cualquier manera, el efecto indeseado de esta variación inesperada del contexto sería que la unidad de control, al no haber sido inicialmente diseñada para detectar este cambio en las condiciones de ejecución, al identificar que actualmente la instrucción "halt" se encuentra en ejecución, emita prematuramente la orden de detención para el procesador, lo cual a su vez tendría como consecuencia la temprana e incorrecta finalización de la instrucción que involucra el uso de la FPU y/o de las instrucciones que se encuentran a continuación de ella en el programa y también puedan verse afectadas por posibles riesgos durante su ejecución. Esto se debe a que, al requerir un mayor número de ciclos de reloj para completarse que el anticipado, la CPU se estaría deteniendo antes de que las instrucciones alcancen a ejecutar sus últimas etapas (ver figura 78). La única solución posible para evitar este riesgo consiste en demorar la decodificación de la instrucción "halt" hasta que a lo sumo sólo una instrucción del programa posea pendiente de ejecución un único ciclo en su etapa de ejecución (tercera etapa del ciclo de ejecución de las instrucciones). Este ciclo puede incluir dos alternativas: el último ciclo de la etapa de ejecución de una instrucción que necesita realizar una operación aritmética en punto flotante en la FPU, o bien una etapa de ejecución de ciclo único de cualquiera de las demás instrucciones que conforman el repertorio del procesador. De este modo, lograrían garantizarse condiciones de ejecución similares al escenario planteado inicialmente como "normal", en el cual la única etapa de ejecución pendiente una vez transmitida la orden de detención sería aquélla asociada a la propia instrucción "halt". Así podrá asegurarse sin ningún tipo de duda que la orden de alto de la CPU no será emitida prematuramente por la unidad de control y que el programa no finalizará su ejecución hasta que absolutamente todas sus instrucciones se hayan completado hasta sus últimas etapas. Finalmente, aunque resulte redundante porque ya se hizo referencia a esto previamente, cabe aclarar que la etapa de la instrucción "halt" directamente afectada y demorada por la presencia de este riesgo es la segunda de su ciclo de ejecución, es decir, la etapa de decodificación, ya que sería precisamente aquí el momento en el cual la unidad de control identificaría la naturaleza de esta instrucción e intentaría emitir la orden de detención a todas las unidades del procesador. En las figuras 79 y 80 se presentan ejemplos del progreso de la ejecución de dos programas distintos a través del tiempo cuando existe este tipo de atascos en el cauce del procesador.

En la siguiente tabla se ofrece una breve síntesis sobre la naturaleza de los distintos tipos de riesgos representados en cada una de las próximas figuras, dedicándose un espacio para identificar y destacar especialmente las etapas de los ciclos de ejecución de las instrucciones particular y directamente involucradas y/o afectadas en cada escenario según corresponda. Se notará que fueron delineadas y dispuestas una e incluso varias figuras en algunos casos para analizar y explicar cada riesgo presentado en la introducción previa, incluso aquéllos que, a pesar de haber sido considerados durante el diseño del nuevo procesador, por diversos motivos no se encuentran implementados en la versión final de este último, o al menos la que se dispone hasta el momento.

Tabla 63. Tipos de riesgos conocidos para el cauce segmentado del nuevo procesador representados en las próximas figuras. Etapas de los ciclos de ejecución de las instrucciones directamente involucradas y/o afectadas en cada caso.

<table>
<tr><th rowspan="2">Figura</th><th colspan="2">Riesgo</th><th colspan="2">Etapa</th></tr>
<tr><th>Tipo</th><th>Modalidad</th><th>Tipo A (**)</th><th>Tipo B (***)</th></tr>
<tr><td>65</td><td rowspan="2">Estructural</td><td>Memoria de datos</td><td>4. Acceso a memoria</td><td>4. Acceso a memoria</td></tr>
<tr><td>66</td><td>Banco de registros de uso general</td><td>5. Almacenamiento en registro</td><td>5. Almacenamiento en registro</td></tr>
<tr><td>67</td><td rowspan="5">Dependencia de datos</td><td rowspan="3">Lectura Después de Escritura (RAW)</td><td>5. Almacenamiento en registro</td><td rowspan="3">2. Decodificación</td></tr>
<tr><td>68</td><td rowspan="2">3.2.4. Ejecución en punto flotante (Etapa 4)</td></tr>
<tr><td>69</td></tr>
<tr><td>70</td><td rowspan="2">Escritura Después de Lectura (WAR) (*)</td><td rowspan="2">2. Decodificación</td><td rowspan="2">5. Almacenamiento en registro</td></tr>
<tr><td>71</td></tr>
<tr><td>72</td><td rowspan="2">Escritura Después de Escritura (WAW)</td><td rowspan="2">5. Almacenamiento en registro</td><td rowspan="2">5. Almacenamiento en registro</td></tr>
<tr><td>73</td></tr>
<tr><td>74</td><td rowspan="3">Control</td><td>Incondicional (****)</td><td>2. Decodificación</td><td>2. Decodificación</td></tr>
<tr><td>75</td><td rowspan="2">Condicional</td><td rowspan="2">3.1. Ejecución</td><td rowspan="2">2. Decodificación</td></tr>
<tr><td>76</td></tr>
<tr><td>77</td><td rowspan="2">Fin del programa</td><td rowspan="2">Instrucción “halt”</td><td rowspan="2"></td><td rowspan="2">2. Decodificación</td></tr>
<tr><td>78</td></tr>
</table>

79			3.2.3. Ejecución en punto flotante (Etapa 3)	
80 (*****)			3.2.4. Ejecución en punto flotante (Etapa 4)	
			3.1. Ejecución	

Observaciones:

- Para comprender mejor los siguientes comentarios se recomienda haber observado previamente cada una de las figuras indicadas en esta tabla y leído detenidamente sus respectivas observaciones.
- (*): Esta modalidad de riesgo en particular no posee actualmente implementación alguna en el nuevo procesador ya que no se ha logrado determinar aún ningún escenario de ejecución que presentara un atasco en el cauce de la CPU debido a la existencia de una dependencia de datos de tipo WAR y que justificara la laboriosa tarea de incluir un soporte adicional en el procesador para detectar y gestionar esta clase de situaciones.
- (**): Considérese aquí la existencia de un escenario en el cual la ejecución segmentada del programa se encuentre afectada y comprometida por la presencia de cualquiera de los atascos indicados en esta tabla. Entonces, se clasificaría en ella como “tipo A” a aquella etapa del ciclo de ejecución de una de las instrucciones que están ejecutándose actualmente pero todavía no se han completado, tratándose precisamente la ejecución pendiente de esta etapa en particular de la responsable directa del atasco en cuestión surgido en el cauce del procesador ya que durante su ejecución se alcanzaría la condición necesaria para que la/s instrucción/es atascada/s pueda/n reanudarse. Por ejemplo, en el caso de la figura 65, la clasificación “tipo A” sería perfectamente aplicable para la etapa de acceso a memoria de la instrucción I_2 de aquel programa debido a que exactamente en esta etapa dicha instrucción se encuentra utilizando el sistema de buses conectado a la memoria de datos, impidiendo así que la instrucción I_5 pueda llevar a cabo su propio acceso a dicha memoria, provocándose así un atasco que afecta tanto a I_5 como a todas sus instrucciones posteriores actualmente en ejecución.
- (***): Continuando con la misma situación considerada y presentada en la observación anterior, se clasificaría en ella como “tipo B” a aquella etapa del ciclo de ejecución de una de las instrucciones que se encuentra atascada, es decir, demorada a la espera de que se

completa la ejecución pendiente de la etapa “tipo A” de manera tal que logre cumplirse la condición necesaria para que pueda reanudarse. Tal vez el lector recuerde que durante un atasco pueden encontrarse múltiples instrucciones demoradas en distintas etapas de sus respectivos ciclos de ejecución: en estos casos únicamente se clasificaría como “tipo B” a la etapa que corresponda al ciclo de ejecución de la instrucción que se encuentre ubicada en la primera posición de todas las instrucciones atascadas en el programa, ya que sería ésta la que en realidad fue inicialmente suspendida por su conflicto directo presentado con la etapa “tipo A”, mientras que la detención temporal del resto de las etapas aplazadas se provoca simplemente como consecuencia indirecta del atasco, más precisamente para evitar que se desordene la ejecución del programa y sucedan repercusiones indeseadas como, verbigracia, el surgimiento de nuevos atascos en el cauce del procesador. Por ejemplo, en el caso de la figura 65, la clasificación “tipo B” sería perfectamente aplicable para la etapa de acceso a memoria de la instrucción I_5 de aquel programa debido a que exactamente esta etapa se encuentra directamente en conflicto con la misma etapa del ciclo de ejecución de la instrucción I_2 ya que ambas etapas estarían intentando utilizar el sistema de buses conectado a la memoria de datos de la computadora para acceder a ella. Así, I_5, al ubicarse en un lugar posterior a I_2 en el programa, deberá demorarse en su etapa de acceso a memoria durante un ciclo de reloj hasta que I_2 haya completado su respectivo acceso.

- (****): Si se revisan las observaciones de la figura 74, las mismas mencionan que, al ejecutarse una instrucción de transferencia de control incondicional, en realidad no se provoca atasco alguno en el cauce del procesador, sino que simplemente se cancela la ejecución de la instrucción inmediatamente posterior a la instrucción de salto. Esta instrucción ya había comenzado a ejecutarse hasta un punto tal de haber completado su etapa de búsqueda una vez decodificada la instrucción de transferencia de control, por lo cual deberá interrumpirse prematura y definitivamente el resto de su ciclo de su ejecución luego de que la unidad de control detecte y determine la necesidad indefectible de alterar el flujo de ejecución del programa a causa del salto incondicional. Entonces, en este contexto, se clasificaría como “tipo A” a la etapa de decodificación de la instrucción de transferencia de control incondicional ya que sería ésta la responsable directa de la cancelación de la ejecución actualmente en curso de su instrucción inmediatamente posterior previo al salto y de su reemplazo en el flujo del programa por la instrucción destino de dicho salto. Por su parte, la etapa “tipo B” sería aquella correspondiente a la decodificación de la instrucción interrumpida prematuramente, debido a que a partir de esta etapa comenzaría la cancelación del ciclo de ejecución de dicha instrucción, una vez que se ha logrado determinar que ya no

formará parte del flujo de ejecución del programa. Planteando un caso más concreto, en el programa de la misma figura 74, por ejemplo, a la etapa de decodificación de la instrucción de transferencia de control incondicional I_4 le correspondería la clasificación “tipo A”, y a la etapa de decodificación de la instrucción inmediatamente posterior a ella, I_5, la clasificación “tipo B”.

- (*****): Ésta es la única figura cuyo caso representado cuenta con dos etapas distintas susceptibles de ser clasificadas como “tipo A”. Tal como se describe en sus observaciones pertinentes, esta situación se caracteriza especialmente por la presencia de dos instrucciones del programa ejecutando la última subetapa de su ejecución en punto flotante y su etapa de ejecución respectivamente al mismo tiempo que es llevada a cabo la comprobación inicial por la unidad de control en la etapa de decodificación de la instrucción “halt”. Al determinarse durante esta comprobación el incumplimiento por parte del escenario de ejecución de las condiciones mínimas necesarias para garantizar la emisión segura de la orden de detención para todas las unidades de ejecución del procesador sin comprometer la correcta finalización de las instrucciones del programa, se detiene temporalmente la instrucción “halt” en su etapa de decodificación, posponiéndose la transmisión de la orden de alto hasta que se completen las etapas pendientes de los ciclos de ejecución de las instrucciones anteriores a la instrucción “halt” que sean requeridas para que se alcancen las condiciones que permitan reanudar la decodificación atascada y proceder sin peligro alguno con la transmisión previamente frustrada de la orden de detención. Entonces, a partir de los preceptos anteriores, observando detenidamente la figura 80, se concluye que, para este caso en particular, podrían clasificarse como “tipo A” tanto la última subetapa de la ejecución en punto flotante de la instrucción I_4 como la etapa de ejecución de la instrucción I_7. Aun así, por otra parte también se presentaría la posibilidad de argumentar que a la etapa de ejecución de I_7 no necesariamente le resultaría apropiado este tipo de clasificación ya que, en términos estrictos, si se revisa el estado de la ejecución del programa en caso de transmitirse la orden de detención durante la primera vez que intenta ejecutarse de manera fallida la decodificación de la instrucción “halt”, se establecería que, en realidad, la única responsable del incumplimiento de las condiciones mínimas de ejecución es la instrucción I_4 debido a que, al ser su ciclo de ejecución más lento, complejo y prolongado que el de las demás instrucciones del programa, está incorporando al cauce del procesador etapas adicionales pendientes por atender que no existirían si su ciclo de ejecución no tuviera conexión alguna con la ejecución de operaciones aritméticas en punto flotante en la FPU. Sin embargo, la figura 79 demuestra que existen entornos de ejecución en los cuales la

instrucción “halt” puede proceder a transmitir la orden de detención sin inconvenientes encontrándose otra instrucción del programa anterior a ella ejecutando simultáneamente el último ciclo de su etapa de ejecución en punto flotante. Por lo tanto, aquí yace la justificación para la determinación de clasificar a las dos etapas ya mencionadas de la figura 80 como “tipo A”: a pesar de que en realidad serían la instrucción I_4 y, en particular, la última subetapa de su ejecución en punto flotante, las responsables directas del atasco surgido en el cauce del procesador en la decodificación de la instrucción “halt”, sin la presencia adicional de la etapa de ejecución de la instrucción I_7 completándose al mismo tiempo que ella habría sido imposible que en este escenario se incumplieran las condiciones mínimas de ejecución y la decodificación de “halt” se viera forzada a detener temporalmente su ejecución. Esta definición, naturalmente, puede extrapolarse a cualquier otro caso en el cual se encuentren ejecutando simultáneamente la última subetapa de la ejecución en punto flotante de una instrucción y la etapa de ejecución de otra al mismo tiempo que la unidad de control intenta realizar la comprobación del estado de ejecución del programa durante la decodificación de la instrucción “halt”. Finalmente, por si hace falta aclararlo, en la situación asociada al atasco especialmente representado y analizado en esta figura 80 es la propia etapa de decodificación de la instrucción “halt” a la cual le correspondería la clasificación “tipo B” ya que tanto esta etapa como esta instrucción se tratan efectivamente de las únicas del programa en este contexto de ejecución que se demorarían temporal e indeseadamente como consecuencia del fracaso de la comprobación llevada a cabo por la unidad de control en esta misma etapa para poder transmitir la orden de detención: ninguna de las demás instrucciones del programa debería sufrir retrasos en su ejecución normal a causa de este atasco presente en el cauce del procesador.

Antes de proceder a representar en las siguientes figuras los distintos tipos de atascos posibles que pueden presentarse en el cauce del nuevo procesador durante la ejecución de las instrucciones de un programa, cabe mencionar una última aclaración: aquí únicamente se está presentando para cada caso una situación concreta y particular en la cual se provocará el atasco indicado en la figura. Sin embargo, dicha situación no es única, ya que podrían llegar a existir innumerables escenarios de ejecución distintos que no mantengan ninguna relación con aquéllos representados en las figuras, pero de todas maneras exhiban sus mismas clases de atascos. Sólo se pretende colaborar con el lector para facilitarle su comprensión de la naturaleza de cada atasco a través de representaciones gráficas de casos específicos, pero todavía a un

nivel conceptual: recién en la subsección 3.1.8.7., llamada “Programas de prueba”, se presentarán entornos de prueba más orientados a cuestiones prácticas que involucrarán la ejecución de diversos ejemplos de programas escritos en el lenguaje Assembler.

Tabla 64. Colores utilizados para representar cada una de las etapas del ciclo de ejecución de las instrucciones de los programas de prueba analizados en las próximas figuras.

Etapa	Color
1. Búsqueda	Rojo
2. Decodificación	Violeta
3.1. Ejecución	Verde
3.2. Ejecución en punto flotante	Marrón
4. Acceso a memoria	Azul
5. Almacenamiento en registro	Rosa

3.1.8.1. Dependencias estructurales

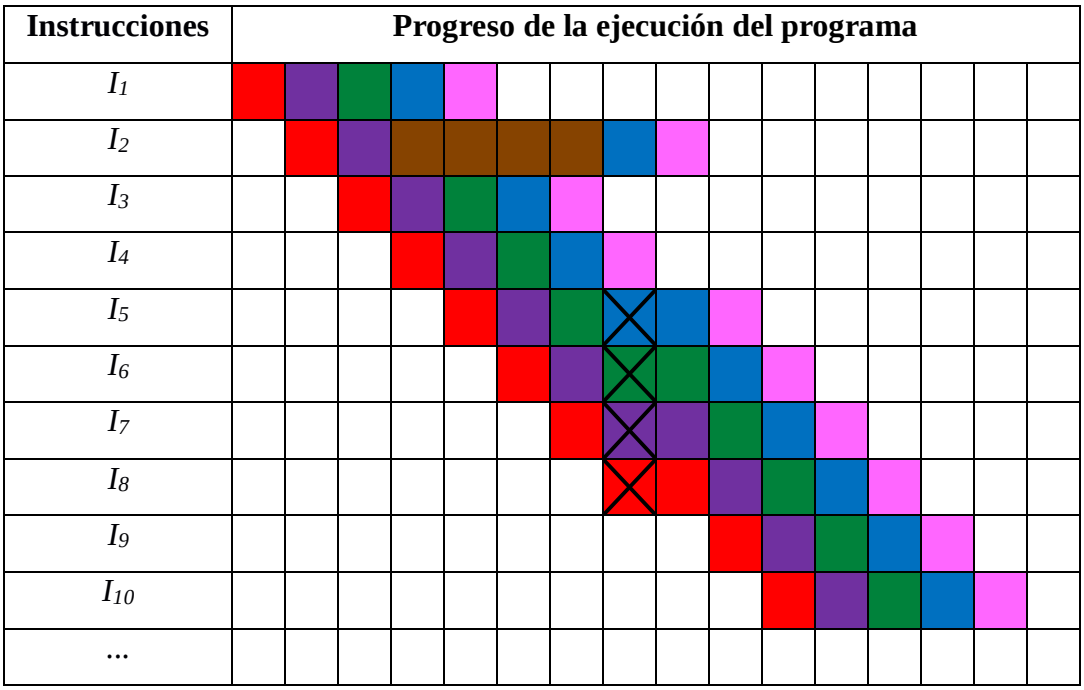


Figura 65. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco de tipo estructural en el cauce del nuevo procesador debido a dos intentos de acceso simultáneos a la memoria de datos de la computadora.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ Tanto la instrucción I_2 como la I_5 están intentando ejecutar su respectiva etapa de acceso a memoria del ciclo de ejecución simultáneamente.
  - ❖ Esta situación no se encuentra permitida por la arquitectura Harvard diseñada para la nueva computadora ya que ésta, como se recordará, solamente posee un único sistema de buses para acceder a la sección de datos de la memoria principal a fin de realizar operaciones de lectura o bien de escritura sobre ella.
  - ❖ Por lo tanto, en este escenario jamás la memoria de datos podrá ser accedida al mismo tiempo por múltiples instrucciones, ya que cada una necesitaría un sistema de buses exclusivo para llevar a cabo su acceso.
  - ❖ Entonces, sólo existe una solución posible para este conflicto: la instrucción I_5, al haber comenzado su ejecución con posterioridad al inicio de la instrucción I_2, deberá detenerse temporalmente hasta que la instrucción I_2 haya logrado completar su etapa de acceso a memoria, durante la cual esta última, en caso de requerirlo, llevará a cabo su propio acceso a la memoria de datos.
  - ❖ Como cabe esperarse, la demora únicamente se prolongará durante un ciclo de reloj, ya que éste es el tiempo requerido por la instrucción I_2 para ejecutar tanto esta etapa como cualquiera de las otras que constituyen su ciclo de ejecución.
  - ❖ Este atasco en el cauce del procesador involucrará tanto a la instrucción I_5 como a todas las instrucciones posteriores a ella que se encuentren actualmente en ejecución: I_6, I_7 e I_8, y podrá comprobarse que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una por cada instrucción afectada, durante el octavo ciclo de reloj.

Instrucciones	**Progreso de la ejecución del programa**																
I_1																	
I_2																	

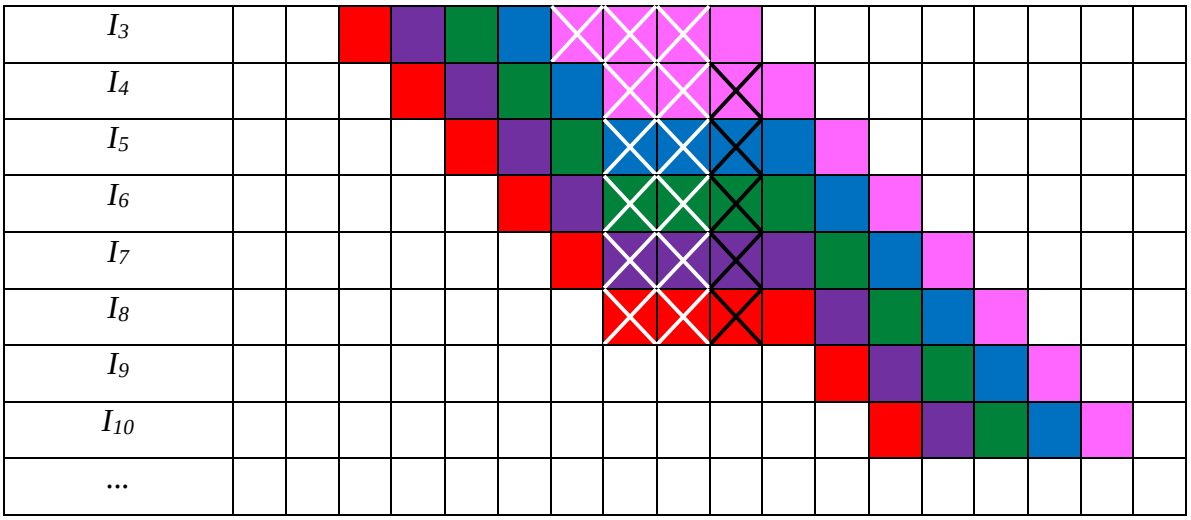


Figura 66. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco de tipo estructural en el cauce del nuevo procesador debido a dos intentos de escritura simultáneos sobre el banco de registros de uso general de la CPU.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ Tanto la instrucción I_3 como la I_4 están intentando ejecutar su respectiva etapa de almacenamiento en registro del ciclo de ejecución simultáneamente.
  - ❖ Esta situación no se encuentra soportada por la estructura actual del procesador ya que éste actualmente posee únicamente una unidad encargada de llevar a cabo esta etapa (unidad de almacenamiento en registro, ver tabla 46). Dicha unidad, a su vez, sólo tiene la capacidad de atender a una instrucción al mismo tiempo.
  - ❖ Además, debe recordarse que uno de los principios de funcionamiento de la segmentación implementada en el cauce del nuevo procesador establece que en ningún momento podrán coexistir múltiples instrucciones ejecutándose simultáneamente en la misma etapa de su ciclo de ejecución ya que, si se presentara esta situación, ya no se trataría de una segmentación, sino de una paralelización de las instrucciones ejecutadas.
  - ❖ Por lo tanto, la suma de ambos factores previamente mencionados determina que la instrucción I_4, al haber comenzado su ejecución con posterioridad al inicio de la instrucción I_3, deberá detenerse temporalmente hasta que la instrucción I_3 haya logrado completar su etapa de almacenamiento en registro, durante la cual esta

última, en caso de requerirlo, llevará a cabo su propio acceso al banco de registros de uso general de la CPU para actualizar alguno de ellos con un nuevo valor.

- ❖ Como cabe esperarse, la demora únicamente se prolongará durante un ciclo de reloj, ya que éste es el tiempo requerido por la instrucción I_3 para ejecutar tanto esta etapa como cualquiera de las otras que constituyen su ciclo de ejecución.
- ❖ Este atasco en el cauce del procesador involucrará tanto a la instrucción I_4 como a todas las instrucciones posteriores a ella que se encuentren actualmente en ejecución: I_5, I_6, I_7 e I_8, y podrá comprobarse que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una por cada instrucción afectada, durante el décimo ciclo de reloj.

- Por su parte, las celdas tachadas con una cruz blanca representan un atasco de otra índole, ya que se debe a una dependencia de datos de tipo WAW entre las instrucciones I_2 e I_3, y será explicado más detenidamente en las figuras 72 y 73.

3.1.8.2. Lectura Después de Escritura

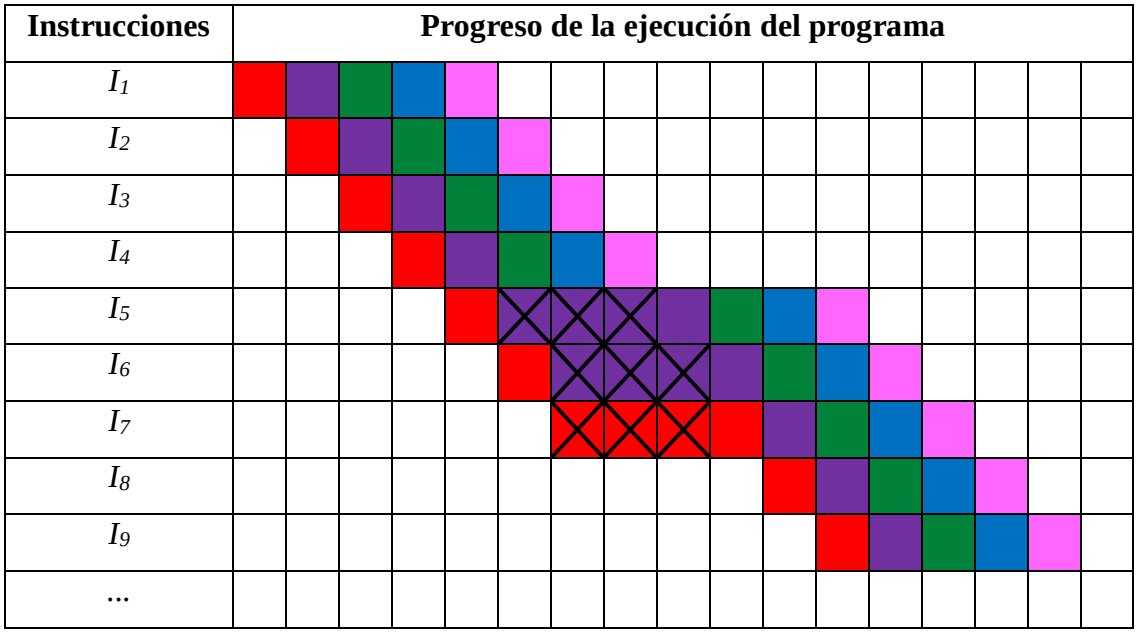


Figura 67. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco en el cauce del nuevo procesador debido a una dependencia de datos de tipo RAW, perteneciendo el registro en conflicto al banco de registros de uso general de la CPU.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
    - La instrucción I_4 tiene como objetivo para la etapa de almacenamiento en registro de su ciclo de ejecución realizar una operación de escritura sobre uno de los registros de uso general de la CPU (puede tratarse de un registro entero o de punto flotante).
    - Asimismo, la instrucción I_5 pretende llevar a cabo durante la etapa de decodificación de su ciclo de ejecución una operación de lectura sobre el mismo registro que se encuentra involucrado en la escritura a ser realizada por la instrucción I_4.
    - Si el cauce del procesador prosiguiera con su ejecución de manera normal, I_5 procedería con su lectura dos ciclos de reloj antes de que I_4 pueda efectuar su propia escritura sobre el registro, por lo cual el valor obtenido por I_5 no sería aquél que I_4 pretende escribir sino uno asignado al registro por alguna instrucción anterior a esta última, o bien un valor declarado para el registro durante la inicialización del programa y que no sufrió ningún tipo de modificación hasta el momento actual.
    - Sin embargo, si se tiene en cuenta el flujo de ejecución natural de un programa secuencial, al haber comenzado I_5 su ejecución con posterioridad al inicio de la instrucción I_4, el valor que lea desde el registro de uso general obligatoriamente deberá coincidir con aquél previamente escrito por I_4, ya que cualquier otro valor provocará un comportamiento no deseado por parte del programa, el cual a su vez le devolverá al usuario un resultado incorrecto.
    - Entonces, para que la ejecución segmentada del programa presente el mismo comportamiento y ofrezca los mismos resultados que la ejecución secuencial, la instrucción I_5 deberá detenerse temporalmente en su etapa de decodificación hasta que la instrucción I_4 haya logrado completar su propia etapa de almacenamiento en registro, durante la cual esta última llevará a cabo su acceso al banco de registros de uso general de la CPU para actualizar alguno de ellos con un nuevo valor, el cual será leído inmediatamente a continuación por la instrucción I_5 en el siguiente ciclo de reloj ya que precisamente éste es el dato que necesita para llevar a cabo su tarea, sin importar su naturaleza particular.
- A diferencia de los riesgos estructurales presentados en las figuras anteriores, aquí el tiempo de demora puede variar en función de la ubicación relativa de las instrucciones en conflicto una respecto de la otra dentro del programa: a mayor proximidad entre ellas, más

prolongada será la duración del atasco presente en el cauce del procesador. Esto se debe a que la distancia entre los instantes de inicio de la ejecución de ambas instrucciones repercutirá sobre el número de etapas y, en consecuencia, ciclos de reloj, que a la instrucción que pretende realizar la escritura sobre el registro conflictivo le restan por completar hasta lograr alcanzar su etapa de almacenamiento en registro, durante la cual actualizará el registro de uso general de la CPU con el dato requerido por la instrucción de lectura, la cual a su vez recién podrá proceder a obtenerlo en el próximo ciclo de reloj.

- Una vez tenida en cuenta la observación anterior, podrá comprobarse en la figura que, para este caso en particular, la demora se prolongará durante exactamente tres ciclos de reloj, ya que éste es el tiempo requerido por la instrucción I_4 para alcanzar y completar su etapa de almacenamiento en registro, cuyo dato a actualizar en el banco de registros de uso general de la CPU resulta imprescindible para que I_5 pueda proseguir normalmente con su ejecución. Este atasco en el cauce del procesador involucrará tanto a la instrucción I_5 como a todas las instrucciones posteriores a ella que se encuentren actualmente en ejecución: I_6 e I_7, y se observará que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada ciclo de reloj demorado, desde el sexto al octavo ciclo en el caso de la instrucción I_5 y, desde el séptimo al noveno ciclo para las instrucciones I_6 e I_7.

Instrucciones	**Progreso de la ejecución del programa**
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_7	
I_8	
I_9	
...	

Figura 68. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco en el cauce del nuevo procesador debido a una dependencia de datos de tipo RAW, siendo ahora el registro en conflicto uno de los registros de uso particular de la CPU.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
    - La instrucción I_4 tiene como objetivo para la etapa de ejecución en punto flotante de su ciclo de ejecución realizar una operación de escritura sobre uno de los registros de uso particular de la CPU: el registro de banderas FPFLAGS, principalmente asociado al estado de las operaciones aritméticas en punto flotante realizadas en la FPU del procesador.
    - Asimismo, la instrucción I_5 pretende llevar a cabo durante la etapa de decodificación de su ciclo de ejecución una operación de lectura sobre este mismo registro, el cual todavía posee una escritura pendiente de ser realizada por la instrucción I_4.
    - Éste es el escenario idóneo para recordar la distribución de las tareas a ser realizadas durante la etapa de ejecución en punto flotante de la instrucción I_4 entre sus cuatro subetapas distintas (ver tabla 50).
    - El motivo por el cual esta información resulta tan importante radica en que, al ser precisamente ésta la etapa en la cual I_4 llevará a cabo su escritura pendiente en el registro FPFLAGS, entonces se vuelve imprescindible conocer exactamente en cuál de los cuatro ciclos de reloj de la etapa se efectuará la actualización del registro para determinar la duración del atasco provocado en el cauce del procesador como consecuencia del conflicto surgido por esta dependencia de datos actualmente sin resolver.
    - Ahora bien, una vez analizada la distribución de las tareas entre las cuatro subetapas que conforman la etapa de ejecución en punto flotante, puede concluirse que la instrucción I_4 recién estará llevando a cabo su escritura pendiente en el registro FPFLAGS durante el último de los cuatro ciclos de reloj de su etapa de ejecución en punto flotante, ya que esta tarea corresponde precisamente a la cuarta subetapa de dicha etapa.

- Entonces, si el cauce del procesador prosiguiera con su ejecución de manera normal, I_5 procedería con su lectura tres ciclos de reloj antes de que I_4 pueda efectuar su propia escritura sobre el registro FPGLAGS, por lo cual el valor obtenido por I_5 no sería aquél que I_4 pretende escribir sino uno asignado al registro por alguna instrucción anterior a esta última, o bien un valor declarado para el registro durante la inicialización del programa y que no sufrió ningún tipo de modificación hasta el momento actual.
- Sin embargo, si se tiene en cuenta el flujo de ejecución natural de un programa secuencial, al haber comenzado I_5 su ejecución con posterioridad al inicio de la instrucción I_4, el valor que lea desde el registro de uso general obligatoriamente deberá coincidir con aquél previamente escrito por I_4, ya que cualquier otro valor provocará un comportamiento no deseado por parte del programa, el cual a su vez le devolverá al usuario un resultado incorrecto.
- Entonces, para que la ejecución segmentada del programa presente el mismo comportamiento y ofrezca los mismos resultados que la ejecución secuencial, la instrucción I_5 deberá detenerse temporalmente en su etapa de decodificación hasta que la instrucción I_4 haya logrado completar el cuarto ciclo de reloj de su propia etapa de ejecución en punto flotante, durante el cual esta última llevará a cabo su acceso al registro FPFLAGS para actualizarlo con un nuevo valor, el cual será leído inmediatamente a continuación por la instrucción I_5 en el siguiente ciclo de reloj ya que precisamente éste es el dato que necesita para llevar a cabo su tarea, sin importar su naturaleza particular.
- Por su parte, las celdas tachadas con una cruz blanca representan un atasco provocado en el cauce del procesador a causa de un riesgo de control cuya presencia, aunque su naturaleza sea absolutamente distinta de aquélla asociada a la dependencia de datos de tipo RAW entre las instrucciones I_4 e I_5 que desencadenó la primera obstrucción y detención temporal en el cauce, no deja de resultar oportuno aclarar que existe como consecuencia de este atasco previo, aunque tal vez no de una manera tan directa como el lector podría esperar ni de la misma forma que en el caso de los atascos representados en las otras figuras.
- A pesar de que las características principales de todos los tipos de riesgos de control conocidos que pueden presentarse en el cauce del procesador son ejemplificadas y explicadas más detalladamente en las figuras 74, 75 y 76, se optó por incluir aquí

un breve apartado adicional para que el usuario comprenda más claramente la relación presente en esta situación en particular entre el atasco RAW y el de control.

- Para empezar, se procederá a proponer a continuación un caso de prueba alternativo a aquél representado previamente en la figura 68:

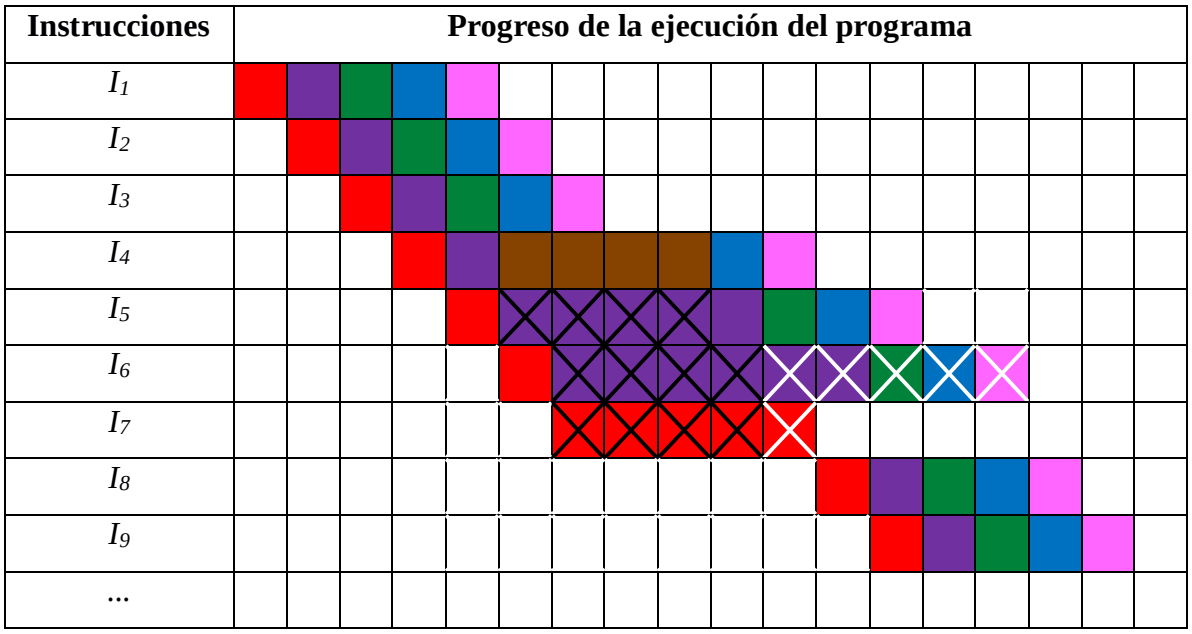


Figura 69. Progreso de la ejecución segmentada de un programa similar a aquél ejecutado en la figura anterior, pero ahora con una modificación en una de las instrucciones que alterará el resto de la ejecución del programa una vez resuelta la dependencia de datos de tipo RAW.

- En este nuevo caso de ejecución, incluso teniendo en cuenta que la dependencia de datos de tipo RAW entre las instrucciones I_4 e I_5 se mantiene inalterable con respecto

al ejemplo de la figura 68, podrá apreciarse una diferencia notoria durante el resto de la ejecución del programa a partir de la resolución de dicha dependencia, es decir, una vez reanudada la ejecución de las instrucciones I_5, I_6 e I_7, anteriormente demoradas debido a la presencia del atasco RAW en el cauce del procesador.

- Más precisamente, se determinó durante la confección de las situaciones cuya ejecución se encuentra representada en las figuras 68 y 69 que la instrucción I_5 debía tratarse de una instrucción de transferencia de control condicional. Asimismo, se estableció que esta conclusión podía extenderse a cualquier caso similar a éste: todas las dependencias de datos de tipo RAW surgidas por un conflicto en torno al contenido del registro FPFLAGS se encontrarán inevitablemente asociadas a la presencia de una instrucción de salto condicional.
- Naturalmente, el lector podría cuestionar las razones que motivaron dicha determinación. Para conocer la respuesta a esta pregunta correspondería llevar a cabo, en primer lugar, un estudio pormenorizado de la tabla 36, en la cual se presentan las características principales del registro de uso particular de la CPU ya conocido como FPFLAGS, ya que éste contiene precisamente el dato cuyo intento de lectura prematura por parte de, en este caso, la instrucción I_5, origina el conflicto entre las instrucciones directamente involucradas en la dependencia, I_4 e I_5 aquí, y su consecuente atasco.
- Las conclusiones obtenidas a partir del estudio anterior deberían permitirle al lector determinar que, para empezar, el dato contenido por el registro FPFLAGS está conformado de hecho por diversos bits, conocidos como banderas, la mayoría de los cuales refleja en cada bit un aspecto distinto acerca del estado de la última operación aritmética en punto flotante llevada a cabo en la FPU del procesador.
- Investigando más exhaustivamente esta conclusión resulta oportuno agregar que, en realidad, la única de estas banderas cuyo valor podría ser relevante durante la ejecución de cualquier programa tanto para el procesador como para el usuario es la bandera F (Punto Flotante). Por su parte, la importancia de los datos ofrecidos por todas las banderas restantes, debido a motivos varios, casi todos vinculados al tipo de arquitectura optado durante el diseño de la nueva computadora, se limita únicamente a un plano informativo, dado que en la práctica no son utilizados, al menos en la versión del procesador actualmente disponible y operativa, por ninguna de las instrucciones del repertorio para llevar a cabo operación o tarea alguna.

- A partir de esta última definición lograría deducirse entonces que esta modalidad particular de dependencias de datos de tipo RAW sólo podría acontecer en caso de encontrarse involucrada una operación de lectura sobre la bandera F del registro de uso particular FPFLAGS de la CPU, sencillamente debido a que en realidad ésta es la única bandera de dicho registro cuyo valor podría resultar de interés para ser leído por alguna instrucción del repertorio.
- Teniendo en cuenta esta nueva información adquirida, el siguiente paso sería proceder a analizar el repertorio de instrucciones propiamente dicho soportado por el nuevo procesador, definido en la tabla 38, de manera tal de identificar e individualizar la denominación de las instrucciones que podrían en particular ser susceptibles de generar un atasco en el cauce del procesador a causa de la dependencia de datos descripta en las observaciones anteriores.
- Para ello, simplemente debería estudiarse la tarea de cada una de las instrucciones y detectar aquéllas que obligatoriamente requieren acceder al registro FPFLAGS y realizar una operación de lectura sobre él a fin de obtener el valor de la bandera F (la única relevante para la versión actual de este procesador) para poder llevar a cabo sus respectivas tareas.
- De este modo, se observa que, en realidad, las únicas instrucciones del repertorio que cumplen las condiciones anteriores son las siguientes: “bfpt” y “bfpf”, en otras palabras, dos instrucciones de transferencia de control condicional capaces de alterar o preservar el flujo de ejecución actual de un programa en función del cumplimiento o incumplimiento de la condición de salto asociada y establecida implícitamente en cada una de ellas: el valor de la bandera F debe ser igual a uno para el caso de la instrucción “bfpt”, e igual a cero para la instrucción “bfpf”.
- Entonces, sería correcto y adecuado asumir que únicamente puede presentarse una dependencia de datos de tipo RAW de características similares a aquéllas representadas en las figuras 68 y 69, es decir, en la cual el conflicto se centre en el valor almacenado en el registro de uso particular FPFLAGS de la CPU, en caso de existir involucrada en ella una instrucción de transferencia de control condicional. Por otra parte, cabe aclarar que esta condición es necesaria pero insuficiente por sí misma: de ninguna manera una instrucción de este tipo debe encontrarse obligatoriamente asociada a la presencia de un atasco RAW de esta naturaleza en el cauce del procesador.

- La consecuencia lógica y esperable de la ejecución de una instrucción de transferencia de control condicional como I_5, sin importar su denominación (“bfpt” o “bfpf”), será el surgimiento de un riesgo de control con su consecuente atasco, cuyos atributos principales ya se encuentran, como se mencionó anteriormente, descriptos en las observaciones de las figuras 75 y 76 (en la figura 74 se detallan las consecuencias de la ejecución de una instrucción de transferencia de control incondicional).
- A pesar de que tanto en la figura 68 como en la 69 se presenta una dependencia de datos de tipo RAW entre las instrucciones I_4 e I_5, tratándose esta última de una transferencia de control condicional cuya ejecución provoca un riesgo de control que demorará la ejecución del programa, dicha instrucción fue alterada entre ambos casos de manera tal que el resultado de la evaluación de la condición de salto varíe y con él, diverjan las características del atasco ocasionado, especialmente sus resoluciones y desenlaces en las dos situaciones.
- Más precisamente, en el caso de la figura 68 no se cumple la condición de salto asociada a la instrucción de transferencia de control condicional I_5, por lo tanto, no se produce salto alguno ni se altera el flujo de ejecución normal del programa. De esta manera, la instrucción I_6, previamente demorada a causa del atasco asociado al riesgo de control, reanuda su ejecución sin inconvenientes, así como también su instrucción inmediatamente posterior, I_7, inicia la primera etapa de su ciclo de ejecución, cuyo comienzo asimismo había sido pospuesto debido a este mismo atasco. Puede encontrarse una presentación y descripción más pormenorizada de esta situación en particular en la figura 75.
- En cambio, en el escenario de la figura 69 se cumple la condición de salto de la instrucción I_5, provocándose efectivamente dicho salto y alterándose el flujo de ejecución normal del programa. Así, la instrucción I_6, previamente demorada, será cancelada terminante y prematuramente antes de completar su ejecución e I_7, cuyo inicio había sido retrasado, jamás comenzará a ejecutarse, dado que ambas fueron eliminadas del flujo original del programa, siendo reemplazadas, en primera instancia, por la instrucción destino del salto, es decir, I_8 en este caso, la cual se ejecutará en lugar de ellas, seguida a su vez por todas las instrucciones que estén ubicadas inmediatamente a continuación de ella en el programa: I_9, I_{10}, I_{11}, etc. Puede encontrarse una presentación y descripción más pormenorizada de esta situación en particular en la figura 76.

- En resumen, la determinación de presentar en las figuras 68 y 69 este tipo de dependencia de datos RAW inevitablemente asociada a la existencia de un consecuente riesgo de control se justifica de la siguiente manera: aunque teóricamente podría argumentarse que esta clase de conflictos con respecto al valor del registro FPFLAGS se encuentra vinculada a la presencia de cualquier tipo de instrucción que intente realizar una operación de lectura sobre dicho registro, en la práctica, al menos en la versión actualmente implementada del procesador, se concluye no sólo que en realidad solamente dos instrucciones del repertorio, "bfpt" y "bfpf", pueden pretender llevar a cabo esta lectura, sino que, además, ambas pertenecen al mismo subtipo de instrucciones: transferencia de control condicional. Ergo, a través de estas dos figuras ya se estarían representando todas las modalidades de las distintas situaciones de ejecución posibles que podrían presentarse en este procesador.
- La afirmación anterior es válida con respecto a las posibles instrucciones que podrían intentar llevar a cabo una operación de lectura sobre el registro FPFLAGS y así, en algunos casos, ocasionar un atasco RAW en el cauce del procesador.
- Sin embargo, todavía faltaría por analizar la otra instrucción directamente involucrada en la dependencia de datos: aquélla cuyo objetivo consiste en realizar una actualización, es decir, una operación de escritura, sobre el registro FPFLAGS, con el propósito de modificar el valor de una o varias de sus banderas.
- Como fue explicado anteriormente, en los casos representados tanto en la figura 68 como en la número 69, este papel estaría siendo desempeñado por la instrucción I_4, tratándose la misma de una instrucción que, independientemente de su denominación y tarea particulares, obligatoriamente requiere utilizar la FPU del procesador para realizar una operación aritmética en punto flotante que le permita llevar a cabo su función asignada, sea cual sea.
- Obviamente, este último requerimiento provocará que el ciclo de ejecución de dicha instrucción sea más intrincado, extenso y, en consecuencia, lento, que aquél asociado a una instrucción que no haga uso alguno de la FPU de la CPU, debido a que, en particular, su etapa de ejecución, conformada por cuatro subetapas independientes, tendrá una duración de cuatro ciclos de reloj en lugar de uno como en los demás casos más tradicionales, a causa de la complejidad de la operación aritmética en punto flotante llevada a cabo en la FPU (ver y comparar tablas 48 y 49).

- Al consistir la función principal del registro FPFLAGS precisamente en reflejar a través de sus banderas distintos atributos acerca del estado de la última operación aritmética en punto flotante llevada a cabo en la FPU del procesador, podría asumirse perfectamente que cualquiera de las instrucciones del repertorio que deba realizar una operación aritmética en punto flotante en la FPU para desempeñar su tarea asociada obligatoriamente tendrá que acceder al registro FPFGLAGS a fin de actualizar todas las banderas que sean necesarias para que pueda exhibirse correctamente el estado de la operación efectuada por dicha instrucción en la FPU, dado que ésta pasaría a ser ahora efectivamente la última operación realizada cuyo estado debe estar informado por el registro FPFLAGS.
- Una apreciación a tener en cuenta es la siguiente: aunque todas las instrucciones que utilicen de una u otra manera la FPU durante su ejecución deberán sobrescribir el registro FPFLAGS, esta actualización no necesariamente afectará a todas las banderas que conforman dicho registro. En primer lugar, la bandera I únicamente posee un próposito asignado en el registro FLAGS. Por su parte, en el registro FPFLAGS permanece sin utilizar, por lo que su valor jamás será actualizado por ninguna instrucción. Por otra parte, las banderas Z, S, O, C, A y P, al encontarse cada una asociada de algún modo al estado de la última operación realizada en la FPU, ineludiblemente estarán afectadas por cualquier sobrescritura sobre el registro FPFLAGS llevada a cabo por una instrucción luego de haber utilizado esta última la FPU.
- Una vez aclarado esto, la atención debería focalizarse en la bandera F: así como previamente se determinó a través de un estudio pormenorizado del repertorio de instrucciones del procesador, definido en la tabla 38, que en realidad sólo dos instrucciones, “bfpt” y “bfpf”, necesitarían acceder al registro FPFLAGS para llevar a cabo una operación de lectura, aquí podría llevarse a cabo un análisis similar, pero ahora para identificar las instrucciones del repertorio que, durante su ejecución, acceden al registro FPFLAGS para sobrescribir en particular la bandera F.
- Observando la descripción de la función de cada instrucción, todas detalladas en el repertorio, se concluye que, en realidad, únicamente las instrucciones “ltf”, “lef” y “eqf” requieren actualizar la bandera F, ya que precisamente su objetivo consiste en asignarle a esta última un valor igual a uno o cero según el cumplimiento o incumplimiento respectivamente de la condición de comparación y evaluación, la cual varía en función de la instrucción ejecutada. Por lo tanto, podría argumentarse

que sólo estas instrucciones sobrescribirán la bandera F durante su acceso al registro FPFLAGS para actualizarlo con el estado de su operación aritmética en punto flotante llevada a cabo en la FPU.

- Aun así, el repertorio de instrucciones soportado por el procesador no ofrece por sí solo una descripción para la tarea a desempeñar por cada instrucción asociada al uso de la FPU con la precisión necesaria como para obtener una interpretación unívoca con respecto al papel que debe cumplir la bandera F durante la ejecución de las distintas instrucciones.
- En otras palabras, podría admitirse como válida, por un lado, la afirmación de que la bandera F no debería ser actualizada por ninguna de estas instrucciones, a excepción, obviamente, de las instrucciones "ltf", "lef" y "eqf" ya que el cumplimiento de sus tareas se encuentra intrínseca e ineludiblemente vinculado con la sobrescritura del valor de dicha bandera.
- Sin embargo, también resultaría inicialmente aceptable interpretar, por otro lado, que todas las instrucciones que utilizan durante su ciclo de ejecución la FPU del procesador para realizar alguna operación aritmética en punto flotante deberían ser capaces de alterar el valor de la bandera F ya que esta última, aunque no exactamente de la misma manera que el resto de las banderas del registro FPFLAGS (exceptuando la bandera I, que no posee ninguna utilidad en este registro), también refleja un atributo sobre el estado de la última operación llevada a cabo en la FPU.
- No obstante, al analizar la definición de las banderas de los registros FLAGS y FPFLAGS del nuevo procesador, establecida en la tabla 36, se puede examinar que, en sus observaciones, se presenta una descripción detallada del propósito y funcionamiento de la bandera F. En particular, allí se determina que, mientras que en el registro FLAGS su valor resulta irrelevante, el mismo es actualizado en el registro FPFLAGS sólo si en la FPU se ejecuta alguna de las instrucciones de comparación especiales incorporadas en el repertorio a tal efecto: precisamente las instrucciones "ltf", "lef" y "eqf" que ya fueron identificadas en los incisos previos.
- De esta manera, el dilema anterior se resuelve del siguiente modo: únicamente la ejecución de las instrucciones de comparación en punto flotante "ltf", "lef" y "eqf" podrá modificar el estado de la bandera F del registro FPFLAGS, mientras que la misma permanecerá inalterable durante la ejecución de cualquiera de las demás instrucciones del repertorio soportado por el procesador, tanto si necesitan utilizar

la FPU para realizar operaciones aritméticas en punto flotante como si no lo requieren.

- Asimismo, se garantiza así que cualquiera de las instrucciones particulares de transferencia de control condicional que deban leer la bandera F para evaluar su condición de salto obtengan efectivamente el valor actualizado por la última instrucción de comparación en punto flotante ejecutada, si es que la hubo. Más precisamente, esta garantía implica la evitación segura del posible riesgo de que la ejecución de cualquier otra instrucción en punto flotante entre la instrucción de comparación y la de salto sobrescriba innecesaria e incorrectamente esta bandera y que termine siendo este último valor aquél que sea leído y tenido en cuenta por la instrucción de transferencia de control para la evaluación de su condición de salto.
- No obstante, cabe mencionar y describir un último percance: el mecanismo para la detección de esta clase de dependencias actualmente implementado y disponible en el nuevo procesador posee algunas limitaciones esperables y hasta incluso razonables considerando la característica general que define a cualquier dependencia de datos. Independientemente de su modalidad en cuestión (RAW, WAR o WAW), la misma se presenta entre dos instrucciones con respecto al orden de acceso para realizar una operación sobre el dato almacenado en un registro de uso general o particular del procesador, tratándose al menos una de las operaciones de una escritura (nunca puede provocarse una dependencia de datos entre dos instrucciones de lectura).
- En el caso particular del registro FPFLAGS, en el cual al fin y al cabo se centra la situación presentada en esta figura, el dato almacenado en dicho registro se conforma por todas las banderas ya conocidas, definidas en la tabla 36, es decir, tanto la bandera F como los flags I, Z, S, C, O, A y P. Como ya habrá podido adivinarse, esto implica que la variante de las dependencias de datos de tipo RAW aquí estudiada podría presentarse en el registro FPFLAGS incluso entre dos instrucciones cuyos intentos de acceso al registro no involucren exactamente la lectura o escritura, según corresponda, de la misma bandera.
- Para comprender mejor esta situación resultaría conveniente exponer un ejemplo a través de una nueva figura tal como se ha hecho hasta ahora para presentar todos los demás tipos de atascos que pueden obstruir temporalmente la ejecución de las instrucciones en el procesador. No obstante, al considerarse que ésta no se trata por sí misma de una nueva modalidad de riesgos de datos sino simplemente de una

curiosidad para ser observada, se ha determinado que la demostración del ejemplo práctico se pospondrá para la siguiente subsección, "3.8.1.7. Programas de prueba", dedicada especialmente a la muestra de la ejecución de diversos casos de prueba conformados por varios programas cuyas instrucciones fueron específicamente escritas en el lenguaje Assembler.

- En cambio, aquí se describirá este asunto conceptualmente de manera tal que el lector pueda comprender la idea que se pretende transmitir sin necesidad de graficar la ejecución de ningún programa de prueba: básicamente se trataría de la misma situación presentada en la figura 69, pero la instrucción I_4, en lugar de tratarse de una de las instrucciones de comparación que durante su ejecución sobrescriben el valor de la bandera F del registro FPFLAGS ("ltf", "lef" o "eqf"), consiste en cualquiera de las demás instrucciones del repertorio soportado por el procesador que también hacen uso de la FPU del procesador para realizar operaciones aritméticas en punto flotante pero sin llevar a cabo actualización alguna del flag F del registro FPFLAGS, aunque sí del resto de las banderas (a excepción del flag I, naturalmente).
- Una vez planteada esta situación, convendría prestar especial atención a la instancia de ejecución del programa en la cual comienza la etapa de decodificación de la instrucción I_5. Esta última permanece siendo una de las instrucciones de transferencia de control condicional cuya condición de salto se encuentra íntimamente asociada al estado de la bandera F ("bfpt" o "bfpf").
- En dicha etapa, tal como en cualquier otro caso normal, procederá a analizarse la existencia de posibles escrituras pendientes por parte de instrucciones anteriores a I_5, capaces de provocar que el o los datos que se pretenden leer aún no se encuentren actualizados hasta el momento con los valores que deberían poseer. Si I_5 procediera con la lectura existiendo operaciones de escritura pendientes de instrucciones previas a ella, entonces la información leída no sería la correcta por las razones ya conocidas: debe respetarse el orden natural de las instrucciones del programa tal como en una ejecución secuencial del mismo, en la cual resultaría incoherente e ilógico que una instrucción lea un dato que todavía no fue escrito por una instrucción anterior.
- Ahora bien, éste se trata de un caso único, en el cual podría argumentarse que, al no involucrar la ejecución de la instrucción I_4 modificación alguna sobre la bandera F del registro FPFLAGS, la instrucción I_5, al requerir la lectura pura y exclusivamente de este flag, no debería esperar a que I_4 complete el cuarto ciclo de su etapa de

ejecución en punto flotante, en la cual actualizaría las banderas Z, S, C, O, A y P de dicho registro. Esto se debe a que, al no sufrir la bandera F modificación alguna, el valor leído por I_5 para evaluar su condición de salto sería el correcto incluso antes de la sobrescritura del registro llevada a cabo por I_4, sin importar su naturaleza particular.

- Sin embargo, el mecanismo de detección para posibles riesgos de datos actualmente implementado en el nuevo procesador todavía no posee la precisión necesaria como para determinar conflictos con respecto a bits individuales y específicos de un registro de la CPU: la dependencia se establece sobre la totalidad del contenido del registro involucrado en el conflicto. Entonces, volviendo a este caso en particular, el detector determinará la necesidad de atascar la ejecución tanto de I_5 como de sus instrucciones subsiguientes (I_6 e I_7, si se mantiene el ejemplo de la figura 69) que se encuentren ejecutándose actualmente hasta que I_4 logre completar su escritura pendiente sobre el registro FPFLAGS. Esta acción por parte del detector será llevada a cabo independientemente de los flags que estén específicamente involucrados en la operación de actualización ya que, como se acaba de describir, la dependencia se establece con respecto a todo el registro en lugar de sus banderas individuales cuya sobrescritura esté específicamente pendiente de ser finalizada.
- Podría argumentarse, acertadamente, que este atasco resulta innecesario, ocasionando una demora indeseada en la ejecución del programa ya que el dato requerido para ser leído por la instrucción I_5, la bandera F, no posee relación alguna con los datos pendientes de escritura por la instrucción I_4, las banderas Z, S, C, O, A y P, más allá de pertenecer todos al mismo registro, FPFLAGS.
- Como consecuencia de esta última observación previa, surge como propuesta para implementar en la próxima versión del procesador una posible mejora de la precisión del detector de riesgos de datos de manera tal que sea capaz de discernir, dentro de un registro, los bits individuales del mismo que resulten objeto de las operaciones de lectura o escritura de las instrucciones entre las cuales podrían provocarse conflictos por dependencias con respecto a dichos bits. De este modo, el cauce del procesador se atascaría únicamente cuando resulte estrictamente necesario, es decir, cuando no sólo el registro en general, sino también al menos uno de sus bits particulares, se encuentre involucrado en las operaciones de lectura o escritura a realizar por las dos instrucciones que estén directamente implicadas en el conflicto.

- Como se recordará, en la figura 67 se representó un posible caso de ejecución de una serie de instrucciones en la CPU en el cual se provocara un riesgo por dependencia de datos de tipo RAW, perteneciendo el registro en conflicto al banco de registros de uso general de la CPU. Al igual que en aquella clase de riesgos, aquí el tiempo de demora puede variar en función de la ubicación relativa de las instrucciones en conflicto una respecto de la otra dentro del programa: a mayor proximidad entre ellas, más prolongada será la duración del atasco presente en el cauce del procesador. Esto se debe a que la distancia entre los instantes de inicio de la ejecución de ambas instrucciones repercutirá sobre el número de etapas y, en consecuencia, ciclos de reloj, que a la instrucción que pretende realizar la escritura sobre el registro conflictivo, FPFLAGS en este caso, le restan por completar hasta lograr alcanzar el cuarto ciclo de su etapa de ejecución en punto flotante, durante el cual actualizará el registro de uso particular de la CPU con el valor requerido por la instrucción de lectura, la cual a su vez recién podrá proceder a obtenerlo en el próximo ciclo de reloj.
- Una vez tenida en cuenta la observación anterior, podrá comprobarse en las figuras 68 y 69 que, para este caso en particular, la demora se prolongará durante exactamente cuatro ciclos de reloj, ya que éste es el tiempo requerido por la instrucción I_4 para alcanzar y completar el cuarto ciclo de su etapa de ejecución en punto flotante, cuyo valor a actualizar en el banco registro FPFLAGS de la CPU resulta imprescindible para que I_5 pueda proseguir normalmente con su ejecución. Este atasco en el cauce del procesador involucrará tanto a la instrucción I_5 como a todas las instrucciones posteriores a ella que se encuentren actualmente en ejecución: I_6 e I_7, y se observará que está representado en ambas figuras a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada ciclo de reloj demorado, desde el sexto al noveno ciclo en el caso de la instrucción I_5 y, desde el séptimo al décimo ciclo para las instrucciones I_6 e I_7.

3.1.8.3. Escritura Después de Lectura

Instrucciones	**Progreso de la ejecución del programa**																	
I_1																		
I_2																		
I_3																		
I_4																		

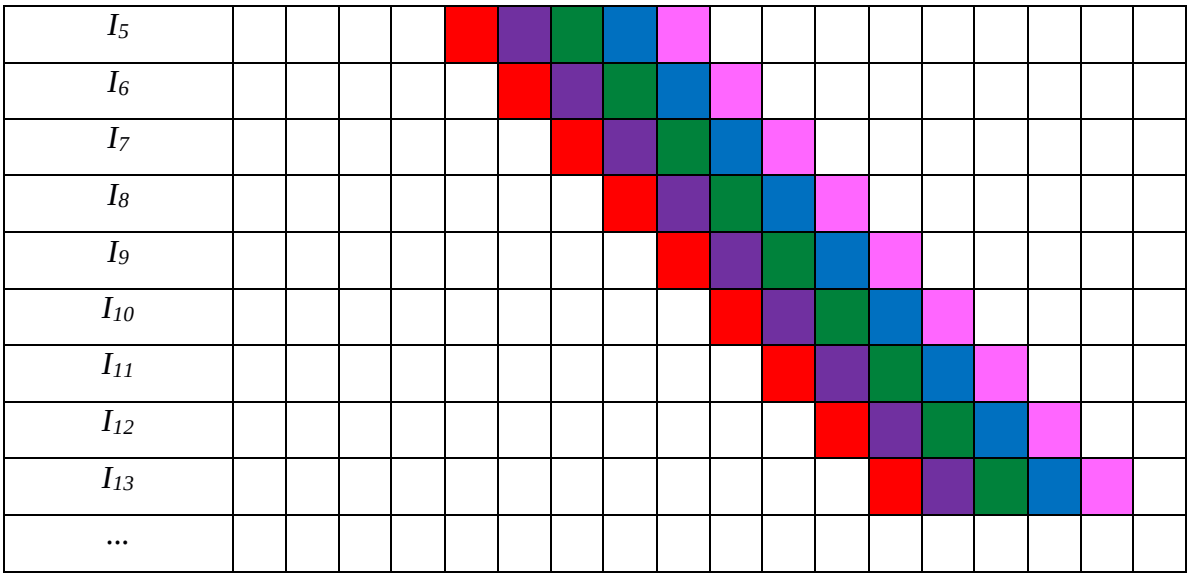


Figura 70. Progreso de la ejecución segmentada de un programa previo al reordenamiento de las instrucciones por parte del ensamblador necesario para evitar la presencia de huecos de retardo vacíos e inutilizados a continuación de las instrucciones de transferencia de control.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ La instrucción I_2 consiste en una transferencia de control incondicional hacia la instrucción I_4.
  - ❖ Entonces, la instrucción I_3 comenzará a ser ejecutada en su etapa de búsqueda hasta que la naturaleza de I_2 sea identificada durante la ejecución de su propia etapa de decodificación.
  - ❖ Al encontrarse las etapas de ambas instrucciones ejecutándose simultáneamente, la detección del salto incondicional de I_2 provocará la cancelación inmediata y prematura de I_3 antes de que pueda seguir completando las cuatro etapas restantes de su ciclo de ejecución.
  - ❖ De esta manera, la ejecución incompleta de I_3 generará un hueco de retardo disponible en el cauce del procesador para que el ensamblador pueda reubicar en él alguna instrucción anterior o posterior del programa que no posea ninguna relación o dependencia con la instrucción I_2 con el objetivo de que cada ciclo de reloj se encuentre óptimamente utilizado y aprovechado, es decir, que posea todas sus

respectivas instrucciones del cauce en ejecución en lugar de demoradas o canceladas.

- ❖ En este caso, como el ensamblador no realizó ningún tipo de reordenamiento en el programa, el hueco de retardo permanece vacío, desaprovechándose una fracción del tiempo de ejecución de los últimos cuatro ciclos de reloj de la instrucción I_3, durante los cuales ésta persiste sin ejecutarse.
- ❖ No obstante, al preservarse el orden de ejecución natural inicial del programa, es decir, todas las instrucciones comienzan su ejecución tal y como fueron dispuestas por el usuario en dicho programa, se reduce la posibilidad de que se presente alguno de los riesgos asociados a la implementación de la segmentación en el cauce del procesador debido a dependencias de datos por inconsistencias y/o descoordinaciones durante el transcurso de la ejecución de las instrucciones.
- ❖ Más concretamente, el desaprovechamiento del hueco de retardo generado por la cancelación prematura de I_3, aunque disminuye el rendimiento del procesador, elimina por completo cualquier tipo de posibilidad de que se presenten atascos durante la ejecución asociados a la existencia de dependencias de datos de tipo WAR.
- ❖ Esto es así porque, al encontrarse las etapas vinculadas a esta clase de dependencia tan lejanas entre sí en el tiempo, se ha comprobado que resulta absolutamente imposible en condiciones normales que la etapa de almacenamiento en registro de una instrucción finalice su actualización de un registro de uso general de la CPU antes de que la etapa de decodificación de otra instrucción anterior a ella haya logrado leerlo.
- ❖ En la próxima figura se observará la manera en la cual la alteración en la secuencia de ejecución natural inicial dispuesta originalmente por el usuario para las instrucciones del programa debido al reordenamiento de las mismas llevado a cabo por el ensamblador puede tener repercusiones muy severas sobre la ejecución del programa hasta el extremo de provocar nuevos tipos de dependencias de datos, con sus consecuentes atascos en el cauce del procesador, que jamás existirían si se respetara el orden de las instrucciones en el programa, independientemente de si la ejecución de estas últimas es llevada a cabo en forma secuencial o segmentada.

- Por su parte, las celdas tachadas con una cruz blanca no poseen relación alguna con las dependencias de datos de tipo WAR analizadas en este escenario, sino que simplemente representan los ciclos de ejecución desaprovechados por la instrucción I_3 debido a su

cancelación prematura como consecuencia de la transferencia de control incondicional asociada a la instrucción I_2. Esta última situación será explicada más detenidamente en la figura 74.

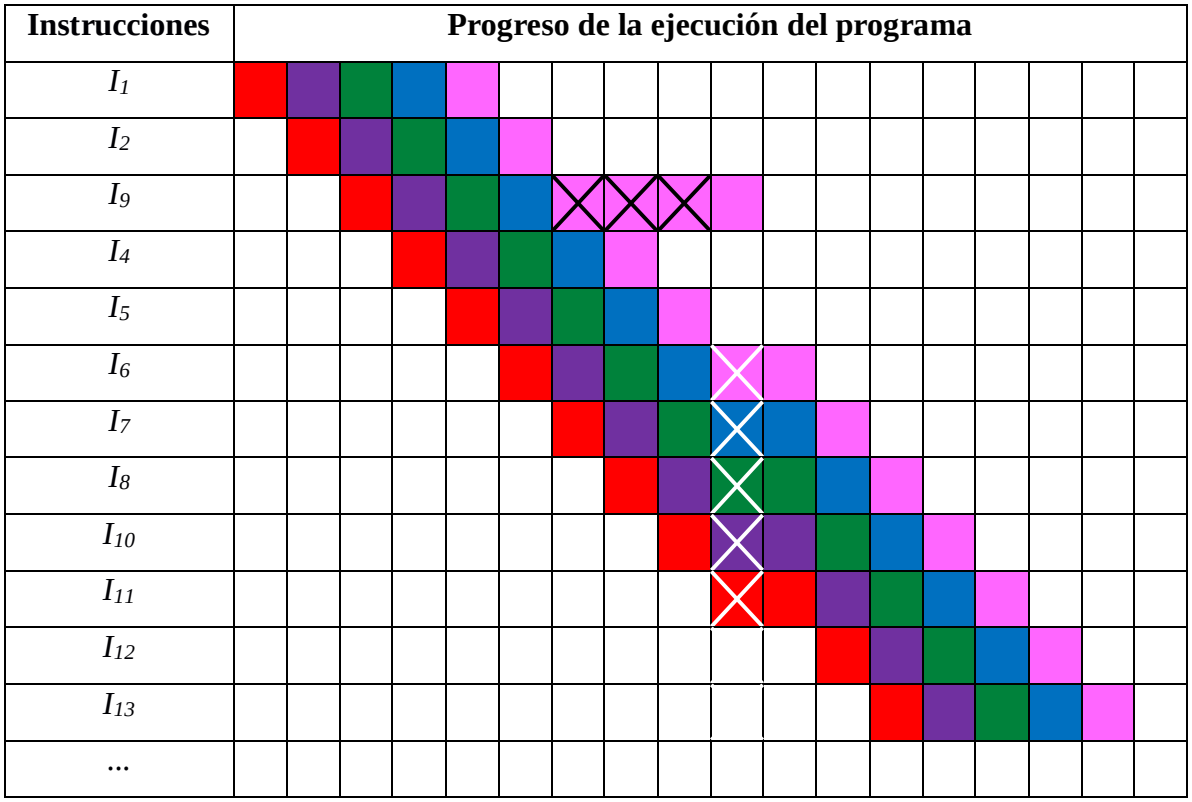


Figura 71. Progreso de la ejecución segmentada del mismo programa ejecutado en la figura anterior, pero ahora una vez aplicada sobre él la técnica de salto retardado, por lo cual surge un atasco en el cauce del nuevo procesador debido a una dependencia de datos de tipo WAR.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ La instrucción I_2 consiste en una transferencia de control incondicional hacia la instrucción I_4.

- No obstante, a diferencia del escenario planteado en la figura 70, aquí el ensamblador había previsto el hueco de retardo que se generaría en el cauce del procesador durante la ejecución del programa debido a la cancelación prematura de la instrucción I_3 a causa de la transferencia de control incondicional asociada a la instrucción I_2.
- Por lo tanto, I_3 jamás comenzará con su ejecución. Al haber anticipado este espacio de tiempo durante el cual existirá una instrucción sin ejecutarse, el ensamblador procederá a realizar un reordenamiento de las instrucciones del programa, ubicando en este hueco disponible otra instrucción del programa anterior o posterior a I_3 no posea ninguna relación o dependencia con la instrucción I_2 con el objetivo de que cada ciclo de reloj se encuentre óptimamente utilizado y aprovechado, es decir, que posea todas sus respectivas instrucciones del cauce en ejecución en lugar de demoradas o canceladas.
- En esta ocasión, el ensamblador ha optado por la instrucción I_9 para reubicarla en el hueco de retardo generado en el espacio de ejecución de la instrucción I_3.
- Tal y como se demostró en la figura 70, si se hubiera preservado el orden de ejecución original de las instrucciones del programa, I_3 habría comenzado a ejecutarse a continuación de I_2, pero su ejecución se habría cancelado prematuramente debido a la transferencia de control desencadenada por la instrucción I_2, desaprovechándose los últimos cuatro ciclos de reloj de la instrucción cancelada.
- En cambio, el reordenamiento de las instrucciones previamente llevado a cabo por el ensamblador posibilitará que sea I_9 en lugar de I_3 la instrucción que comience a ejecutarse a continuación de I_2.
- Al no poseer relación o dependencia alguna con el salto provocado por la ejecución de la instrucción I_2, I_9 se ejecutará en su totalidad sin ningún tipo de interrupción o cancelación debido a la transferencia de control llevada a cabo por I_2.
- De este modo, se aprovechan al máximo todos los ciclos de reloj de este hueco de retardo que de otra manera habrían permanecido inutilizados debido a la cancelación de la ejecución de I_3.
- Desafortunadamente, este reordenamiento previo de las instrucciones también posee sus desventajas:
    - La instrucción I_9 tiene como objetivo para la etapa de almacenamiento en registro de su ciclo de ejecución realizar una operación de escritura sobre

uno de los registros de uso general de la CPU (puede tratarse de un registro entero o de punto flotante).

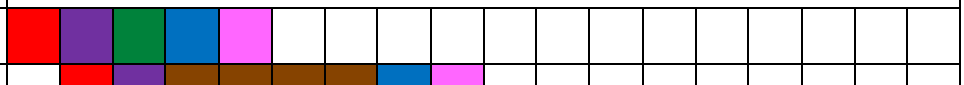
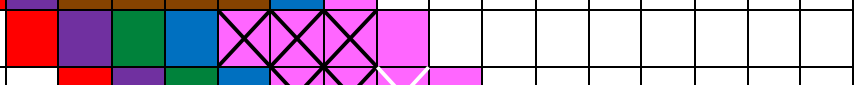
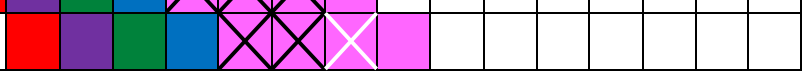
- Asimismo, la instrucción I_8 pretende llevar a cabo durante la etapa de decodificación de su ciclo de ejecución una operación de lectura sobre el mismo registro que se encuentra involucrado en la escritura a ser realizada por la instrucción I_9.
- Si el cauce del procesador prosiguiera con su ejecución de manera normal, I_9 procedería con su escritura un ciclo de reloj antes de que I_8 pueda efectuar su propia lectura sobre el registro, por lo cual el valor obtenido por I_8 sería aquél que fue previamente escrito por I_9.
- Puede deducirse sin inconvenientes que el valor leído por I_8 en esta ocasión es incorrecto ya que I_9 había sido dispuesta por el usuario en el programa, previamente al reordenamiento de sus instrucciones por parte del ensamblador, en una ubicación posterior a I_8, por lo cual el dato que esta última debería leer tendría que ser un valor asignado al registro por alguna instrucción localizada originalmente antes que ella (I_1 a I_7), o bien un valor declarado para el registro durante la inicialización del programa y que no sufrió ningún tipo de modificación hasta el momento actual.
- Entonces, para que la ejecución segmentada del programa reordenado presente el mismo comportamiento y ofrezca los mismos resultados que la ejecución del programa original, ya sea segmentada o secuencial, la instrucción I_9 deberá detenerse temporalmente en su etapa de almacenamiento en registro hasta que la instrucción I_8 haya logrado completar su propia etapa de decodificación, durante la cual esta última llevará a cabo su acceso al banco de registros de uso general de la CPU para leer el valor actual de alguno de ellos, el cual será actualizado inmediatamente a continuación por la instrucción I_9 en el siguiente ciclo de reloj ya que precisamente éste era el dato que I_8 necesitaba obtener para llevar a cabo su tarea, sin importar su naturaleza particular.

- A diferencia de cualquiera de los riesgos presentados en las figuras anteriores, en este caso la relación entre la dependencia de datos de tipo WAR y la implementación de la segmentación en el cauce del procesador no es tan directa. Aquí el conflicto surge como consecuencia no deseada del reordenamiento de las instrucciones del programa llevado a cabo por el ensamblador para solucionar el desaprovechamiento de los ciclos de reloj de la

instrucción ubicada inmediatamente a continuación de una instrucción de transferencia de control y cuya ejecución será prematuramente cancelada a causa del salto provocado por esta última. Obviamente, en una ejecución secuencial del programa no sería necesario llevar a cabo reordenamiento alguno ya que la naturaleza de la instrucción de salto sería identificada por la unidad de control en la etapa de decodificación con suma anterioridad al inicio de la ejecución de la instrucción siguiente a ella, por lo cual ésta no comenzaría a ejecutarse. Ergo, jamás se cancelaría, y sus ciclos de reloj podrían ser utilizados sin ningún inconveniente para la ejecución de la instrucción destino del salto.

- Con respecto a la duración del atasco, podrá comprobarse en la figura que, para este caso en particular, la demora se prolongará durante exactamente tres ciclos de reloj, ya que éste es el tiempo requerido por la instrucción I_8 para alcanzar y completar su etapa de decodificación, cuyo dato a leer a partir del banco de registros de uso general de la CPU resulta imprescindible para que I_9 pueda proseguir normalmente con su ejecución y sobrescribirlo. Al tratarse de una situación tan particular ya que, debido al reordenamiento de las instrucciones, I_9 se encuentra ubicada en un lugar muy anterior al que originalmente debería, este atasco en el cauce del procesador involucrará únicamente a I_9, mientras que todas las instrucciones "posteriores" a ella en el programa reordenado que se encuentren actualmente en ejecución, I_4 a I_8, proseguirán ejecutándose normalmente. Finalmente, se observará que el atasco está representado en la figura a través de todas las celdas tachadas con una cruz negra, una por cada ciclo de reloj demorado, desde el séptimo al noveno ciclo en el caso de la instrucción I_9.
- Por su parte, las celdas tachadas con una cruz blanca representan un atasco de otra índole, asociado a un conflicto estructural entre las instrucciones I_9 e I_6 debido a que ambas están intentando realizar simultáneamente una escritura sobre el banco de registros de uso general de la CPU. Como se recordará, esta situación ya fue previamente considerada y descripta en la figura 66.

3.1.8.4. Escritura Después de Escritura

Instrucciones	**Progreso de la ejecución del programa**
I_1	
I_2	
I_3	
I_4	

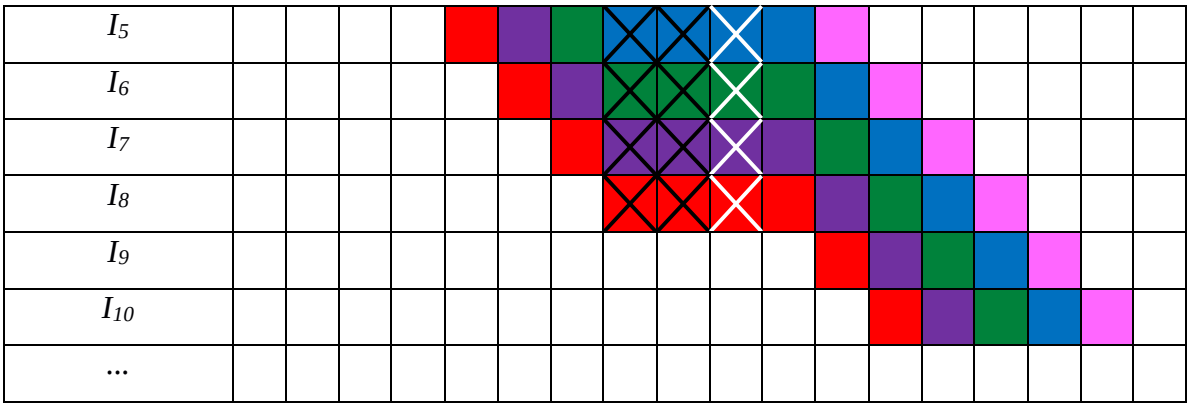


Figura 72. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco en el cauce del nuevo procesador debido a una dependencia de datos de tipo WAW.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ La instrucción I_2 tiene como objetivo para la etapa de almacenamiento en registro de su ciclo de ejecución realizar una operación de escritura sobre uno de los registros de punto flotante de uso general de la CPU.
  - ❖ Asimismo, la instrucción I_3 pretende llevar a cabo durante la etapa de almacenamiento en registro de su propio ciclo de ejecución una operación de escritura sobre el mismo registro que se encuentra involucrado en la actualización a ser realizada por la instrucción I_2.
  - ❖ Si el cauce del procesador prosiguiera con su ejecución de manera normal, I_3 procedería con su escritura dos ciclos de reloj antes de que I_2 pueda efectuar su propia actualización sobre el registro, por lo cual el valor que permanecerá asignado al registro luego de la finalización de ambas instrucciones no sería aquél escrito por I_3, sino el dato actualizado por I_2, ya que ésta estaría siendo la última en acceder a dicho registro para sobrescribirlo.
  - ❖ Sin embargo, si se tiene en cuenta el flujo de ejecución natural de un programa secuencial, al haber comenzado I_3 su ejecución con posterioridad al inicio de la instrucción I_2, el valor que permanezca en el registro de punto flotante de uso general obligatoriamente deberá coincidir con aquél escrito por I_3 y no por I_2 ya que, en caso

contrario, se obtendrá un comportamiento no deseado por parte del programa, el cual a su vez le devolverá al usuario un resultado incorrecto.

- ❖ Entonces, para que la ejecución segmentada del programa presente el mismo comportamiento y ofrezca los mismos resultados que la ejecución secuencial, la instrucción I_3 deberá detenerse temporalmente en su etapa de almacenamiento en registro hasta que la instrucción I_2 haya logrado completar su propia etapa de almacenamiento en registro, durante la cual esta última llevará a cabo su acceso al banco de registros de uso general de la CPU para actualizar alguno de sus registros de punto flotante con un nuevo valor, el cual será sobrescrito inmediatamente a continuación por la instrucción I_3 en el siguiente ciclo de reloj ya que precisamente éste es el dato que necesita que contenga el registro a fin de poder actualizarlo con la plena seguridad de que va a ser su propio valor aquél que permanezca asignado a dicho registro durante el resto del programa una vez finalizada la ejecución de ambas instrucciones, al menos hasta que se alcance otra instrucción posterior a estas dos que realice una nueva modificación sobre él.

- A diferencia de los riesgos estructurales presentados en las figuras anteriores, aquí el tiempo de demora puede variar en función de la ubicación relativa de las instrucciones en conflicto una respecto de la otra dentro del programa: a mayor proximidad entre ellas, más prolongada será la duración del atasco presente en el cauce del procesador. Esto se debe a que la distancia entre los instantes de inicio de la ejecución de ambas instrucciones repercutirá sobre el número de etapas y, en consecuencia, ciclos de reloj, que a la primera instrucción que pretende realizar la escritura sobre el registro conflictivo le restan por completar hasta lograr alcanzar su etapa de almacenamiento en registro, durante la cual actualizará el registro de punto flotante de uso general de la CPU con el dato requerido para ser sobrescrito por la segunda instrucción de escritura, la cual a su vez recién podrá proceder a realizar la nueva actualización en el próximo ciclo de reloj.
- Una vez tenida en cuenta la observación anterior, podrá comprobarse en la figura que, para este caso en particular, la demora se prolongará durante exactamente dos ciclos de reloj, ya que éste es el tiempo requerido por la instrucción I_2 para alcanzar y completar su etapa de almacenamiento en registro, cuyo dato a actualizar en el banco de registros de uso general de la CPU, más precisamente, en uno de sus registros de punto flotante, resulta imprescindible para que I_3 pueda proseguir normalmente con su ejecución y sobrescribirlo con su propio valor. Este atasco en el cauce del procesador involucrará tanto a la instrucción I_3 como a todas las instrucciones posteriores a ella que se encuentren actualmente en

ejecución: I_4, I_5, I_6, I_7 e I_8, y se observará que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada ciclo de reloj demorado, desde el séptimo al noveno ciclo en el caso de la instrucción I_3 y, desde el octavo al noveno ciclo para las instrucciones I_6 e I_7.

- Por su parte, las celdas tachadas con una cruz blanca representan un atasco de otra índole, asociado a un conflicto estructural entre las instrucciones I_3 e I_4 debido a que ambas están intentando realizar simultáneamente una escritura sobre el banco de registros de uso general de la CPU. Como se recordará, esta situación ya fue previamente considerada y descripta en la figura 66.
- Considerando el inciso previo a este último, podrá comprobarse en la figura que aquí se presenta un fenómeno exclusivo de este tipo de atascos:
    - ❖ Para este caso en particular, la demora en la ejecución de la instrucción I_3 se prolongará durante exactamente tres ciclos de reloj, ya que éste es el tiempo requerido por la instrucción I_2 para alcanzar y completar su etapa de almacenamiento en registro, cuyo dato a actualizar en el banco de registros de uso general de la CPU, más precisamente, en uno de sus registros de punto flotante, resulta imprescindible para que I_3 pueda proseguir normalmente con su ejecución y sobrescribirlo con su propio valor.
    - ❖ Este atasco en el cauce del procesador involucrará tanto a la instrucción I_3 como a todas las instrucciones posteriores a ella que se encuentren actualmente en ejecución: I_4, I_5, I_6, I_7 e I_8, y se observará que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada ciclo de reloj demorado, desde el séptimo al noveno ciclo en el caso de la instrucción I_3 y, desde el octavo al noveno ciclo para las instrucciones I_4, I_5, I_6, I_7 e I_8.
    - ❖ No obstante, a diferencia del caso de la instrucción I_3, la duración del retraso en la ejecución de las instrucciones I_4, I_5, I_6, I_7 e I_8 será de dos ciclos de reloj en lugar de tres. Esto se debe al mecanismo optado por el autor de este informe para la implementación de la resolución para este tipo de dependencia de datos en particular.
    - ❖ Para explicar este mecanismo, se podría comenzar por ofrecer un escenario que resulte claro y sencillo para contrastarlo con la situación aquí presente:

- El escenario más recomendable correspondería a aquél provocado como consecuencia de las dependencias de datos de tipo RAW, el cual ya fue ejemplificado en las figuras 67, 68 y 69.
- Si se observa a éstas detenidamente, se comprobará que en ambos escenarios tanto la primera como la segunda instrucción atascadas, I_5 e I_6, se encuentran demoradas en la misma etapa de su respectivo ciclo de ejecución: la etapa de decodificación.
- Por consiguiente, si una vez resuelta la dependencia las dos instrucciones reanudaran su ejecución simultáneamente, entonces ambas intentarían ejecutar al mismo tiempo su etapa demorada.
- Al tratarse esta última de la misma etapa del ciclo de ejecución para las dos instrucciones, se presentaría un nuevo atasco en el cauce del procesador ya que, como se recordará, en una ejecución segmentada jamás pueden múltiples instrucciones ejecutar simultáneamente la misma etapa de su ciclo de ejecución.
- Entonces, la segunda instrucción (I_6) debería detener nuevamente su ejecución durante un ciclo de reloj hasta que la primera instrucción (I_5) complete su propia etapa de decodificación.
- Este atasco en el cauce del procesador involucraría tanto a esta segunda instrucción como a todas las instrucciones posteriores a ella que se encontraran en ejecución en ese momento, únicamente I_7 en los casos de las figuras 67, 68 y 69, ya que se comportaría de una manera muy similar a los atascos estructurales explicados en las figuras 65 y 66.
- Sin embargo, tal como se mencionó anteriormente, en el caso del potencial atasco estructural asociado a las figuras 67, 68 y 69 la etapa en conflicto del ciclo de ejecución de las instrucciones sería la etapa de decodificación, y el recurso de hardware en conflicto, la unidad de control de la CPU, ya que dicha unidad es la encargada de ejecutar esta etapa para cualquier instrucción (ver tabla 46) y, por lo tanto, a ella estarían intentando acceder simultáneamente las dos instrucciones conflictivas (I_5 e I_6).
- En cambio, en los atascos estructurales representados en las figuras 65 y 66, la etapa en conflicto del ciclo de ejecución de las instrucciones es la etapa de acceso a memoria y la de almacenamiento en registro respectivamente, y el recurso en conflicto en cada caso, la memoria de datos de la computadora y

el banco de registros de uso general de la CPU, ya que en cada figura se representa una de las posibles variantes distintas presentadas en esta clase de atasco, tal como fue explicado previamente a la exposición de todas estas figuras.

- A fin de evitar la presencia de este nuevo atasco estructural en la etapa de decodificación durante la ejecución de un caso similar a aquél representado en las figuras 67, 68 y 69, se planteó la necesidad de prolongar la demora en la ejecución tanto de la segunda instrucción atascada (I_6) como de sus instrucciones subsiguientes retrasadas (I_7) hasta un ciclo de reloj después de reanudada la ejecución normal de la primera instrucción demorada (I_5).
- De este modo, esta última (I_5) ya habría finalizado la ejecución de su etapa de decodificación para el momento en el cual la segunda instrucción (I_6) reanudara su ejecución e intentara ejecutar su propia etapa de decodificación.
- No obstante, al haberse suspendido la ejecución de la segunda instrucción (I_6) y sus instrucciones posteriores (I_7) un ciclo de reloj después de la detención en la ejecución de la primera instrucción demorada (I_5), se concluye que el tiempo de duración de la demora es igual para todas las instrucciones involucradas en el atasco, aunque sus instantes de inicio y fin no necesariamente coincidan entre sí para dichas instrucciones.
- Ahora bien, el siguiente paso sería examinar nuevamente los distintos tipos de ciclos de ejecución que pueden presentar las instrucciones del repertorio del procesador (ver tablas 48 y 49).
- Una vez hecho esto, se comprobará que, independientemente de si las instrucciones requieren o no realizar operaciones aritméticas en punto flotante en la FPU de la CPU, su ciclo de ejecución permanecerá idéntico hasta la etapa de decodificación, presentándose una bifurcación recién a partir de la siguiente etapa (ejecución), la cual podrá exhibir diferentes variantes en su duración y su estructura en función de la naturaleza de la instrucción ejecutada.
- Regresando a los casos representados en las figuras 67, 68 y 69, se concluye que, afortunadamente, al tratarse precisamente la etapa de decodificación de aquélla cuya ejecución se encuentra demorada en las primeras dos instrucciones atascadas (I_5 e I_6) existe la certeza absoluta de que, para cualquier escenario posible en el cual se presente una dependencia de datos

de tipo RAW, siempre las dos primeras instrucciones retrasadas estarán demoradas en su etapa de decodificación independientemente de su naturaleza particular, por lo cual de una u otra manera en cualquier escenario se terminará presentando una situación similar a aquélla descripta en los incisos anteriores.

- Asimismo, se han planteado y ejecutado un elevado número de casos de prueba con distintos escenarios en los cuales incluso han llegado a presentarse otros tipos de atascos al mismo tiempo que la dependencia de datos RAW, y en todos ellos se obtuvo la misma conclusión: al provocarse el atasco de tipo RAW en una etapa tan temprana del ciclo de ejecución de las instrucciones, resulta imposible que la presencia de otra clase de atasco al mismo tiempo que esta dependencia de datos altere la secuencia de ejecución establecida y explicada en los incisos anteriores.
- En consecuencia, puede establecerse como conclusión final que, en toda situación en la cual se presente una dependencia de datos RAW, el tiempo de duración de la demora presentada en las instrucciones atascadas siempre será el mismo, aunque los instantes de inicio y fin del retraso no necesariamente coincidan entre sí para todas las instrucciones retrasadas.
- Nótese que, durante la enumeración y descripción de los distintos tipos de riesgos asociados a la implementación de la segmentación en el cauce del procesador, previamente a la exposición de todas estas figuras, en ningún momento se realizó mención alguna a la posible presencia de atascos estructurales debido a intentos de acceso simultáneos por parte de múltiples instrucciones a la unidad de control de la CPU durante la etapa de decodificación de sus respectivos ciclos de ejecución.
- Esto se debe a que, al menos en la práctica, por todas las razones previamente expuestas en los incisos anteriores, jamás se produce este último tipo de atascos estructurales en el cauce del procesador.
- Solamente se incluye aquí una breve explicación de esta clase de atascos porque la misma resulta necesaria para que el lector comprenda la razón por la cual el autor de este informe ha determinado los distintos tiempos de demora para las instrucciones involucradas en atascos debido a dependencias de datos de tipo RAW o WAW según el caso.

❖ Una vez comprendido el escenario de ejecución surgido como consecuencia de la existencia de una dependencia de datos de tipo RAW, se puede proceder a explicar las diferencias que éste posee respecto a la situación provocada a causa de la presencia de una dependencia de datos de tipo WAW:

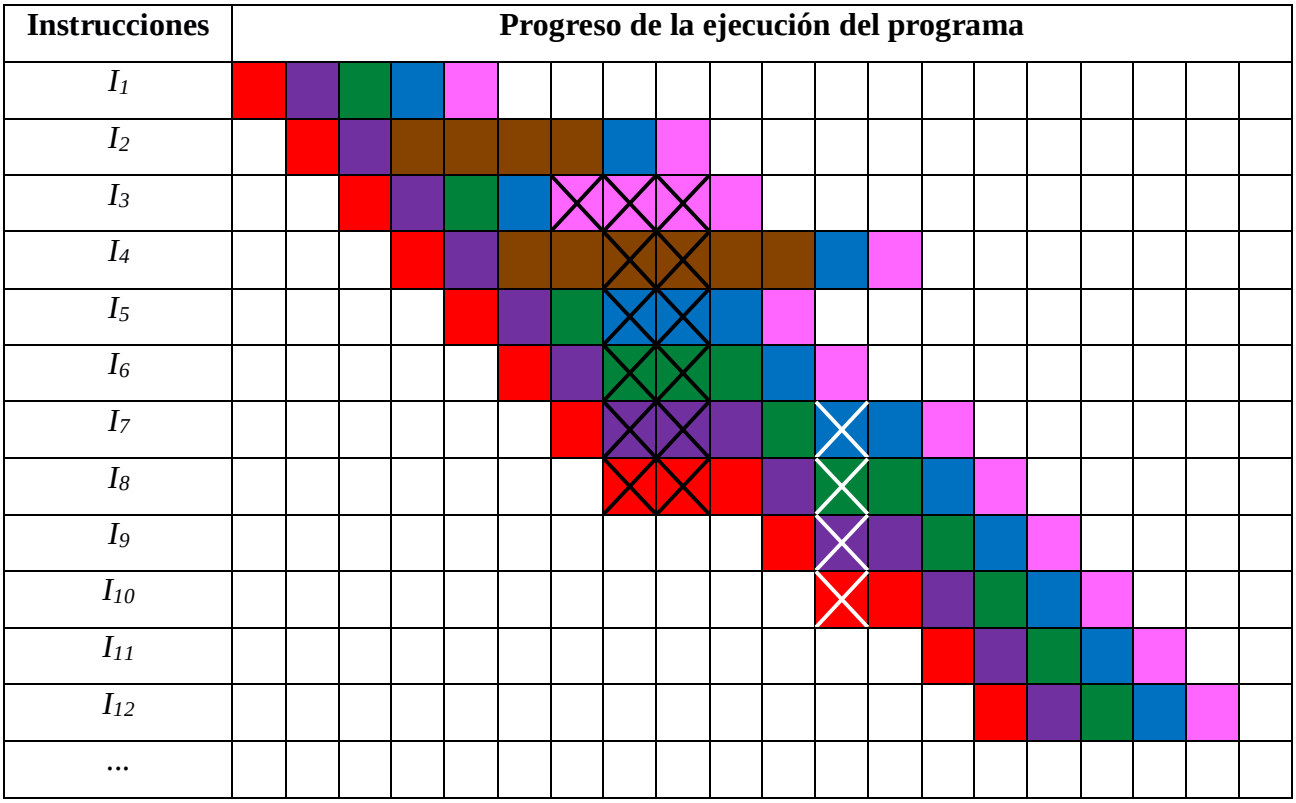


Figura 73. Progreso de la ejecución segmentada de un programa distinto al de la figura anterior, durante la cual también se produce un atasco en el cauce del nuevo procesador debido a una dependencia de datos de tipo WAW.

- Se representan en las dos últimas gráficas anteriores, es decir, las figuras 72 y 73, las ejecuciones de dos programas de prueba durante las cuales surge una dependencia de datos de tipo WAW en el cauce del procesador.
- Cada una de ellas presenta en su ejecución, posteriormente a la resolución de su dependencia y la finalización de su atasco surgido, un desenlace muy diferente, como consecuencia de la distinta naturaleza particular de las primeras dos instrucciones afectadas por la demora.

- Si se observa a ambas figuras detenidamente, se comprobará que, en ambos escenarios, la primera instrucción atascada (I_3) se encuentra demorada en la misma etapa de su respectivo ciclo de ejecución: la etapa de almacenamiento en registro.
- Sin embargo, la situación cambia drásticamente cuando se procede a analizar el caso de la segunda instrucción atascada (I_4): mientras que en la figura 72 dicha instrucción se encuentra demorada en la etapa de almacenamiento en registro de su ciclo de ejecución, en la figura 73 la misma está retrasada en el tercer ciclo de reloj de su etapa de ejecución en punto flotante (tercera subetapa).
- Podrá observarse que en la figura 72 ambas instrucciones (I_3 e I_4) reanudan su ejecución simultáneamente. Al encontrarse atascadas en la misma etapa de su ciclo de ejecución (almacenamiento en registro en este caso), entonces se provoca inmediatamente a continuación de la reanudación un atasco de tipo estructural en el cauce del procesador durante un ciclo de reloj debido a dos intentos de escritura simultáneos sobre el banco de registros de uso general de la CPU.
- Esta situación es idéntica a aquélla representada en la figura 66. De hecho, tanto en ella como en la figura 72 se está analizando la ejecución de exactamente el mismo programa durante la cual, obviamente, se provocan los mismos tipos de atascos en los mismos instantes de tiempo. La única diferencia entre ambas figuras radica en que, mientras que en una de ellas se realiza un mayor hincapié sobre el atasco estructural, en la otra la atención principal se encuentra concentrada en la dependencia de datos de tipo WAW.
- Por su parte, la figura 73 es sólo uno de los múltiples casos de ejecución posibles en los cuales, a diferencia de la figura 72 y de cualquier escenario provocado como consecuencia de una dependencia de datos de tipo RAW (por ejemplo, las figuras 67, 68 y 69), las primeras dos instrucciones atascadas (I_3 e I_4) no se encuentran demoradas en la misma etapa de su ciclo de ejecución.
- Antes de proceder a explicar más detalladamente las consecuencias provocadas por esta diferencia, se puede comenzar planteando una de las pocas similitudes compartidas por las figuras 72 y 73: en ambos casos, las primeras dos instrucciones atascadas (I_3 e I_4) reanudan su ejecución

simultáneamente. Además, en los dos escenarios se presenta un atasco de tipo estructural en el cauce del procesador en algún momento posterior a la reanudación de la ejecución del programa una vez resuelta la dependencia de datos de tipo WAW, el cual provocará una nueva demora de un ciclo de reloj adicional no sólo en la instrucción del programa directamente involucrada en dicha dependencia sino también en todas las instrucciones posteriores a ella que se encuentren en ejecución en ese momento.

- Aquí el fenómeno más interesante e importante para analizar no consiste en el atasco en sí, dado que todas las clases de atascos estructurales ya fueron explicadas y ejemplificadas anteriormente en las figuras 65 y 66, sino en su naturaleza particular, el instante de tiempo en el cual se provoca y en las instrucciones que se encuentran en conflicto.
- En primer lugar, a diferencia del caso de la figura 72, en la figura 73 el recurso de hardware involucrado en el atasco estructural no es el banco de registros de uso general de la CPU sino la memoria de datos de la computadora. En otras palabras, la etapa conflictiva del ciclo de ejecución de las instrucciones no es la de almacenamiento en registro, sino la de acceso a memoria. Esta situación es muy similar a aquélla previamente descripta en la figura 65.
- Con respecto al instante de tiempo en el cual se provoca este atasco estructural, existe el siguiente contraste con respecto al caso de la figura 72: al no encontrarse en la figura 73 las dos primeras instrucciones demoradas debido a la dependencia de datos de tipo WAW (I_3 e I_4) atascadas en la misma etapa de su ciclo de ejecución, no se provocará inmediatamente a continuación de la reanudación de la ejecución del programa ninguna clase de atasco estructural en el cauce del procesador, por lo cual ambas instrucciones podrán continuar normalmente, al menos inicialmente, con su ejecución junto con el resto de las instrucciones posteriores (I_5, I_6, I_7 e I_8) a ellas que se encontraban atascadas.
- No obstante, tres ciclos de reloj después de la finalización del atasco WAW y la consecuente reanudación de la ejecución de las instrucciones demoradas (I_3, I_4, I_5, I_6, I_7 e I_8), se presenta este nuevo atasco estructural en el cauce del procesador debido a dos intentos de acceso simultáneos a la memoria de datos de la computadora.

- Como última diferencia entre ambas figuras, se recordará que, en la figura 72, las instrucciones en conflicto en el atasco estructural allí provocado eran las dos primeras instrucciones que se encontraban demoradas como consecuencia de la dependencia de datos de tipo WAW (I_3 e I_4).
- Esto se debía a que ambas estaban atascadas en la misma etapa de su ciclo de ejecución y, al intentar llevarla a cabo simultáneamente una vez reanudada su ejecución normal, se provocaba el conflicto estructural entre ellas por intentar acceder al mismo tiempo al banco de registros de uso general de la CPU en su etapa de almacenamiento en registro, la etapa previamente demorada.
- Sin embargo, en la figura 73 las instrucciones directamente responsables del atasco son la segunda y la quinta dentro del conjunto de instrucciones que se encontraban demoradas debido a la dependencia de datos de tipo WAW (I_4 e I_7).

❖ A partir de las explicaciones previas, se puede fundamentar sintéticamente el mecanismo optado por el autor de este informe para resolver las dependencias de datos de tipo WAW de la siguiente manera:

- A diferencia de las dependencias de datos de tipo RAW, en los atascos WAW no existen garantías absolutas de que las dos primeras instrucciones atascadas se encuentren demoradas en la misma etapa de su ciclo de ejecución.
- Ergo, no puede asegurarse que siempre vaya a provocarse un atasco estructural inmediatamente a continuación de la resolución de la dependencia de datos y la reanudación de la ejecución de las instrucciones atascadas, con la consecuente demora de un ciclo de reloj adicional hasta que el recurso de hardware en conflicto haya sido liberado por la primera de las dos instrucciones en conflicto, tal como se ha indicado en las figuras 65, 66 y 72.
- Por lo tanto, en el caso de los atascos WAW no sería recomendable replicar la metodología de resolución para las dependencias de RAW de demorar automáticamente un ciclo de reloj adicional la ejecución de todas las instrucciones atascadas excepto la primera de ellas.
- Esto se debe a que, si se presentara una situación similar a aquélla representada en la figura 73 en la cual las dos primeras instrucciones

demoradas no se encontraran obstruidas en la misma etapa de su ciclo de ejecución, entonces no ocurriría atasco estructural alguno: todas las instrucciones demoradas podrían inicialmente reanudar su ejecución normal sin inconvenientes.

- Por lo cual, si se imitara el mecanismo implementado para resolver las dependencias RAW, al imponerse obligatoriamente la demora de un ciclo de reloj adicional previamente explicada, este tiempo estaría siendo desperdiciado innecesariamente para las instrucciones demoradas ya que éstas podrían utilizarlo sin problemas para proseguir ejecutando normalmente una de las etapas de su ciclo de ejecución.
- Entonces, se ha tomado para todos los casos asociados a las dependencias de datos de tipo WAW la determinación de reanudar la ejecución de absolutamente todas las instrucciones demoradas como consecuencia de cada dependencia que se presente durante la ejecución de un programa en forma simultánea e inmediatamente a continuación de la resolución de la dependencia surgida.
- De esta manera, se pueden aprovechar todos los ciclos de reloj para todas las instrucciones demoradas sin asumir equivocadamente de antemano la presencia de ningún atasco estructural que luego no se concrete en tiempo de ejecución.
- Después, durante la ejecución de cada programa en particular se determinarán los pasos a seguir en función del caso en cuestión:
    - ✓ Si es una situación similar a aquélla estudiada en la figura 72, en la cual las dos primeras instrucciones demoradas debido al atasco WAW se encuentran retrasadas en la misma etapa de su ciclo de ejecución, entonces sí se provocará un atasco estructural, semejante pero no idéntico a aquél que ocurriría en cualquier caso asociado a las dependencias de datos RAW si todas las instrucciones atascadas reanudaran su ejecución simultáneamente, tal y como fue explicado en las observaciones anteriores. Dicho atasco detendrá nuevamente la ejecución de las instrucciones afectadas durante un ciclo de reloj inmediatamente a continuación de la resolución de la dependencia WAW.

- ✓ En cambio, si se trata de una situación similar a aquélla analizada en la figura 73, en la cual las dos primeras instrucciones atascadas debido al atasco WAW no se encuentran demoradas en la misma etapa de su ciclo de ejecución, no se provocará ningún atasco estructural inmediatamente a continuación de la resolución de la dependencia WAW. En consecuencia, todas las instrucciones demoradas podrán reanudar su ejecución libre y normalmente hasta que se origine un nuevo atasco, el cual, en caso de efectivamente surgir, se tratará por su parte de una situación absolutamente independiente de la dependencia WAW recientemente solucionada que deberá contar con su propio mecanismo de resolución.

3.1.8.5. Riesgos de control

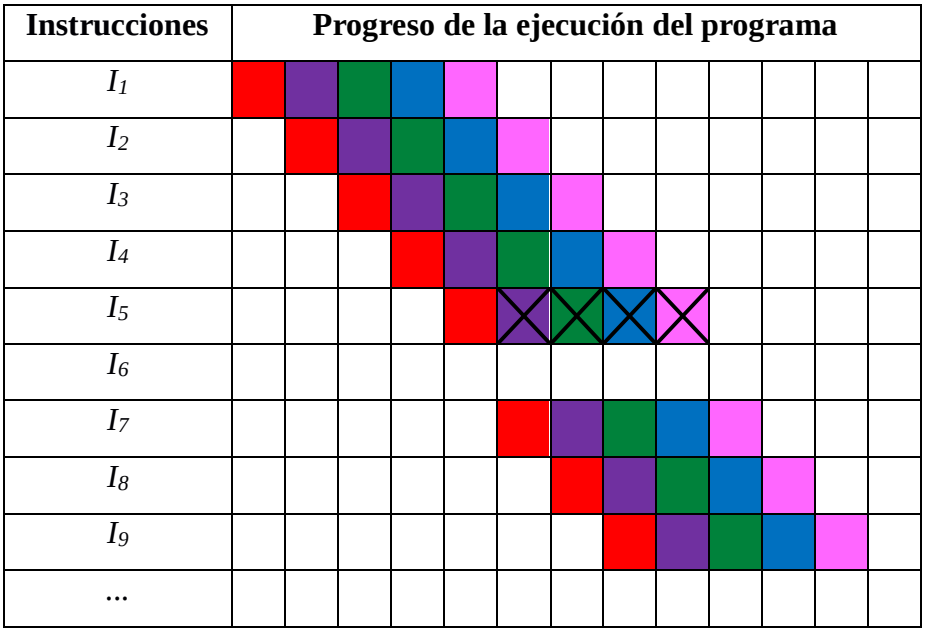


Figura 74. Progreso de la ejecución segmentada de un programa durante la cual se cancela prematuramente la ejecución de una instrucción en el nuevo procesador debido a la presencia de una instrucción de transferencia de control incondicional.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.

- Aquí se plantea la siguiente situación:
    - La instrucción I_4 consiste en una transferencia de control incondicional hacia la instrucción I_7.
    - Entonces, la instrucción I_5 comenzará a ser ejecutada en su etapa de búsqueda hasta que la naturaleza de I_4 sea identificada durante la ejecución de su propia etapa de decodificación por la unidad de control de la CPU.
    - Al encontrarse las etapas de ambas instrucciones ejecutándose simultáneamente, la detección del salto incondicional de I_4 provocará la cancelación inmediata y prematura de I_5 antes de que pueda seguir completando las cuatro etapas restantes de su ciclo de ejecución.
    - De esta manera, la ejecución incompleta de I_5 generará un hueco de retardo disponible en el cauce del procesador para que el ensamblador pueda reubicar en él alguna instrucción anterior o posterior del programa que no posea ninguna relación o dependencia con la instrucción I_4 con el objetivo de que cada ciclo de reloj se encuentre óptimamente utilizado y aprovechado, es decir, que posea todas sus respectivas instrucciones del cauce en ejecución en lugar de demoradas o canceladas.
    - En este caso, como el ensamblador no realizó ningún tipo de reordenamiento en el programa, el hueco de retardo permanece vacío, desaprovechándose una fracción del tiempo de ejecución de los últimos cuatro ciclos de reloj de la instrucción I_5, durante los cuales ésta permanece sin ejecutarse.
    - Se afirma que el desaprovechamiento de estos ciclos de reloj es únicamente parcial y no total porque, a pesar de que la instrucción I_5 no se está ejecutando en ellos por la cancelación precoz, si lo están haciendo el resto de las instrucciones del programa actualmente en ejecución al mismo tiempo que ella: I_2, I_3, I_4, I_7, I_8 e I_9 ejecutan alguna de sus etapas en alguno o varios de los ciclos de reloj que solían pertenecer al ciclo de ejecución de I_5 antes de ser tempranamente cancelado por I_4.
    - Podrá recordarse que, tal como se indica en la tabla 45, tanto la unidad de control como la ALU de la CPU poseen la capacidad de acceder al Puntero de Instrucción (IP) en las etapas de decodificación y ejecución respectivamente a fin de actualizarlo con la dirección en la memoria de instrucciones de la instrucción destino del salto en caso de alterarse el flujo de ejecución normal del programa debido a la ejecución de una instrucción de transferencia de control incondicional o bien el cumplimiento de una condición de salto de una instrucción de transferencia de control condicional.

- ❖ De esta manera, siempre resulta recomendable adelantar la resolución de la condición de salto a la etapa más temprana posible del ciclo de ejecución de la instrucción de transferencia de control.
- ❖ Esto se debe a que cualquier demora adicional indeseada en la evaluación de dicha condición irremediablemente provocará un atasco en el cauce del procesador que afectará a las instrucciones posteriores a la instrucción de salto que se encuentren en ejecución en ese momento. Entonces, a mayor espera, más prolongada será también la duración de este atasco con el consecuente perjuicio sobre el rendimiento de la CPU, como se podrá comprobar en la próxima figura.
- ❖ Por su parte, como ya se ha mencionado previamente, las instrucciones de transferencia de control incondicional se caracterizan por no poseer condición de salto alguna o, en otras palabras, tener una condición que siempre se cumple para todos los casos de ejecución posibles.
- ❖ Por lo tanto, afortunadamente para este tipo de riesgos de control, la evaluación del salto, que en realidad no es tal porque no existe condición alguna para evaluar, puede adelantarse sin inconvenientes hasta la etapa de decodificación de la instrucción de transferencia de control incondicional, durante la cual la unidad de control, al determinar que el salto deberá ser llevado a cabo indefectiblemente, procederá automática e inmediatamente a actualizar el registro IP con la dirección en memoria de la instrucción destino del salto.
- ❖ A pesar de todos los esfuerzos realizados para resolver la condición de salto con la mayor prontitud posible, al quedar absolutamente inutilizado el hueco de retardo asociado a la instrucción cancelada, el tiempo de ejecución total del programa se verá retrasado e incrementado irremediablemente en un ciclo de reloj. Esto cambiaría si se reordenaran las instrucciones de manera tal de asignarle una tarea útil a dicho hueco ocupándolo con alguna instrucción que se ejecute independientemente de la alteración en el flujo de ejecución del programa, tal como se ejemplificó en la figura 71.

- A partir de esta figura se verifica la afirmación expresada durante la explicación general ofrecida para esta clase de riesgos de control antes de la exposición de todas las figuras: en ningún escenario asociado a la presencia de un riesgo de control debido a la ejecución de una instrucción de transferencia de control incondicional puede afirmarse que existan atascos propiamente dichos en el cauce del procesador como consecuencia de la alteración en el flujo de ejecución normal del programa. Esto se debe a que, en realidad, ninguna de

las instrucciones posteriores a la instrucción de salto que se encuentren ejecutándose en ese instante sufrirá demora alguna en su ejecución: únicamente la instrucción ubicada inmediatamente a continuación de ella padecerá secuelas, aunque éstas no estarán vinculadas con un retraso temporal en su ejecución sino directamente con una cancelación prematura y terminante de la misma ya que, una vez efectuado el salto, dejará de pertenecer al flujo de ejecución del programa.

- Una vez tenida en cuenta la observación anterior, podrá comprobarse en la figura que, para este caso en particular, el riesgo de control afectará exactamente a cuatro ciclos de reloj, ya que cada uno de los cuales se corresponde con una de las etapas que le quedaban por completar a la instrucción I_5 de su ciclo de ejecución antes de ser cancelada prematuramente por el salto provocado a causa de la instrucción de transferencia de control incondicional I_4 durante su etapa de decodificación. Esto cambiaría si la instrucción cancelada, I_5 en este caso, requiriera realizar una operación aritmética en punto flotante en la FPU del procesador ya que, como se recordará, su ciclo de ejecución está constituido por un mayor número de etapas que el de una instrucción que no necesita hacer uso de la unidad de punto flotante (ver y comparar las tablas 48 y 49), por lo cual la cantidad de ciclos desaprovechados debido al riesgo de control también será superior a la del escenario representado en esta figura.
- Este riesgo involucrará únicamente a la instrucción I_5, sin afectar en absoluto a las instrucciones posteriores a ella, I_7, I_8 e I_9 en este caso, que ejecuten algunas de sus etapas en los mismos ciclos de reloj parcialmente desaprovechados a causa de la cancelación temprana de I_5, y se observará que está representado en la figura a través de todas las celdas tachadas con una cruz negra, una por cada ciclo de reloj desperdiciado por la instrucción cancelada precozmente: en esta ocasión, desde el sexto al noveno ciclo para la instrucción I_5.

Instrucciones	**Progreso de la ejecución del programa**														
I_1															
I_2															
I_3															

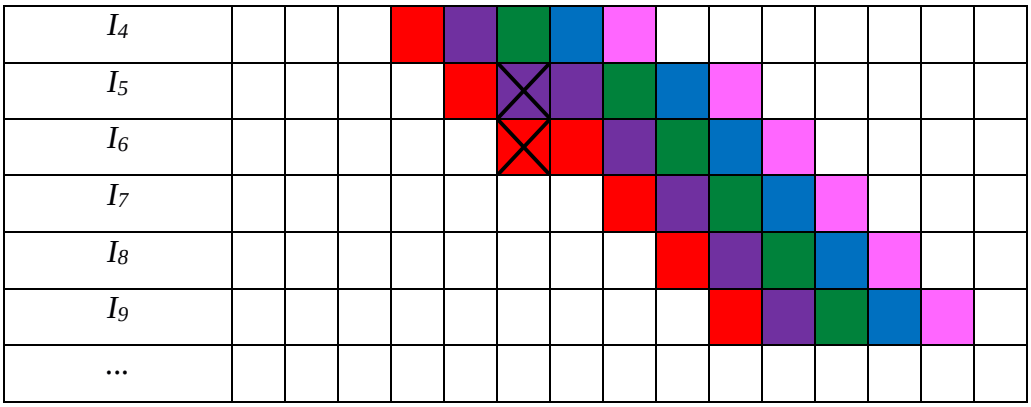


Figura 75. Progreso de la ejecución segmentada de un programa durante la cual se produce un atasco en el cauce del nuevo procesador debido al incumplimiento de la condición de salto de una instrucción de transferencia de control condicional.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ La instrucción I_4 consiste en una transferencia de control condicional hacia la instrucción I_7.
  - ❖ Entonces, la instrucción I_5 comenzará a ser ejecutada en su etapa de búsqueda hasta que la naturaleza de I_4 sea identificada durante la ejecución de su propia etapa de decodificación por la unidad de control de la CPU.
  - ❖ La detección del salto condicional asociado a I_4 provocará prontamente un atasco en el cauce del procesador, el cual demorará la ejecución de la instrucción inmediatamente posterior a la instrucción de salto, I_5 en este caso, en su etapa de decodificación, hasta que la ALU logre evaluar la condición de salto.
  - ❖ Asimismo, la intervención obligatoria de la ALU para comprobar y resolver esta condición implica que esta evaluación recién podrá ser llevada a cabo en la etapa de ejecución de la instrucción de transferencia de control condicional.
  - ❖ Además, este atasco le impedirá a la instrucción siguiente a I_5, es decir, I_6, comenzar la ejecución de su etapa de búsqueda ya que la unidad de control de la CPU actualmente desconoce si ambas instrucciones continuarán perteneciendo al flujo de ejecución del programa una vez evaluada la condición de salto.

- Si se prosiguiera con la ejecución de estas instrucciones, las mismas podrían realizar durante alguna de las etapas de su ciclo de ejecución cambios al estado de los registros de la CPU o la memoria de datos de la computadora que resultarían irreversibles en caso de determinarse que dichas instrucciones no deben formar parte del flujo de ejecución del programa. Por lo tanto, el comportamiento y el resultado ofrecidos por el programa no terminarían siendo los correctos ni los esperados por el usuario.
- En función del resultado de la evaluación, la ALU determinará los próximos pasos a seguir: alterar el flujo de ejecución del programa si se cumple la condición o mantenerlo en caso contrario.
- En el escenario particularmente reflejado en esta figura, la ALU resuelve que no se cumple la condición de salto, por lo cual no se produce modificación alguna en el Puntero de Instrucción (IP) de la CPU ni, obviamente, en el flujo de ejecución del programa: I_5 reanuda normalmente su ejecución demorada en su etapa de decodificación mientras que su instrucción posterior, I_6, inicia su ejecución tardíamente en su etapa de búsqueda. Luego, las demás instrucciones restantes del programa, I_7, I_8 e I_9 en este caso, comenzarán sus respectivas ejecuciones de a una por vez en cada nuevo ciclo de reloj según corresponda, tal como ocurre en cualquier ejecución segmentada de un programa, hasta que acontezca nuevamente, en caso de ser así, cualquiera de los riesgos descriptos en esta subsección del informe.
- Podrá recordarse que, tal como se indica en la tabla 45, tanto la unidad de control como la ALU de la CPU poseen la capacidad de acceder al Puntero de Instrucción (IP) en las etapas de decodificación y ejecución respectivamente de una instrucción de transferencia de control a fin de actualizarlo con la dirección de inicio en la memoria de instrucciones de la instrucción destino del salto en caso de alterarse el flujo de ejecución normal del programa debido a la ejecución de una instrucción de salto incondicional o bien el cumplimiento de una condición de salto de una instrucción de transferencia de control condicional.
- De esta manera, siempre resulta recomendable adelantar la resolución de la condición de salto a la etapa más temprana posible del ciclo de ejecución de la instrucción de transferencia de control.
- Esto se debe a que cualquier demora adicional indeseada en la evaluación de dicha condición irremediablemente provocará un atasco en el cauce del procesador que afectará a las instrucciones posteriores a la instrucción de salto que se encuentren en

ejecución en ese momento. Entonces, a mayor espera, más prolongada será también la duración de este atasco con el consecuente perjuicio sobre el rendimiento de la CPU.

- ❖ Por su parte, como ya se ha mencionado previamente, las instrucciones de transferencia de control condicional poseen una condición de salto que necesariamente deberá ser evaluada ya que el resultado de dicha evaluación repercutirá severamente sobre el flujo de ejecución del programa, más precisamente, sobre las instrucciones que se ejecuten a continuación de la instrucción de salto.
- ❖ Por lo tanto, para este tipo de riesgos de control, la evaluación del salto puede adelantarse sin inconvenientes únicamente hasta la etapa de ejecución de la instrucción de transferencia de control condicional, ya que, tal como lo indica la tabla 46, la ALU, al ser la unidad de la CPU encargada de ejecutar esta etapa, recién aquí podría intervenir para llevar a cabo una operación aritmética cuya complejidad excede ampliamente las prestaciones de cálculo aritmético-lógico ofrecidas por la unidad de control, asignada para gestionar la etapa anterior del ciclo de ejecución (decodificación).
- ❖ Esta operación aritmética será requerida para evaluar la condición de salto indicada en la instrucción de transferencia de control, ya que su resultado servirá para comprobar el cumplimiento de la condición: al determinar que la misma no se cumple en este caso, la ALU no llevará a cabo ninguna otra acción aparte de reanudar la ejecución de la instrucción atascada, I_5 en este caso, y comenzar la ejecución de la próxima instrucción, I_6, que no había podido iniciarse anteriormente a causa de este atasco.

- A diferencia del escenario reflejado en la figura 74 como consecuencia de la ejecución de una instrucción de transferencia de control incondicional, aquí sí existirá un atasco en toda regla una vez detectada la naturaleza de la instrucción de transferencia de control condicional en su etapa de decodificación. Esto se debe a que ahora el salto no podrá ser llevado a cabo inmediatamente en esta misma etapa, sino que deberá proseguirse la ejecución de esta instrucción durante un ciclo de reloj adicional, hasta alcanzar la etapa de ejecución, para que la ALU pueda evaluar la condición de salto, determinar su cumplimiento o incumplimiento y alterar el flujo de ejecución del programa en caso de ser necesario.
- Una vez tenida en cuenta la observación anterior, podrá comprobarse en la figura que, para esta situación en particular, la demora se prolongará durante exactamente el mismo ciclo de

reloj al cual se hizo referencia en el párrafo anterior, ya que éste es el tiempo requerido por la instrucción I_4 para alcanzar y completar su etapa de ejecución, cuya evaluación de su condición de salto por parte de la ALU resulta imprescindible para que I_5 pueda proseguir normalmente con su ejecución o ser cancelada prematura y terminantemente en función de la determinación de la ALU, una vez evaluada su condición, respecto a la permanencia o expulsión de esta instrucción del flujo de ejecución del programa.

- Si se revisan nuevamente en la tabla 45 las tareas a ser ejecutadas en cada una de las etapas del ciclo de ejecución de las instrucciones para este procesador, se notará que, a pesar de que la ALU posee la capacidad de acceder y modificar el registro IP en la etapa de ejecución a fin de alterar el flujo de ejecución del programa en caso de cumplirse la condición de salto de una instrucción de transferencia de control condicional, no ocurre lo mismo con la FPU en la etapa de ejecución de punto flotante. Esto se debe a que, si se observa el repertorio de instrucciones soportado por la CPU, detallado en la tabla 38, se comprobará que ninguna de las condiciones de salto establecidas por las distintas instrucciones de transferencia de control disponibles en el repertorio requiere realizar una operación aritmética en punto flotante para su evaluación y resolución. Por lo tanto, las prestaciones ofrecidas por la ALU resultarán ampliamente satisfactorias para evaluar cualquier condición de salto que se presente durante la ejecución de un programa. De esta manera, se garantiza que la intervención de la FPU jamás será necesaria para determinar el cumplimiento o incumplimiento de una condición de salto y, en consecuencia, nunca existirá un escenario de ejecución en el cual la instrucción de transferencia de control condicional posea en su ciclo de ejecución una etapa de ejecución en punto flotante tal como se indica en la tabla 49 para las instrucciones que requieren utilizar la FPU para realizar una operación aritmética en punto flotante. Por ello, la situación representada en esta figura resulta la única posible para esta clase de riesgos de control, volviéndose innecesaria la tarea de considerar cualquier otra alternativa de ejecución.
- Este atasco en el cauce del procesador involucrará únicamente a la instrucción I_5 porque no existen instrucciones posteriores a ella que se encuentren actualmente en ejecución: técnicamente no puede afirmarse que la instrucción I_6 esté atascada ya que su ejecución, a pesar de encontrarse directamente afectada por el atasco, aún no ha comenzado, sino que simplemente su inicio acontecerá en un instante de tiempo posterior a aquél originalmente anticipado. No obstante, se observará que tanto el atasco propiamente dicho en I_5 como el retraso en el comienzo de la ejecución de I_6 están representados en la figura a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada

ciclo de reloj demorado, tratándose en este caso del sexto ciclo tanto para la instrucción I_5 como para la instrucción I_6.

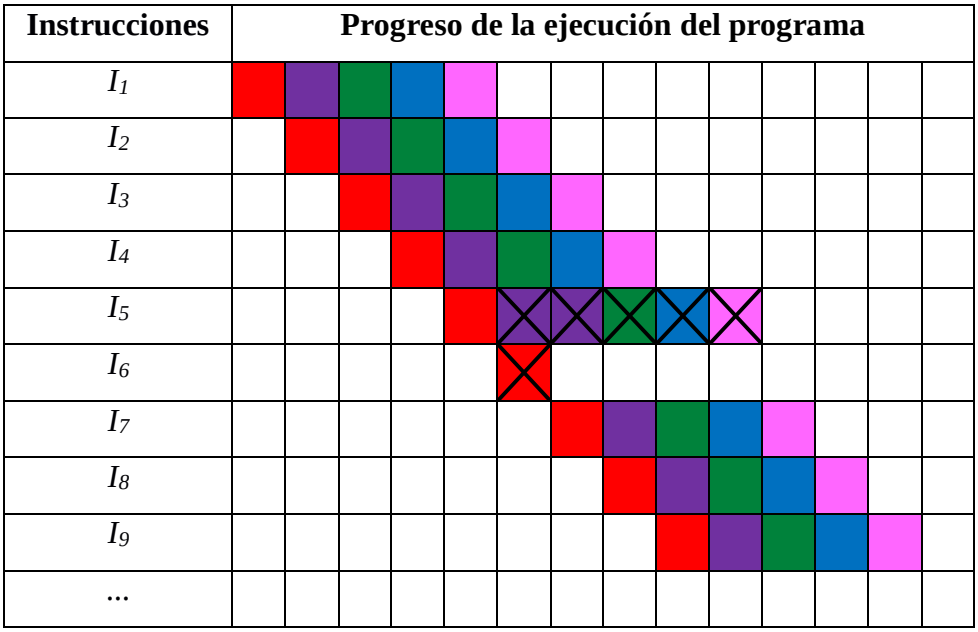


Figura 76. Progreso de la ejecución segmentada de un programa durante la cual se cancela prematuramente la ejecución de una instrucción y se produce un atasco en el cauce del nuevo procesador debido al cumplimiento de la condición de una instrucción de salto condicional.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se plantea la siguiente situación:
  - ❖ La instrucción I_4 consiste en una transferencia de control condicional hacia la instrucción I_7.
  - ❖ Entonces, la instrucción I_5 comenzará a ser ejecutada en su etapa de búsqueda hasta que la naturaleza de I_4 sea identificada durante la ejecución de su propia etapa de decodificación por la unidad de control de la CPU.
  - ❖ La detección del salto condicional asociado a I_4 provocará prontamente un atasco en el cauce del procesador, el cual demorará la ejecución de la instrucción inmediatamente posterior a la instrucción de salto, I_5 en este caso, en su etapa de decodificación, hasta que la ALU logre evaluar la condición de salto.

- Asimismo, la intervención obligatoria de la ALU para comprobar y resolver esta condición implica que esta evaluación recién podrá ser llevada a cabo en la etapa de ejecución de la instrucción de transferencia de control condicional.
- Además, este atasco le impedirá a la instrucción siguiente a I_5, es decir, I_6, comenzar la ejecución de su etapa de búsqueda ya que la unidad de control de la CPU actualmente desconoce si ambas instrucciones continuarán perteneciendo al flujo de ejecución del programa una vez evaluada la condición de salto.
- Si se prosiguiera con la ejecución de estas instrucciones, las mismas podrían realizar durante alguna de las etapas de su ciclo de ejecución cambios al estado de los registros de la CPU o la memoria de datos de la computadora que resultarían irreversibles en caso de determinarse que dichas instrucciones no deben formar parte del flujo de ejecución del programa. Por lo tanto, el comportamiento y el resultado ofrecidos por el programa no terminarían siendo los correctos ni los esperados por el usuario.
- En función del resultado de la evaluación, la ALU determinará los próximos pasos a seguir: alterar el flujo de ejecución del programa si se cumple la condición o mantenerlo en caso contrario.
- En el escenario particularmente reflejado en esta figura, la ALU resuelve que se cumple la condición de salto, por lo cual procederá inmediatamente a modificar el Puntero de Instrucción (IP) de la CPU, lo cual, obviamente, alterará en consecuencia el flujo de ejecución del programa: la instrucción I_5, hasta este momento demorada en su etapa de decodificación, al dejar de formar parte del flujo, será definitivamente cancelada sin posibilidad alguna de que reanude la ejecución de las cuatro etapas pendientes de su ciclo de ejecución. Por su parte, su instrucción posterior, I_6, padecerá un destino similar: el inicio de la ejecución de su etapa de búsqueda, el cual había sido originalmente postergado a causa de este atasco, jamás sucederá, siendo el mismo reemplazado por el comienzo de la ejecución de la instrucción destino del salto, I_7 en este caso, la cual pasará a pertenecer al flujo de ejecución del programa en su lugar. Luego, las demás instrucciones restantes del programa, I_8 e I_9 en este caso, comenzarán sus respectivas ejecuciones de a una por vez en cada nuevo ciclo de reloj según corresponda, tal como ocurre en cualquier ejecución segmentada de un programa, hasta que acontezca nuevamente, en caso de ser así, cualquiera de los riesgos descriptos en esta subsección del informe.

- Podrá recordarse que, tal como se indica en la tabla 45, tanto la unidad de control como la ALU de la CPU poseen la capacidad de acceder al Puntero de Instrucción (IP) en las etapas de decodificación y ejecución respectivamente de una instrucción de transferencia de control a fin de actualizarlo con la dirección de inicio en la memoria de instrucciones de la instrucción destino del salto en caso de alterarse el flujo de ejecución normal del programa debido a la ejecución de una instrucción de salto incondicional o bien el cumplimiento de una condición de salto de una instrucción de transferencia de control condicional.
- De esta manera, siempre resulta recomendable adelantar la resolución de la condición de salto a la etapa más temprana posible del ciclo de ejecución de la instrucción de transferencia de control.
- Esto se debe a que cualquier demora adicional indeseada en la evaluación de dicha condición irremediablemente provocará un atasco en el cauce del procesador que afectará a las instrucciones posteriores a la instrucción de salto que se encuentren en ejecución en ese momento. Entonces, a mayor espera, más prolongada será también la duración de este atasco con el consecuente perjuicio sobre el rendimiento de la CPU.
- Por su parte, como ya se ha mencionado previamente, las instrucciones de transferencia de control condicional poseen una condición de salto que necesariamente deberá ser evaluada ya que el resultado de dicha evaluación repercutirá severamente sobre el flujo de ejecución del programa, más precisamente, sobre las instrucciones que se ejecuten a continuación de la instrucción de salto.
- Por lo tanto, para este tipo de riesgos de control, la evaluación del salto puede adelantarse sin inconvenientes únicamente hasta la etapa de ejecución de la instrucción de transferencia de control condicional, ya que, tal como lo indica la tabla 46, la ALU, al ser la unidad de la CPU encargada de ejecutar esta etapa, recién aquí podría intervenir para llevar a cabo una operación aritmética cuya complejidad excede ampliamente las prestaciones de cálculo aritmético-lógico ofrecidas por la unidad de control, asignada para gestionar la etapa anterior del ciclo de ejecución (decodificación).
- Esta operación aritmética será requerida para evaluar la condición de salto indicada en la instrucción de transferencia de control, ya que su resultado servirá para comprobar el cumplimiento de la condición: al determinar que la misma se cumple en este caso, la ALU se encargará asimismo de actualizar el registro IP para que

apunte a la instrucción destino del salto, I_7 en este caso, ya que a partir de este momento será la próxima ejecución en ejecutarse, y cancelar la ejecución de la instrucción atascada, I_5. I_7 empezará a ejecutarse a partir del instante de tiempo que originalmente le correspondía al inicio de la instrucción I_6, cuyo comienzo, inicialmente pospuesto a causa de este atasco, ahora jamás ocurrirá por haberse determinado que no formará parte del flujo de ejecución del programa.

- A diferencia del escenario reflejado en la figura 74 como consecuencia de la ejecución de una instrucción de transferencia de control incondicional, aquí sí existirá un atasco en toda regla una vez detectada la naturaleza de la instrucción de transferencia de control condicional en su etapa de decodificación. Esto se debe a que ahora el salto no podrá ser llevado a cabo inmediatamente en esta misma etapa, sino que deberá proseguirse la ejecución de esta instrucción durante un ciclo de reloj adicional, hasta alcanzar la etapa de ejecución, para que la ALU pueda evaluar la condición de salto, determinar su cumplimiento o incumplimiento y alterar el flujo de ejecución del programa en caso de ser necesario.
- Una vez tenida en cuenta la observación anterior, podrá comprobarse en la figura que, para esta situación en particular, la demora se prolongará durante exactamente el mismo ciclo de reloj al cual se hizo referencia en el párrafo previo, ya que éste es el tiempo requerido por la instrucción I_4 para alcanzar y completar su etapa de ejecución, cuya evaluación de su condición de salto por parte de la ALU resulta imprescindible para que I_5 pueda proseguir normalmente con su ejecución o ser cancelada prematura y terminantemente en función de la determinación de la ALU, una vez evaluada su condición, respecto a la permanencia o expulsión de esta instrucción del flujo de ejecución del programa.
- No obstante, existe otro fenómeno manifestado en este escenario que es muy similar al presentado en la situación exhibida en la figura 74 como consecuencia de la ejecución de una instrucción de transferencia de control incondicional: la ejecución de la instrucción inmediatamente posterior a la instrucción de salto condicional, I_5 en este caso, es prematuramente cancelada, esta vez por la ALU en lugar de la unidad de control de la CPU, en su etapa de decodificación antes de que pueda completar tanto ésta como las tres etapas restantes de su ciclo de ejecución. Cada una de estas etapas canceladas constituirá uno de los ciclos de reloj parcialmente desaprovechados debido a su inutilización por parte de esta instrucción incompleta. En esta oportunidad, la cancelación será llevada a cabo inmediatamente a continuación de la resolución del atasco, ya que precisamente durante el

transcurso de este último la ALU determinó la expulsión de I_5 del flujo de ejecución del programa a causa del cumplimiento de la condición de salto.

- La conclusión más evidente que puede obtenerse a partir de la observación anterior es que, dentro de las posibles alternativas de ejecución que pueden desarrollarse ante la presencia de un riesgo de control, ésta tendrá la repercusión más negativa de todas sobre el rendimiento del procesador. Esta afirmación se fundamenta en el descubrimiento de que en realidad esta situación consiste simplemente en una combinación de las dos modalidades anteriormente expuestas en las figuras 74 y 75 en la cual se acumulan los inconvenientes de ambas entre sí.
- Esto se comprueba al intentar determinar el incremento adicional en el tiempo de ejecución total del programa en caso de cumplirse la condición de salto de una instrucción de transferencia de control condicional. El mismo será de dos ciclos de reloj respecto a un caso de ejecución segmentada ideal en la cual no ocurra atasco alguno: el primer ciclo añadido se deberá al hueco de retardo inutilizado y vacío asociado a la instrucción cancelada prematuramente debido a la alteración provocada en el flujo de ejecución del programa, el cual ya fue explicado para el caso de las instrucciones de transferencia de control incondicional en la figura 74. Por su parte, el segundo ciclo estará vinculado a la duración del atasco surgido en el cauce del procesador a causa de la demora necesaria para evaluar la condición de salto de la instrucción, tal como se describió para el caso representado en la figura 75, en el cual no se cumplía la condición de la instrucción de transferencia de control condicional, pero el atasco se presentaba de todos modos, igual que aquí.
- Si se revisan nuevamente en la tabla 45 las tareas a ser ejecutadas en cada una de las etapas del ciclo de ejecución de las instrucciones para este procesador, se notará que, a pesar de que la ALU posee la capacidad de acceder y modificar el registro IP en la etapa de ejecución a fin de alterar el flujo de ejecución del programa en caso de cumplirse la condición de salto de una instrucción de transferencia de control condicional, no ocurre lo mismo con la FPU en la etapa de ejecución de punto flotante. Esto se debe a que, si se observa el repertorio de instrucciones soportado por la CPU, detallado en la tabla 38, se comprobará que ninguna de las condiciones de salto establecidas por las distintas instrucciones de transferencia de control disponibles en el repertorio requiere realizar una operación aritmética en punto flotante para su evaluación y resolución. Por lo tanto, las prestaciones ofrecidas por la ALU resultarán ampliamente satisfactorias para evaluar cualquier condición de salto que se presente durante la ejecución de un programa. De esta manera, se garantiza que la intervención de la FPU jamás será necesaria para determinar el cumplimiento o

incumplimiento de una condición de salto y, en consecuencia, nunca existirá un escenario de ejecución en el cual la instrucción de transferencia de control condicional posea en su ciclo de ejecución una etapa de ejecución en punto flotante tal como se indica en la tabla 49 para las instrucciones que requieren utilizar la FPU para realizar una operación aritmética en punto flotante. Por ello, la situación representada en esta figura resulta la única posible para esta clase de riesgos de control, volviéndose innecesaria la tarea de considerar cualquier otra alternativa de ejecución.

- Este atasco en el cauce del procesador involucrará únicamente a la instrucción I_5 porque no existen instrucciones posteriores a ella que se encuentren actualmente en ejecución: técnicamente no puede afirmarse que la instrucción I_6 esté atascada ya que su ejecución, a pesar de encontrarse directamente afectada por el atasco, aún no ha comenzado, sino que simplemente su inicio acontecerá en un instante de tiempo posterior a aquél originalmente anticipado.
- De igual manera, no puede aseverarse que la instrucción I_6 haya sido cancelada porque en realidad nunca comenzó su ejecución: esta última simplemente fue reemplazada por la instrucción que pasó a pertenecer al flujo de ejecución del programa en su lugar, es decir, la instrucción destino del salto, I_7 en este caso. A diferencia de la instrucción I_5, la determinación de la ALU de no ejecutar I_6 no tendrá repercusiones de ningún tipo sobre el rendimiento del procesador.
- Podrá observarse que tanto el atasco y la posterior cancelación de I_5 como el retraso en el comienzo de la ejecución de I_6, a pesar de tratarse cada una de una situación diferente, al encontrarse todas ellas directa e igualmente vinculadas con el riesgo de control ocurrido como consecuencia de la ejecución de una instrucción condicional cuya condición de salto se cumple, están representadas en la figura a través de todas las celdas tachadas con una cruz negra, una para cada instrucción afectada y por cada ciclo de reloj demorado, tratándose en este caso del sexto ciclo y desde el séptimo hasta el décimo ciclo para el atasco y la cancelación respectivamente de la instrucción I_5 y del sexto ciclo para la demora en el inicio de la ejecución de la instrucción I_6.

3.1.8.6. Atascos de fin del programa

Instrucciones	**Progreso de la ejecución del programa**											
I_1												
I_2												

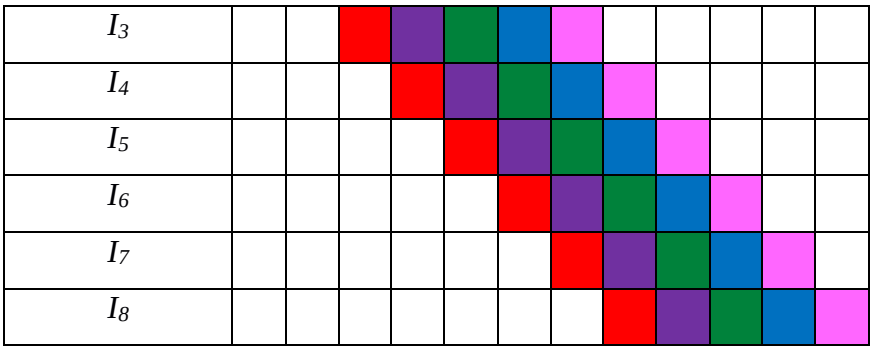


Figura 77. Progreso de la ejecución segmentada de un programa cuya finalización es correcta, en un escenario con condiciones normales sin ningún tipo de inconvenientes.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones de las instrucciones ejecutadas del caso de prueba, representa el transcurso de un ciclo de reloj.
- Aquí se presenta la siguiente situación:
  - I_8 es una instrucción "halt". Si no se recuerda su propósito, podrá revisarse el repertorio de instrucciones de la CPU definido en la tabla 38, según el cual ésta se trata de una instrucción que se escribe y ejecuta por única vez al final del programa para lograr la detención efectiva del procesador.
  - Su ciclo de ejecución es idéntico al de cualquier otra instrucción que no requiere utilizar la FPU de la CPU para realizar una operación aritmética en punto flotante: cinco etapas cuya ejecución se llevará a cabo durante el transcurso de un ciclo de reloj para cada una (ver tabla 48).
  - Se debe prestar una especial atención a la etapa de decodificación de esta instrucción, ya que durante la misma la unidad de control, encargada de su ejecución, detectará su naturaleza e inmediatamente procederá aquí mismo a emitir una orden de detención a todas las unidades restantes del procesador que se ocupan de ejecutar las demás etapas correspondientes tanto a este tipo de ciclo de ejecución como a aquél perteneciente a las instrucciones que necesitan llevar a cabo una operación aritmética en punto flotante en la FPU.
  - En síntesis, a partir de la información proporcionada por la tabla 46, puede afirmarse que todas las siguientes unidades de ejecución de la CPU recibirán simultáneamente desde la unidad de control la orden de detención: la unidad de búsqueda (etapa de búsqueda), la ALU o unidad aritmético-lógica (etapa de ejecución), la FPU o unidad

de punto flotante (etapa de ejecución en punto flotante), la unidad de acceso a memoria (etapa de acceso a memoria) y la unidad de almacenamiento en registro (etapa de almacenamiento en registro).

- Aunque pueda resultar evidente al lector, sería oportuno aclarar aquí que la misma unidad de control se verá igualmente afectada por esta orden transmitida, procediendo a detenerse inmediatamente a continuación de la emisión de esta última, es decir, una vez finalizada la ejecución de la etapa de decodificación de la instrucción "halt", I_8 en esta figura, durante el ciclo de reloj actual. Esta prontitud en la detención se debe a que, al conocerse exactamente la naturaleza de esta instrucción, se asume que, luego del comienzo de su ejecución, en caso de no ocurrir ningún fenómeno, asociado a la presencia de una instrucción de transferencia de control incondicional o condicional, que altere el flujo de ejecución del programa (ver figuras 74 y 76), no debería quedar ninguna instrucción por iniciarse. De hecho, en el supuesto de que la etapa de decodificación de la instrucción "halt" empiece su ejecución efectiva, es decir, sin interrupción alguna a causa de la existencia de un atasco provocado por algún conflicto con una instrucción anterior, se pueden extraer ciertas observaciones extremadamente relevantes.
- En primer lugar, examinando en las figuras 74 y 76 las distintas alternativas que podrían alterar el flujo de ejecución normal del programa, cancelando prematuramente, en este caso, la instrucción "halt", se concluye fácilmente que el caso en el que la ejecución del programa se encuentre en un estado más avanzado correspondería a aquél en el cual la instrucción inmediatamente anterior a "halt" se trate de una transferencia de control condicional cuya condición de salto se cumpla (ver figura 76). Si esto sucediera, este cumplimiento sería detectado por la ALU en su etapa de ejecución, procediendo en consecuencia a modificar ciertamente el flujo original del programa a través de la actualización del registro IP de la CPU con la dirección de inicio de la instrucción destino del salto, dado que ésta ocuparía la primera ubicación en el nuevo flujo. Por lo tanto, dicha instrucción comenzará a efectuarse prontamente en el próximo ciclo de reloj, mientras que la decodificación demorada de "halt", al pertenecer ahora esta última instrucción al flujo previamente descartado, en esta ocasión no se llevará a cabo, anulándose definitivamente la ejecución tanto de ésta como de todas las etapas restantes de la instrucción "halt" que aún se encontraban pendientes.

- Ergo, la ejecución efectiva de la etapa de decodificación de la instrucción “halt” asegura por sí sola la inexistencia de futuros atascos del programa que cuenten con la capacidad de alterar el flujo de ejecución del programa, es decir, aquéllos que surjan en el cauce como consecuencia de los riesgos de control presentados en la subsección anterior (“3.1.8.5. Riesgos de control”). Por lo tanto, en esta instancia la certeza ya es absoluta: al compartir todos los distintos tipos posibles de ciclos de ejecución de las instrucciones la misma estructura interna hasta su etapa de decodificación (comprobar y comparar figuras 48 y 49), esta instrucción “halt” será la última del programa en decodificarse. Por consiguiente, se garantiza que la unidad de control, encargada exclusivamente de efectuar esta tarea, ya no será necesaria durante el resto de la ejecución del programa y podrá ser desactivada sin inconvenientes.
- Por su parte, se considera que cualquiera de los otros atascos ya analizados en las subsecciones anteriores no representaría inconveniente alguno en esta situación en particular. Esto se debe a que su aparición, si ocurriera, simplemente retrasaría la finalización del ciclo de ejecución de la instrucción “halt”, pero en ningún caso la terminaría tempranamente ni la expulsaría del flujo del programa, con el peligro que implicaría una transmisión indeseada de la orden de detención del procesador por parte de la unidad de control ya que correspondería a una instrucción que jamás debería haberse comenzado a ejecutar y que, por lo tanto, no tendría que repercutir de ningún modo sobre el estado de las unidades de ejecución de la CPU. Próximamente se explicará en esta misma subsección, “5.1.8.6. Atascos de fin del programa”, el mecanismo desarrollado para garantizar la ausencia de nuevos riesgos de cualquier índole en el cauce del procesador una vez finalizada la ejecución efectiva de la etapa de decodificación de la instrucción “halt”, así como también las razones que motivaron al autor del informe a diseñarlo, implementarlo e incorporarlo en el TDA 1819.
- Ahora bien, a pesar de que el resto de las unidades de ejecución de la CPU reciben la orden de detención simultáneamente, cada una de ellas la acatará y ejecutará en forma efectiva en diferentes instantes de tiempo, según la ubicación de su etapa gestionada en el ciclo de ejecución de las instrucciones. Este mecanismo funciona de la siguiente manera:
  - Al tratarse la instrucción “halt”, I_8 en este caso, de la última del programa, y al encontrarse ésta ya ejecutando su etapa de decodificación, puede

garantizarse, al igual que en el caso de la unidad de control y su respectiva etapa administrada y asociada del ciclo de las instrucciones (decodificación), que bajo ninguna circunstancia existirá una nueva ejecución de la etapa de búsqueda durante el resto del programa. Por lo tanto, la unidad de búsqueda puede detenerse inmediatamente una vez recibida la orden de alto ya que jamás volverá a ser requerida por ninguna de las instrucciones cuya ejecución todavía se encuentre en curso. En otras palabras, tanto la unidad de búsqueda como la de control se detendrán al mismo tiempo, al finalizar el ciclo de reloj durante el cual se efectúa la decodificación de I_8 y se transmite la orden de parada.

- Una situación similar ocurre con la FPU ya que, antes de emitir la orden de detención, la unidad de control comprueba que todas las instrucciones actualmente en ejecución previas a la instrucción "halt", I_5, I_6 e I_7 en esta figura, hayan completado sus ciclos de ejecución al menos hasta que a lo sumo una sola de ellas se encuentre llevando a cabo actualmente su etapa de ejecución o el último ciclo o subetapa de su etapa de ejecución en punto flotante según el tipo de instrucción (ver tablas 48 y 49). Asimismo, la disposición interna de la estructura del ciclo de ejecución propio de la instrucción "halt" indica que la misma no requiere hacer uso alguno de la FPU (observar tabla 56). Por lo tanto, puede garantizarse que esta última no volverá a ser utilizada durante el resto de la ejecución del programa por ninguna otra instrucción una vez finalizado el ciclo de reloj en el cual se transmite la orden de alto. Ergo, esta unidad podrá detenerse sin inconvenientes junto con las unidades de búsqueda y de control de la CPU al terminar dicho ciclo de reloj.
- Con respecto a la ALU, cabría esperarse, a partir de la mención que se hizo en la observación anterior acerca de la comprobación de la unidad de control de la CPU sobre el estado de ejecución de todas las instrucciones actualmente activas previamente a la emisión de la orden de detención del procesador, que la única etapa de ejecución pendiente para completar antes de finalizar el programa se trate de aquélla perteneciente a la misma instrucción "halt".
- Como consecuencia de esto, la ALU, encargada de ejecutar esta etapa, debería detenerse al finalizar el ciclo de reloj posterior a aquél durante el

cual fue emitida la orden de parada por parte de la unidad de control ya que en ese momento ya se habría acabado de completar la etapa de ejecución de la instrucción “halt”, la cual sería la última del programa.

- Sin embargo, como se habrá adivinado a este punto, la única tarea útil de la instrucción “halt” es llevada a cabo durante su etapa de decodificación, siendo emitida aquí la orden de detención para todas las unidades de ejecución del procesador: el resto de las etapas del ciclo de ejecución no poseen ninguna utilidad para esta instrucción ni se lleva a cabo acción alguna en ellas.
- Por lo tanto, se ha optado por adelantar este suceso de manera tal que la ALU se detenga simultáneamente junto a las unidades de búsqueda y de control y la FPU del procesador, es decir, al término del ciclo de reloj durante el cual fue emitida la orden de detención, en la etapa de decodificación de la instrucción “halt” (I_8). Aunque técnicamente estaría quedando una etapa de ejecución pendiente por completar antes de finalizar el programa, ésta sería aquélla correspondiente a la misma instrucción “halt” y, tal como se mencionó en la observación anterior, durante ella no se lleva a cabo tarea alguna, por lo cual, en términos estrictos, la ALU ya no necesitaría encontrarse activa durante su ejecución.
- Por su parte, la parada de la unidad de acceso a memoria se gestiona de una manera similar a la detención de la ALU. Se recuerda que, si la unidad de control procede con la transmisión de la orden de alto, se debe a la certeza absoluta de que a lo sumo una instrucción del programa está ejecutando su etapa de ejecución o el último ciclo de su etapa de punto flotante durante el ciclo de reloj actual. En consecuencia, puede deducirse, conociendo que todas las instrucciones, independientemente de su tipo, comparten exactamente las mismas dos etapas para su ciclo de ejecución tras la conclusión de su etapa de ejecución (revisar tablas 48 y 49), que habrá a lo sumo dos instrucciones cuyas etapas de acceso a memoria todavía se encuentren pendientes de ejecución una vez transmitida la orden de parada hasta la detención efectiva de todas las unidades de ejecución del procesador y la finalización definitiva del programa: la primera, en caso de existir, sería la instrucción que esté actualmente completando su etapa de ejecución, ya sea en la ALU o la FPU (última subetapa de su ejecución en punto flotante),

y la segunda, siempre presente, se trataría de la propia instrucción “halt”, encargada de detener la ejecución del procesador asignando en su etapa de decodificación a la unidad de control la tarea de emitir la orden de detención para todas las unidades de ejecución de la CPU.

- La etapa de acceso a memoria de la primera de estas dos instrucciones, cuya existencia, como se mencionó, puede no ocurrir, se ejecutará indefectiblemente a continuación del ciclo de decodificación de la instrucción “halt” durante la cual se emite la orden de detención para todas las unidades de ejecución del procesador. Recordar que, independientemente de si la etapa de ejecución de la instrucción actual haya involucrado o no el uso de la FPU, siempre después de ella en su ciclo de ejecución se encontrará la etapa de acceso a memoria. Por ejemplo, precisamente en esta figura la instrucción I_7, inmediatamente anterior a la instrucción “halt”, I_8, está completando su etapa de ejecución mientras la unidad de control emite la orden de detención durante la etapa de decodificación de I_8. Entonces, en el ciclo de reloj siguiente se llevará a cabo la etapa de acceso a memoria de I_7, la cual, en caso de tratarse de una instrucción de transferencia de datos desde o hacia la memoria de la computadora, es decir, una carga (“load”) o un almacenamiento (“store”), requerirá obligatoriamente de la gestión de la unidad de acceso a memoria para, valga la redundancia, acceder a la memoria de datos ya sea para realizar una operación de lectura o escritura según la naturaleza de la instrucción.
- ¿LA ETAPA DE ACCESO A MEMORIA INVOLUCRA SOLO A LA MEMORIA DE DATOS O TAMBIÉN A LA PILA Y LOS PERIFÉRCOS DE ENTRADA-SALIDA E INSTRUCCIONES? CORREGIR ACÁ Y REVISAR TABLAS 42, 45, 47 Y TAL VEZ ALGUNA OTRA. REVISAR SUBSECCIÓN 3.1.7. IDENTIFICAR CORRECTAMENTE SECCIONES DE MEMORIA QUE PUEDEN CONTENER INSTRUCCIONES (INSTRUCCIONES Y SUBRUTINAS) Y DATOS (¿INTERRUPCIONES Y PERIFÉRICOS DE ENTRADA-SALIDA?, DATOS Y PILA).
- Por su parte, la etapa de acceso a memoria de la segunda instrucción (“halt”) se completará con una posterioridad de dos ciclos de reloj respecto a la emisión de la orden de detención, es decir, inmediatamente después de la finalización de la primera etapa de acceso a memoria pendiente, si es que

esta última hubiera existido. Recordar que esta orden se estaría transmitiendo durante la etapa de decodificación de la instrucción "halt", por lo cual, conociendo la estructura de su ciclo de ejecución, aún quedarían dos etapas para poder alcanzar el acceso a memoria pendiente. Nuevamente, si se observa esta figura, se comprobará que la instrucción "halt" del programa, I_8, ejecuta su etapa de acceso a memoria inmediatamente a continuación de la finalización de esta misma etapa para su instrucción anterior, I_7, y dos ciclos de reloj después de la ejecución de la etapa de decodificación de la misma instrucción I_8 durante la cual se emitió la orden de detención. I_8 efectuó su etapa de ejecución entre la decodificación y el acceso a memoria, tal y como lo dispone su ciclo de ejecución (ver tabla 48).

- Sin embargo, se hizo referencia previamente al concepto de que en realidad no se lleva a cabo tarea alguna en la etapa de acceso a memoria de la instrucción "halt", por lo cual podría detenerse anticipadamente la unidad de acceso a memoria, tal como se hizo con la ALU, ya que no será necesaria para gestionar la ejecución de esta etapa pendiente.
- Aun así, no puede afirmarse lo mismo respecto a la primera etapa: al desconocerse inicialmente la naturaleza de la instrucción a cuyo ciclo de ejecución pertenece previo a su decodificación, la unidad de acceso a memoria obligatoriamente deberá permanecer activa durante la ejecución de esta etapa, ya que su presencia podría ser requerida para llevar a cabo alguna tarea útil en ella.
- Entonces, esta unidad de ejecución de la CPU recién podrá ser detenida al finalizar el ciclo de reloj posterior a aquél durante el cual fue emitida la orden de detención: así se garantiza que dicha unidad permanezca activa durante la etapa de acceso a memoria, si es que la hubo, de la instrucción que se encontraba en ese momento ejecutando su etapa de ejecución o el último ciclo de su ejecución en punto flotante. Un ciclo de reloj después de la desactivación de la unidad de acceso a memoria se efectuará la etapa de acceso a memoria aún pendiente de la instrucción "halt", pero al no llevarse a cabo ninguna tarea útil en ella, no existiría ningún inconveniente si aquí la unidad de acceso a memoria ya se encontrara inactiva.
- Finalmente, considerando una vez más la comprobación realizada por la unidad de control de la CPU antes de emitir la orden de detención para todas

las unidades de ejecución del procesador, y teniendo en cuenta que cualquier tipo de ciclo de ejecución definido para las instrucciones se encuentra conformado por las mismas etapas, acceso a memoria y almacenamiento en registro, una vez completada su etapa de ejecución, haya sido esta última o no en punto flotante (revisar tablas 48 y 49), puede concluirse que existirán, a lo sumo, tres instrucciones cuya etapa de almacenamiento en registro se encuentre pendiente de ejecución una vez transmitida la orden de parada hasta la detención efectiva del procesador y la finalización definitiva del programa: la primera de todas estaría completando su etapa de acceso a memoria simultáneamente junto a la transmisión de la orden de alto por parte de la unidad de control, la segunda, finalizando en este mismo momento su etapa de ejecución o el último ciclo de su ejecución en punto flotante según corresponda en función de su naturaleza, y la tercera y última, sería la propia instrucción "halt", encargada de detener la ejecución del procesador asignando en su etapa de decodificación a la unidad de control la tarea de emitir la orden de detención para todas las unidades de ejecución de la CPU.

- Como se explicó, la primera de estas tres instrucciones, cuya existencia puede ocurrir o no según el estado de la ejecución de las instrucciones activas en el procesador en el instante durante el cual es emitida la orden de detención, sería aquélla que se encuentre completando su etapa de almacenamiento en memoria en el ciclo de reloj de la transmisión de la orden de parada. Entonces, prosiguiendo con su ciclo de ejecución normal, continuaría con la realización de su etapa de almacenamiento de registro en el siguiente ciclo de reloj, finalizando aquí la totalidad de su ejecución. Precisamente en esta misma figura la instrucción I_6, ubicada dos posiciones antes de la instrucción I_8 ("halt"), se encuentra asociada a esta situación: podrá observarse que ejecuta su etapa de acceso a memoria simultáneamente junto a la etapa de decodificación de la instrucción "halt", en la cual la unidad de control emite la orden de detención. Luego, lleva a cabo su etapa de almacenamiento en registro un ciclo de reloj después, finalizando su ciclo de ejecución y completando la primera de las etapas de almacenamiento en registro que se encontraban pendientes luego de la transmisión de la orden de alto para que todas las unidades de ejecución del procesador puedan proceder a detenerse definitivamente.

- Por otra parte, la segunda instrucción, cuya presencia, de una manera similar a la primera, puede o no suceder según el caso de ejecución en cuestión, llevaría a cabo su etapa de ejecución o la última subetapa de su ejecución en punto flotante según el tipo de instrucción, al mismo tiempo que la transmisión de la orden de detención. Entonces, prosiguiendo con su ciclo de ejecución normal, independientemente de si la instrucción involucró o no el uso de la FPU, completaría su etapa de almacenamiento de registro dos ciclos de reloj después, finalizando aquí la totalidad de su ejecución. Puede apreciarse un ejemplo en esta misma figura: la instrucción I_7, inmediatamente anterior a la instrucción I_8 ("halt"), completa su etapa de ejecución simultáneamente junto a la etapa de decodificación de I_8, durante la cual la unidad de control emite la orden de detención a todas las unidades de ejecución del procesador. En el siguiente ciclo de reloj, I_7 procede con su etapa de acceso a memoria y recién en el próximo, es decir, dos ciclos después de la transmisión de la orden de alto, podría completar su propia etapa de almacenamiento en registro, siendo ésta la segunda de aquéllas que se encontraban pendientes luego de la transmisión de la orden de alto para que todas las unidades de ejecución del procesador puedan proceder a detenerse definitivamente.
- Por último, la tercera instrucción, siempre presente para cualquier caso de ejecución, es la propia instrucción de detención del procesador, "halt": prosiguiendo normalmente con las etapas de su ciclo de ejecución, al emitir la orden de alto durante su etapa de decodificación, recién tres ciclos de reloj después de la transmisión podría completar su propia etapa de almacenamiento en registro, deteniéndose aquí efectivamente todas las unidades de ejecución de la CPU y finalizando así definitivamente la ejecución del programa. En la situación representada en esta figura, se verifica que I_8, la instrucción "halt", completa su etapa de almacenamiento en registro durante el último ciclo de reloj de la ejecución del programa, el cual a su vez transcurre tres ciclos después de la realización de la etapa de decodificación en la cual la unidad de control transmite la orden de detención al resto de las unidades de ejecución de la CPU. Finalmente, puede concluirse que este almacenamiento en registro correspondiente a la instrucción "halt", al ejecutarse en el último ciclo de reloj del programa, se

trata sin ningún lugar a dudas de la última etapa de almacenamiento en registro que se encontraba pendiente luego de la transmisión de la orden de alto para que todas las unidades de ejecución del procesador puedan proceder a detenerse definitivamente.

- Como curiosidad, sin duda el lector se habrá percatado de que estas últimas dos instrucciones, I_7 e I_8 para el caso de ejecución representado en la figura, coinciden exactamente con aquéllas mencionadas durante la definición de las etapas de acceso a memoria aún pendientes una vez emitida la orden de detención para que la CPU se encuentre en condiciones de ejecutar esta orden y pararse, lo cual resulta sensato considerando que las etapas de acceso a memoria y almacenamiento en registro se encuentran dispuestas en el ciclo de ejecución de cualquier instrucción, sin importar si hace uso o no de la FPU, de manera consecutiva.
- A partir de las exposiciones previas, la conclusión más lógica y razonable sería determinar que la unidad de almacenamiento en registro, encargada de gestionar la ejecución de esta etapa del ciclo de ejecución de las instrucciones, deberá permanecer activa, luego de la emisión de la orden de detención para el procesador, hasta que todas las etapas de almacenamiento en registro pendientes hayan podido ser atendidas y completadas ya que la presencia de esta unidad resulta imprescindible para que puedan ejecutarse y finalizar correctamente, de manera tal que el banco de registros de uso general de la CPU, accedido durante esta etapa, siempre sea actualizado con los valores correctos y así tanto el estado último del procesador como el resultado final del programa coincidan con los esperados por el usuario.
- Como consecuencia de esto, la unidad de almacenamiento en registro tendría que detenerse tres ciclos después de la transmisión de la orden de detención, es decir, al finalizar el último ciclo de reloj de la ejecución del programa, durante el cual se completa la etapa de almacenamiento en registro de la instrucción “halt”, la última etapa pendiente de ejecución de este tipo para poder detener el procesador sin inconvenientes.
- Sin embargo, se puede aplicar aquí una definición similar a aquélla que se tuvo en cuenta para determinar el instante de detención de la unidad de acceso a memoria: en realidad no se lleva a cabo tarea alguna en la etapa de almacenamiento en registro de la instrucción “halt”, por lo cual podría

desactivarse anticipadamente la unidad de almacenamiento en registro ya que no será necesaria para gestionar la ejecución de esta etapa pendiente.

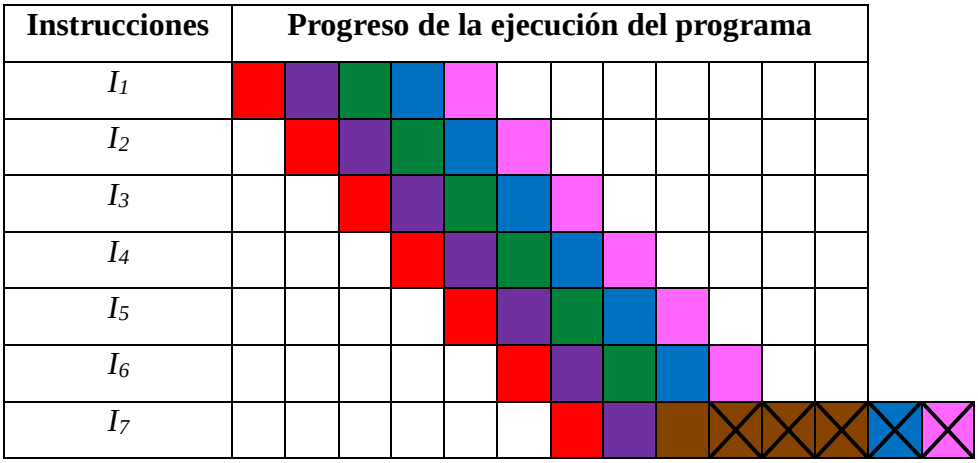
- Aun así, no puede afirmarse lo mismo respecto a las primeras dos etapas de almacenamiento en registro pendientes: al desconocer inicialmente la naturaleza de las instrucciones a cuyos ciclos de ejecución pertenecen previo a sus decodificaciones, la unidad de almacenamiento en registro obligatoriamente deberá permanecer activa durante la ejecución de estas etapas, ya que su presencia podría ser requerida para llevar a cabo alguna tarea útil en alguna de ellas o incluso en ambas.
- Entonces, esta unidad de ejecución de la CPU recién podrá ser detenida al finalizar el segundo ciclo de reloj posterior a aquél durante el cual fue emitida la orden de detención: así se garantiza que dicha unidad permanezca activa durante las etapas de almacenamiento en registro, si es que las hubo, de las dos instrucciones que se encontraban en ese momento ejecutando su etapa de ejecución o el último ciclo de su ejecución en punto flotante, y/o completando su etapa de acceso a memoria. Un ciclo de reloj después de la desactivación de la unidad de almacenamiento en registro se efectuará la etapa de almacenamiento en registro aún pendiente de la instrucción “halt”, pero al no llevarse a cabo ninguna tarea útil en ella, no existiría ningún inconveniente si aquí esta unidad ya se encontrara inactiva.
- Téngase en cuenta que la detención anticipada y prematura de la ALU y las unidades de acceso a memoria y almacenamiento en registro fue una elección estrictamente personal del autor de este informe para simplificar el diseño de la computadora, limitando el tiempo de ejecución de las unidades de ejecución del procesador tanto como sea posible sin alterar el comportamiento ni el resultado final ofrecido por el programa y esperado por el usuario. De ninguna manera esta decisión tendrá repercusión alguna, ya sea positiva o negativa, sobre el rendimiento ofrecido por la CPU.
- Se habrá notado que, al mencionar todas las instrucciones que poseían una etapa de acceso a memoria y/o almacenamiento en registro pendiente de ejecución, jamás se consideró la posibilidad de que existiera una instrucción completando su etapa de búsqueda y/o decodificación al mismo tiempo que la transmisión de la orden de detención para todas las unidades de ejecución del procesador. La razón para esto es muy sencilla: al emitirse esta orden

durante la ejecución de la instrucción “halt”, conociendo que esta última siempre se tratará de la última instrucción del programa ya que precisamente su tarea es finalizar correctamente este último deteniendo el procesador encargado de ejecutarlo, se garantiza que nunca se ejecutará una instrucción con posterioridad a ella. Entonces, ninguna instrucción podrá encontrarse en su etapa de búsqueda durante la transmisión de la orden de detención de la instrucción “halt” en su etapa de decodificación porque, si la hubiera, significaría que inició su ejecución en un instante posterior al comienzo de “halt”, invalidando el propósito de esta instrucción como última del programa para poder finalizarlo adecuadamente. Asimismo, jamás existirá una instrucción ejecutándose en su etapa de decodificación simultáneamente junto al envío de la orden de detención porque, al ocurrir esta transmisión durante la etapa de decodificación de la instrucción “halt”, se estaría ante la presencia de dos instrucciones del programa completando una idéntica etapa de su ciclo de ejecución al mismo tiempo, violando las reglas que rigen la ejecución segmentada de un programa, según las cuales podrán coexistir múltiples instrucciones ejecutándose simultáneamente siempre y cuando en cada ciclo de reloj se encuentren completando etapas distintas de su ciclo de ejecución.

- También puede comprobarse que, tanto en ésta como en las tres próximas figuras, todas asociadas al análisis del progreso de la finalización de un programa ejecutado en forma segmentada, la última instrucción, I_8, efectivamente finaliza su ciclo de ejecución en el último de los ciclos de reloj representados en la figura (es decir, en alguna de las celdas de la última columna). En cambio, en todas las figuras anteriores, la instrucción final de la serie ejecutada se completa en el anteúltimo ciclo de reloj de la figura (es decir, en alguna de las celdas de la anteúltima columna). Esta distinción fue establecida deliberadamente por el autor de este informe para que el lector comprenda lo siguiente: mientras que en estas tres figuras el término de la última instrucción siempre finaliza la ejecución del programa, ya que en todos los casos se tratará de una instrucción “halt” encargada de detener la ejecución del procesador, en los demás escenarios representados esto no necesariamente es así, dado que la última instrucción de las restantes figuras podría no ser una instrucción “halt” y el programa tal vez no termine su

ejecución una vez finalizado el análisis de su progreso en cada figura; no obstante, una vez solucionado el riesgo de la segmentación estudiado en ella, el avance del resto de la ejecución del programa resulta irrelevante, por lo cual se omite su representación en la figura en cuestión.

- Uno de los conceptos más destacables expuestos en la explicación del mecanismo utilizado por el procesador para, una vez emitida la orden de detención por la unidad de control, desactivar cada una de las unidades de ejecución en diferentes instantes de tiempo según la ubicación de su etapa gestionada en el ciclo de ejecución de las instrucciones, consiste en la comprobación llevada a cabo por la unidad de control durante la misma etapa de decodificación de la instrucción "halt" previamente a la transmisión de la orden de alto. Como se recordará, durante dicha validación se verificaba que existiera a lo sumo una sola instrucción en el programa ejecutándose actualmente en su etapa de ejecución o, en caso de requerir el uso de la FPU para realizar una operación aritmética en punto flotante, en el último ciclo de reloj de su etapa de ejecución en punto flotante.
- Es posible que en este momento el lector todavía no comprenda la importancia que posee esta comprobación para garantizar una correcta finalización del programa, durante la cual todas y cada una de las instrucciones de este último logren concluir exitosamente su ciclo de ejecución antes de proceder con la detención del procesador. En esta figura, al encontrarse efectivamente una única instrucción, I_7, ejecutando su etapa de ejecución durante la verificación realizada por la unidad de control en la etapa de decodificación de la instrucción "halt", I_8, la validación resulta exitosa, por lo cual no existiría inconveniente alguno para terminar el programa adecuadamente.
- Para explicar mejor este concepto, se representa a continuación un escenario de ejecución opuesto al de la figura anterior, en el cual ahora fracase la verificación realizada por la unidad de control:



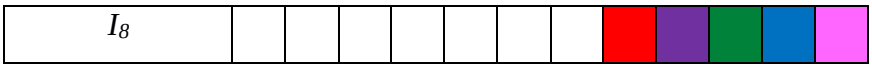


Figura 78. Progreso de la ejecución segmentada de un programa cuya finalización es prematura e incorrecta, dentro de un escenario con condiciones inusuales debido a la presencia de una instrucción que requiere utilizar la FPU del nuevo procesador.

- Como se observará, en esta figura el número de instrucciones del programa es idéntico al de la figura 77 y, además, la última instrucción, I_8 para ambos casos, cumple la misma tarea: finalizar el programa deteniendo el procesador encargado de ejecutarlo. En otras palabras, se trata de una instrucción "halt", igual que en el escenario representado en dicha figura 77.
- No obstante, en la figura 77 la estructura del ciclo de ejecución de la anteúltima instrucción, I_7 para ambas figuras, es exactamente igual al del resto de las instrucciones ejecutadas en aquel programa, especialmente la propia instrucción "halt", ninguna de las cuales requiere utilizar la FPU del procesador para realizar operaciones aritméticas en punto flotante: cinco etapas con una duración de un ciclo de reloj cada una; en particular, la etapa de ejecución puede completarse en un único ciclo (ver tabla 48).
- Sin embargo, en esta nueva figura, la misma instrucción I_7, ubicada inmediatamente antes de la instrucción "halt", es decir, I_8, requiere realizar en la FPU de la CPU una operación aritmética en punto flotante, por lo cual la estructura de su ciclo de ejecución será más compleja y su duración, más larga que la del resto de las instrucciones del programa, en particular, la propia instrucción "halt", que no utilizan en absoluto la FPU para sus tareas a cumplir. Ahora, la etapa de ejecución en punto flotante asociada al ciclo de ejecución de I_7 recién podrá completarse durante el transcurso de cuatro ciclos de reloj, por lo cual el tiempo de ejecución total de I_7 será de ocho ciclos (ver tabla 49). Al compararse con los cinco ciclos de necesarios para ejecutar las demás instrucciones, se concluye que existirán tres ciclos de reloj de diferencia entre la duración de la ejecución de ambos tipos de instrucciones.
- Esta diferencia entre las estructuras de los ciclos de ejecución de la instrucción "halt" (I_8) y su predecesora inmediata (I_7), aunque en un principio parezca trivial, en realidad esto no es así en absoluto. De hecho, si todas las instrucciones del

repertorio soportado por el procesador compartieran el mismo ciclo de ejecución, se garantizaría para cualquier escenario posible el mismo desenlace que el ocurrido en la figura 77. Esto, a su vez, volvería innecesaria la comprobación del estado actual de las instrucciones activas anteriores a la instrucción "halt", I_5, I_6 e I_7 en este caso, por parte de la unidad de control antes de emitir la orden de detención para todas las unidades de ejecución del procesador: sin importar el programa ejecutado, siempre será la instrucción inmediatamente anterior a "halt", I_7 en esta ocasión, la única que se encuentre en este momento completando su etapa de ejecución, cuya duración sería de un único ciclo. Por lo tanto, la validación llevada a cabo por la unidad de control resultará exitosa para todos los casos, perdiendo así su razón de ser: no tiene sentido alguno comprobar una condición si se conoce de antemano que se cumplirá incondicionalmente.

- A partir de la observación anterior puede determinarse una nueva conclusión respecto al mecanismo explicado anteriormente, utilizado para desactivar cada una de las unidades de ejecución del procesador en diferentes instantes de tiempo según la ubicación de su etapa gestionada en el ciclo de ejecución de las instrucciones: únicamente puede garantizarse su correcto funcionamiento por sí solo, es decir, sin que la unidad de control realice una previa comprobación del estado actual de la ejecución de las instrucciones anteriores a la instrucción "halt", para cualquier escenario, en caso de que todas las instrucciones del programa ejecutado compartan la misma estructura para su ciclo de ejecución.
- Sin embargo, al poder presentar las instrucciones del repertorio para este procesador dos posibles estructuras alternativas para sus ciclos de ejecución en función de su naturaleza, más precisamente, si necesitan realizar o no operaciones aritméticas en punto flotante en la FPU de la CPU (ver tablas 48, 49 y 56), existiría la alarmante posibilidad de que el mecanismo desactive alguna o varias unidades de ejecución del procesador prematuramente, es decir, antes de que hayan podido gestionar todas las etapas pendientes de atención de los ciclos de ejecución de las instrucciones anteriores a la instrucción "halt" que todavía se encuentran ejecutándose al mismo tiempo que ella.
- La consecuencia de esta desactivación prematura sería exactamente la esperada (e indeseada): las etapas pendientes no podrán ser atendidas ni gestionadas por sus respectivas unidades de ejecución ya detenidas temprana y erróneamente, por lo cual no se podrá llevar a cabo en ellas ninguna tarea útil en caso de ser necesario o, peor

aún, se realizarán de una manera errónea, comprometiendo todas las actividades efectuadas por las instrucciones anteriores del programa. En cualquier caso, este último no podrá ofrecerle al usuario el comportamiento ni el resultado correcto.

- En esta figura se puede comprobar, en particular, el hipotético progreso de la ejecución de la anteúltima instrucción, I_7, una vez emitida la orden de detención por la unidad de control durante la etapa de decodificación de la instrucción "halt", en caso de no llevarse a cabo previamente la comprobación del estado actual de la ejecución tanto de I_7 como del resto de las instrucciones activas durante la validación.
- Como se recordará, en la figura anterior, al tener esta misma instrucción un ciclo de ejecución cuya estructura era idéntica al del resto de las instrucciones del programa, dicha instrucción se encontraba ejecutando su etapa de ejecución durante la decodificación de la instrucción "halt", por lo cual la comprobación llevada a cabo por la unidad de control resultaba exitosa, procediendo así sin inconvenientes con la emisión de la orden de detención para todas las unidades de ejecución del procesador.
- No obstante, en esta figura, como el ciclo de ejecución de la instrucción I_7 es más prolongado que el del resto de las instrucciones del programa, ahora se encontrará ejecutando en este caso el primer ciclo de su etapa de ejecución en punto flotante, por lo cual aquí no se cumplirá la condición evaluada durante la comprobación, según la cual, tal como se explicó para la figura anterior, todas las instrucciones del programa actualmente en ejecución previas a la instrucción "halt" (I_5, I_6 e I_7) tienen que haber completado sus respectivos ciclos de ejecución hasta tal punto que a lo sumo sólo una de ellas se encuentre en el momento de la validación ejecutando su etapa de ejecución o el último ciclo de su etapa de ejecución según el tipo de estructura del ciclo de ejecución para cada instrucción verificada.
- Por lo tanto, si este proceso de comprobación no se encontrara implementado en la unidad de control para que pueda efectuarlo antes de transmitir la orden de alto, el programa proseguiría con su ejecución tal como lo muestra esta figura: la orden sería efectivamente transmitida durante la etapa de decodificación de la instrucción "halt" a pesar de que no se cumplen las condiciones que garanticen la correcta detención del procesador junto con su respectiva finalización del programa.
- En particular, la FPU de la CPU será detenida inmediatamente una vez concluido el ciclo de reloj durante el cual fue transmitida la orden de detención. En otras palabras,

la FPU se desactivará cuando la anteúltima instrucción del programa, I_7, finalice el primer ciclo de su etapa de ejecución en punto flotante.

- De este modo, I_7 se verá impedida de terminar su ciclo de ejecución satisfactoriamente, dado que, cuando intente proceder con el segundo ciclo de su etapa de ejecución en punto flotante, la FPU ya habrá sido deshabilitada por el procesador, tal como fue determinado en el mecanismo de desactivación progresiva para todas las unidades de ejecución a través del tiempo.
- Algo similar ocurriría con el resto de las etapas del ciclo de ejecución de I_7, acceso a memoria y almacenamiento en registro: no sólo la instrucción I_8, "halt", completará su propio ciclo de ejecución y finalizará el programa antes de que esta instrucción sea capaz siquiera de intentar ejecutar estas etapas sino que, incluso si las alcanzara, no podría efectuarlas ya que a esta altura el procesador ya habrá desactivado también tanto la unidad de acceso a memoria como la unidad de almacenamiento en registro, por lo cual estas etapas permanecerían incapaces de ser atendidas y gestionadas en caso de tener que realizarse alguna tarea útil en ellas.
- Dicho en otras palabras, la ejecución de I_7 será interrumpida antes de concluirse y jamás podrá ser reanudada debido a la desactivación prematura e incorrecta de las unidades de ejecución del procesador y la posterior detención total de este último junto con la finalización del programa.
- Efectivamente, podrá comprobarse en esta figura que se han tachado con una cruz negra cada una de las celdas correspondientes a las etapas de la instrucción I_7 que no han podido ejecutarse debido a la detención temprana de la FPU, la unidad de acceso a memoria y la unidad de almacenamiento en registro luego de la emisión de la orden de alto. Se han incluido las etapas de acceso a memoria y almacenamiento en registro en celdas especiales, separadas y ubicadas por fuera del progreso de la ejecución del programa, ya que, como se explicó anteriormente, cuando la instrucción "halt", I_8, completa su ciclo de ejecución, el procesador se detiene por completo y el programa finaliza antes de que I_7 siquiera haya logrado alcanzar e intentar efectuar estas etapas, por lo cual técnica y formalmente estas últimas no formarían parte de dicho progreso.
- A causa de la ejecución incompleta de la instrucción I_7, puede afirmarse que, si el procesador ejecutara este programa tal como lo muestra esta figura, es decir, aplicando el mecanismo de desactivación progresiva para todas las unidades de ejecución, pero sin una comprobación previa del estado actual de la ejecución de las

instrucciones anteriores a “halt” activas en el momento de la validación, entonces el comportamiento ofrecido por el programa será incorrecto dado que I_7 no podrá finalizar su operación aritmética en punto flotante ni tampoco almacenar su resultado en el banco de registros de uso general en punto flotante del procesador. En consecuencia, dicho resultado no se encontrará disponible para ser utilizado por el usuario durante el resto de la ejecución del programa según juzgue conveniente.

- Toda la explicación previa debería servirle al usuario para comprender la motivación detrás de la comprobación llevada a cabo por la unidad de control previo a la emisión de la orden de detención: la condición a evaluar y cumplir fue establecida por el autor de este informe de manera tal de garantizar, una vez transmitida dicha orden, un escenario de ejecución cuyo estado sea similar a aquél representado en la figura 77, en la cual, como se recordará, todas las instrucciones del programa compartían la misma estructura para su ciclo de ejecución. Éste, a su vez, es el requisito imprescindible para asegurar el correcto funcionamiento del mecanismo para desactivar paulatinamente cada una de las unidades de ejecución del procesador, atendiendo y gestionando todas las etapas pendientes de los ciclos de ejecución de las instrucciones previas a la instrucción “halt” actualmente activas antes de detener totalmente el procesador y finalizar el programa ejecutado.
- El estado del escenario de ejecución, mencionado en la observación anterior como condición central para determinar el éxito o fracaso de la comprobación realizada por la unidad de control, se encuentra conformado, más precisamente, por el tipo y la cantidad de etapas pendientes de atención para las instrucciones anteriores a la instrucción “halt” que aún permanezcan en ejecución una vez finalizado el ciclo de reloj durante el cual se transmita la orden de detención, en la etapa de decodificación de la instrucción “halt”, I_8 en este caso.
- Observando la figura 77, se determina que el estado del escenario de ejecución luego de concluida la decodificación de la instrucción I_8, “halt”, se encuentra compuesto por las siguientes etapas pendientes:
    - Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción “halt”, por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
    - Dos etapas de acceso a memoria: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a “halt”, I_7 en este caso, y otra perteneciente al de la propia instrucción “halt”, por lo que en esta última

etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de acceso a memoria para su gestión.

- Tres etapas de almacenamiento en registro: dos correspondientes a los ciclos de ejecución de las instrucciones inmediatamente anteriores a "halt", I_6 e I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

❖ En cambio, esta nueva figura exhibe un estado ligeramente distinto para su escenario de ejecución una vez transmitida la orden de detención por la unidad de control, constituido por las siguientes etapas pendientes:

- Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
- Tres subetapas de ejecución, cada una correspondiente al segundo, tercer y cuarto ciclo respectivamente de la etapa de ejecución en punto flotante del ciclo de ejecución perteneciente a la instrucción inmediatamente anterior a "halt", I_7 en este caso.
- Dos etapas de acceso a memoria: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de acceso a memoria para su gestión.
- Tres etapas de almacenamiento en registro: dos correspondientes a los ciclos de ejecución de las instrucciones inmediatamente anteriores a "halt", I_6 e I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

❖ Tal como fue explicado, el objetivo de la comprobación llevada a cabo por la unidad de control antes de emitir la orden de detención es garantizar que las condiciones de ejecución para los escenarios en los cuales los ciclos de ejecución de las instrucciones no coincidan entre sí, como en el caso, por ejemplo, de esta figura, sean similares a aquéllas asociadas a los programas en los cuales todas las instrucciones poseen la misma estructura para su ciclo de ejecución, tal como se ejemplificó en la figura 77.

- ❖ La similitud en las condiciones de ejecución implica más precisamente que puede existir, durante la finalización de un programa, una vez transmitida la orden de ejecución, una cantidad de etapas pendientes para cada tipo de éstas que resulte, a lo sumo, equivalente a su respectivo número de etapas aún por atender que restaban en el primero de los dos escenarios explicados anteriormente, en el cual existía un único ciclo de ejecución común a todas las instrucciones. Como se recordará, ésta era la cantidad de etapas pendientes para cada tipo en este escenario, representado en la figura 77: una etapa de ejecución, dos etapas de acceso a memoria y tres etapas de almacenamiento en registro.
- ❖ Puede notarse una clara diferencia entre ambos escenarios de ejecución descriptos previamente: la presencia de las tres subetapas de la ejecución en punto flotante en el segundo caso que no existen en absoluto en el primero.
- ❖ Entonces, al menos para el caso representado en esta figura en particular, la comprobación realizada por la unidad de control debería estar orientada, inicialmente, a eliminar de este escenario las tres subetapas pendientes por atender de la etapa de ejecución en punto flotante de la instrucción I_7, es decir, aquélla ubicada inmediatamente antes de la instrucción "halt". De este modo, una vez finalizada la transmisión de la orden de detención, esta instrucción también concluiría el último ciclo de su etapa de ejecución en punto flotante, de la misma manera en la cual, en la figura 77, la instrucción I_7, análoga a ella, completa su propia etapa de ejecución, procediendo así en ambos escenarios con el comienzo de la ejecución de su etapa de acceso a memoria pendiente.
- ❖ No se deben malinterpretar las palabras anteriores: la manera correcta de eliminar estas etapas pendientes del escenario no sería cancelando su ejecución sino atendiéndola, manteniendo activa la FPU del procesador para poder gestionar sus tareas que de otro modo, sin la comprobación por parte de la unidad de control, permanecerían sin poder llevarse a cabo debido a la desactivación prematura tanto de la FPU como del resto de las unidades de ejecución de la CPU, con las consecuencias ya conocidas y manifestadas previamente.

- Ahora bien, el siguiente y último paso para completar esta explicación sería representar nuevamente la ejecución del mismo caso exhibido en esta figura, pero ahora con el proceso de comprobación plenamente incorporado y operativo en la unidad de control del procesador:

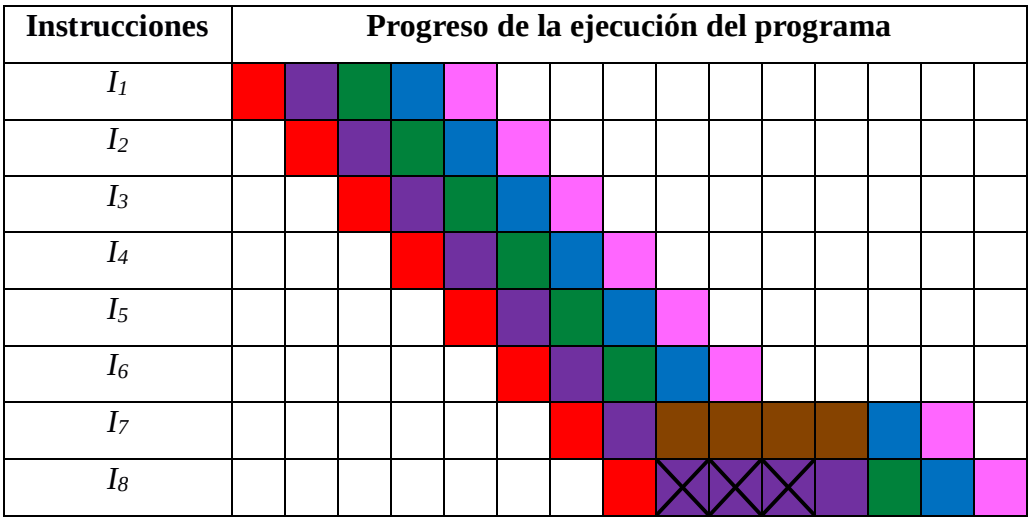


Figura 79. Progreso de la ejecución segmentada de un programa dentro de un escenario idéntico al de la figura anterior, pero cuya finalización es ahora correcta debido a la presencia de un atasco en el cauce del procesador en la última instrucción del programa.

- Independientemente de la naturaleza particular del programa cuya ejecución se representa en esta figura, que puede o no estar conformado por las mismas instrucciones que el programa ejecutado en la figura 78, el aspecto a destacar aquí son las estructuras de los ciclos de ejecución de todas las instrucciones que pertenecen a la secuencia de esta figura, las cuales coinciden exactamente cada una con la estructura del ciclo perteneciente a su respectiva instrucción del programa de la figura 78: los ciclos de ejecución de las instrucciones I_1 de ambas figuras coinciden entre sí, y lo mismo ocurre con el resto de las instrucciones: I_2, I_3, I_4, etc., hasta alcanzar la última instrucción, "halt", es decir, I_8.
- Una vez más, correspondería analizar particularmente la instrucción inmediatamente anterior a "halt", I_7, cuyo ciclo de ejecución, nuevamente, difiere de aquél asociado a las demás instrucciones del programa ya que requiere realizar una operación aritmética en punto flotante en la FPU del procesador. Ergo, su etapa de ejecución en punto flotante será tres ciclos más larga si se la compara con la duración de una etapa de ejecución llevada a cabo por cualquier instrucción que no involucre uso alguno de la FPU. Por lo cual, su ciclo de ejecución será más prolongado y lento que el del resto de las instrucciones, una consideración muy

importante a tener en cuenta para determinar el momento exacto en el cual deberá emitir la orden de detención para todas las unidades de ejecución del procesador y comenzar así con la finalización del programa, de manera tal que esta última se lleve a cabo correctamente, garantizando que todas las instrucciones completen hasta la última etapa de sus ciclos de ejecución antes de la detención total de la CPU.

- Ya se explicaron para el caso anterior, es decir, la figura 78, todos los riesgos que podrían ocurrir si no se realizara una comprobación durante la etapa de decodificación de la instrucción "halt", antes de transmitir la orden de detención, sobre el estado de la ejecución actual de las instrucciones anteriores a "halt" que aún permanecieran en ejecución (I_5, I_6 e I_7).
- Por lo tanto, aquí directamente se procederá a describir exactamente el procedimiento llevado a cabo por la unidad de control durante esta comprobación de manera tal de garantizar que las condiciones de ejecución para este programa sean, al menos, equivalentes a aquéllas existentes para un caso como el representado en la figura 77, en el cual todas las instrucciones comparten la misma estructura para su ciclo de ejecución.
- En primer lugar, durante la decodificación de la instrucción "halt", I_8, una vez identificada la naturaleza de la misma, la unidad de control comenzará con la validación del estado actual de las instrucciones activas en el programa anteriores a la instrucción "halt" y las comparará con las condiciones mínimas esperadas para determinar si la ejecución del programa se encuentra lo suficientemente avanzada como para poder proceder con la transmisión de la orden de detención sin comprometer la correcta finalización de las instrucciones que aún no se hayan completado.
- Cabe aclararse que, por su parte, la instrucción "halt", aunque forme parte del escenario de ejecución a ser comprobado por la unidad de control, no necesita ser verificada en forma particular ya que, al ser su presencia obligatoria y su ubicación, fija para cualquier programa, su ciclo de ejecución permanecerá inalterable, así como sus etapas pendientes de ejecución después de la emisión de la orden de detención.
- En cambio, las instrucciones anteriores del programa, al variar según el escenario de ejecución, también podrían presentar diferencias para cada situación entre las estructuras de sus ciclos de ejecución y la ubicación de cada uno de dichos ciclos en el programa, con las distintas alternativas posibles para la cantidad y el tipo de etapas

pendientes que cada instrucción aún posea una vez transmitida la orden de detención. Por lo tanto, sobre estas últimas instrucciones deberá concentrarse el análisis y la verificación de la unidad de control, aunque obviamente también tendrán que considerarse las etapas pendientes de la instrucción "halt", siempre fijas para cualquier escenario, a fin de determinar el estado actual de la ejecución del programa de esta figura y compararlo con las condiciones de un programa cuyas instrucciones tengan un ciclo de ejecución uniforme, como en el caso representado en la figura 77, a fin de evaluar si se puede proceder con la emisión de la orden de alto para todas las unidades de ejecución del procesador.

- Rápidamente se describirá nuevamente el estado del escenario de la figura 77 una vez finalizada la transmisión de la orden de detención, cuyas condiciones de ejecución están determinadas por las siguientes etapas pendientes: una etapa de ejecución, asociada a la instrucción I_8 ("halt"); dos etapas de acceso a memoria, vinculadas a las instrucciones I_7 e I_8, y tres etapas de almacenamiento en registro, correspondientes a las instrucciones I_6, I_7 e I_8.
- Las condiciones de ejecución del caso de esta figura, por su parte, si se transmitiera la orden de alto durante el mismo ciclo de la decodificación inicial de la instrucción "halt", es decir, sin realizar comprobación alguna sobre el estado de la ejecución de las instrucciones previas, sería el mismo que aquél detallado para la figura 78: una etapa de ejecución, asociada a la instrucción I_8 ("halt"); tres subetapas de ejecución en punto flotante, vinculadas a la instrucción I_7; dos etapas de acceso a memoria, pertenecientes a las instrucciones I_7 e I_8; y tres etapas de almacenamiento en registro, correspondientes a las instrucciones I_6, I_7 e I_8.
- Se explicó anteriormente que, al estar destinada la comprobación llevada a cabo por la unidad de control a garantizar condiciones de ejecución para cualquier escenario que resulten, en el peor de los casos, equivalentes a aquéllas presentes en un programa cuyas instrucciones compartan la misma estructura para su ciclo ejecución.
- Expresado más claramente y aplicado concretamente a este caso, la cantidad total de etapas pendientes de atención a partir de la finalización de la transmisión de la orden de detención en esta figura debe ser igual o menor para cada tipo de etapa que el número de etapas pendientes en el mismo instante de tiempo para el escenario de la figura 77. Por su parte, al encontrarse en la misma posición relativa de su respectivo ciclo de ejecución (antes del acceso a memoria) y tener la misma duración

(un ciclo de reloj), se acepta la equivalencia entre una etapa de ejecución y el último ciclo de una etapa de ejecución en punto flotante.

- Entonces, al menos para este escenario, la motivación y la finalidad de la comprobación deberían estar orientadas a prevenir la posible transmisión prematura de la orden de detención por parte de la unidad de control. Para ello, tendrá que demorar forzosamente la ejecución de la etapa de decodificación de la instrucción "halt", I_8, hasta que I_7 pueda ejecutar los ciclos de su etapa de ejecución en punto flotante que sean necesarios para que este programa cumpla con las condiciones de ejecución suficientes a fin de garantizar el correcto funcionamiento del mecanismo de desactivación progresiva para las unidades de ejecución del procesador.
- Efectivamente, esto es lo que ocurre durante la ejecución del programa en esta nueva figura con la comprobación plenamente incorporada y operativa en la unidad de control: durante la validación se evalúa el estado de la ejecución del programa a partir de la finalización del ciclo de reloj actual y se detecta que el número de etapas pendientes de atención supera el máximo permitido, ya que, además de la etapa de ejecución de I_8, las etapas de acceso a memoria de I_7 e I_8 y las etapas de almacenamiento en registro de I_6, I_7 e I_8, existen tres subetapas de ejecución en punto flotante asociadas a la instrucción I_7 que no se presentan en absoluto en un escenario de ejecución como el de la figura 77.
- Por lo cual, la comprobación falla y, como consecuencia, la ejecución de la etapa de decodificación de la instrucción "halt", I_8, será temporalmente suspendida hasta que hayan logrado finalizarse por completo estas tres etapas de ejecución en punto flotante de I_7.
- Podrá observarse que cada una de las celdas asociadas a los ciclos de reloj durante los cuales la ejecución de la instrucción "halt" permanece atascada, tres en total, ha sido tachada con una cruz negra, más precisamente, desde el noveno hasta el decimoprimer ciclo de la ejecución total del programa para la instrucción I_8.
- Naturalmente, mientras la instrucción "halt" continúe demorada, no sólo I_7, sino todas las demás instrucciones anteriores a I_8 que se encuentren actualmente activas (I_5 e I_6) podrán proseguir normalmente con su ejecución, tal como ocurre en los demás casos vinculados al resto de los distintos tipos de atascos que pueden presentarse en el cauce del procesador detallados en esta subsección del informe, ya que de ninguna manera su avance representaría riesgo alguno de complicar la correcta finalización del programa.

- Una vez que la instrucción I_7 se dispone a completar el último ciclo de la etapa de ejecución en punto flotante de su ciclo de ejecución, la FPU del procesador, encargada de gestionar su tarea y habiendo sido previamente notificada por la unidad de control sobre el fracaso de su comprobación realizada, le envía a esta última unidad una nueva señal para indicarle que I_7 ya ha finalizado las tres primeras subetapas de su ejecución en punto flotante, por lo cual ya están garantizadas las condiciones mínimas necesarias para reanudar la ejecución de la instrucción "halt", demorada hasta el momento en su etapa de decodificación, y emitir la orden de detención para iniciar la desactivación paulatina de todas las unidades de ejecución de la CPU sin que el procesador se detenga antes de que todas las instrucciones hayan completado su ejecución.
- Al recibir esta señal, la unidad de control, ahora sin realizar comprobación alguna dado que la FPU ya le garantizó que se cumplían las mínimas condiciones de ejecución, continúa con la etapa de decodificación hasta el momento aplazada de la instrucción "halt" y transmite la orden de parada, procediéndose a continuación con el mecanismo ya explicado para deshabilitar gradualmente las unidades de ejecución del procesador.
- La unidad de búsqueda, la unidad de control, la ALU y la FPU de la CPU se detendrán inmediatamente a continuación de la finalización de la etapa de decodificación de la instrucción "halt" durante la cual fue emitida la orden de detención e iniciado el mecanismo de desactivación para las unidades de ejecución del procesador, lo cual es correcto dado que a partir de este instante ya no quedarán más instrucciones en el programa cuyos ciclos de ejecución posean etapas pendientes de atención ya sea de búsqueda, decodificación o ejecución, sean estas últimas o no en punto flotante. Solamente la instrucción "halt" tendría una etapa de ejecución sin efectuar, pero, como ya se explicó, no es requerida la ALU para gestionarla ya que esta instrucción en particular no lleva a cabo ningún trabajo útil en ella.
- La unidad de acceso a memoria se deshabilitará una vez finalizada la ejecución de la etapa de acceso a memoria de la instrucción I_7. Esto también es razonable ya que ésta era en efecto la última instrucción del programa que podía llegar a realizar alguna tarea útil en este tipo de etapa: la instrucción "halt" no efectúa ninguna labor en esta etapa en particular de su ciclo de ejecución.

- Por último, la unidad de almacenamiento en registro se desactivará luego de concluida la ejecución de la etapa de almacenamiento en registro de la instrucción I_7. Al igual que para la unidad anterior, esto tiene sentido ya que efectivamente ésta era la última instrucción del programa que podía llegar a realizar alguna tarea útil en este tipo de etapa: la instrucción "halt" no efectúa ninguna labor en esta etapa en particular de su ciclo de ejecución.
- Finalmente, una vez alcanzada y completada la etapa de almacenamiento en registro de la instrucción "halt", todas las unidades del procesador se habrán detenido y el programa habrá finalizado definitivamente, pero no sin antes haberse gestionado absolutamente todas las etapas pendientes de atención pertenecientes a los ciclos de ejecución de las instrucciones anteriores a "halt" que aún se encontraban activas luego del envío de la orden de detención.
- Aunque la unidad de control no compruebe nuevamente el estado de la ejecución de las instrucciones anteriores a "halt", en este apartado el autor del informe se tomará la libertad de hacerlo a fin de verificar que, tal como se esperaba y aseguraba, las condiciones de ejecución del programa sean ahora al menos equivalentes a las presentadas en la figura 77 una vez finalizada la transmisión de la orden de detención durante la etapa de decodificación de I_8.
- Entonces, analizando esta nueva figura, es decir, la número 79, se determina que ahora las etapas pendientes de ejecución luego de emitida la orden de alto que había sido previamente demorada son las siguientes:
  - Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
  - Dos etapas de acceso a memoria: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de acceso a memoria para su gestión.
  - Dos etapas de almacenamiento en registro: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

- ❖ Tal vez la diferencia más interesante con respecto al escenario de la figura 77 sea la ausencia de la etapa de almacenamiento en registro pendiente correspondiente al ciclo de ejecución de la instrucción I_6. Esta divergencia se debe sencillamente a que, al haber continuado I_6 con su ejecución normal mientras la instrucción "halt", I_8, permanecía atascada, una vez que fueron alcanzadas las condiciones de ejecución suficientes para la reanudación de "halt", I_6 ya había finalizado.
- ❖ Más allá de este detalle, que al fin y al cabo resulta irrelevante porque, como se había anticipado, no afecta en absoluto a la correcta finalización del programa, se verifica que efectivamente la comprobación, si se llevara a cabo, resultaría exitosa ya que las condiciones de ejecución para este programa ahora no sólo son equivalentes a las de la figura 77, sino incluso superiores: se presenta el mismo número pendiente de etapas de ejecución (una) y de acceso a memoria (dos), pero la cantidad de etapas de almacenamiento en registro pendientes es aquí menor (dos en lugar de tres). Esto simplemente significa que la ejecución del programa se encuentra aun más avanzada en este caso, debido a la finalización ya acontecida y satisfactoria de la instrucción I_6, la cual todavía está activa en la figura 77 luego de la finalización de la etapa de decodificación de la instrucción "halt", durante la cual fue enviada la orden de alto.

- Aunque la situación anterior, u otra cuyo comportamiento exhiba características similares al de ella, sea la causante con mayor frecuencia de la presencia de esta clase de atascos, existen otros escenarios distintos, más inusuales, pero que aun así pueden tener la misma capacidad de ocasionar una demora transitoria en la etapa de decodificación de la instrucción "halt" con el objetivo de aplazar temporalmente la transmisión de la orden de alto para todas las unidades de ejecución de la CPU de modo tal de garantizar una finalización correcta para todas las instrucciones del programa, durante la cual estas últimas logren completar la totalidad de las etapas de sus respectivos ciclos de ejecución antes de la detención efectiva, definitiva y absoluta del procesador.
- La ejecución de las instrucciones de un programa conformadas por ciclos de ejecución cuyas estructuras sean tales que se termine presentando durante las instancias finales de dicha ejecución uno de los posibles escenarios infrecuentes introducidos en el párrafo anterior se ilustra a continuación en la siguiente figura. Su inclusión adicional en este informe se debe a que el autor consideró que podría resultarle útil al lector a fin de reforzar los conceptos definidos en las observaciones de las figuras 77, 78 y 79 para explicar la naturaleza de esta clase de riesgo en particular, ya que su existencia se encuentra

absolutamente limitada al entorno de ejecución de este nuevo procesador, siendo la misma ignota para cualquiera de las CPU más conocidas (se habrá notado que, por ejemplo, en el simulador WinMIPS64 jamás se presenta durante la ejecución de un programa un atasco comparable a éste en la instrucción “halt”):

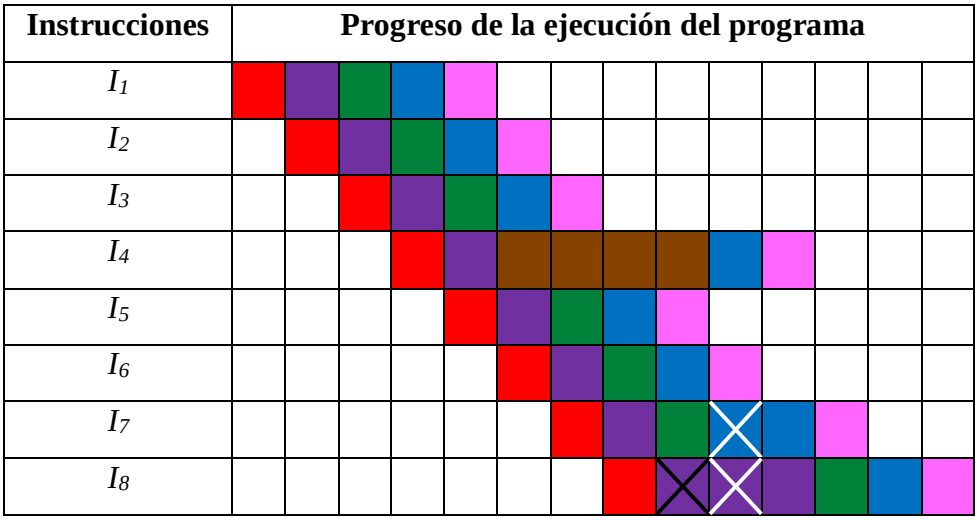


Figura 80. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

- Si se realiza una comparación entre este programa y aquél ejecutado en la figura 79, se comprobará rápidamente que, aunque la cantidad de instrucciones permanezca inalterable entre ambos casos, existen algunas diferencias con respecto a las estructuras de los ciclos de ejecución que conforman a cada una de ellas en particular.
- Para empezar, la instrucción I_4, es decir, la cuarta del programa en este caso, aquí requiere realizar una operación aritmética en punto flotante en la FPU del procesador, por lo cual la estructura de su ciclo de ejecución será, como ya se conoce, más larga y compleja y su duración, más prolongada, con respecto al resto de las instrucciones del programa o, si se prefiere, su respectiva instrucción I_4 análoga en el programa de la figura 79, ninguna de las cuales involucra en absoluto el uso de la FPU para cumplir sus tareas (ver tablas 48 y 49).

- Sin embargo, la instrucción ubicada inmediatamente antes de la instrucción “halt”, I_7 en este caso, la anteúltima del programa, posee aquí un ciclo de ejecución cuya estructura es idéntica a la de las demás instrucciones del programa, obviamente exceptuando a I_4. En cambio, si se recuerda, la instrucción I_7 ejecutada en la figura 79 necesitaba utilizar la FPU del procesador para realizar una operación aritmética en punto flotante, por lo cual su ciclo de ejecución era más prolongado y lento que el del resto de las instrucciones de aquel programa.
- En conclusión, podría afirmarse que, para este programa, simplemente se han intercambiado entre sí las instrucciones I_4 e I_7 de la figura 79 o, mejor dicho, las estructuras de sus respectivos ciclos de ejecución (recordar que para esta clase de atascos la naturaleza en sí de las instrucciones involucradas resulta irrelevante).
- Por su parte, la última instrucción del programa, I_8 en este caso, continúa siendo exactamente la misma que en el escenario de la figura 79: la instrucción “halt”, encargada de finalizar la ejecución del programa a través de la detención total del procesador. Obviamente, la estructura del ciclo de ejecución de esta instrucción tampoco se modifica, manteniéndose igual a la de cualquier otra instrucción que no requiera llevar a cabo operaciones aritméticas en punto flotante en la FPU de la CPU.
- Como cabrá esperarse, esta modificación en el posicionamiento de la instrucción “especial”, por expresarlo de alguna manera, debido a la particularidad presentada por la estructura de su ciclo de ejecución respecto a la del resto de las instrucciones, tendrá profundas repercusiones sobre la ejecución del programa y, en particular, el mecanismo adecuado que debe aplicarse para garantizar la correcta finalización del mismo.
- Tal como ocurría en el caso de la figura 79, la unidad de control del procesador procederá durante la etapa de decodificación de la instrucción I_8, “halt”, a comprobar el estado de ejecución actual de las instrucciones anteriores del programa que aún permanezcan activas (I_4, I_5, I_6 e I_7) para determinar las condiciones del escenario de ejecución para este programa una vez finalizada la hipotética transmisión de la orden de detención en esta etapa y establecer si, al menos, se alcanza el estado de ejecución mínimo necesario para permitir dicha transmisión sin comprometer la correcta finalización de todas las instrucciones del programa.
- La primera divergencia notable entre ambos escenarios consiste en que aquí la instrucción I_4, al ser su duración más prolongada, todavía se encuentra en ejecución

durante la comprobación llevada a cabo por la unidad de control. En cambio, en la figura 79, como el ciclo de ejecución de la instrucción I_4 era igual al del resto de las instrucciones del programa (excepto I_7, obviamente, ya que era ésta en aquel caso la instrucción cuyo ciclo de ejecución era distinto al de las otras), ya se había completado para el momento en el cual se realizaba la verificación, por lo cual no formaba parte de esta última.

- Si se analiza aún más detenidamente en esta figura el ciclo de reloj durante el cual se intenta completar por primera vez la ejecución de la etapa de decodificación de la instrucción "halt" y la unidad de control del procesador lleva a cabo la validación previa a la emisión de la orden de detención, podrá comprobarse otra diferencia que presenta este escenario respecto al de la figura 79, e incluso a la situación representada en la figura 77, en la cual, como se recordará, todas las instrucciones del programa compartían el mismo ciclo de ejecución.
- Más precisamente, la segunda diferencia entre esta figura y cualquiera de los escenarios anteriormente representados para describir esta clase de atasco (figuras 77, 78 y 79) radica en que aquí, durante la validación de la unidad de control, se encuentran en ejecución simultáneamente el último ciclo de la etapa de ejecución en punto flotante y la etapa de ejecución de los ciclos de ejecución de las instrucciones I_4 e I_7 respectivamente.
- Al haberse determinado la equivalencia entre ambas etapas/subetapas debido a su idéntica duración (un ciclo de reloj) y ubicación relativa con respecto al fin de su correspondiente ciclo de ejecución (antepenúltima etapa/subetapa de la instrucción), se presenta una situación extremadamente particular en este caso, la cual podrá comprenderse mejor una vez que se estudie y exponga en forma pormenorizada el estado de la ejecución que tendría el programa de esta figura una vez emitida la orden de alto si la unidad de control prosiguiera con su transmisión normalmente, es decir, sin realizar comprobación alguna ni contar con ningún tipo de garantías sobre el cumplimiento de las condiciones mínimas de ejecución de forma tal de no detener prematuramente el procesador y finalizar el programa antes de que todas sus instrucciones hayan completado hasta las últimas etapas de sus respectivos ciclos de ejecución.
- Una vez aclarado esto, se describirá a continuación el estado propiamente dicho para el escenario de ejecución del programa propuesto en esta figura una vez transmitida la orden de detención por la unidad de control, tal como es obtenido y extraído por

esta última luego de analizar cada una de las instrucciones aún activas. Dicho estado se encontrará constituido por las siguientes etapas pendientes:

- Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
- Tres etapas de acceso a memoria: la primera, pertinente a la instrucción cuyo ciclo de ejecución es más extenso debido a su utilización de la FPU de la CPU (I_4 en este caso); la segunda, correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt" (I_7 en este caso); y la última, perteneciente al ciclo de ejecución de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de acceso a memoria para su gestión.
- Cuatro etapas de almacenamiento en registro: la primera, pertinente a la instrucción cuyo ciclo de ejecución es más extenso debido a su utilización de la FPU de la CPU (I_4 en este caso); la segunda y la tercera, correspondientes a los ciclos de ejecución de las instrucciones inmediatamente anteriores a "halt", I_6 e I_7 en este caso; y la última, perteneciente al ciclo de ejecución de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

❖ Si se observa nuevamente la figura, podrá notarse especialmente que no todas las etapas de almacenamiento en registro pendientes serán atendidas por la unidad de almacenamiento en registro en el mismo orden en el cual fueron definidas en el párrafo anterior. Esto se debe, nuevamente, a la presencia de un ciclo de ejecución inusual y foráneo como aquél asociado a la instrucción I_4, el cual, al tener una duración más prolongada que el resto, provocará que no todas las instrucciones del programa finalicen en la misma secuencia en la cual comenzaron: la propia I_4, por ejemplo, se completará después que I_5 e I_6, a pesar de haberse iniciado antes que ellas. Naturalmente, esta diferencia entre las estructuras de los ciclos de ejecución no provocaría desorden alguno si el programa se ejecutara en forma secuencial en lugar de segmentada. No obstante, por otra parte, en la modalidad de ejecución secuencial tampoco existiría ninguna necesidad de este tipo de comprobaciones y atascos ya que, al ejecutarse cada instrucción de manera individual, iniciándose solamente una vez finalizada su instrucción inmediatamente anterior, para cualquier

escenario siempre quedarán solamente tres etapas pendientes de atención una vez transmitida la orden de detención: la ejecución, el acceso a memoria y el almacenamiento en registro pertenecientes al ciclo de ejecución de la propia instrucción "halt", en ninguna de las cuales se lleva a cabo tarea útil alguna. Por lo tanto, para cualquier programa ejecutado secuencialmente se está asegurada la correcta finalización de todas sus instrucciones independientemente de la coincidencia o diferencia entre las estructuras de sus respectivos ciclos de ejecución.

- Retornando al análisis del estado del escenario de ejecución presentado por el programa de esta figura una vez finalizada la transmisión de la orden de detención, se observará, a diferencia del caso exhibido en la figura 79, la ausencia de subetapas de ejecución en punto flotante pendientes de atención. Esta situación resulta ventajosa para este nuevo escenario respecto al anterior ya que lo vuelve más semejante a aquel mostrado en la figura 77, el cual actúa como referencia para determinar el cumplimiento de las condiciones mínimas de ejecución ya mencionadas y conocidas.
- No obstante, se advierte ahora en este escenario la presencia de dos nuevas etapas pendientes que no se encontraban en la figura 77 ni tampoco en la 79: un acceso a memoria y un almacenamiento en registro, ambos pertenecientes al ciclo de ejecución de la instrucción I_4.
- Como podrá adivinarse, esta discrepancia se debe simplemente al cambio en el posicionamiento de la instrucción cuyo ciclo de ejecución es más prolongado que el de las demás instrucciones del programa: al tratarse esta instrucción de la anteúltima del programa ejecutado en la figura 79, es decir, I_7, su ejecución comenzaba solamente un ciclo de reloj antes del inicio de la instrucción "halt", por lo cual, en el momento de la comprobación llevada a cabo por la unidad de control, aún se encontraba en una de las etapas más tempranas de su ciclo de ejecución, más precisamente, en la primera subetapa de su ejecución en punto flotante.
- Asimismo, como ya se mencionó, en la figura 79, al ser el ciclo de ejecución de I_4 idéntico al del resto de las instrucciones del programa que no involucran el uso de la FPU para realizar operaciones en punto flotante, esta instrucción ya se habrá completado antes de la finalización de la transmisión de la orden de detención, por lo cual no tendrá ninguna etapa pendiente que deba ser tenida en cuenta para la comprobación de la unidad de control sobre el estado del escenario de ejecución.

- En cambio, en esta nueva figura, por un lado, al no ubicarse la instrucción asociada al uso obligatorio de la FPU, I_4 en este caso, en una posición tan cercana en el programa a la instrucción "halt", existirá una distancia muy superior entre el comienzo de ambas instrucciones, exactamente cuatro ciclos de reloj, por lo tanto, su ejecución se encontrará en una etapa considerablemente más avanzada para el momento en el cual la unidad de control realice su respectiva comprobación. Más precisamente, estará completando la cuarta y última subetapa de su ejecución en punto flotante.
- Por otra parte, al tratarse esta instrucción de la única del programa que necesita llevar a cabo operaciones en punto flotante, puede garantizarse entonces que no quedarán subetapas de ejecución en punto flotante pendientes por atender una vez finalizada la transmisión de la unidad de control. No obstante, aún permanecerán por gestionar las etapas de acceso a memoria y almacenamiento en registro de la misma I_4, las cuales ya habían sido completadas en la figura 79 por pertenecer en aquel escenario a un ciclo de ejecución más breve, sin ningún tipo de vínculo con el uso de la FPU del procesador.
- La presencia adicional de estas etapas de acceso a memoria y almacenamiento en registro pendientes de atención pertenecientes al ciclo de ejecución de la instrucción I_4 provocará el fracaso de la comprobación llevada a cabo por la unidad de control durante la decodificación de la instrucción "halt", dado que la cantidad de etapas pendientes una vez transmitida la orden de detención, si esta última efectivamente se emitiera, superaría el número máximo permitido de etapas para cada tipo, el cual, como se recordará, fue determinado durante el análisis de la ejecución del programa planteado para la figura 77: una etapa de ejecución (en este apartado coinciden los escenarios de ambas figuras), dos etapas de acceso a memoria (en esta figura son tres) y tres etapas de almacenamiento en registro (aquí son cuatro).
- En consecuencia, a la unidad de control no le quedará ninguna otra alternativa que demorar la ejecución de la etapa de decodificación de la instrucción "halt" hasta que logren completarse al menos un acceso a memoria y un almacenamiento en registro de aquéllos que aún se encuentran por atender de manera tal que, al menos, la cantidad de etapas pendientes sea igual entre este escenario y el de la figura 77 y se cumplan así las condiciones mínimas de ejecución requeridas para garantizar la correcta finalización de la totalidad de las instrucciones del programa antes de la detención total del procesador.

- Naturalmente, al igual que en la figura 79, esta demora únicamente afectará a la instrucción "halt", I_8: todas las instrucciones anteriores del programa podrán continuar ejecutándose con normalidad, no sólo porque su ejecución no compromete en absoluto la finalización del programa sino también debido a que de hecho resulta imprescindible que prosigan completando sus etapas pendientes a fin de que dicho programa alcance las condiciones necesarias para que I_8 pueda reanudar su ejecución suspendida.
- Se aprecia en esta figura que la celda asociada a la presencia del atasco en la decodificación de la instrucción "halt" está tachada con una cruz negra. También puede notarse que la duración del mismo se prolonga durante un único ciclo de reloj, más precisamente, el noveno ciclo de la ejecución del programa.
- Si la orden de detención pudiera emitirse durante el siguiente ciclo de reloj, es decir, el décimo del programa, una vez finalizada la transmisión se observa que también en este mismo ciclo fueron atendidos y completados el acceso a memoria y el almacenamiento en registro de las instrucciones I_4 e I_6 respectivamente, que hasta el momento se encontraban sin ejecutar. Ergo, las etapas pendientes de ejecución serían ahora las siguientes:
  - Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
  - Dos etapas de acceso a memoria: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso, y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de acceso a memoria para su gestión.
  - Tres etapas de almacenamiento en registro: la primera, pertinente a la instrucción cuyo ciclo de ejecución es más extenso debido a su utilización de la FPU de la CPU (I_4 en este caso); la segunda, correspondiente al ciclos de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso; y la última, perteneciente al ciclo de ejecución de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

- En conclusión, una vez cumplido el primer ciclo de demora de la decodificación de la instrucción “halt” el estado del escenario de ejecución del programa cumpliría las condiciones mínimas para que la instrucción “halt” reanude su ejecución y la unidad de control emita la orden de detención, ya que una vez finalizada ésta la cantidad de etapas pendientes sería exactamente idéntica para cada tipo entre esta figura y la figura 77, utilizada como referencia para realizar la comprobación.
- Sin embargo, durante este mismo ciclo surge la presencia de un nuevo atasco cuya naturaleza es completamente diferente, ya que no se debe a un incumplimiento de las condiciones mínimas de ejecución para poder proceder sin inconvenientes con la finalización del programa sino a una dependencia de tipo estructural entre las instrucciones I_4 e I_7 debido a que ambas pretenden ejecutar simultáneamente su etapa de acceso a memoria. Esto no sólo está prohibido dado que tanto la unidad de acceso a memoria como el sistema de buses asociado a la memoria de datos de la computadora únicamente pueden atender una petición de acceso a la vez, sino porque también se incumpliría uno de los principios que rigen la ejecución segmentada de un programa, según el cual jamás pueden coexistir dos instrucciones completando una etapa idéntica de sus respectivos ciclos de ejecución al mismo tiempo.
- Este atasco estructural también se prolonga durante un solo ciclo de reloj, ya que éste es el tiempo necesario para que I_4 proceda con su propia etapa de acceso a memoria pendiente, liberando el uso del sistema de buses conectado a la memoria de datos a la instrucción I_7 a fin de que esta última pueda llevar a cabo su respectivo acceso a ella, en caso de necesitarlo. Las celdas asociadas a la presencia de este atasco se encuentran tachadas con una cruz blanca, una para cada instrucción afectada: I_7 e I_8, ambas durante el décimo ciclo de la ejecución general del programa.
- A pesar de no estar directamente implicada en la dependencia estructural originada entre las instrucciones I_4 e I_7, la instrucción “halt” también se encontrará involucrada en la demora dado que en cualquier atasco estructural todas las instrucciones posteriores a la última de las dos instrucciones responsables de dicho conflicto y atasco (I_7 en este caso) son detenidas por el procesador a fin de evitar nuevas dependencias estructurales que puedan surgir en un futuro, tal como se demuestra y explica en la figura 65, dedicada especialmente al estudio de la presencia de este tipo de atascos en el cauce del procesador.

- Como consecuencia de todas las explicaciones previas, la instrucción “halt” se demorará en su etapa de decodificación, en la cual se transmitirá la orden de detención para todas las unidades de ejecución del procesador, durante un ciclo de reloj adicional debido a la existencia de este nuevo atasco estructural, a pesar de que dicho atraso no sería estrictamente necesario para este caso dado que, no sólo ya se cumplen las condiciones de ejecución necesarias para emitir sin inconvenientes la orden de alto, sino que, además, al tratarse de la última instrucción del programa, se garantiza que no se iniciará la ejecución de una nueva instrucción a continuación de “halt”, por lo cual bajo ninguna circunstancia podría originarse una nueva dependencia estructural durante el resto de la ejecución del programa.
- Finalmente, una vez completada la etapa de acceso a memoria pendiente del ciclo de ejecución de la instrucción I_4, se resolverá la dependencia estructural y este atasco concluirá: I_7 podrá proceder con su propio acceso a memoria previamente demorado a causa de dicha dependencia, e I_8 reanudará la ejecución aplazada de su etapa de decodificación, por lo que ahora sí indefectiblemente la unidad de control emitirá durante este ciclo de reloj la orden de detención para todas las unidades de ejecución del procesador.
- Al igual que en el caso de la figura 79, la unidad de control no llevará a cabo nuevamente en esta instancia del programa una comprobación, previa a la transmisión de la orden de alto, sobre el estado de la ejecución de las instrucciones anteriores a “halt” que todavía permanezcan activas.
- Esto se debe a que, en el instante de tiempo en el cual intentó ejecutarse por primera vez la etapa de decodificación de la instrucción “halt” y la unidad de control realizó aquella comprobación inicial que determinó la necesidad de demorar temporalmente la emisión de la orden de detención para evitar una finalización prematura e incorrecta del programa, no estaban garantizadas aún las condiciones de ejecución mínimas para el escenario de ejecución en esa instancia. En otras palabras, se desconocía de antemano si la cantidad de etapas pendientes de atención una vez emitida la orden de alto superaba o no el número máximo permitido de etapas por gestionar para cada tipo de etapa.
- En cambio, una vez resuelta la dependencia estructural, la unidad de control recibirá una señal por parte de la FPU y la ALU del procesador, encargadas de ejecutar la última etapa de cada una de las dos instrucciones en conflicto antes de que se

originara el atasco, es decir, el cuarto ciclo de la etapa de ejecución en punto flotante de I_4 por un lado y, por otro, la etapa de ejecución de I_7.

- Tanto la FPU como la ALU, habiendo sido previamente alertadas por la unidad de control sobre el fracaso de su comprobación realizada y la consecuente imposibilidad de proseguir con la ejecución de la etapa de decodificación de la instrucción "halt" para transmitir la orden de detención, estarán preparadas para enviarle una nueva señal tan pronto como se resuelva la dependencia estructural entre I_4 e I_7, cuyo atasco asociado perdura, como ya fue mencionado, durante un ciclo de reloj, por lo cual una vez transcurrido este intervalo de tiempo a partir del origen de la dependencia se llevará a cabo el envío de esta señal por parte de la FPU y la ALU.
- Por sí sola, la resolución de la dependencia estructural alcanza para garantizar el cumplimiento de las condiciones mínimas de ejecución necesarias para proceder con la transmisión de la orden de detención. Ya se demostró previamente que dichas condiciones se alcanzaban incluso antes del surgimiento de este nuevo tipo de atasco, por lo cual ahora, al encontrarse aún más avanzado el estado de la ejecución general del programa, deberían incluso superarse, resultando así innecesario que la unidad de control realice cualquier tipo de comprobación similar a aquélla que llevó a cabo inicialmente antes de la primera demora de la decodificación de "halt".
- Aun así, tal como se hizo para el caso de la figura 79, aquí una vez más el autor de este informe analizará el estado del escenario de ejecución que presentaría el programa una vez transmitida la orden de detención durante la etapa de decodificación de la instrucción "halt", la cual se completaría inmediatamente a continuación de la finalización del atasco estructural. Se observa que, durante la transmisión, son también atendidas simultáneamente la etapa de almacenamiento en registro de I_4 y la de acceso a memoria de I_7. Por lo tanto, se concluye que las etapas pendientes de ejecución serían ahora las siguientes:
  - Una etapa de ejecución, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la ALU para su gestión.
  - Una etapa de acceso a memoria, correspondiente al ciclo de ejecución de la propia instrucción "halt", por lo cual no se realiza ninguna tarea útil en ella ni tampoco es requerida la unidad de acceso a memoria para su gestión.

- Dos etapas de almacenamiento en registro: una correspondiente al ciclo de ejecución de la instrucción inmediatamente anterior a "halt", I_7 en este caso; y otra perteneciente al de la propia instrucción "halt", por lo que en esta última etapa en particular no se realizan tareas útiles ni tampoco es requerida la unidad de almacenamiento en registro para su gestión.

- Previsiblemente, en esta instancia tan avanzada de la ejecución del programa no sólo se alcanzan las condiciones mínimas estipuladas para emitir la orden de detención sino que incluso se superan: si se realiza una comparación con el estado del escenario de ejecución de la figura 77 una vez finalizada la transmisión de la orden de alto, se comprobará que ahora en este caso es menor tanto la cantidad de etapas pendientes por atender de acceso a memoria (una en esta figura contra dos en la figura 77) como así también de almacenamiento en registro (dos en esta figura contra tres en la figura 77), aunque el número de etapas de ejecución pendientes sea exactamente el mismo (uno en ambas figuras).
- Volviendo a la descripción del progreso de la ejecución del programa representado en esta figura, una vez resuelta la dependencia estructural y reanudada la ejecución de la instrucción "halt", se completará la etapa de decodificación de esta última, previamente demorada, y se transmitirá finalmente durante ella la orden de detención para todas las unidades de ejecución del procesador, procediéndose a continuación con el mecanismo ya explicado para deshabilitar progresivamente dichas unidades.
- La unidad de búsqueda, la unidad de control, la ALU y la FPU de la CPU se detendrán inmediatamente a continuación de la finalización de la etapa de decodificación de la instrucción "halt" durante la cual fue emitida la orden de detención e iniciado el mecanismo de desactivación para las unidades de ejecución del procesador, lo cual es correcto dado que a partir de este instante ya no quedarán más instrucciones en el programa cuyos ciclos de ejecución posean etapas pendientes de atención ya sea de búsqueda, decodificación o ejecución, sean estas últimas o no en punto flotante. Solamente la instrucción "halt" tendría una etapa de ejecución sin efectuar, pero, como ya se explicó, no es requerida la ALU para gestionarla ya que esta instrucción en particular no lleva a cabo ningún trabajo útil en ella.
- La unidad de acceso a memoria se deshabilitará una vez completada la etapa de ejecución de la instrucción "halt", es decir, I_8. Esto también es razonable ya que la

última etapa de acceso a memoria pendiente por atender luego de la desactivación de esta unidad sería aquélla perteneciente al ciclo de ejecución de la propia instrucción “halt”, pero, de una manera similar al caso de la etapa de ejecución, y como también fue explicado anteriormente, no es requerida la unidad de acceso a memoria para gestionar esta etapa pendiente en especial ya que ningún trabajo útil es llevado a cabo en ella por esta instrucción en particular.

- Por último, la unidad de almacenamiento en registro se desactivará luego de concluida la ejecución de la etapa de acceso a memoria de la instrucción “halt”. Al igual que para la unidad anterior, y todas las unidades previas a ella, esto tiene sentido ya que la última etapa de almacenamiento en registro pendiente por atender luego de la desactivación de esta unidad sería aquélla perteneciente al ciclo de ejecución de la propia instrucción “halt”, pero, como también se expuso previamente, no es requerida la unidad de almacenamiento en registro para gestionar esta etapa pendiente en especial ya que ningún trabajo útil es llevado a cabo en ella por esta instrucción en particular.
- Finalmente, una vez alcanzada y completada la etapa de almacenamiento en registro de la instrucción “halt”, todas las unidades del procesador se habrán detenido y el programa habrá finalizado definitivamente, pero no sin antes haberse gestionado absolutamente todas las etapas pendientes de atención pertenecientes a los ciclos de ejecución de las instrucciones anteriores a “halt” que aún se encontraban activas luego del envío de la orden de detención.
- Ahora bien, a partir de una comparación postrimera entre este escenario y el de la figura 79 puede extraerse una valiosa conclusión, la cual, sin embargo, no resulta aplicable para el caso de la dependencia estructural acontecida en esta nueva figura entre las instrucciones I_4 e I_7 sino únicamente para el atasco presentado en la instrucción “halt”, I_8 en este caso, a causa del fracaso obtenido como resultado de la comprobación realizada por la unidad de control durante la decodificación de esta última instrucción.
- Dicha conclusión consiste en que, en cualquier caso, el tiempo durante el cual la ejecución de la etapa de decodificación de la instrucción “halt” permanezca demorada variará en función de la ubicación relativa de la instrucción cuyo ciclo de ejecución sea particularmente extenso, complejo y lento debido a su necesidad de utilizar la FPU de la CPU para realizar una operación aritmética en punto flotante, con respecto a la instrucción “halt”, dentro del programa: a mayor proximidad entre

ambas, más prolongada será la duración del atasco surgido en el cauce del procesador. Esto se debe a que la distancia entre los instantes de inicio de la ejecución de las dos instrucciones repercutirá sobre el número de etapas y, en consecuencia, ciclos de reloj, que a la instrucción particularmente duradera, directamente vinculada al uso de la FPU, le restan por completar hasta que el programa pueda alcanzar las condiciones mínimas de ejecución necesarias para resolverse el atasco y así permitirse la reanudación de la etapa de decodificación demorada de la instrucción "halt", durante la cual se transmitirá la orden de detención para todas las unidades de ejecución de la CPU.

- Si se presta atención, podrá apreciarse en este último apartado una notoria similitud entre el tipo de atascos especialmente abordado en la subsección actual y aquél originado como consecuencia de la presencia de una dependencia de datos RAW entre dos instrucciones de un programa, cuyas distintas modalidades posibles ya fueron ejemplificadas en las figuras 67, 68 y 69. Al igual que aquí, en esos casos también se concluía que la magnitud de la extensión de la demora de cualquiera de las distintas variantes de los atascos RAW dependía directamente de la ubicación relativa entre las instrucciones en conflicto dentro del programa, determinándose que a mayor proximidad entre ellas, más amplio sería dicho intervalo de duración. Asimismo, se había determinado que la fundamentación era muy similar a la esgrimida en el inciso previo: conforme se acercaran entre sí los instantes de inicio de la ejecución de ambas instrucciones, se incrementaría la cantidad de etapas y, por ende, el número de ciclos de reloj, pendientes de completar del ciclo de ejecución por parte de la instrucción de escritura hasta lograr alcanzar su etapa de almacenamiento en registro, durante la cual almacenaría en uno de los registros de uso general de la CPU el dato requerido por la instrucción de lectura para poder acceder a él en su etapa de decodificación de manera correcta en lugar de prematura.
- La conclusión anterior resulta especialmente útil para establecer la siguiente afirmación, en un principio válida exclusivamente para los riesgos de tipo RAW: si la proximidad entre las dos instrucciones conflictivas repercute directamente sobre el tiempo de duración del atasco generado a causa de dicho conflicto, habiéndose determinado que, a mayor lejanía, más acotada y reducida resulta la duración de la demora, entonces obviamente podría postularse la existencia de una distancia límite. Ésta funcionaría, tal como su nombre lo indica, a modo de extremo, el cual, en caso de ser superado, aunque las instrucciones operen sobre el mismo dato, ya sea para

leerlo o escribirlo según corresponda, su ubicación entre sí en el programa sería lo suficientemente distante como para que no se provocara atasco alguno en el cauce del procesador dado que el dato implicado en la dependencia de tipo RAW ya habría sido escrito por la primera instrucción involucrada en el conflicto antes de que la segunda intentara llevar a cabo su propia operación de lectura sobre él.

- Esta aseveración, aunque parezca trivial, permite deducir a su vez que, al compartir este nuevo atasco representado en estas últimas figuras previas el mismo fenómeno que los riesgos RAW con respecto a la relación directa entre la proximidad de las instrucciones y el tiempo de duración del atasco generado, entonces no debería existir ningún inconveniente en extender la aplicación de la afirmación anterior a este atasco provocado solamente en la instrucción "halt". De este modo, podría argumentarse que, así como en los riesgos de tipo RAW sólo pueden encontrarse directamente involucradas dos instrucciones que se encuentren dentro de un determinado nivel de proximidad entre sí en el orden de ejecución del programa, cuyo extremo estaría definido por una distancia límite que sería válida para cualquier caso en el cual se presente alguna de las variantes ya conocidas e introducidas en las figuras 67, 68 y 69, una situación similar debe ocurrir aquí, para esta nueva clase de atascos.
- Por lo tanto, el último objetivo propuesto para esta subsección sería determinar de manera exacta y precisa la transición entre dos casos de prueba que involucren cada uno la ejecución de un programa que presente una instrucción que realice una operación aritmética en punto flotante en la FPU del procesador cuya distancia con respecto a la instrucción "halt" varíe en cada escenario de manera tal que, durante la transición, dicha variación en la ubicación relativa entre ambas instrucciones posibilite la desaparición del atasco provocado en el cauce del procesador en la instrucción final del programa.
- Así, podría establecerse numéricamente la distancia que delimite axiomáticamente la presencia de este nuevo tipo de atascos durante la ejecución segmentada de cualquier programa conocido, teniendo en cuenta en todo momento algunas consideraciones especiales que serán expuestas en el próximo inciso. En un principio, a partir de una simple observación del código del programa y un sencillo cálculo matemático para determinar la superación o no de esta distancia límite, el lector sería capaz de predecir en forma certera el cumplimiento o incumplimiento según corresponda de las condiciones mínimas necesarias para emitir la orden de

detención destinada a todas las unidades de ejecución de la CPU y, en consecuencia, la ausencia o existencia respectivamente de un posible atasco en la instrucción “halt” de dicho programa.

- Debe tenerse en cuenta que el principio anterior sólo es estrictamente aplicable en un entorno de ejecución cuyas condiciones cumplan ciertos requisitos esenciales. En otras palabras, no puede negarse que, desafortunadamente al menos desde una perspectiva conceptual, existirán seguramente algunos programas cuya distancia entre su instrucción “halt” y aquélla vinculada al uso de la FPU no logre superar el valor límite y, sin embargo, no presenten este nuevo atasco analizado en la subsección actual durante la ejecución de la etapa de decodificación de la instrucción “halt”. De la misma manera, indudablemente se encontrarán casos de ejecución en los cuales, a pesar de que la separación entre las mismas dos instrucciones establecidas y definidas anteriormente alcanza a exceder la distancia límite, la instrucción “halt” deberá inevitablemente demorar su ejecución antes de que se cumplan las condiciones para que pueda proceder a emitir la orden de detención para todas las unidades de ejecución del procesador.
- Una de las razones por las cuales se afirma que existe la posibilidad de que algunos programas posean una separación entre su instrucción “halt” y su instrucción vinculada directamente al uso de la FPU del procesador de tal magnitud que no exceda la distancia límite, pero aun así no se presente durante la ejecución de los mismos esta nueva modalidad de atasco en la instrucción “halt”, radica en que podría surgir cualquier otro tipo de riesgo de la segmentación de una naturaleza diferente en el cauce. No obstante, este último atasco, si se presentara, debería obligatoriamente repercutir únicamente sobre la ejecución de instrucciones situadas en una ubicación posterior a la instrucción que realiza la operación aritmética en punto flotante.
- Resulta irrelevante si la instrucción “halt” en particular se encuentra visiblemente involucrada o no en la demora provocada por el atasco anteriormente descripto, dado que el objetivo aquí sería solamente lograr un retraso en la ejecución de la etapa de decodificación de esta misma instrucción, permitiéndose mientras tanto que la instrucción que hace uso de la FPU continúe ejecutándose normalmente. Dicho retraso, a su vez, tiene que prolongarse durante el tiempo suficiente como para que consigan alcanzarse las condiciones de ejecución requeridas antes de que la

instrucción “halt” comience dicha etapa de decodificación, de modo tal de evitar efectivamente el aparecimiento de este riesgo.

- Pueden mencionarse dos fenómenos capaces de implicar un retraso con estas características: el primero es simplemente una interrupción efectiva y temporal de la instrucción “halt” que demore su ejecución actualmente en curso, siendo éste el más intuitivo y fácil de comprender: en esta situación sencillamente dicha instrucción pertenece al conjunto de instrucciones que todavía se encuentran activas en el cauce del procesador y cuya ejecución deberá demorarse hasta que el conflicto en cuestión, ya sea estructural, de datos o de control, logre resolverse, tal como ocurría y se ha analizado para todos los tipos de atascos anteriormente expuestos. Por otro lado, el segundo fenómeno consiste en una dilación en el inicio de la ejecución de la instrucción “halt” aún sin comenzar en este caso, debiéndose la existencia del retraso a un atasco previo tal que aplazó la ejecución de instrucciones anteriores a ella: básicamente un escenario muy similar al anterior, aunque aquí la instrucción “halt” no pertenecería al conjunto de instrucciones directamente involucradas en el atasco y afectadas por la consecuente demora. No obstante, cabe recordar que todas las instrucciones que formen parte de este conjunto siempre deben comenzar su ejecución en un instante de tiempo posterior al inicio de la instrucción que realiza la operación aritmética en punto flotante para que ésta pueda proseguir con su propia ejecución sin inconvenientes. Si esto último no se cumpliera, una vez resuelto el riesgo de la segmentación y reanudada la ejecución de las instrucciones demoradas seguirían sin alcanzarse las condiciones mínimas de ejecución del programa en cuestión ya conocidas, presentándose así la obligación de detenerse temporalmente la instrucción “halt” en su etapa de decodificación hasta que las mismas logren alcanzarse.
- Por su parte, un motivo válido a esgrimir para argumentar la posible presencia de ciertos programas cuya separación existente entre su instrucción “halt” y su instrucción encargada de realizar una operación aritmética en punto flotante en la FPU del procesador supere la distancia límite, pero de todos modos presente esta nueva modalidad de atasco en la instrucción “halt”, se debe a la ejecución de un tipo particular de instrucciones: las de transferencia de control. Aquí reviste especial importancia la distinción entre el orden de las instrucciones durante el ensamblaje y la ejecución de un programa. En otras palabras, aunque a través de la observación simple de un determinado programa lograra determinarse que la distancia entre las

dos instrucciones ya mencionadas supera el valor límite, debe prestarse especial atención a la presencia de posibles alteraciones del flujo de ejecución normal de dicho programa a causa de la ejecución de cualquier instrucción de transferencia de control que pertenezca a él. Dichas alteraciones, en caso de efectivizarse, podrían repercutir de diversas maneras sobre el orden de ejecución de las instrucciones durante el resto del programa, poseyendo algunas la capacidad de involucrar drásticas consecuencias, al menos con respecto a los aspectos cuyo estudio revista especial interés en la subsección actual.

- Particularmente, podría encontrarse involucrada entre dichas consecuencias la eliminación de algunas de las instrucciones originalmente ubicadas en el flujo de ejecución luego de la instrucción directamente vinculada al uso de la FPU del procesador y antes de la instrucción “halt”, cuya secuela más inmediata sería una disminución en la distancia inicialmente calculada entre ambas instrucciones. Si esto sucediera y además antes de la ejecución del programa se hubiera determinado que la distancia original previamente mencionada superaba la distancia límite, existiría el peligro de que durante la ejecución efectiva de la instrucción “halt” se presente exactamente la misma modalidad de atasco estudiada en este apartado del informe y cuyo surgimiento había sido inicialmente asumido como imposible de ocurrir en el cauce segmentado debido a que, tal como se había establecido, la distancia límite era inferior a la separación anteriormente calculada entre las dos instrucciones durante el ensamblaje del programa. Por lo tanto, en caso de presentarse esta situación, una vez finalizado el programa y conociéndose por consiguiente el flujo y la secuencia de ejecución seguidos por sus instrucciones, correspondería efectuar un nuevo análisis del orden de las mismas, pero ahora centrándose en su disposición efectiva en dicho flujo. Se debe recordar que, en el ensamblaje, al desconocerse aún en aquel entonces el flujo de ejecución que presentaría efectivamente el programa, el cálculo de la distancia entre las dos instrucciones fue llevado a cabo asumiéndose el escenario más sencillo y predecible de todos, en el cual todas las instrucciones se ejecutarían en el mismo orden en el cual se encontraban dispuestas en el programa; presunción que, al igual que las conclusiones obtenidas a partir de ella, demostró ser errónea luego de realizar y descubrir la ejecución real del programa. En cambio, a partir del nuevo estudio de la disposición de todas las instrucciones en función de su auténtico orden de ejecución en el programa, se determinaría que la separación primeramente medida entre ambas instrucciones se redujo a través de la ejecución

del programa como consecuencia de la/s alteración/es acontecida/s en el flujo cuyas características coincidieran con las propiedades definidas al comienzo de este inciso hasta tal punto que, una vez realizado nuevamente el cálculo de la separación a partir de la nueva posición ocupada por la instrucción que realiza la operación aritmética en punto flotante en el flujo de ejecución modificado, el valor obtenido para la separación no alcanzó a superar la distancia límite. Ergo, básicamente se estaría retornando a una situación en la cual, refutando las conjeturas originales, aparecería una vez más el atasco en la instrucción "halt" en el cauce del procesador, tal como se había indicado anteriormente que sucedería para la mayoría de las situaciones en las cuales la distancia entre las dos instrucciones ya conocidas y establecidas no lograba exceder el valor límite.

- Asimismo, otra de las ramificaciones que podría presentar una alteración en el flujo de ejecución del programa potencialmente provocada por la ejecución de una instrucción de transferencia de control incluiría, por ejemplo, la exclusión directa y absoluta de la instrucción que hace uso de la FPU del mismo; o bien, a diferencia del inciso anterior, un aumento en lugar de una disminución de la distancia entre esta instrucción y la instrucción "halt", en este caso a causa de una posible incorporación de nuevas instrucciones a ejecutar entre la instrucción que realiza la operación aritmética en punto flotante y la instrucción "halt". Si ocurriera el primer caso, es decir, si se removiera del flujo de ejecución la instrucción vinculada al uso de la FPU, entonces la ejecución del programa en cuestión padecería modificaciones tan radicales que básicamente resultaría idéntica a la de cualquier programa cuyas instrucciones comparten exactamente una estructura idéntica para el ciclo de ejecución de todas ellas, tal como el caso representado en la figuar 77. Al garantizarse en esta clase de situaciones la inexistencia de percance alguno que pudiera comprometer la correcta finalización del programa, del mismo modo aquí se encuentra en condiciones de asegurar que el cambio en el flujo del programa cuyos efectos sean aquéllos propuestos para este caso conduciría a la ausencia incondicional de este nuevo tipo de atasco en la instrucción "halt", aun si durante el ensamblaje se hubiera determinado que la separación entre la instrucción "halt" y aquélla que utiliza la FPU no alcanzaba a superar la distancia límite.
- Por otra parte, en caso de presentarse la segunda alternativa planteada en el inciso anterior, es decir, si la alteración en el flujo de ejecución involucrara un incremento en la lejanía existente entre ambas instrucciones, existiría la posibilidad de que

durante la ejecución efectiva de la instrucción “halt” se presentara exactamente la misma modalidad de atasco aquí estudiada y cuya aparición había sido originalmente asumida como inevitable de ocurrir en el cauce segmentado debido a que la separación inicialmente calculada entre ambas instrucciones antes de la ejecución del programa era menor a la distancia límite. Por lo tanto, en caso de presentarse este escenario, una vez finalizado el programa y conociéndose por consiguiente el flujo y la secuencia de ejecución seguidos por sus instrucciones, correspondería efectuar un nuevo análisis del orden de las mismas, pero ahora centrándose en su disposición efectiva en dicho flujo. Se debe recordar que, en el ensamblaje, al desconocerse aún en aquel entonces el flujo de ejecución que presentaría efectivamente el programa, el cálculo de la distancia entre las dos instrucciones fue llevado a cabo asumiéndose el escenario más sencillo y predecible de todos, en el cual todas las instrucciones se ejecutarían en el mismo orden en el cual se encontraban dispuestas en el programa; presunción que, al igual que las conclusiones obtenidas a partir de ella, demostró ser errónea luego de realizar y descubrir la ejecución real del programa. En cambio, a partir del nuevo estudio de la disposición de todas las instrucciones en función de su auténtico orden de ejecución en el programa, se determinaría que la separación primeramente medida entre ambas instrucciones se acrecentó a través de la ejecución del programa como consecuencia de la/s alteración/es acontecida/s en el flujo cuyas características coincidieran con las propiedades definidas en la introducción de esta alternativa en la observación anterior hasta tal punto que, una vez realizado nuevamente el cálculo de la separación a partir de la nueva posición ocupada por la instrucción que realiza la operación aritmética en punto flotante en el flujo de ejecución modificado, el valor obtenido resultó tener la magnitud suficiente como para superar la distancia límite. Ergo, básicamente se estaría retornando a una situación en la cual, refutando las conjeturas originales, no aparecería en absoluto el atasco en la instrucción “halt” en el cauce del procesador, tal como se había indicado anteriormente que sucedería para la mayoría de las situaciones en las cuales la distancia entre las dos instrucciones ya conocidas y establecidas lograba exceder el valor límite.

- Durante la descripción introductoria y general ofrecida al comienzo de la presente subsección 3.1.8. para todos los riesgos de la segmentación que pueden presentarse en el cauce del procesador, había sido llevada a cabo una distinción especial para el caso de los riesgos de control, provocados a causa de la ejecución de instrucciones

de transferencia de control. Dicha distinción involucraba, por un lado, a las instrucciones cuyo salto asociado era incondicional y, por otro, a aquéllas que contaban con una condición de salto capaz de determinar, en función de su cumplimiento o incumplimiento, si el flujo de la ejecución del programa debía alterarse o proseguir normalmente. La razón de esta diferenciación se basaba en que, por un lado, ambos subtipos de instrucciones no presentaban un comportamiento idéntico durante su etapa de ejecución. Más precisamente, la inexistencia de una condición de salto a evaluar en una instrucción de salto incondicional conlleva por sí sola la consecuencia de no tener que realizar ninguna otra tarea en su etapa de ejecución (ver figura 74). No obstante, en una instrucción de transferencia de control condicional, al encontrarse implicada una condición de salto cuya evaluación es imprescindible para determinar si dicho salto debe efectuarse efectivamente o no, requerirá ineludiblemente realizar alguna operación aritmética en su etapa de ejecución para evaluar, a partir del resultado obtenido, el cumplimiento o incumplimiento de la condición y luego alterar o no el flujo de ejecución del programa según la conclusión extraída de la evaluación (ver figuras 75 y 76). De hecho, se recordará que había sido establecido en aquella misma subsección que la tarea de evaluar la condición de salto en la etapa de ejecución provocaría inevitablemente un atasco en el cauce del procesador que demoraría la ejecución durante un ciclo de reloj de todas las instrucciones posteriores a la instrucción de transferencia de control condicional que se encontraran activas en ese momento. Este atasco, obviamente, deberá ser especialmente considerado en caso de encontrarse ubicado entre la instrucción que utiliza la FPU del procesador, si dicha instrucción existiera, y la instrucción “halt” del programa; y, además, previo a la ejecución del programa se hubiera determinado que la separación entre ambas instrucciones no alcanzaba a superar la distancia límite y que, por lo tanto, el atasco exclusivo de la instrucción “halt” aparecería indefectiblemente en el cauce del procesador. Esto se debe a que, tal como se había mencionado anteriormente, existe la posibilidad de que cualquier tipo de atasco surgido durante la ejecución del programa entre estas dos últimas instrucciones retrase el comienzo de la etapa de decodificación de la instrucción “halt” hasta un punto tal que se logren cumplir las condiciones de ejecución mínimas para que esta instrucción pueda proceder normalmente con la emisión de la orden de detención destinada a todas las unidades de ejecución del procesador sin comprometer la correcta finalización del programa.

De este modo, no se presentaría atasco alguno como consecuencia de la ejecución de la etapa de decodificación de la instrucción “halt”, refutando la hipótesis planteada antes de iniciar la ejecución del programa.

- No obstante, el mayor interés de la situación expuesta en la observación anterior yace en un fenómeno compartido de igual modo una vez finalizada la ejecución de la etapa de decodificación de una instrucción de transferencia de control incondicional o bien luego de completarse la etapa de ejecución de una instrucción de salto condicional cuya condición asociada resulta cumplirse una vez evaluada la misma en dicha etapa. Aunque las consecuencias para ambos casos se encuentran reflejadas y detalladas en las figuras 74 y 76 respectivamente, podría definirse aquí que básicamente consisten en una alteración en el flujo de ejecución actual del programa, la cual a su vez implicará, como ya se conoce, la cancelación de la instrucción inmediatamente posterior a la instrucción de transferencia de control responsable de haber ocasionado el salto en cuestión. Esta terminación prematura se provocará en las dos situaciones a partir de la etapa de decodificación de la instrucción cancelada, ya habiéndose encontrado afectada en el caso de la ejecución de una instrucción de transferencia de control condicional por una demora generada previamente debido a que la misma no podía continuar ejecutándose hasta que la evaluación de la condición de salto involucrada hubiera logrado determinar si resultaba efectivamente correcto y adecuado que esta instrucción se ejecutara o bien, como ocurrió en este caso, debe efectuarse el salto y eliminarse la instrucción del flujo de ejecución del programa. Por su parte, se había afirmado en las observaciones redactadas para ambas figuras que la acción de cancelar tempranamente la ejecución de una instrucción no constituía técnicamente un atasco en el cauce del procesador ya que en realidad en ningún momento se estaba demorando la ejecución de alguna de las instrucciones del programa que estuvieran activas en aquel instante sino simplemente descartando una instrucción que ya había comenzado a ejecutarse, pero aún no había finalizado. Sin embargo, y ahora centrándose con mayor énfasis en los aspectos cuyo estudio se encuentra especialmente relacionado con la modalidad de atasco presentada más recientemente aquí, deberá tenerse en cuenta un concepto expuesto previamente en las figuras 54 y 55, así como también al principio de la subsección 3.1.8., más precisamente a lo largo de la breve descripción redactada sobre los riesgos de la segmentación que podrían presentarse en el cauce de la CPU a causa de la ejecución de instrucciones de transferencia de control incondicional.

Este concepto establecía que, durante la ejecución segmentada de cualquier programa, toda instrucción que comienza a ejecutarse, a excepción de la primera, incrementa en un ciclo de reloj adicional el tiempo de ejecución total del programa, siempre y cuando su ciclo de ejecución sea idéntico al de su instrucción previamente ejecutada. No obstante, esta apreciación final resulta irrelevante aquí, ya que el interés radica únicamente en su instante de inicio y no en la estructura ni la duración de su ciclo de ejecución, por lo que las conclusiones que se obtengan en este apartado serán igualmente válidas para cualquier instrucción tempranamente anulada, independientemente de si la misma involucra o no la realización de una operación aritmética en punto flotante en la FPU. Ahora bien, la cancelación prematura de una instrucción implica que su ejecución fue incorrectamente inicializada, por lo cual, basándose en el concepto primeramente definido, podría afirmarse entonces que en realidad este error estaría implicando como consecuencia un ciclo de reloj desperdiciado que aumentó innecesariamente el tiempo total del programa y repercutirá negativamente sobre el rendimiento del procesador. A su vez, esto conllevaría que, si la instrucción cancelada se situara entre la instrucción que utiliza la FPU de la CPU y la instrucción “halt”, su terminación tendría repercusiones similares a aquéllas vinculadas a la presencia de un atasco entre ambas instrucciones: podría retrasarse la ejecución de la etapa de decodificación de la instrucción “halt” hasta tal punto que existiría la posibilidad de que, contradiciendo la hipótesis planteada previamente a la ejecución del programa, la separación entre las dos instrucciones no alcance a superar la distancia límite, pero aun así no aparezca el atasco característico de la instrucción “halt” dado que la instrucción que efectúa la operación aritmética en punto flotante avanzó con su ejecución hasta una etapa lo suficientemente avanzada de su ciclo de ejecución como para que se cumplan las condiciones mínimas necesarias que logren garantizar que la unidad de control pueda proceder sin demora alguna con la emisión de la orden de detención para todas las unidades de ejecución del procesador sin comprometer la correcta finalización de todas las instrucciones del programa que se encuentren activas en ese momento. En resumen, durante cualquier análisis detallado que se realice a posteriori de la ejecución de un programa deberán tenerse las mismas consideraciones y precauciones para examinar la cancelación temprana de una instrucción que aquéllas recomendadas para la inspección de un atasco tradicional presente en el cauce segmentado del procesador.

- Asimismo, la terminación prematura del ciclo de ejecución de una instrucción como consecuencia de una alteración en el flujo normal del programa provocada por una instrucción de transferencia de control incondicional o condicional implica que dicha instrucción cancelada ya no será tenida en cuenta durante el análisis llevado a cabo por la unidad de control de la CPU en la etapa de decodificación de la instrucción "halt". Como se recordará, el objetivo de este análisis es determinar si el programa cumple con los requisitos de ejecución mínimos requeridos para emitir la orden de detención a todas las unidades de ejecución del procesador sin comprometer la correcta finalización de las instrucciones que aún se encuentren ejecutándose en el cauce del procesador. El motivo por el cual la instrucción anulada no es relevante para la comprobación realizada por la unidad de control yace en que había sido establecido durante la descripción de las condiciones de ejecución requeridas junto con su respectivo procedimiento de análisis que únicamente debían considerarse las etapas pendientes por atender de los ciclos de ejecución de las instrucciones todavía activas. Por lo tanto, la instrucción cancelada y excluida del flujo del programa no formaría parte de este último conjunto: al anularse, dejaría de estar en ejecución, no le restarían etapas pendientes por ejecutar y debería ser ignorada, descartada y apartada por completo del proceso de verificación de las condiciones. En otras palabras, a efectos prácticos cualquier instrucción anulada prematuramente será tratada por la unidad de control durante la comprobación del mismo modo que todas las instrucciones del programa cuya ejecución ya se hubiera completado o bien jamás hubiera comenzado a causa de algún salto en el programa que directamente haya evitado por completo su inclusión en el flujo de ejecución, a diferencia de la instrucción inhabilitada, que al menos había pertenecido brevemente a él y comenzado su ejecución antes de su abrupta y temprana expulsión. Esta excepción realizada durante la validación de las condiciones previa a emitir la orden de detención es mencionada aquí debido a que constituye uno de los posibles motivos por los cuales podría presentarse una situación inversa a cualquiera de las descriptas en las observaciones anteriores. El caso sería la siguiente: asumiendo que la separación dispuesta en un programa entre la instrucción "halt" y aquélla vinculada al uso de la FPU fue calculada antes de ejecutar el programa y establecida como suficiente para superar la distancia límite, una vez ejecutado efectivamente el programa se detecta que, tal como había sido previsto, la ejecución de la instrucción "halt" no se encontró particularmente afectada por ningún tipo de atasco originado

por el incumplimiento de las condiciones requeridas para que la unidad de control pueda iniciar la detención de la ejecución del procesador. No obstante, si luego se decidiera realizar un análisis pormenorizado del flujo de ejecución del programa, podría arribarse, erróneamente, a la conclusión de que esto no debería haber sido así ya que durante la ejecución del programa la distancia entre ambas instrucciones disminuyó hasta tal punto que ya no alcanzaba a superar la distancia límite, por lo cual tendría que haberse presentado un atasco en la instrucción "halt" hasta que la instrucción que realiza la operación aritmética en punto flotante en la FPU haya progresado con la realización de las etapas de su ciclo de ejecución que hubieran resultado necesarias para que se cumplieran las condiciones mínimas de ejecución del programa y la unidad de control pudiera emitir la orden de detención para todas las unidades de ejecución del procesador. La causa por la cual se obtiene esta conclusión equivocada podría radicar precisamente en que el cálculo de la distancia límite no tiene en cuenta casos particulares como éste en los cuales una instrucción comienza la etapa de búsqueda de su ciclo de ejecución pero, al intentar efectuar la siguiente etapa (decodificación), es cancelada precozmente, por lo cual tanto esta nueva etapa como las restantes que conforman el ciclo de ejecución de la instrucción anulada dejan de tenerse en cuenta durante la validación de las condiciones de ejecución del programa. Sin embargo, al haberse iniciado y completado al menos una de sus etapas antes de su terminación anticipada, esta instrucción, al igual que todas las instrucciones en un cauce segmentado, inevitablemente incrementará en un ciclo de reloj el tiempo de ejecución total del programa y asimismo provocará un retraso de un ciclo adicional para el comienzo de la etapa de decodificación de la instrucción "halt" encargada de validar las condiciones de ejecución del programa y emitir, si éstas se cumplieran, la orden de detención. Ergo, éste se trata de un caso absolutamente excepcional, capaz de disminuir y aumentar simultáneamente la probabilidad de la aparición del atasco ya conocido en la instrucción "halt" y que, por lo tanto, deberá ser especialmente considerado durante el análisis del flujo de ejecución del programa ya que fácilmente podría llevar al lector a arribar a conclusiones precipitadas y erróneas sin alcanzar a comprender adecuadamente los motivos que lo indujeron a cometer dichos errores.

- Sería conveniente mencionar aquí algunas consideraciones a tener en cuenta con respecto al mecanismo requerido para realizar el cálculo de la distancia referenciada en las observaciones anteriores: en primer lugar, en caso de que solamente una de

las instrucciones del programa en cuestión necesite realizar una operación aritmética en punto flotante en la FPU del procesador, entonces obvia y simplemente deberá determinarse la separación existente entre dicha instrucción y la instrucción “halt” a fin de comprobar si excede o no la distancia límite y así verificar si se presentará o no un atasco en la instrucción “halt”. Se recuerda que la conclusión obtenida de esta comprobación será concluyente siempre y cuando se cumplan además las condiciones también establecidas previamente: ausencia de cualquier otro tipo de atasco entre la instrucción directamente vinculada al uso de la FPU y la instrucción “halt” que provoque una demora en la ejecución de esta última; e inexistencia de instrucciones de transferencia de control que alteren el flujo de ejecución del programa de manera tal que se modifique la distancia inicialmente esperada entre ambas instrucciones durante el ensamblaje de dicho programa.

- Por su parte, si el programa no presenta instrucciones que realicen uso alguno de la FPU de la CPU para ejecutar operaciones aritméticas en punto flotante, entonces resulta innecesario llevar a cabo cualquier tipo de cálculo para determinar una distancia entre dos instrucciones del programa, tal como sí era requerido hacerse en el caso del inciso anterior. Básicamente, esto se debe a que precisamente este nuevo tipo de atasco en la instrucción “halt” descripto en la subsección actual del informe surge, tal como ya fue definido, a causa de la ejecución de una instrucción que realiza una operación aritmética en punto flotante y cuyo ciclo de ejecución, al ser más complejo y prolongado que el del resto de las instrucciones del repertorio, incrementa el tiempo de ejecución total de esta instrucción. Dicho incremento es de tal magnitud que existe la posibilidad, si la instrucción se encuentra entre las últimas del programa en ser ejecutadas, de que ésta complete sus postremas etapas luego de la finalización de la instrucción “halt”. A su vez, si esto sucediera, ocurriría el riesgo de que tanto la emisión de la orden de alto, transmitida durante la decodificación de la instrucción “halt”, como la consecuente detención de las unidades de ejecución del procesador necesarias para completar las etapas pendientes de la instrucción que requiere utilizar la FPU se efectúen antes de que dichas etapas se hayan atendido y ejecutado. Ergo, la mera ausencia de instrucciones en un programa que realicen operaciones aritméticas en punto flotante bastará para garantizar sin ningún tipo de duda que, en cualquier situación, al presentar todas las instrucciones del programa una estructura idéntica para su ciclo de ejecución, la instrucción “halt” siempre será la última del programa en finalizar su ejecución. Por lo tanto, siempre se cumplirán

aquí las condiciones de ejecución necesarias para que la instrucción “halt” pueda proceder normalmente con la correspondiente transmisión de su orden de detención total sin que se presente en ningún momento este tipo de atasco en el cauce segmentado de la CPU.

- Tal vez la situación más complicada esté conformada por un programa que se encuentre constituido por múltiples instrucciones que ejecutan operaciones aritméticas en punto flotante en la FPU de la CPU. Aun así, la implementación del mecanismo de resolución no tendría que resultar demasiado difícil: aquí simplemente deberá procederse como punto de partida para realizar el cálculo de la distancia a la instrucción que se encuentre más cercana a la instrucción “halt” de entre todas aquéllas que se encuentren directamente vinculadas a la utilización de la FPU. Expresado en otras palabras, al tratarse la instrucción “halt” de la última del programa en iniciar su ejecución, entonces la distancia a calcular sería aquella existente entre ella y la postrimera instrucción del programa que realice una operación aritmética en punto flotante en la FPU dentro del flujo efectivamente seguido a través de la ejecución del programa. Es imprescindible insistir y enfatizar el concepto de que la conclusión obtenida del cálculo anterior con respecto a la presencia o ausencia de este nuevo tipo de atasco en el cauce del procesador será válida y aplicable siempre y cuando la instrucción del programa que hace uso de la FPU cuya ubicación sea la más cercana a la instrucción “halt” en el ensamblaje represente realmente la última instrucción de este tipo en completarse durante la ejecución del programa. Esto se encuentra íntimamente relacionado con las condiciones establecidas previamente para determinar la veracidad irrefutable de este cálculo realizado: el orden de ensamblaje de las instrucciones debe coincidir con su secuencia de ejecución. Naturalmente, sigue manteniéndose el requerimiento de que no puede existir ninguna otra clase de atasco entre la última instrucción que realiza una operación aritmética en punto flotante y la instrucción “halt” que sea capaz de retrasar la ejecución de esta última hasta tal punto que el nuevo atasco característico de ella no se provoque en el cauce segmentado debido al cumplimiento de las condiciones de ejecución del programa necesarias para que pueda procederse con la emisión de la orden de detención para todas las unidades de ejecución del procesador sin comprometer la correcta finalización del programa ni de ninguna de sus instrucciones activas durante la transmisión de dicha orden.

- Por todas las razones previamente mencionadas, se recomienda, para cualquier caso en el que se presente en el programa al menos una instrucción que requiera utilizar la FPU de la CPU para llevar a cabo una operación aritmética en punto flotante, complementar el cálculo realizado de la distancia existente entre esta instrucción y la instrucción “halt” con un estudio de la secuencia de ejecución segmentada de las instrucciones del programa, o al menos de una representación teórica del flujo de ejecución esperado para las mismas, a fin de analizar si se presenta efectivamente este nuevo atasco en el cauce del procesador al intentar efectuarse la decodificación de la instrucción “halt”. Dicho análisis consistiría en la implementación del mismo mecanismo expuesto y utilizado anteriormente: en primer lugar, clasificar y determinar la cantidad de etapas pendientes de atender pertenecientes a los ciclos de ejecución de todas las instrucciones del programa que todavía no se hubieran completado una vez emitida la orden de detención por la unidad de control durante la etapa de decodificación de la instrucción “halt” si esta etapa se ejecutara sin demora alguna. Luego, debería comprobarse que se cumplan las condiciones mínimas de ejecución, es decir, que la cantidad calculada no exceda el número máximo de etapas pendientes de cada tipo permitidas, la cual había sido determinada, como se recordará, a partir del estudio del estado de la ejecución, luego de transmitirse la orden de alto, de un programa que presenta el mismo ciclo de ejecución para todas sus instrucciones, representado en la figura 77 y descripto en las observaciones de esta misma figura así como también de la figura 78: una etapa de ejecución, dos de acceso a memoria y tres de almacenamiento en registro. No obstante, debe tenerse en cuenta que tanto la etapa de ejecución como una de las etapas de acceso a memoria y otra de las de almacenamiento en registro corresponden al ciclo de ejecución de la propia instrucción “halt”, por lo que, en realidad, sólo quedarían genuina y estrictamente a lo sumo una etapa de acceso a memoria y dos de almacenamiento en registro cuya atención pueda encontrarse todavía pendiente después de la transmisión de la orden de parada y sean capaces de formar parte de los ciclos de ejecución de otras instrucciones del programa aún sin finalizar que sean anteriores y distintas de la instrucción “halt”. Del mismo modo, podría concluirse entonces que, al hallarse necesariamente vinculada cada una de estas dos etapas de almacenamiento en registro a un ciclo de ejecución distinto y al estar la etapa de acceso a memoria obligatoriamente asociada a uno de estos dos ciclos (más precisamente, tal como lo muestra la figura 77, a aquél cuyo instante de

inicio haya sido el más reciente de ambos), el número de instrucciones activas en el cauce del procesador una vez emitida la orden de detención no puede ser superior a dos, o tres si se incluyera la instrucción “halt”. Se notará que esta última conclusión no consideró durante su análisis la estructura del ciclo de ejecución de las instrucciones sin finalizar, es decir, incluso si se cumpliera la condición aquí planteada, en caso de que alguna de estas instrucciones precisara realizar una operación aritmética en punto flotante en la FPU de la CPU, puede comprobarse el ejemplo exhibido en las figuras 78 y 79 para verificar que la cantidad adicional de ciclos de reloj requeridos por esta instrucción para completarse garantiza que deberá demorarse temporalmente la decodificación de la instrucción “halt” a fin de evitar que la instrucción vinculada al uso de la FPU sea detenida prematura y erróneamente durante la finalización del programa y la detención del procesador. Esta demora se prolongará exactamente hasta que la instrucción que hace uso de la FPU efectúe y termine la tercera subetapa de su ejecución en punto flotante, logrando asegurarse así, al igual que en cualquiera de los demás escenarios ya conocidos, que la orden de alto no interrumpa anticipadamente ninguna de las instrucciones del programa que todavía se estén ejecutando.

- Desafortunamente, podría afirmarse entonces que la tarea de calcular la separación entre las dos instrucciones a fin de compararla con la distancia límite y extraer conclusiones, aunque útil, no es capaz de ofrecer definiciones ni soluciones categóricas, funcionando pura y exclusivamente a modo de indicio para anticipar si existe una mayor probabilidad, según el resultado obtenido de la verificación, de que se presente o no el nuevo atasco en el cauce del procesador durante la ejecución de la instrucción “halt” del programa.
- Retornando y prosiguiendo con el análisis del caso representado en la figura 80, se determina que la distancia crítica a ser calculada para obtener el valor límite buscado sería aquélla existente entre las instrucciones I_4 e I_8. Naturalmente, la justificación para esta determinación consiste en que I_4 se trata de la única instrucción del programa allí propuesto y ejecutado que utiliza la FPU del procesador para realizar una operación aritmética en punto flotante y, en consecuencia, posee un ciclo de ejecución especialmente complejo y prolongado, requiriendo un mayor número de ciclos de reloj para completarse que las demás instrucciones del programa y resultando directamente responsable del atasco provocado en la instrucción “halt” a causa del incumplimiento de las condiciones mínimas de ejecución para proceder

normalmente con la finalización de dicho programa. Por su parte, I_8 es precisamente la instrucción "halt", la última del programa, encargada de la terminación correcta de este último emitiendo la orden de detención para todas las unidades de ejecución del procesador.

- Por su parte, se había establecido que el tiempo de duración del atasco propio de la instrucción "halt" en el programa ejecutado en la figura 80 era exactamente igual a un único ciclo de reloj, tratándose éste en realidad del noveno ciclo de la ejecución total del programa, dado que el siguiente ciclo demorado, es decir, el décimo del programa, se debía a la aparición de un riesgo estructural a causa de múltiples intentos de acceso simultáneos a la memoria de datos de la computadora, en este caso por parte de las instrucciones I_4 e I_7. Asimismo, este último atasco no sólo involucraba a la instrucción "halt", I_8 en aquella figura, sino también a su instrucción inmediatamente anterior, es decir, I_7, la cual aún se encontraba en ejecución allí en ese momento y era directamente responsable, junto con la instrucción I_4, también activa en aquel instante, de la existencia de dicho atasco en el cauce segmentado del procesador, por las razones aquí mismo ya mencionadas.
- Una de las características más apreciables de la demora cuya presencia afecta exclusivamente a la ejecución de la instrucción "halt" del programa propuesto para la figura 80, previamente identificada como I_8, consiste en que su duración, si se la analiza detenidamente, involucra un intervalo de tiempo extremadamente reducido. De hecho, al menos en la versión de la segmentación actualmente implementada en el cauce de este procesador, únicamente pueden surgir atascos cuyo lapso se encuentre definido por una cantidad entera y, obviamente, positiva, de ciclos de reloj. Por lo tanto, bajo ningún concepto podrán acontecer dilaciones que se extiendan durante un espacio de tiempo menor a un ciclo de reloj, tratándose este último a su vez precisamente del espacio de tiempo a través del cual la instrucción "halt" permanece demorada en el caso de la figura 80 estrictamente a causa de la existencia de este nuevo tipo de riesgo, cuyo origen se encuentra intrínsecamente asociado a la ejecución de la etapa de decodificación de la susodicha instrucción. Ergo, se estaría definitivamente en condiciones de afirmar sin lugar a dudas que la duración de este mismo retraso en el ejemplo de la figura 80 es la más breve posible para cualquier caso imaginable.
- Por consiguiente, a partir de la observación anterior se extraería una conclusión muy importante con respecto a la distancia allí existente entre las instrucciones I_4 e I_8,

cuyo cálculo ya había sido establecido como crítico para determinar el valor límite deseado: dicha distancia ya se encuentra, en el peor de los casos, muy cercana del límite que se pretende averiguar. En el mejor de ellos, la figura 80 ya estaría reflejando efectivamente la situación límite buscada, en la cual las instrucciones I_4 e I_8 poseen una separación entre sí insuficiente como para lograr que se satisfagan las condiciones de ejecución requeridas para evitar la aparición del atasco ya conocido en el cauce de la CPU, asociado a la ejecución de la etapa de decodificación de la instrucción "halt". No obstante, esta separación alcanza para que I_4 sea capaz de progresar con su ciclo de ejecución hasta un punto tal que, cuando la unidad de control realiza la comprobación del estado de la ejecución del programa en el momento previamente a emitir la orden de detención para todas las unidades del procesador, a I_4 sólo le restaría por completar una subetapa de su ejecución en punto flotante para que la transmisión de la orden pueda proceder sin que I_4 finalice prematuramente. Como ya se sabe, cualquiera de estas subetapas tiene una duración de un ciclo de reloj (ver tabla 49), el cual coincide y explica a la perfección el tiempo de demora de este mismo tipo de atasco en el escenario de prueba de la figura 80. Si esta última hipótesis fuera válida y se estuviera realmente ante el mejor de los casos, la distancia que se calcule y obtenga resultaría ser la más amplia posible que continúe posibilitando la aparición de la clase de atasco ya mencionada y especialmente estudiada en la presente subsección en el cauce del procesador.

- El afán tan imperioso por averiguar y establecer con exactitud este límite se debe, como se recordará, a que el valor que se obtenga permitiría demarcar automáticamente la separación entre los casos en los cuales podría presentarse este nuevo tipo de atasco y aquellas situaciones en las que definitivamente jamás habrá posibilidad alguna de la aparición del mismo en el cauce del procesador. De este modo, ya no sería necesaria, en condiciones normales, la tarea de realizar un análisis conceptual como se hizo en todas las figuras de la actual subsección para evaluar el cumplimiento o incumplimiento de las condiciones mínimas de ejecución para que la instrucción "halt" pueda proceder normalmente con su ejecución, sin atasco o dilación alguna en su etapa de decodificación.
- A continuación, se procederá con el cálculo propiamente dicho de la distancia crítica del programa ejecutado en la figura 80. Para ello se utilizarán los dos mecanismos previamente explicados: por un lado, se determinará el valor a partir del orden de la

ubicación de las instrucciones durante el proceso de ensamblaje y, por otro, se analizará pormenorizadamente el flujo de ejecución del programa una vez finalizado el mismo para comprobar si se mantiene la distancia originalmente calculada y se acepta la hipótesis inicialmente planteada, sea cual fuere, dado que aún no se posee información suficiente como para predecir a partir de una determinada distancia si puede esperarse o no la aparición de un atasco durante la decodificación de la instrucción "halt".

- Para emplear el primer método de cálculo, sería recomendable comenzar recordando el panorama planteado inicialmente en la figura 80. Éste consistía en que, por un lado, existe una única instrucción del programa, I_4, cuyo ciclo de ejecución es excepcionalmente largo por encontrarse destinada al uso de la FPU de la CPU para ejecutar una operación aritmética en punto flotante. Por su parte, la instrucción "halt" es I_8, la cual, si el programa posee el comportamiento esperado, debería ser la última en ejecutarse. Luego, se debería proceder a observar el orden de las instrucciones del programa, principalmente I_4 e I_8, antes de que comience su ejecución: I_4 ocupa el cuarto lugar e I_8, la octava y postrema ubicación. A partir de esta nueva información, un sencillo cálculo permite determinar que la distancia crítica entre I_4 e I_8 es igual a cuatro instrucciones o, en otras palabras, que I_4 es la instrucción inmediatamente anterior a la trasantepenúltima del programa, o bien la quinta del mismo si se invirtiera el orden de ejecución de todas las instrucciones. Del mismo modo, otra posible lectura o interpretación del resultado obtenido a partir del cálculo realizado consistiría en que hay tres instrucciones situadas entre I_4 e I_8, siendo en este caso las mismas I_5, I_6 e I_7.
- Ahora se implementará el segundo mecanismo, el cual implica un análisis pormenorizado del flujo de ejecución seguido por el programa una vez finalizado el mismo, el cual ya se encuentra representado en la misma figura 80. Este estudio deberá centrarse especialmente en el intervalo de tiempo transcurrido entre la finalización de la etapa de búsqueda de la instrucción I_4 y el primer intento llevado a cabo para comenzar la etapa de decodificación de la instrucción I_8, ya que cualquier fenónemo que altere la ejecución normal del programa podría modificar la distancia originalmente calculada por medio del método utilizado en la observación anterior. Así, puede concluirse que, al no existir atasco alguno en el cauce del procesador durante el período mencionado debido a conflictos originados por dependencias estructurales, de datos o de control entre dos de las instrucciones

del programa, todas ellas ya presentadas y explicadas en las subsecciones anteriores del informe, ni tampoco ejecutarse ningún tipo de instrucción de transferencia de control entre I_4 e I_8 que modifique el flujo de ejecución seguido por el programa en este espacio de tiempo, la distancia inicialmente determinada en el ensamblaje del programa permanece inalterable. Por lo tanto, la hipótesis que se hubiera planteado antes de conocer el progreso de la ejecución del programa sería válida: si se predijo que la decodificación de la instrucción "halt" no se atascaría, así será, mientras que, si se anticipó la situación contraria, entonces la decodificación se verá demorada hasta que se cumplan las condiciones mínimas de ejecución del programa para que pueda efectuarse en forma segura. Próximamente se ofrecerán ejemplos en los cuales esto no ocurra: las distancias críticas calculadas en ambos mecanismos no coincidirán entre sí, con la alarmante pero latente posibilidad de que la hipótesis planteada inicialmente en el ensamblaje del problema con respecto a la presencia o ausencia del atasco durante la decodificación de la instrucción "halt" termine rechazándose en caso de fallar su comprobación durante el análisis del flujo de la ejecución del programa.

- Un comentario general con respecto al primer intento de comenzar la etapa de decodificación de la instrucción "halt", es decir, I_8 en el caso del programa aquí analizado, es la siguiente: dicho intento podría ser exitoso y por lo tanto no repetirse a lo largo del resto de la ejecución del programa bajo estudio, o bien, como ocurre en el mismo caso de prueba propuesto en la figura 80, resultar infructuoso a causa de la existencia de algún atasco que surja en el cauce del procesador durante la frustrada decodificación. La naturaleza de este último es irrelevante, pudiendo encontrarse vinculada al incumplimiento de las condiciones mínimas de ejecución necesarias para comenzar la finalización del programa y la detención de la CPU, o bien estar asociada a cualquier otro conflicto debido a alguna dependencia sin resolver entre dos instrucciones, ya sea de carácter estructural, de datos o de control. Ambas variantes poseen la misma capacidad de abortar cualquier intento de decodificación de la instrucción "halt" que pretenda efectuarse mientras el atasco en cuestión se halle en el cauce y, obviamente, esta instrucción se encuentre directamente afectada por el retraso ocasionado por él. Si se presentara esta última situación y la etapa de decodificación se viera obligada a demorarse temporalmente, podría predecirse indudablemente que ocurrirá al menos un nuevo intento en algún instante posterior de la ejecución del programa una vez que el riesgo responsable

del atasco haya conseguido solucionarse y la instrucción “halt” en particular esté en condiciones de reanudar su ejecución aplazada. De hecho, en el programa planteado a modo de ejemplo en la mencionada figura se pueden localizar tres intentos de inicializar la etapa de decodificación de I_8, ubicados uno inmediatamente a continuación del otro: los dos primeros fallan respectivamente en virtud de la aparición de un atasco como consecuencia el incumplimiento de las condiciones para la finalización del programa y otro provocado por una dependencia estructural entre las instrucciones I_4 e I_7. Únicamente luego de haberse resuelto ambos riesgos, el resultado del tercer intento logra ser satisfactorio y la decodificación de I_8 puede proceder y finalizar con normalidad. Aun así, se recuerda que el intervalo de tiempo de interés para el análisis del flujo de ejecución del programa concluye exactamente en el instante de inicio del primer intento de realizar la decodificación de la instrucción “halt”, independientemente de si la misma se ejecuta sin inconveniente alguno o si por el contrario surge un conflicto que fuerce transitoriamente su demora. El comentario ofrecido en el párrafo actual fue concebido simplemente para evitar que el lector padezca confusiones durante la determinación del instante correcto para la finalización del susodicho período de estudio de entre todos los valores de tiempo posibles si existieran múltiples intentos en el programa de comenzar la decodificación de la instrucción “halt”. Del mismo modo, este instante también podría referirse como aquél en el cual termina la etapa de búsqueda de la misma instrucción “halt”. Al igual que cualquier otra etapa del ciclo de ejecución, la búsqueda de una instrucción en particular sólo puede finalizarse en una única ocasión, por lo cual esta expresión para definir el instante sería unívoca y, por lo tanto, más fácil le permitiría al usuario detecta

- Tal vez otra expresión igualmente adecuada para referirse a este instante sería el momento de finalización de la etapa de búsqueda de la instrucción “halt”, el cual sería exactamente idéntico al anterior, pero la expresión resultaría unívoca y por lo tanto, más fácil de comprender y menos susceptible de prestarse a confusiones, aunque con la desventaja de que sería más difícil encontrar una relación directa entre dicha etapa y la presencia de esta atasco, asociado estrictamente a la decodificación de la instrucción “halt” en lugar de la búsqueda. Por ello, se plantean ambas expresiones para el instante en cuestión y se propone al usuario que opte por aquélla que le resulte más conveniente ya que ambas son igualmente aceptables y se refieren al mismo instante de ejecución del programa debido a que la decodificación de la

instrucción “halt” comienza inmediatamente a continuación de que finaliza su etapa de búsqueda. Como se sabe, una se encuentra enseguida de la otra dentro de la estructura del ciclo de ejecución de dicha instrucción (ver tabla 48).

- Por su parte, otro enfoque válido a considerar durante este análisis pormenorizado consistiría en calcular directamente el número de ciclos que conforman el intervalo de duración transcurrido entre el instante de finalización de la instrucción I_4 y el de inicio del primer intento de decodificación de la instrucción I_8. Éste, como se recordará, es precisamente el espacio de tiempo que resulta especialmente relevante para estudiar el comportamiento del programa y así establecer conclusiones con respecto a la aparición del atasco asociado a la imposibilidad temporal de la unidad de control de emitir la orden de detención para todas las unidades de ejecución del procesador. Si se analiza la figura 80, y se considera que cada columna equivale al transcurso de un ciclo de reloj, independientemente del período de reloj que posea la CPU en particular, puede concluirse que transcurren exactamente cuatro ciclos de reloj entre los dos instantes previamente planteados. Este valor coincide perfectamente con la distancia calculada inicialmente entre las instrucciones I_4 e I_8 mediante el primer mecanismo, antes de conocer el flujo de la ejecución del programa, y asimismo confirma la conclusión establecida anteriormente: la ausencia de cualquier fenómeno que altere la ejecución normal del programa durante el intervalo de tiempo ya conocido y estudiado con profundidad posibilita que la separación entre ambas instrucciones permanezca inalterable durante la ejecución del programa con respecto a la distancia evaluada en el ensamblaje del programa.
- ¿EL INTERVALO DE TIEMPO DE INTERÉS NO DEBERÍA SER DESDE EL FIN DE LA ETAPA DE BÚSQUEDA DE I4 HASTA EL COMIENZO DE LA ETAPA DE DECODIFICACIÓN DE I8? ¿CUÁL ES LA RELACIÓN ENTRE LA DISTANCIA CRÍTICA OBTENIDA POR MEDIO DEL PRIMER MECANISMO Y LA CANTIDAD DE CICLOS DE RELOJ TRANSCURRIDOS ENTRE AMBAS ETAPAS EN EL FLUJO DE EJECUCIÓN (SEGUNDO MECANISMO)? REVISAR.
- VERIFICAR EN EL SET DE INSTRUCCIONES LAS OPERACIONES DE DESPLAZAMIENTO DE BITS. ¿SON DE DESPLAZAMIENTO (SE DESCARTAN LOS BITS QUE SUPERAN LOS EXTREMOS) O DE ROTACIÓN (SE CONSERVAN TODOS LOS BITS COMO SI AMBOS EXTREMOS DEL OPERANDO ESTUVIERAN INTERCONECTADOS)?

- Cabe aclarar que, en este programa, los únicos aspectos que resultan genuinamente relevantes son la estructura del ciclo de ejecución de la instrucción I_4 y la naturaleza particular de la instrucción I_8. En el primer caso, debido a que la extraordinariamente prolongada duración de I_4 es directamente responsable del posible atasco cuya aparición se pretende predecir a partir del cálculo de la distancia crítica y su posterior comparación con el valor límite aún desconocido, mientras que I_8, al tratarse de la instrucción "halt", será la que se encuentre directamente afectada por la demora asociada a dicho atasco. Con respecto al resto de las instrucciones, su intrascendencia es tal que podría llevarse a cabo una nueva ejecución del mismo programa de la figura 80 intercambiando entre sí, por ejemplo, las instrucciones I_5 e I_7 con las instrucciones I_1 e I_2 respectivamente. El análisis ulterior debería determinar, una vez finalizado el programa, que las instrucciones localizadas entre I_4 e I_8 en el flujo de ejecución son ahora las instrucciones que solían ser I_1, I_6 e I_2 en el programa original. Sin embargo, siempre y cuando la cantidad de instrucciones que separan a I_4 de I_8 permanezca inalterable, no existiría diferencia alguna con respecto al primer escenario planteado, dado que dicha cantidad constituye la única información cuyo estudio y conocimiento poseen trascendencia aquí ya que sólo este parámetro repercutirá sobre la distancia crítica que se desea calcular: cualquier otra variante no modificará en absoluto el valor obtenido luego de efectuar el cálculo matemático del mismo.
- En esta instancia se cuenta con argumentos considerables que favorecen la hipótesis que sostiene el concepto de que el valor límite buscado se trata exactamente de la separación crítica entre las instrucciones I_4 e I_8 del programa representado y ejecutado en la figura 80, calculada y establecida en la observación anterior. Principalmente, el motivo más relevante consiste en que, tal como ya fue explicado, la duración del atasco exclusivo de la etapa de decodificación de la instrucción "halt" requerido para una correcta finalización del programa es la más breve posible y concebible, por lo cual no habría manera alguna de ampliar la distancia entre la instrucción que ejecuta una operación aritmética en punto flotante en la FPU del procesador, I_4 en este caso, y la instrucción "halt", I_8 en este programa, de un modo tal que este atasco continúe presentándose en el cauce del procesador y demorando la ejecución de la decodificación de la instrucción "halt". En otras palabras, cualquier incremento en la distancia entre ambas instrucciones, aunque sea el mínimo posible, es decir, una única unidad, debería alcanzar para que la instrucción

vinculada al uso de la FPU progrese con la etapa de ejecución en punto flotante de su ciclo de ejecución hasta un punto tal que la cantidad de etapas del mismo que aún permancen sin atender debería ser lo suficientemente reducida como para que logren cumplirse las condiciones mínimas de ejecución del programa requeridas para que la unidad de control pueda proceder a emitir la orden de detención total durante la etapa de decodificación de la instrucción "halt". Si esto sucediera, la transmisión se llevaría a cabo sin retraso alguno sin verse retrasada a causa de este nuevo tipo de riesgo.

- No obstante, a fin de asegurar con una certeza absoluta que efectivamente ya se ha averiguado el valor buscado de la distancia límite, el último paso sería comprobar empíricamente la hipótesis previa planteando el siguiente escenario de ejecución:

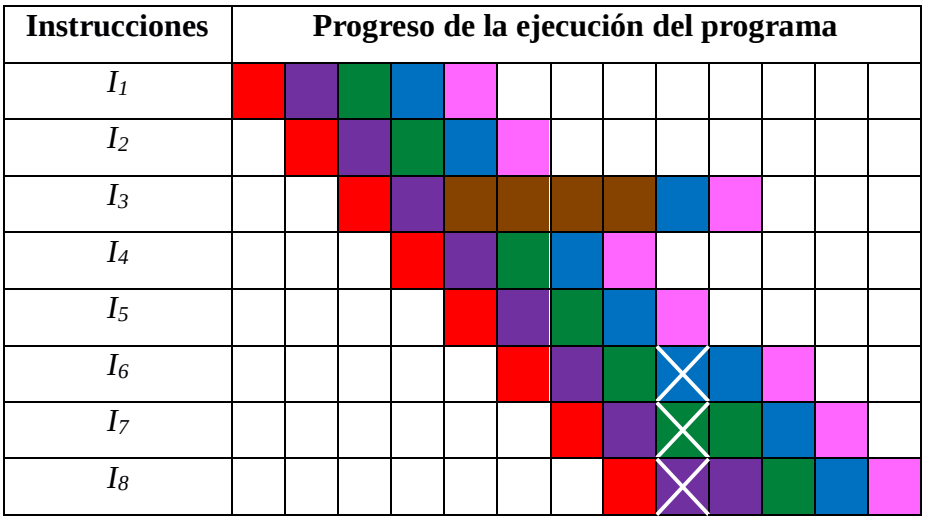


Figura 81. Progreso de la ejecución segmentada de un programa dentro de un escenario similar al de la figura 77, pero cuyas condiciones de prueba fueron ahora forzadas hasta el máximo límite permitido, es decir, sin alcanzar la situación representada en la figura anterior.

- En esta última figura simplemente se representó la situación planteada conceptualmente durante la argumentación anterior con respecto a la alta probabilidad de que la figura 80 constituya el caso límite buscado. Es decir, con el objetivo de incrementar en el mínimo valor posible la distancia crítica que separa a las instrucciones I4 e I8 y comprobar si los cambios en el flujo de la ejecución del nuevo programa coinciden con los esperados, se decidió intercambiar entre sí las

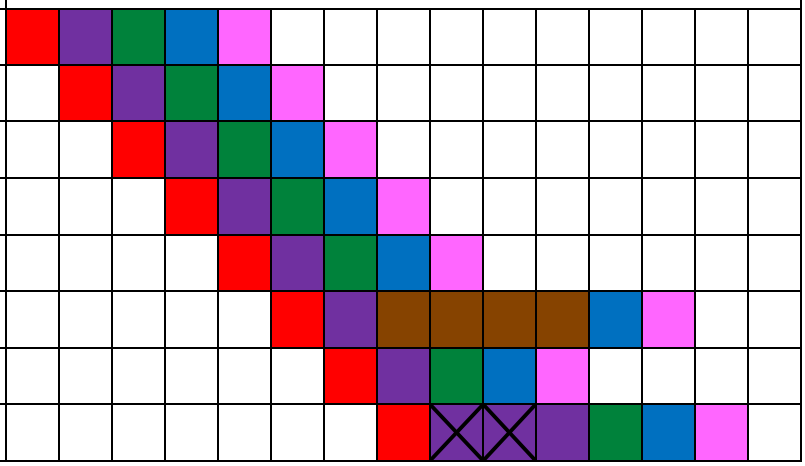
instrucciones I3 e I4. Por lo tanto, como cabía esperarse, en esta ocasión es I3 la instrucción que realiza una operación aritmética en punto flotante a través de la FPU del procesador y posee un ciclo de ejecución más complejo y prolongado debido a las cuatro subetapas que conforman su ejecución en punto flotante. Mientras tanto, ahora I4 es una instrucción cuyo ciclo y tiempo de ejecución es idéntico al de cualquier otra instrucción del programa, excepto, obviamente, la mencionada instrucción I3.

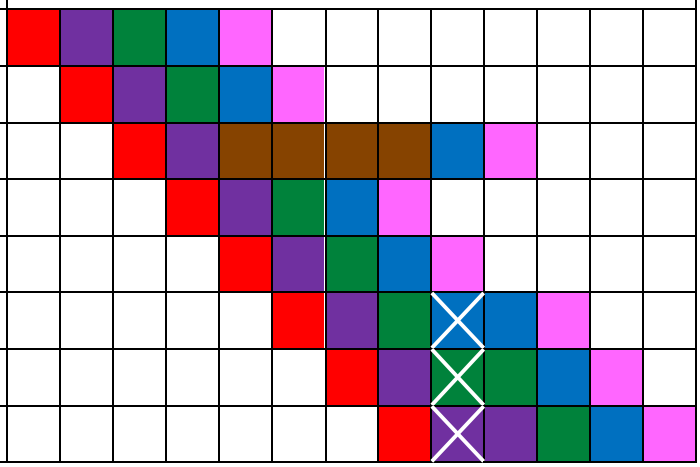
- Como se recordará, la distancia crítica a calcular en cualquier programa es aquélla vinculada entre la instrucción directamente vinculada al uso de la FPU y la instrucción "halt". En el caso de la figura 80, esto implicaba determinar la separación existente entre las instrucciones I4, es decir, la cuarta del programa e I8, es decir, la octava del programa. En este nuevo caso, al retroceder en una sola posición la ubicación de la instrucción que realiza la operación aritmética en punto flotante (aquí es I3, es decir, la tercera del programa, mientras que en la figura 80 era la cuarta), pero mantener inalterable el lugar de la instrucción "halt" (en ambos casos es I8, es decir, la octava de los dos programas), sólo cabe esperarse que la distancia crítica se incremente en una unidad en este nuevo caso con respecto al de la figura 80.
- Asimismo, se enfatiza que la naturaleza particular de las instrucciones I3 e I4 resulta irrelevante en el estudio de este apartado. Es decir, aunque aquí se propuesto a modo de ejemplo la ejecución de un nuevo programa en el cual se intercambiaran entre sí las instrucciones I3 e I4, podría haberse planteado sin convenientes en lugar de cada una de ellas cualquier otra instrucción del repertorio soportado por el procesador, aunque sí se encuentran obligadas a compartir la misma estructura para el ciclo de ejecución que el de las instrucciones que reemplacen: la instrucción que se sitúe en el lugar de I3 debe ejecutar una operación aritmética en punto flotante y la instrucción que sustituya a I4 tiene que poseer un ciclo de ejecución con una etapa de ejecución simple que dure un único ciclo de reloj. Naturalmente, esto no puede aplicarse a la instrucción I8, la cual obligatoriamente debe ser una instrucción "halt" que finalice el programa y detenga el procesador.
- Aunque se afirmó en la observación anterior que el propósito particular de la instrucción I4 es irrelevante aquí, debe tenerse especial consideración en caso de tratarse de una instrucción de transferencia de control, ya que, como fue

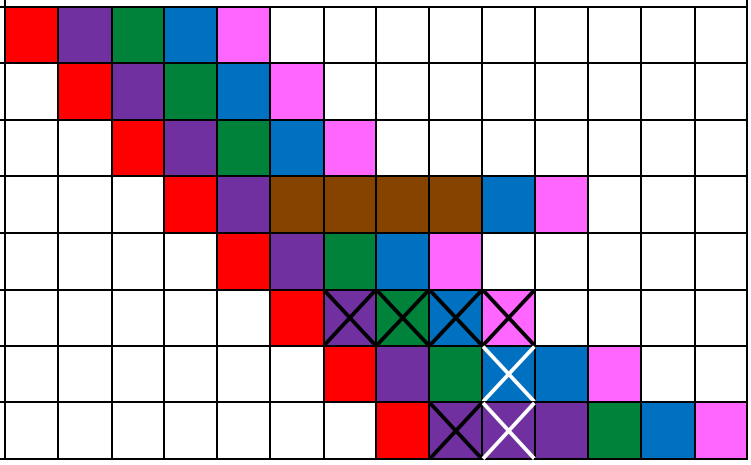
mencionado, este tipo de instrucciones posee la capacidad, bajo ciertas condiciones, de alterar el flujo de ejecución normal del programa y aumentar o disminuir

- Efectivamente, a través de un sencillo cálculo puede determinarse que la distancia crítica, es decir, aquélla ubicada entre las instrucciones I3 e I8, es igual UTILIZAR AMBOS MÉTODOS
- Asimismo, al consistir I_4 en la instrucción del programa planteado en la figura 80 cuyo ciclo de ejecución es excepcionalmente largo e I_8 en la instrucción "halt" del mismo programa, y también al no existir ningún tipo de instrucción de transferencia de control ubicada entre ambas que altere el flujo de ejecución normal del programa, podría calcularse sin mayores inconvenientes que la distancia entre I_4 e I_8 es igual a cuatro instrucciones o, en otras palabras, que I_4 es la instrucción inmediatamente anterior a la trasantepenúltima del programa, o bien la quinta del mismo si se invirtiera el orden de ejecución de todas las instrucciones. Del mismo modo, otra posible lectura o interpretación del resultado obtenido a partir del cálculo realizado consistiría en que hay tres instrucciones situadas entre I_4 e I_8, siendo en este caso las mismas I_5, I_6 e I_7, aunque aquí en realidad resulta irrelevante la denominación y la naturaleza particular de cada instrucción. Es decir, podría llevarse a cabo una nueva ejecución del programa intercambiando entre sí, por ejemplo, las instrucciones I_5 e I_7 con las instrucciones I_1 e I_2 respectivamente. El análisis posterior debería determinar, una vez finalizado el programa, que las instrucciones localizadas entre I_4 e I_8 en el flujo de ejecución son ahora las instrucciones que solían ser I_1, I_6 e I_2 en el programa original. Sin embargo, siempre y cuando la cantidad de instrucciones que separan a I_4 de I_8 permanezca inalterable, no existiría diferencia alguna con respecto al primer escenario planteado, dado que dicha cantidad constituye la única información cuyo estudio y conocimiento poseen trascendencia aquí ya que sólo este parámetro repercutirá sobre la distancia crítica que se desea calcular: cualquier otra variante no modificará en absoluto el valor obtenido luego de efectuar el cálculo matemático del mismo.
-
- ¿SIGUIENTES PASOS A REALIZAR? ¿CUÁL SERÍA O PODRÍA SER EL INTERVALO DE INSTRUCCIONES CRÍTICAS A ANALIZAR ENTONCES?
- INCLUIR MENCIONES A LAS NUEVAS FIGURAS EN LAS OBSERVACIONES PREVIAS SEGÚN CORRESPONDA.

- COMPROBAR SI HACE FALTA AGREGAR NUEVAS FIGURAS PARA REPRESENTAR TODAS LAS VARIANTES POSIBLES DE INCUMPLIMIENTO DE LA RELACIÓN ENTRE EL ATASCO Y LA DISTANCIA MÍNIMA.
- ¿CADA CICLO DE RELOJ DE DEMORA EQUIVALE A UNA UNIDAD MÁS QUE SE DEBE AGREGAR A LA DISTANCIA CALCULADA? COMPROBAR. REALIZAR EJEMPLOS. ARGUMENTAR.
- ¿DURANTE UNA EJECUCIÓN LA DISTANCIA CRÍTICA REAL ESTARÍA DADA POR LA CANTIDAD DE CICLOS DE RELOJ ENTRE EL INICIO DE LA INSTRUCCIÓN FPU Y EL DE LA INSTRUCCIÓN HALT MÁS LOS CICLOS DEMORADOS POR LOS ATASCOS QUE HUBIERA ENTRE AMBAS INSTRUCCIONES? ¿SERÍA EQUIVALENTE A LA CANTIDAD DE CICLOS DE RELOJ ENTRE EL INICIO DE LA INSTRUCCIÓN FPU Y EL DE LA ETAPA DE DECODIFICACIÓN DE LA INSTRUCCIÓN HALT, CUANDO SE EMITE LA ORDEN DE DETENCIÓN? ANALIZAR.
- ¿EXISTE ALGUNA POSIBILIDAD DE QUE SURJA UN ATASCO EN EL CAUCE UNA VEZ FINALIZADA LA EJECUCIÓN EFECTIVA DE LA ETAPA DE DECODIFICACIÓN DE LA INSTRUCCIÓN “HALT”? REVISAR Y, SI ES NECESARIO, CORREGIR PÁGINA 181.

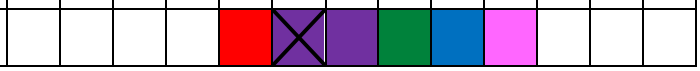
Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_7	
I_8	

Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_7	
I_8	

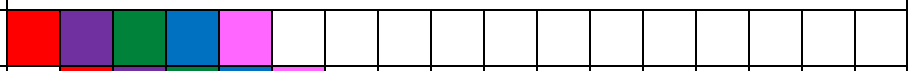
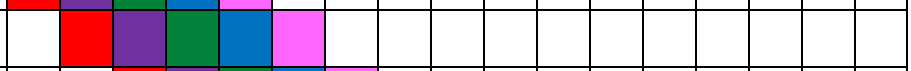
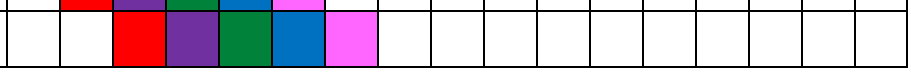
Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_1	
I_8	

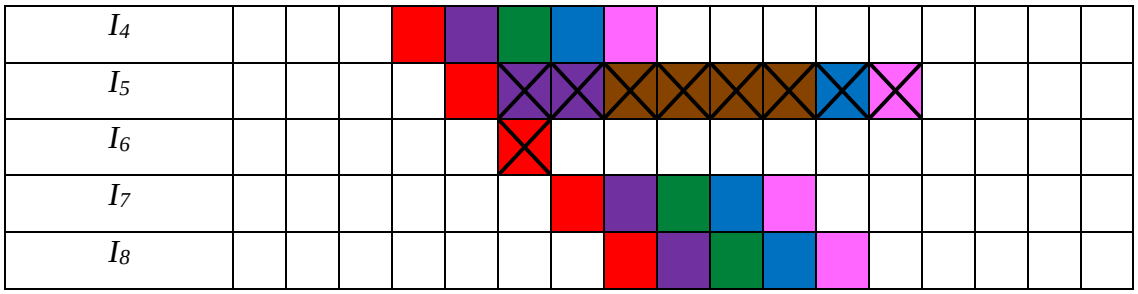
Instrucciones			Progreso de la ejecución del programa														
I_1																	
I_2																	
I_3																	
I_4																	
I_5																	
I_6																	
I_7																	
I_8																	

Instrucciones	Progreso de la ejecución del programa													
I_1														
I_2														
I_3														
I_4														
I_5														
I_6														
I_7														
I_8														
I_9														

Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_7	
I_8	

Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	
I_4	
I_5	
I_6	
I_7	
I_8	

Instrucciones	Progreso de la ejecución del programa
I_1	
I_2	
I_3	



- ¿DETERMINACIÓN EMPÍRICA DE LA DISTANCIA LÍMITE?
- AGREGAR UNA FIGURA EN LA QUE APAREZCAN DOS INSTRUCCIONES FPU PARA DEMOSTRAR CÓMO CALCULAR LA DISTANCIA CRÍTICA Y COMPARARLA CON EL LÍMITE.

3.1.8.7. Programas de prueba

Una vez analizadas todas las figuras presentadas anteriormente entre las subsecciones “3.1.8.1. Dependencias estructurales” y “3.1.8.6. Atascos de fin del programa” para explicar los tipos de riesgos conocidos susceptibles de surgir durante la ejecución de un programa a causa de la implementación de la segmentación en el cauce del procesador, fácilmente el lector podría esgrimir una objeción razonable al respecto. La misma consistiría en que, al encontrarse expresadas las instrucciones de los programas ejecutados en dichas figuras en forma simbólica, es decir, a través de la denominación I_N, siendo N el número de la instrucción ejecutada (I_1, I_2, I_3, etc.), sin ofrecer mayores precisiones, los casos de prueba representados se vuelven demasiado conceptuales y abstractos, dificultándose así su comprensión.

Ergo, aun manteniéndose en el plano teórico, se propone aquí como una siguiente etapa más avanzada de la explicación previa la muestra de ejemplos concretos del progreso de la ejecución esperada a través del tiempo de programas específicos. Éstos fueron especialmente escritos en el lenguaje Assembler, a fin de comprobar y analizar los fenómenos que acontezcan

durante el avance de su ejecución conforme se presenten cada uno de los distintos riesgos de la segmentación anteriormente presentados.

Naturalmente, para la programación del código de estos nuevos programas se utilizaron únicamente las directivas exclusivamente soportadas por el ensamblador de la nueva computadora, las cuales se encuentran definidas en la tabla 37.

En la introducción de este informe se planteó que el proyecto aquí documentado había sido originalmente concebido para su uso académico en una cátedra universitaria conformada por estudiantes avanzados que ya habían adquirido nociones básicas sobre Assembler, definido en aquella sección introductoria como un lenguaje de programación de bajo nivel, durante el transcurso de algunas de las asignaturas anteriores de su plan de estudios. Incluso, ya habían utilizado dicho lenguaje con el objetivo de escribir programas cuyas instrucciones pudieran ser ejecutadas por otros procesadores similares al TDA 1819. De hecho, se conoce que ya han interactuado con los simuladores MSX88 y WinMIPS64, cuyas CPUs descriptas, el Intel 8088 y el MIPS64, constituyeron referencias fundamentales para el autor de este informe, quien se basó en gran medida en su diseño durante la concepción y el desarrollo del nuevo procesador.

Asimismo, se mencionó durante la introducción la consideración inicialmente tenida en cuenta de que, por otra parte, los alumnos de la cátedra ya habían tenido en materias precedentes un contacto previo con algún lenguaje de descripción de hardware, preferiblemente el VHDL, para definir el diseño de circuitos digitales relativamente sencillos. Del mismo modo, se tuvo presente que también en dichas cátedras ya habían interactuado con al menos un simulador, preferentemente el Active-HDL, para comprobar el comportamiento y funcionamiento de los mismos circuitos digitales previamente diseñados por sus propios medios mediante el lenguaje de descripción de hardware que hayan seleccionado y utilizado a tal efecto.

Aunque en aquel momento todavía no se había manifestado el argumento que motivaba la recomendación de contar con una experiencia anterior en el uso del lenguaje VHDL y el simulador Active-HDL en particular, después se definió y explicó a lo largo de la sección “2. Entorno de trabajo” que precisamente éstas habían sido las herramientas respectivamente elegidas por el autor del informe para describir el modelo de la nueva computadora y posteriormente comprobar mediante diversas simulaciones que su comportamiento se correspondiera con el esperado. En consecuencia, cualquier familiarización previa que el lector tuviera con dichas herramientas facilitaría considerablemente el proceso de aprendizaje que inevitablemente deberá atravesar para poder interactuar correcta y adecuadamente con este proyecto, ya sea para programar la CPU mediante código escrito en el lenguaje Assembler o bien realizar tareas más avanzadas como trabajar sobre el propio diseño VHDL de la

computadora, mejorando alguna o varias de sus funcionalidades actuales e incluso agregando otras nuevas.

En resumen, se estableció como prerrequisito para una lectura satisfactoria y productiva de este informe la existencia de algunas nociones mínimas sobre la codificación de programas para microprocesadores y el diseño de circuitos digitales mediante lenguajes de programación de bajo nivel y de descripción de hardware respectivamente, recoméndadose particularmente poseer una destreza previa en el empleo del Assembler en el primer caso y del VHDL en el segundo.

Como ya se habrá notado, en estas secciones relativamente tempranas e iniciales del informe aún no se ha realizado mención alguna sobre ninguno de los atributos principales del diseño definido para el TDA 1819 a través del lenguaje de descripción de hardware optado para el proyecto, VHDL. Únicamente se establecieron en las tablas 37 y 38 las directivas y el repertorio de instrucciones respectivamente válidos y soportados por el ensamblador de la computadora y la unidad de control de la CPU, encargándose el primero de traducir las instrucciones en lenguaje Assembler a código máquina a partir de, principalmente, los códigos de operación definidos en la tabla 39 para todas las instrucciones del repertorio de manera tal que la unidad de control pueda, por su parte, decodificar e interpretar la denominación de la próxima instrucción a ejecutar y así coordinar y gestionar su ejecución propiamente dicha.

En consecuencia, al menos en esta instancia del informe, en caso de ya tener, tal como inicialmente se esperaba, cierta familiarización previa con algún lenguaje de programación de bajo nivel, no debería resultar excesivamente complicada para el lector la tarea de desarrollar por sí mismo programas capaces de ser ejecutados en esta nueva computadora, puesto que, del mismo modo que fue mencionado anteriormente, el TDA 1819 se encuentra basado en gran medida en las descripciones de los diseños de los procesadores Intel 8088 y MIPS64, ofrecidas en los simuladores MSX88 y WinMIPS64 respectivamente, ambos ya asumidos como conocidos para el lector. No obstante, como también se sabrá, al estar los atributos principales de estas CPUs asociados a dos tipos de arquitecturas de computadoras tan distintos, CISC en el caso del Intel 8088 y RISC para el WinMIPS64 (se puede ver en la tabla 1 una comparación general entre ambas arquitecturas), entonces las directivas y los repertorios de instrucciones compatibles con dichos procesadores también serán drásticamente disímiles entre sí. Por lo tanto, aunque se sostenga la afirmación de que el nuevo procesador fue diseñado teniendo en cuenta tanto la arquitectura del Intel 8088 como la del MIPS64, indudablemente en el diseño final del TDA 1819 una de las dos arquitecturas tendrá una mayor preponderancia sobre la otra, ya que sus tipos, CISC y RISC respectivamente, son básica y esencialmente incompatibles entre

ellos. Esta cuestión ya fue planteada, abordada y resuelta en la introducción de esta misma sección del informe, “3. TDA 1819”, así como también plasmada en la tabla 35, en la cual se determinó que la arquitectura de la nueva computadora, aunque presentara algunos atributos característicos de la arquitectura CISC, era predominantemente de tipo RISC. Entonces, se recomienda que el lector se encuentre especialmente adiestrado y versado en el uso del WinMIPS64 ya que, al compartir fundamentalmente el mismo tipo de arquitectura (RISC), el TDA 1819 y el MIPS64 presentarán amplias similitudes entre sus respectivos conjuntos de directivas y repertorios de instrucciones, cuyo conocimiento resulta indispensable para poder programar dichos procesadores a través del lenguaje Assembler. De este modo, si ya se han escrito y ejecutado programas Assembler en el MIPS64, debería ser mínimo el esfuerzo de adaptación para comenzar a realizar lo mismo en el TDA 1819 a partir de las nuevas directivas y repertorio de instrucciones asociados a este último. Por su parte, el Intel 8088, al fundamentarse su modelo en una arquitectura de computadora CISC, radicalmente distinta de aquélla en la cual se basa el diseño de la computadora de este proyecto, no tendrá una relevancia ni un peso tan importantes como el MIPS64 en el proceso de aprendizaje necesario para una correcta programación del TDA 1819. No obstante, aunque no tan enfáticamente, se aconseja asimismo contar con una interacción relativamente reciente con el MSX88 dado que algunas de las características de la descripción del Intel 8088 dispuestas en aquel simulador, que en cierto modo involucran a las directivas y el repertorio de instrucciones, también han sido consideradas, aunque en menor medida que los atributos del MIPS64 descritos en el WinMIPS64, durante el diseño de la arquitectura de la nueva computadora.

No obstante, en caso de que la lectura de los programas y sus subsecuentes ejecuciones incluidas en los casos de prueba propuestos a continuación resulte insuficiente para adquirir una metodología de programación apropiada para esta nueva CPU, se ha dedicado una subsección completa del informe, “3.3. Programación”, al estudio pormenorizado de todos los usos conocidos se les pueden asignar, a partir de sus funciones definidas en las tablas 37 y 38 respectivamente, tanto a las directivas válidas para el ensamblador del nuevo procesador como a las diversas instrucciones que conforman el repertorio soportado por el TDA 1819. Para ello, se ofrecen allí a modo de ejemplo diversos programas que posean objetivos y tareas prácticas y específicas y que, a fin de alcanzarlas y cumplirlas, utilicen distintas combinaciones de directivas e instrucciones que incluyan operandos variados en cantidad así como también en tipo para que el lector disponga de todas las alternativas posibles para poder elegir en cualquier ocasión aquélla que le resulte más conveniente durante la implementación de sus propios programas en el lenguaje Assembler para ser posteriormente ejecutados en el TDA 1819.

Por su parte, las descripciones que acompañarán las próximas figuras se centrarán más precisamente en los eventos más relevantes que sucedan en cada uno de los ciclos de reloj de la ejecución de los distintos programas, principalmente los acontecimientos que actúen como detonantes para la presencia de cualquiera de los atascos conocidos en el cauce segmentado del procesador: conflictos estructurales o de datos entre instrucciones, ejecución de instrucciones que requieren un tratamiento especial (transferencia de control, instrucciones "halt", entre otras), etc.

A pesar de que puede apreciarse una clara transición desde las nociones abstractas de los casos propuestos para su ejecución en las seis subsecciones anteriores hacia las funcionalidades más concretas de los programas que se plantearán a continuación aquí mismo, se sostiene la afirmación de que aún este informe se mantiene en un plano eminentemente teórico porque se ha optado por omitir por el momento la presentación de la simulación efectiva de la ejecución de los mismos en el nuevo procesador, para lo cual ya se requería directamente el uso del simulador Active-HDL. Por el contrario, esta acción se pospondrá hasta alcanzar la subsección "3.4. Simulación", en la cual, a modo anticipatorio, se propondrán varios programas de prueba para simular su ejecución en el TDA 1819, algunos de los cuales coincidirán exactamente con los ejemplos planteados en esta misma subsección. El progreso de las simulaciones que se lleven a cabo será capturado en diversas imágenes y exhibido al lector para que éste compruebe que el comportamiento de la nueva computadora durante la ejecución de dichos programas se corresponde ciertamente con el esperado y, además, aprenda a llevar a cabo nuevas simulaciones por sí mismo a fin de disponer del pleno uso de todas las capacidades y herramientas del proyecto para utilizarlo y/o adaptarlo para alcanzar sus propias metas personales.

Asimismo, cabe aclarar que, durante el desarrollo de estos programas, se priorizó el diseño de situaciones particulares en las cuales se asegurara en forma indudable y absoluta la presencia de cada uno de los distintos tipos de riesgos estudiados en los distintos casos, en detrimento del significado y el sentido de la función general cumplida por cada programa. Expresado en otras palabras, resultaría vano y fútil intentar encontrar un propósito o utilidad práctica para alguno de los programas aquí presentados porque en realidad ninguno de ellos la posee: únicamente existen para ofrecer y representar ejemplos concretos que colaboren con la comprensión por parte del lector de los distintos tipos conocidos de riesgos asociados a la segmentación implementada en el cauce del nuevo procesador.

Antes de proceder con la presentación propiamente dicha del progreso de la ejecución de los nuevos programas, se recordará que, durante la introducción general de la subsección

“3.1.8. Riesgos de la segmentación y atascos en el cauce”, se ofreció en la tabla 63 un resumen de cada tipo y modalidad particular de los posibles riesgos conocidos para el cauce segmentado del nuevo procesador junto con la identificación de la figura especialmente concebida para representarlo conceptual y posteriormente entre las subsecciones “3.1.8.1. Dependencias estructurales” y “3.1.8.6. Atascos de fin del programa”. Ahora bien, durante la concepción de los nuevos programas de prueba, se determinó, en primer lugar, que resultaba altamente improbable la existencia de casos de ejecución reales en los cuales se presentara pura y exclusivamente el tipo de riesgo que pretendiera representarse en dicho caso. Además, previo a la escritura de esta subsección, ya había sido determinada inicial y originalmente como deseable para el autor la meta de combinar aquí varias clases de atascos en un único ámbito de prueba de manera tal de disminuir el número de escenarios de estudio totales y así, presumiblemente, facilitarle al lector el análisis y la comprensión de los mismos al tener la capacidad de concentrar su atención en una cantidad más reducida de ejemplos (seis programas representados a través de ocho figuras en total) si se la compara con el número de casos expuestos en las seis subsecciones anteriores (dieciséis figuras).

Aun así, fue inevitable la reiteración del mismo tipo de riesgo de la segmentación entre distintos casos de ejecución. Esto se debe a motivos empíricos asociados a la ejecución de cualquier programa concreto en un entorno real: por ejemplo, en la amplia mayoría de las situaciones que se presenten, las primeras instrucciones del programa suelen ser de transferencia de datos, más precisamente destinadas a cargar en los registros de uso general de la CPU algunos o todos los valores declarados y almacenados respectivamente en la sección de datos del programa y de la memoria principal de la computadora. Del mismo modo, las instrucciones inmediatamente posteriores a ellas usualmente leen dichos valores directamente desde los registros recientemente cargados, a fin de realizar algún tipo de tarea a partir de ellos. Ergo, al ser tan cercana la proximidad entre las instrucciones de transferencia de datos y las de lectura de estos registros, resulta extremadamente alta la probabilidad de que alguna de estas últimas intente leer un valor a partir de un registro antes de que el mismo haya sido efectivamente cargado y escrito en él por la instrucción de transferencia de datos que, en circunstancias normales (es decir, en una ejecución secuencial del programa en lugar de segmentada), ya debería haberlo traído desde la memoria de datos dado que se ubica en una posición anterior en el programa que la instrucción de lectura. La lógica consecuencia de este percance será entonces la presencia de un atasco de tipo RAW, particularmente la modalidad del mismo mostrada en la figura 67 y descrita a través de sus observaciones. Podrían ofrecerse otros casos, pero se confía en que el lector asimilará mejor estos conceptos a través de la práctica

y la experiencia propia que mediante explicaciones teóricas, comenzando por el estudio y análisis de los próximos ejemplos y concluyendo con la programación y la posterior simulación de la ejecución de escenarios de prueba propios en el nuevo procesador.

Entonces, se ha optado por armar y presentar dos tablas distintas. La primera, a fin de definir los enlaces existentes entre las figuras conceptuales, planteadas en las subsecciones anteriores ya mencionadas para representar de manera simbólica cada uno de los distintos tipos de riesgos de la segmentación, y definidas en la tabla 63, y los nuevos casos de ejecución aquí propuestos. Al presentarse varias modalidades de atascos durante la ejecución de un mismo caso, se comprobará la presencia de conexiones múltiples entre cada una de aquellas figuras abstractas y los ejemplos propuestos en esta subsección. Por su parte, la segunda tabla realiza el proceso inverso al anterior: vincula los casos de prueba propuestos a continuación con las figuras de las subsecciones previas. Del mismo modo que en la primera tabla, se observará la existencia de un mismo programa de ejemplo acoplado a varias figuras conceptuales distintas. Así, el lector podrá observar cualquier situación, cuya comprensión podría complicarse inicialmente debido a la existencia de un entramado extramadamente complejo de múltiples conexiones entre las figuras simbólicas de la subsecciones anteriores del informe y los próximos ejemplos, a través de dos perspectivas distintas: una en la cual se toman a las figuras conceptuales como orígenes y a los nuevos casos como terminaciones de dichas conexiones, y otra, en la cual se invierte el sentido, es decir, el inicio y el fin, de estos mismos enlaces. Así, el análisis individual de cada punto de vista particular resulta indudablemente más sencillo que la perspectiva general, que abarcaría todos los vínculos entre las figuras sin matiz o consideración alguna. De esta manera, el lector podrá ubicarse en cada perspectiva para estudiar la red de enlaces según lo necesite, facilitándosele dicho proceso de estudio considerablemente.

Por último, se habrá apreciado en los párrafos anteriores una sutil distinción terminológica: mientras que a los casos conceptuales de ejecución propuestos en las subsecciones precedentes se los definió unívocamente como “figuras”, los escenarios de prueba planteados aquí fueron referenciados mediante el término “ejemplos”, un vocablo cuya interpretación posible y válida resulta considerablemente más ambigua. Esto se debe a que, en realidad, como se comprobará a continuación, cada ejemplo exhibido en la presente subsección comprende tanto el programa escrito en el lenguaje Assembler como la figura diseñada para representar el progreso de su ejecución segmentada en el nuevo procesador. Por su parte, también se observará próximamente que un mismo programa puede estar vinculado a más de una figura distinta ya que, aunque las instrucciones a ejecutar permanezcan inicialmente inalterables, en algunas ocasiones fueron llevadas a cabo múltiples iteraciones de la ejecución

de las mismas, cambiando ligeramente entre cada repetición el entorno y las condiciones de prueba. A su vez, a causa de estos cambios propuestos se provocaron y detectaron alteraciones significativas en el progreso de la ejecución del programa analizado, obviamente estas últimas sin influencia directa alguna por parte del autor del informe. Dichas alteraciones no sólo abarcaron variaciones en el flujo de ejecución seguido durante cada reiteración, sino que también involucraron aspectos más delicados e importantes, desde modificaciones en la naturaleza del tipo de riesgo para cuyo estudio particular había sido original y principalmente desarrollado el programa en cuestión hasta incluso la aparición de nuevas clases de atascos pura y exclusivamente en alguna iteración en especial que no se manifestaron en ninguna de las demás ejecuciones de dicho programa. De este modo, se justifica la necesidad de representar cada iteración realizada de la ejecución de un mismo programa en una figura separada e independiente cuyo análisis y observaciones sean absolutamente atribuibles y propios de ella. Ergo, indudablemente logrará notarse seguidamente la coexistencia de varios ejemplos distintos cuyo programa de prueba resulta ser exactamente idéntico, pero difieren entre sí las figuras que exponen los progresos de sus variadas y repetidas ejecuciones, cada una de las cuales, así como se mencionó, fue realizada dentro de distintos entornos de prueba y, por lo tanto, cuenta con características peculiares y únicas que permitirán distinguirla claramente del resto de las reiteraciones graficadas y exhibidas, aun teniendo en cuenta siempre que el programa ejecutado en todas ellas no varía en un principio de ninguna manera.

Tabla 81. Enlaces existentes entre las figuras conceptuales representadas en las seis subsecciones anteriores y los ejemplos de prueba propuestos próximamente. Tipo y modalidad del riesgo de la segmentación involucrado en cada conexión.

Riesgo		**Figura conceptual**	**Ejemplo (****)**	
Tipo	**Modalidad**		**Programa**	**Figura**
Estructural	Memoria de datos	65	84	85
			96 (*****)	97 (*****)
	Banco de registros de uso general	66	86	87
			90	92
Dependencia de datos	Lectura Después de Escritura (RAW)	67	84	85
			86	87
			88	89

			90 (*****)	91 (*****)
			90 (*****)	92 (*****)
			93	94
			93	95
			96 (*****)	97 (*****)
		68	88	89
		69	88	89
	Escritura Después de Lectura (WAR) (*)	70	90	91
		71	90	92
	Escritura Después de Escritura (WAW)	72	86	87
		73	96	97
Control	Incondicional	74	84	85
			90	91
	Condicional	75	86	87
			88	89
		76	86	87
			88	89
			93	94
			93	95
Fin del programa	Instrucción “halt”	77 (**)	84	85
			86	87
			88	89
			90	91
			90	92
		78 (***)	93	94
		79	93	95
		80	96	97

Observaciones:

- (*): Tal como se mencionó en las observaciones de la tabla 63, tras varias simulaciones realizadas de la ejecución segmentada de diversos programas de prueba en el nuevo procesador no logró determinarse ninguna situación real en la cual se presentara en el cauce de la CPU un atasco a causa de la existencia de una dependencia de datos de tipo WAR (Escritura Después de Lectura) entre dos instrucciones. Asimismo, se había argumentado

anteriormente, durante la explicación general de esta modalidad de riesgos, ofrecida al comienzo de la sección "3.8. Riesgos de la segmentación y atascos en el cauce", y se continúa sosteniendo aquí, que este fenómeno se debe, principalmente, a que el ensamblador asociado al TDA 1819 no posee implementada la técnica para la solución de riesgos de control conocida como salto retardado, definida a su vez como un mecanismo que le permite al ensamblador la posibilidad de reordenar instrucciones en un programa previo a su ejecución a fin de mejorar el rendimiento del procesador ocupando huecos de retardo que surjan en el cauce como consecuencia de la presencia de atascos de control, cuyas características son absolutamente distintas a las de los riesgos de tipo WAR, ya que su origen se encuentra íntimamente asociado a la presencia de instrucciones de transferencia de control, las cuales requerirán inevitablemente que la ejecución del programa se interrumpa transitoriamente mientras evalúan cada una de sus respectivas condiciones de salto y determinan el flujo de ejecución del programa a seguir. El tiempo de demora de estos atascos y el número de sus consecuentes huecos de retardo generados variarán en función de la naturaleza y posterior resolución del riesgo en cuestión, las cuales dependerán de si se trata de una instrucción de transferencia de control incondicional o condicional y, en este último caso, si se cumple o no la condición de salto estipulada en dicha instrucción (pueden verse todas las variantes posibles de los riesgos de control en las figuras 74, 75 y 76). A su vez, se había sido definido el reordenamiento de las instrucciones de un programa previo a su ejecución como el único evento conocido que podría causar esta clase de atascos en el cauce segmentado del procesador. Del mismo modo, la técnica de salto retardado, al involucrar precisamente la realización de esta misma clase de acción, fue establecida inmediatamente como una indudable potencial responsable de estos atascos. Ergo, al no encontrarse dicha técnica incluida en el ensamblador implementado para la versión actual del TDA 1819, no habría motivo alguno por el cual debieran acontecer riesgos de esta índole durante la ejecución de un programa. Entonces, para no complicar innecesariamente la descripción de un diseño ya de por sí extremadamente complejo, se optó por omitir, al menos por el momento, la incorporación de un soporte en el TDA 1819 para la detección y solución de atascos de tipo WAR. Además, al fin y al cabo, sólo tendría sentido implementarlo de manera conjunta con la metodología de salto retardado debido a que únicamente a partir del reordenamiento de instrucciones llevado a cabo por esta última podrían presentarse casos concretos en los cuales se comprobaría el correcto funcionamiento del detector que haya resuelto diseñarse y añadirse al nuevo procesador. A pesar de que en la subsección "3.4.4. Riesgos de la segmentación del cauce: Ejemplos" se

plantearán y representarán diversos ejemplos prácticos de la simulación efectiva en el TDA 1819 mediante el simulador Active-HDL de la ejecución segmentada de programas escritos en el lenguaje Assembler, aquí ya se estaría en condiciones de anticipar que de ninguna manera tendría sentido alguno esperar encontrar en dicha subsección algún programa que presente durante su ejecución este tipo y modalidad específicas de atasco. Ergo, podría concluirse esencialmente que, en esta misma subsección actual, particularmente a través del programa propuesto en la figura 90 y ejecutado en dos ocasiones distintas, cada una exhibida en las figuras 91 y 92, finalizaría estrictamente el estudio y análisis de los riesgos causados por dependencias de datos de tipo WAR.

- (**): En caso de no recordarse, la figura 77 no describía por sí misma la existencia de ningún tipo de atasco en el cauce del nuevo procesador, sino que simplemente representaba la correcta finalización de un programa absolutamente ordinario, común y corriente. Más concretamente, la "normalidad" de dicho programa, al menos en el apartado que respecta a la clase de riesgo que se pretendía introducir en aquella figura, bautizado como "fin del programa", consistía en que el mismo no se encontraba conformado durante las instancias finales de su ejecución por ninguna instrucción que requiriera utilizar la FPU del procesador para realizar alguna operación aritmética en punto flotante durante su etapa de ejecución. Podrá observarse que ninguna de las ejecuciones de los ejemplos enlazados al subtipo de atasco representado conceptualmente en la figura 77, es decir, las figuras 85, 87, 89, 91 y 92, presentan entre sus últimas cinco instrucciones ejecutadas alguna instrucción cuya etapa de ejecución esté constituida por cuatro subetapas de un ciclo de reloj cada una: todas las etapas de ejecución de dichas instrucciones son simples y tradicionales y poseen una duración de un único ciclo de reloj para completarse. De hecho, la figura 77 únicamente había sido presentada a modo introductorio en la subsección "3.1.8.6. Atascos de fin del programa" a fin de presentar un programa común, cuya finalización se llevara a cabo sin ningún conveniente, y compararla así con otros casos especiales en los cuales, a causa de que una de las últimas cinco instrucciones ejecutadas requería ejecutar una operación aritmética en la FPU y tenía un ciclo de ejecución más extenso, complejo y lento que el resto de las instrucciones, en especial la instrucción "halt", había graves problemas en la finalización de los programas analizados porque la instrucción "halt", cuyo ciclo de ejecución es como el de cualquier instrucción tradicional y, por ende, más corto que el de la instrucción FPU, emitía la orden de detención para todas las unidades de ejecución del procesador, antes de que terminara de ejecutarse dicha instrucción, por lo cual ésta se interrumpía y cancelaba prematuramente, antes de completar su ejecución. Así se justificaba

la concepción de esta clase de atasco, inventada por el autor del informe especialmente para esta clase de atascos y previamente desconocida para cualquier otro procesador que tuviera implementada la técnica de segmentación en su respectivo cauce. Asimismo, la presencia de una proporción tan elevada de los ejemplos representados en la subsección actual que se encuentran enlazados a la modalidad de atasco representada en esta figura conceptual puede fundamentarse a partir de la muy baja probabilidad de que alguna de las últimas cinco instrucciones de un programa asociado a una situación real requiera ejecutar una operación aritmética en la FPU de la CPU. De hecho, ninguna de las ejecuciones de las figuras incluidas en los enlaces fue concebida específicamente con la finalidad de representar esta modalidad de atasco: simplemente surgieron naturalmente mientras cumplían otros objetivos. No puede decirse lo mismo de las demás modalidades: tanto la figura 94 como la 95 y la 97 fueron planteadas especialmente para representar los subtipos de atascos de fin del programa especialmente asociados a ella en esta tabla.

- (***): De una manera similar a los riesgos provocados a causa de dependencias de datos de tipo WAR, explicados aquí mismo en la primera observación, no será posible la simulación real de la ejecución de programas concretos en el TDA 1819 a través del simulador Active-HDL en los cuales se presente la modalidad de atascos de fin del programa representada en la figura conceptual número 78. No obstante, aquí los motivos no se basan en alguna funcionalidad deseada para el nuevo procesador pero aún no implementada actualmente, sino simplemente en que, a modo recordatorio, la figura 78 representaba las consecuencias adversas de la finalización de la ejecución de un programa en una CPU que no tuviera incorporado detector alguno para anticiparse y resolver situaciones asociadas a esta clase de riesgos. Básicamente, dicho programa terminaba prematuramente, ya que la instrucción “halt” emitía la orden de detención para todas las unidades de ejecución del procesador antes de que se hubieran alcanzado las condiciones de ejecución adecuadas para hacerlo. Por lo tanto, todas las instrucciones que estuvieran haciendo en aquel momento alguna operación aritmética en la FPU del procesador, al ser su ciclo de ejecución más lento y extenso que el de la instrucción “halt”, se interrumpirían en un instante previo a completarse, resultando incorrecto tanto el comportamiento como la respuesta finalmente ofrecidas por el programa en cuestión. Dado que el TDA 1819 posee efectivamente un detector especialmente diseñado para tratar esta clase de riesgos, se garantiza absolutamente la nula posibilidad de que pueda llegar a presentarse durante la ejecución de algún programa en este nuevo procesador una situación como la que fue descripta en la figura 78: todos los programas ejecutados finalizarán correctamente ya que se demorará la ejecución de la

instrucción “halt” todo el tiempo que se necesario para asegurar que únicamente se emita la orden de detención una vez que se verifique el cumplimiento de las condiciones necesarias para que la misma se transmita sin interrumpir prematuramente ninguna de las instrucciones que aún se encuentren actualmente en ejecución. Entonces, se vuelve imposible simular mediante el Active-HDL la simulación de la ejecución un programa de prueba en esta computadora que se comporte tal como lo se espera en la figura 78: cualquier programa ejecutado únicamente podrá finalizar de una manera similar a las modalidades representadas en las figuras conceptuales número 77, 79 u 80. A pesar de que en la subsección “3.4.4. Riesgos de la segmentación del cauce: Ejemplos” se plantearán y representarán diversos ejemplos prácticos de la simulación efectiva en el nuevo procesador mediante el simulador Active-HDL de la ejecución segmentada de programas implementados y escritos en el lenguaje Assembler, aquí ya se estaría en condiciones de anticipar que de ninguna manera tendría sentido alguno esperar en dicha subsección algún programa que presente durante su ejecución este tipo y modalidad particulares de atasco. Ergo, podría concluirse esencialmente que, una vez más, al igual que en el caso de los atascos causados por dependencias de datos de tipo WAR, en esta misma subsección actual, particularmente a través del programa planteado en la figura 93 y ejecutado en una única ocasión, representada en la figura 94, finalizaría estrictamente el estudio y análisis de la variante de atascos de fin del programa exhibida y descripta conceptualmente en la figura 78.

- (****): Aquí puede comprobarse la definición ofrecida en el último párrafo de la introducción previa a esta tabla: cada ejemplo presentado aquí estará conformado por un programa de prueba escrito en el lenguaje Assembler, acompañado por su respectiva figura planteada para representar su ejecución segmentada teóricamente esperada en el nuevo procesador. En la tabla, cada combinación distinta establecida entre un programa y una figura define un ejemplo diferente. Cabe aclararse asimismo que los valores asignados a la columna “Programa” corresponden en realidad a los distintos números utilizados para identificar a las figuras que presentan la implementación del código de cada programa (por ejemplo, el programa 84 se encuentra definido en la figura 84, el programa 86, en la figura 86, y así sucesivamente), mientras que el contenido de la columna “Figura” resulta, por su parte, más sencillo de interpretar: cada número simplemente se trata del identificador para la figura en cuestión, definido en el epígrafe de la misma. Si se realizara un análisis minucioso de la tabla, podrá concluirse que se han desarrollado seis programas distintos, mientras que para representar su ejecución fueron modeladas en total ocho figuras diferentes. Entonces, se deduciría, acertadamente, que algunos programas poseen más de

una ejecución distinta representada y asociada, tal como había sido anticipado anteriormente. De hecho, puede observarse claramente que el programa 90 se encuentra vinculado a dos ejecuciones distintas del mismo, representadas en las figuras 91 y 92, mientras que el programa 93 también fue ejecutado en dos ocasiones bajo condiciones de prueba distintas, cuyos progresos de ejecución esperados fueron plasmados respectivamente en las figuras 94 y 95. Por su parte, los programas 84, 86, 88 y 96 únicamente fueron ejecutados una vez cada uno, hallándose su correspondiente ejecución representada en las figuras 85, 87, 89 y 97 respectivamente. En síntesis, la cantidad total de ejemplos propuestos es igual al número de combinaciones distintas entre programas y figuras definidas en la tabla, es decir, ocho. Obviamente, en este apartado se está haciendo referencia estrictamente a las figuras presentes en esta subsección, omitiéndose la consideración de las figuras conceptuales vinculadas a las subsecciones anteriores ya que no poseen relación alguna con el tópico expuesto en esta observación en particular.

- (*****): Estos ejemplos presentan en más de una ocasión el tipo y modalidad de atascos asociados al enlace indicado en esta observación. Más precisamente, el programa 96 presenta en múltiples oportunidades durante su ejecución, representada en la figura 97, un atasco estructural a causa de intentos de acceso simultáneos por parte de dos instrucciones distintas del programa, así como también presenta en varios instantes diferentes de su ejecución una obstrucción en su cauce debido a un conflicto entre dos instrucciones como consecuencia de una dependencia de datos de tipo RAW, particularmente la modalidad en la cual el dato conflictivo se encuentra almacenado en un registro de uso general de la CPU, tal como fue representada en la figura conceptual número 67, en la subsección “3.1.8.2. Lectura Después de Escritura”. Del mismo modo, el programa 90, ejecutado dos veces, cada una bajo condiciones de prueba distintas y representada en las figuras 91 y 92, presenta en ambas oportunidades exactamente esta idéntica variante de atasco de datos de tipo RAW. Aún así, al encontrarse esta tabla a definir los enlaces existentes entre las figuras conceptuales de las subsecciones anteriores y los ejemplos aquí expuestos, particularmente los tipos y modalidades de riesgos involucrados en cada conexión, independientemente de la cantidad de veces que el ejemplo en cuestión presentara el mismo riesgo, ya que con sólo una alcanzaría para establecerse el enlace, se optó por omitirse la reiteración del mismo ejemplo en múltiples oportunidades dentro de la misma modalidad de riesgo ya que se determinó que podría entorpecer la interpretación por parte del lector de la red de enlaces cuyo entremado pretendió describirse en esta tabla. En la presentación individual de cada ejemplo se precisará más detenidamente el número exacto de ocasiones en las cuales cada

tipo y modalidad de riesgo se presenta en la ejecución del programa representada en la figura en cuestión, así como también el instante de tiempo exacto en el cual se provoca cada uno de ellos.

Tabla 82. Enlaces existentes entre los ejemplos de prueba propuestos próximamente y las figuras conceptuales representadas en las seis subsecciones anteriores. Tipo y modalidad del riesgo de la segmentación involucrado en cada conexión.

Ejemplo		**Figura conceptual**	**Riesgo**	
Programa	**Figura**		**Tipo**	**Modalidad**
84	85	65	Estructural	Memoria de datos
		67	Dependencia de datos	Lectura Después de Escritura (RAW)
		74	Control	Incondicional
		77	Fin del programa	Instrucción “halt”
86	87	66	Estructural	Banco de registros de uso general
		67	Dependencia de datos	Lectura Después de Escritura (RAW)
		72	Dependencia de datos	Escritura Después de Escritura (WAW)
		75	Control	Condicional
		76	Control	Condicional
		77	Fin del programa	Instrucción “halt”
88	89	67	Dependencia de datos	Lectura Después de Escritura (RAW)
		68	Dependencia de datos	Lectura Después de Escritura (RAW)
		69	Dependencia de datos	Lectura Después de Escritura (RAW)
		75	Control	Condicional
		76	Control	Condicional
		77	Fin del programa	Instrucción “halt”

90	91	67	Dependencia de datos	Lectura Después de Escritura (RAW)
		70	Dependencia de datos	Escritura Después de Lectura (WAR)
		74	Control	Incondicional
		77	Fin del programa	Instrucción “halt”
90	92	66	Estructural	Banco de registros de uso general
		67	Dependencia de datos	Lectura Después de Escritura (RAW)
		71	Dependencia de datos	Escritura Después de Lectura (WAR)
		77	Fin del programa	Instrucción “halt”
93	94	67	Dependencia de datos	Lectura Después de Escritura (RAW)
		76	Control	Condicional
		78	Fin del programa	Instrucción “halt”
93	95	67	Dependencia de datos	Lectura Después de Escritura (RAW)
		76	Control	Condicional
		79	Fin del programa	Instrucción “halt”
96	97	65	Estructural	Memoria de datos
		67	Dependencia de datos	Lectura Después de Escritura (RAW)
		73	Dependencia de datos	Escritura Después de Escritura (WAW)
		80	Fin del programa	Instrucción “halt”

REVISAR LAS OBSERVACIONES DE LA TABLA 81.

MODIFICAR TODAS LAS REFERENCIAS PARA QUE TENGAN EN CUENTA LA NUEVA FIGURA 81.

¿CREAR NUEVO EJEMPLO PARA LA MODALIDAD DE ATASCO DE LA FIGURA 81 O AGREGARLA EN UN EJEMPLO YA EXISTENTE?
AL IGUAL QUE LA FIGURA 77, EN LA FIGURA 81 NO SE PRESENTA EL ATASCO FIN DEL PROGRAMA, PERO SÍ AYUDA A COMPRENDERLO MEJOR. AGREGAR UNA OBSERVACIÓN PARA EXPLICARLO MEJOR.

CORREGIR LOS EPÍGRAFES DE LAS FIGURAS 83 A 97.

¿LAS MISMAS OBSERVACIONES SON PARA LAS DOS TABLAS?

¿POR QUÉ SON LAS ÚLTIMAS CINCO INSTRUCCIONES DEL PROGRAMA LAS QUE DETERMINAN LA PRESENCIA DE UN ATASCO "HALT"? ¿QUÉ OCURRE SI ES LA SEXTA LA QUE USA LA FPU? REVISAR EXPLICACIÓN DE LA FIGURA 77 Y SI HABRÍA QUE CORREGIRLA PARA QUE TENGA EN CUENTA ESTO (CAMBIAR LA SEXTA INSTRUCCIÓN DE ATRÁS PARA ADELANTE DEL PROGRAMA PARA QUE USE LA FPU Y VERIFICAR QUE NO HAYA ATASCO "FIN DEL PROGRAMA").

REALIZAR CÁLCULOS DISTANCIA ENTRE FPU Y HALT PARA TODOS LOS CASOS Y AVERIGUAR SI SE EXCEDE O NO EL LÍMITE. ACLARAR EN LOS CASOS QUE SEA NECESARIO QUE EL CÁLCULO SE HACE A PARTIR DE LA ÚLTIMA INSTRUCCIÓN FPU.
¿AGREGAR CASO EN EL CUAL SE EXCEDA EL LÍMITE PERO OCURRA ATASCO Y/O CASO EN EL CUAL NO SE SUPERE EL LÍMITE PERO NO OCURRA ATASCO EN LA INSTRUCCIÓN HALT?

Tabla 83. Colores utilizados para representar cada una de las etapas del ciclo de ejecución de las instrucciones de los programas de prueba analizados en las próximas figuras.

Etapa	Color

1. Búsqueda		Rojo
2. Decodificación		Violeta
3.1. Ejecución		Verde
3.2. Ejecución en punto flotante		Marrón
4. Acceso a memoria		Azul
5. Almacenamiento en registro		Rosa

```
        .data
num1:   .word   20
num2:   .word   –80
num3:   .float  –4.0
num4:   .float  1.5
res1:   .word   0
res2:   .float  0.0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lf f1, num3(r0)
        lf f2, num4(r0)
        addf f3, f1, f2
        dadd r3, r1, r2
        dsub r4, r1, r2
        slt r5, r2, r1
        xorr r6, r2, r1
        dsli r7, r3, 5
        dmul r8, r4, r3
        jmp fin
        ddiv r9, r8, r7
        subf f4, f1, f2
```

```
fin:    sw r7, res1(r0)
        sf f3, res2(r0)
        halt
```

Figura 84. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

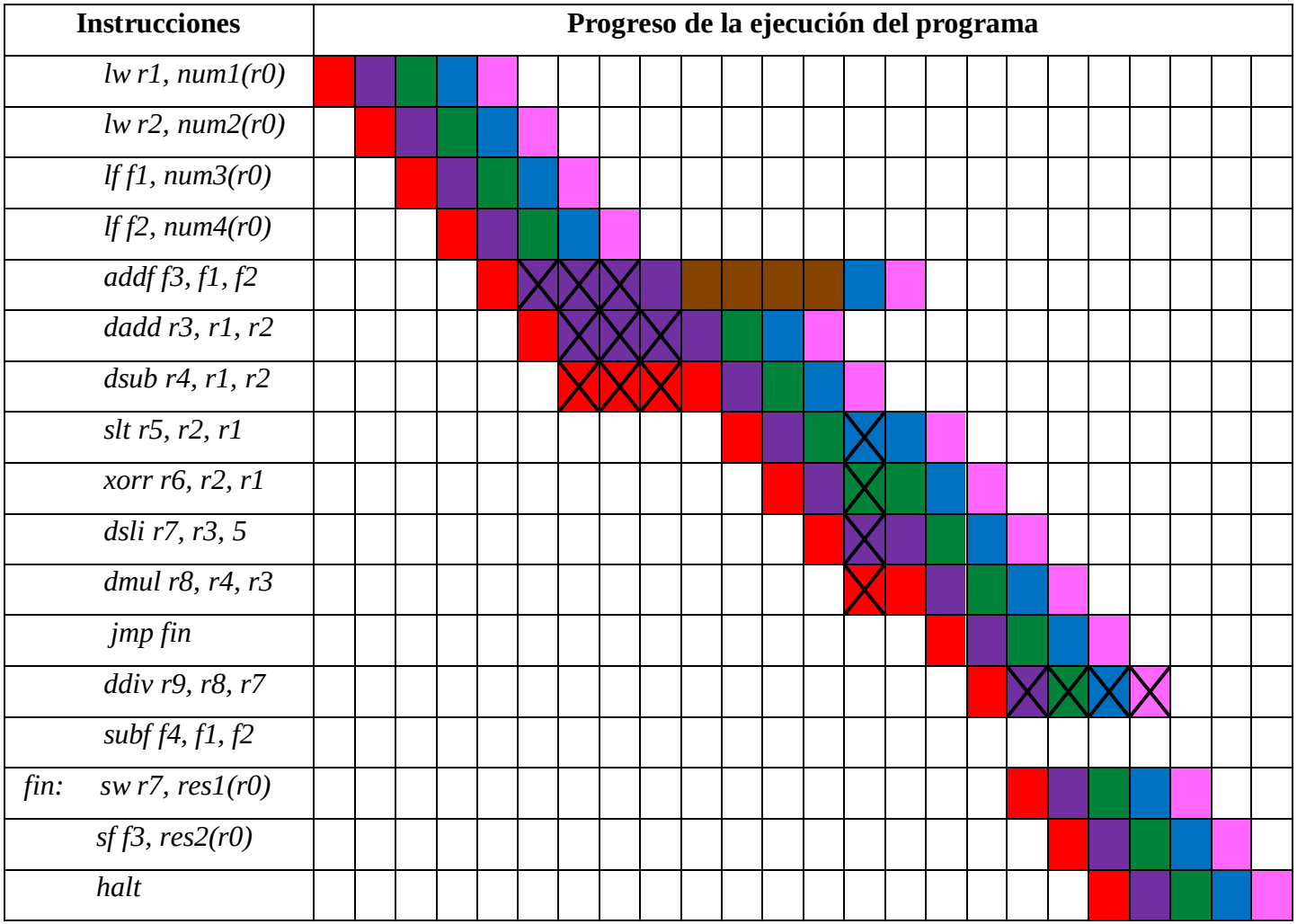
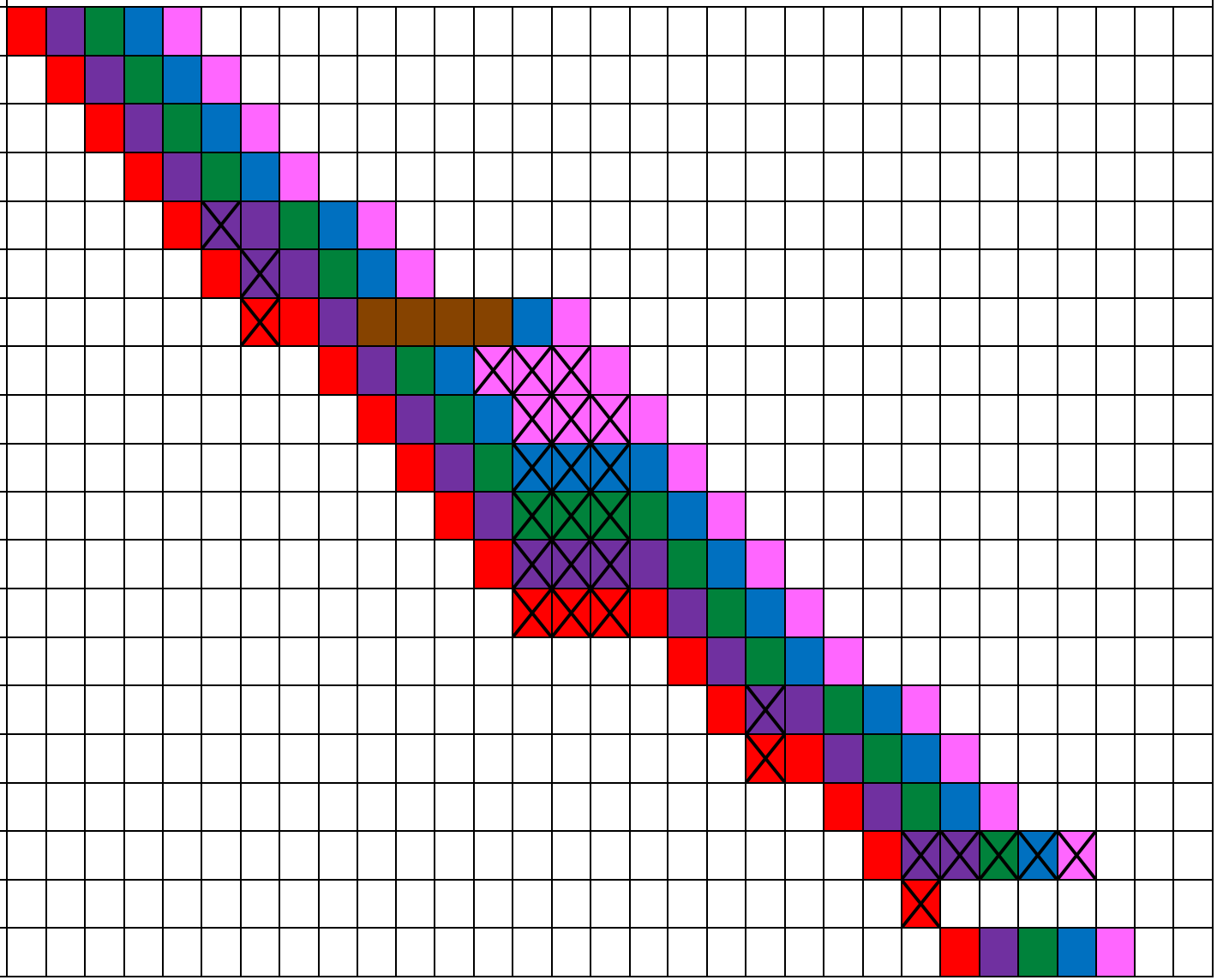


Figura 85. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

```
        .data
num1:   .word    35
num2:   .word    70
num3:   .float   –4.0
num4:   .float   –1.5
num5:   .float   –9.5
res1:   .word    0
res2:   .word    0
res3:   .word    0
res4:   .float   0.0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lf f1, num3(r0)
        lf f2, num4(r0)
        dadd r3, r1, r2
        dsub r4, r1, r2
        mulf f3, f2, f1
        lf f3, num5(r0)
        dmul r5, r1, r3
        andr r6, r2, r4
        dsri r7, r3, 4
        negr r8, r4
        ori r9, r5, 0xF0
        beqz r3, salto
        ddiv r10, r4, r6
        dslsi r11, r8, 2
salto:  bne r1, r5, fin
        sw r5, res1(r0)
        sf f3, res4(r0)
fin:    sw r8, res2(r0)
        sw r11, res3(r0)
        halt
```

Figura 86. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

Instrucciones	Progreso de la ejecución del programa
lw r1, num1(r0)	
lw r2, num2(r0)	
lf f1, num3(r0)	
lf f2, num4(r0)	
dadd r3, r1, r2	
dsub r4, r1, r2	
mulf f3, f2, f1	
lf f3, num5(r0)	
dmul r5, r1, r3	
andr r6, r2, r4	
dsri r7, r3, 4	
negr r8, r4	
ori r9, r5, 0xF0	
beqz r3, salto	
ddiv r10, r4, r6	
dslsi r11, r8, 2	
salto: bne r1, r5, fin	
sw r5, res1(r0)	
sf f3, res4(r0)	
fin: sw r8, res2(r0)	

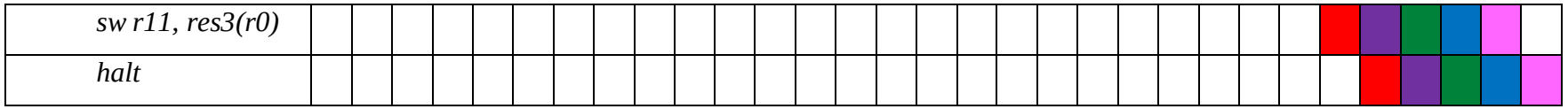
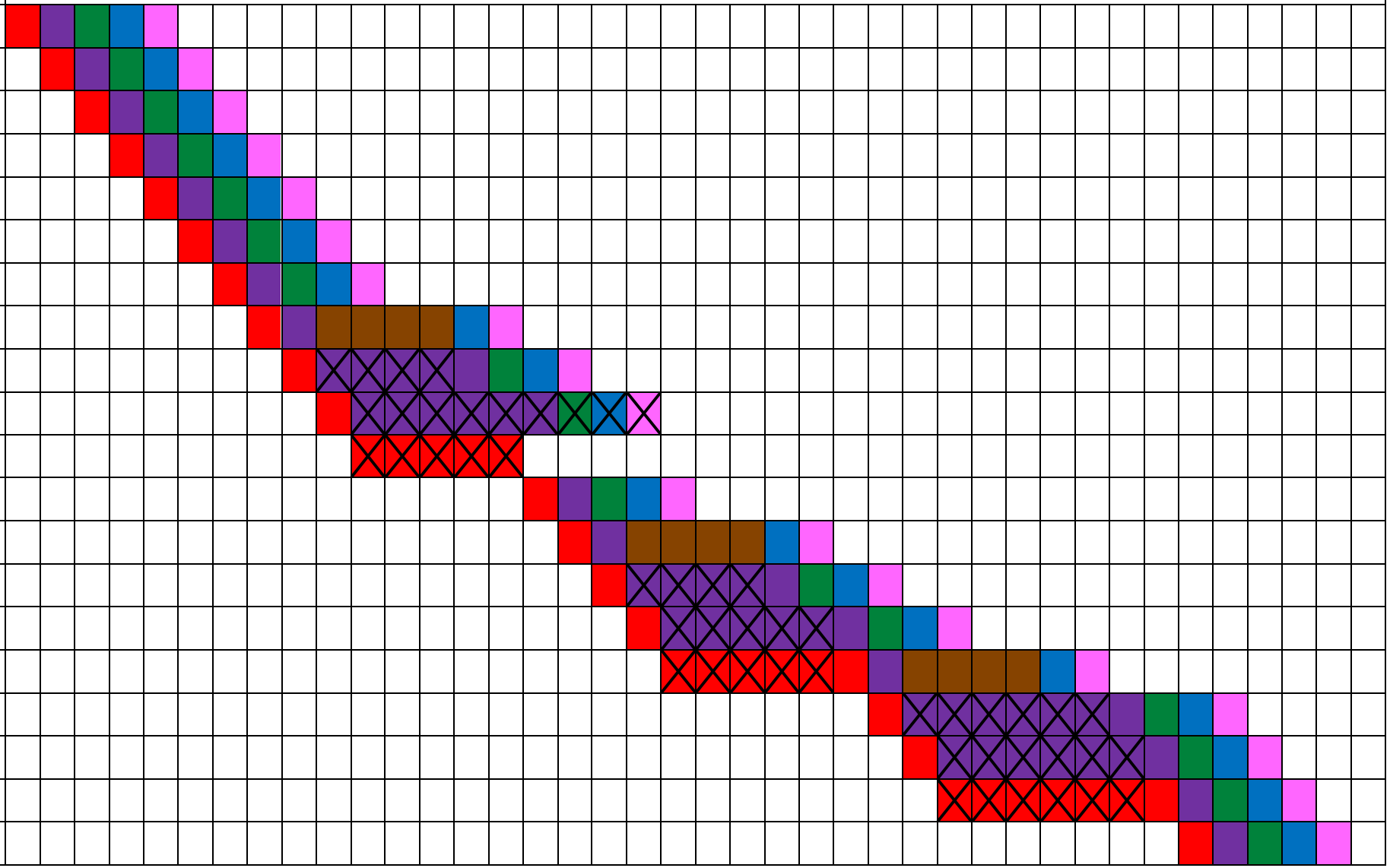


Figura 87. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

```
        .data
num1:   .word   –8
num2:   .word   5
num3:   .float  7.5
num4:   .float  –1.5
res1:   .word   0
res2:   .word   0
res3:   .float  0.0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lf f1, num3(r0)
        lf f2, num4(r0)
        daddi r3, r1, 18
        dsub r4, r2, r1
        andi r5, r2, 0x1F
        lef f1, f2
        bfpf salto
        dmul r6, r3, r1
        notr r7, r2
salto:  mff f3, f1
        divf f4, f1, f2
        bfpt fin
        dadd r8, r1, r0
        ti f5, f2
        mrf r9, f5
        sw r9, res2(r0)
fin:    sw r3, res1(r0)
        sf f3, res3(r0)
        halt
```

Figura 88. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

Instrucciones	Progreso de la ejecución del programa
lw r1, num1(r0)	
lw r2, num2(r0)	
lf f1, num3(r0)	
lf f2, num4(r0)	
daddi r3, r1, 18	
dsub r4, r2, r1	
andi r5, r2, 0x1F	
lef f1, f2	
bfpf salto	
dmul r6, r3, r1	
notr r7, r2	
salto: mff f3, f1	
divf f4, f1, f2	
bfpt fin	
dadd r8, r1, r0	
ti f5, f2	
mrf r9, f5	
sw r9, res2(r0)	
fin: sw r3, res1(r0)	
sf f3, res3(r0)	

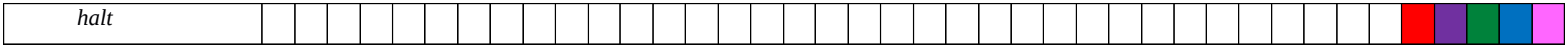


Figura 89. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

```
        .data
num1:   .word   15
num2:   .word   3
num3:   .word   –10
res1:   .word   0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lw r3, num3(r0)
        dsli r4, r1, r2
        dsub r5, r2, r1
        dadd r6, r3, r2
        jmp salto
        negr r7, r2
salto:  dsr r8, r6, r2
        slt r9, r1, r6
        daddi r10, r5, –1
        dmul r11, r3, r5
        ori r3, r0, 0xFF
        sw r6, res1(r0)
        halt
```

Figura 90. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

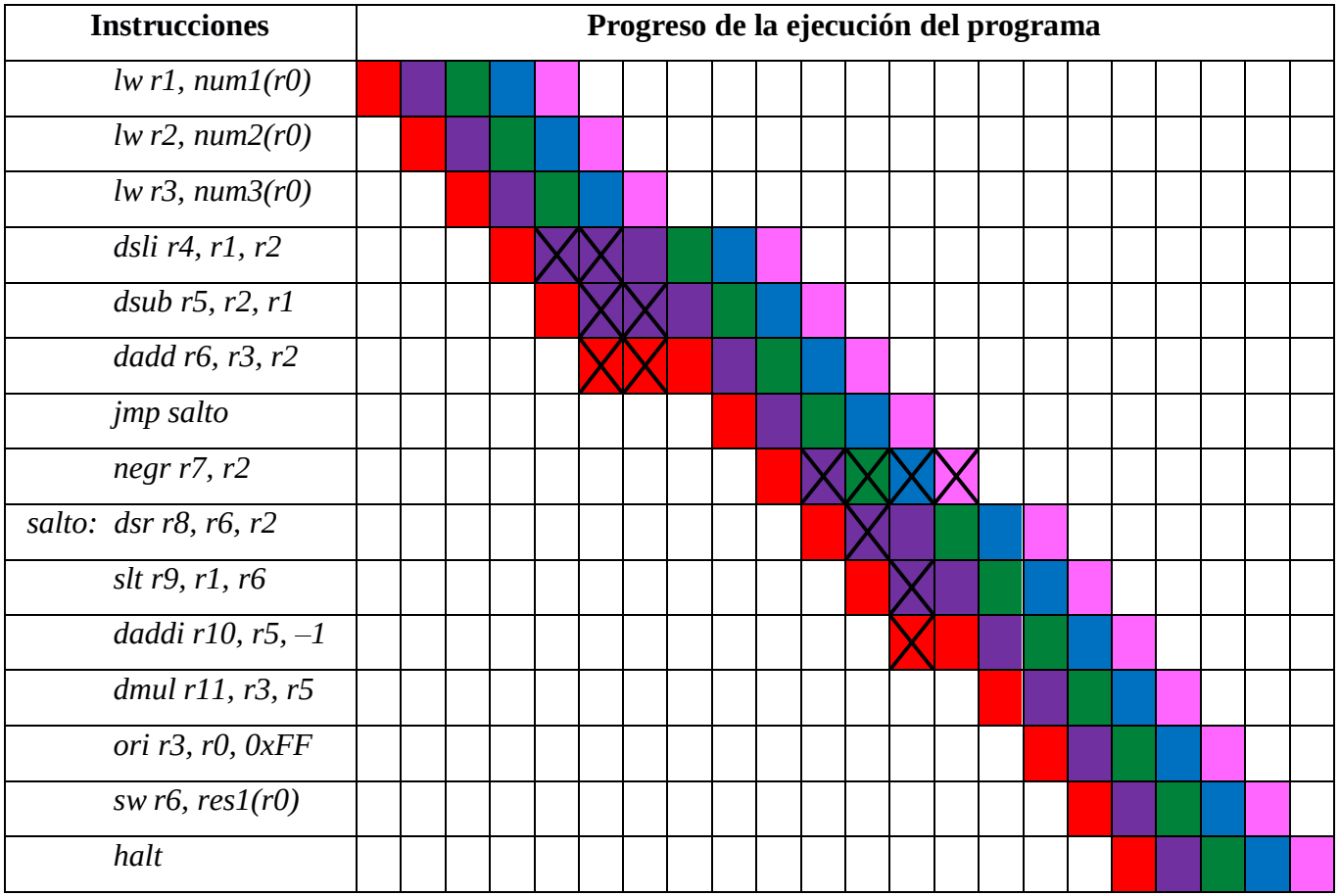


Figura 91. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

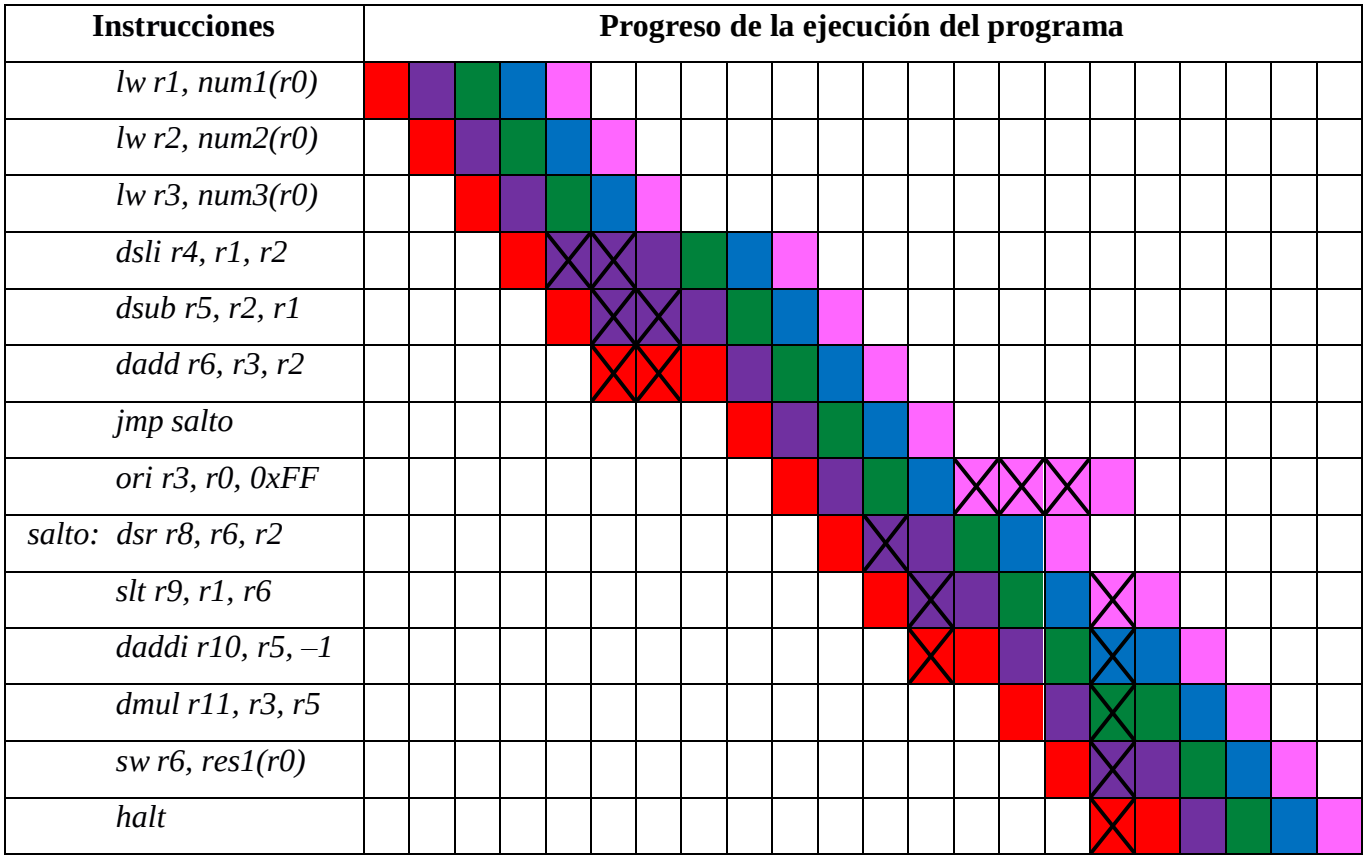


Figura 92. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

```
        .data
num1:   .word   50
num2:   .word   –1
num3:   .word   35
num4:   .float  6.75
num5:   .float  2.0
num6:   .float  –9.25
res1:   .word   0
res2:   .float  0.0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lw r3, num3(r0)
        lf f1, num4(r0)
        lf f2, num5(r0)
        lf f3, num6(r0)
        notr r4, r2
        ddiv r5, r1, r3
        beqz r4, fin
        slti r6, r5, 1
        subf f4, f3, f1
fin:    sw r5, res1(r0)
        sf f3, res2(r0)
        mulf f5, f1, f2
        halt
```

Figura 93. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

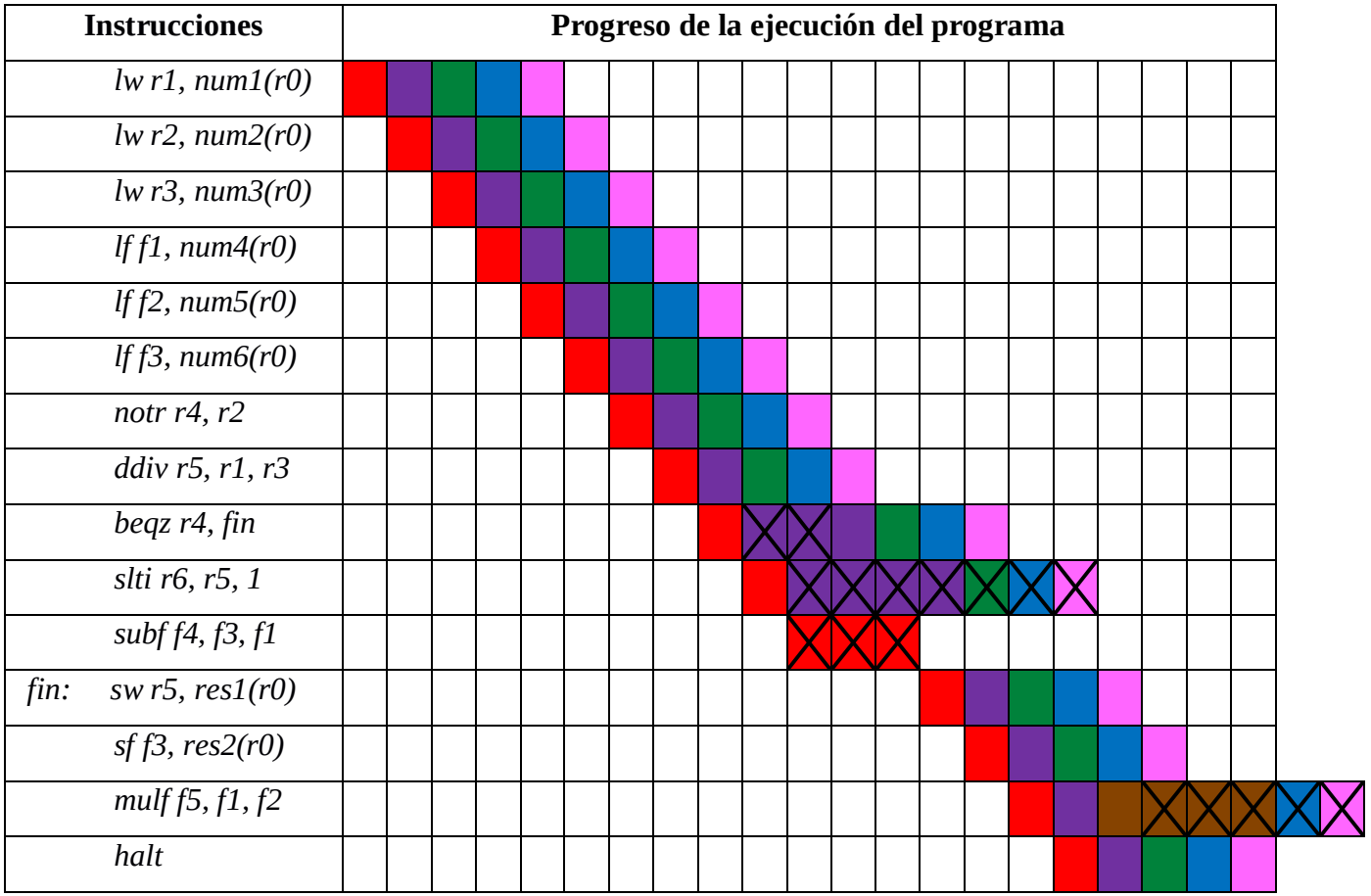


Figura 94. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

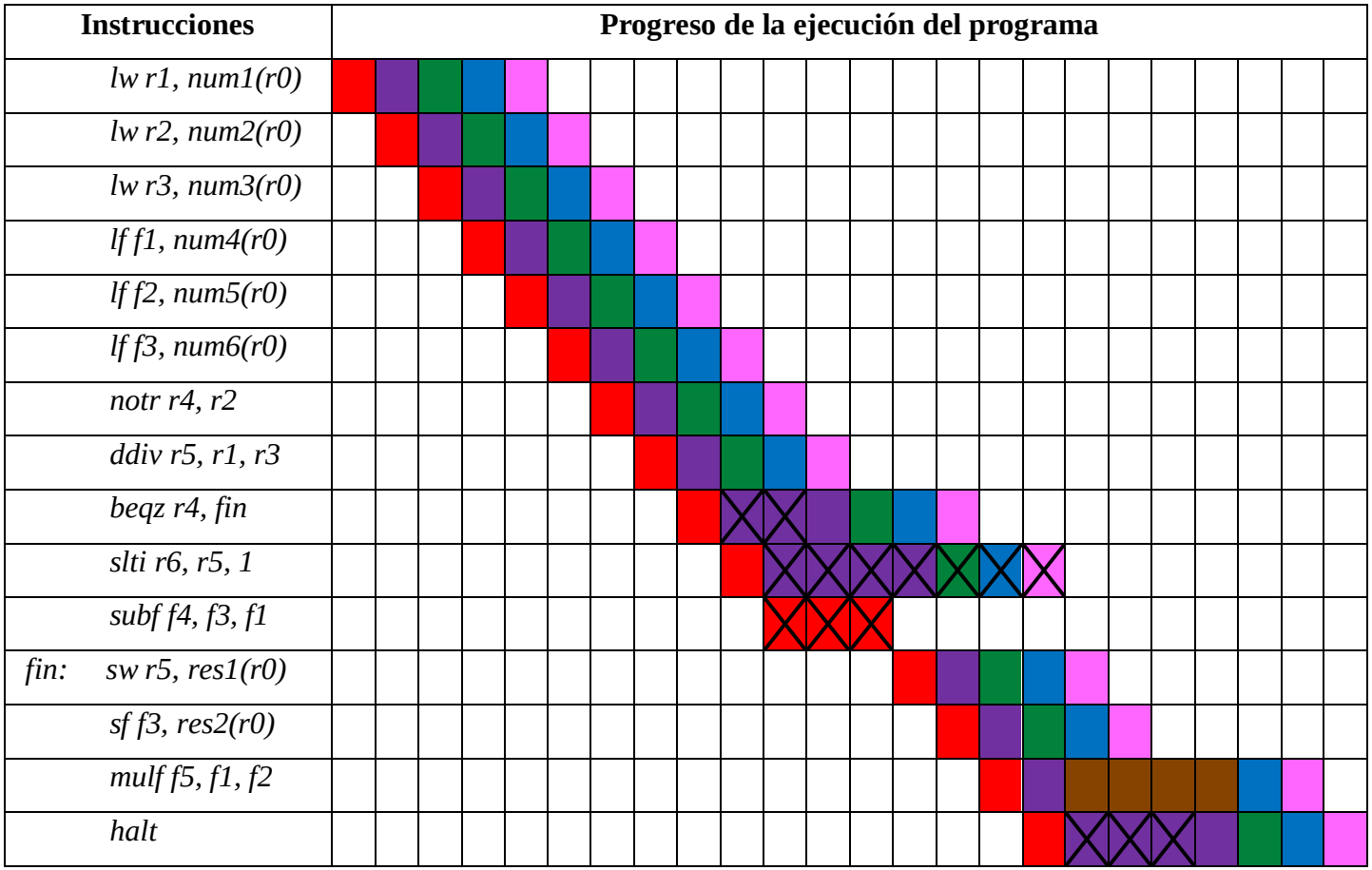


Figura 95. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

```
        .data
num1:   .word    12
num2:   .word    –48
num3:   .float    0.5
num4:   .float    –5.0
num5:   .float    3.5
res1:   .word    0
res2:   .word    0
res3:   .float    0.0
        .code
        lf f1, num3(r0)
        lf f2, num4(r0)
        lw r1, num1(r0)
        lw r2, num2(r0)
        addf f3, f1, f2
        ddiv r3, r2, r1
        ti f4, f2
        divf f5, f2, f1
        lf f5, num5(r0)
        ltf f1, f2
        orr r4, r3, r2
        dsub r5, r1, r3
        daddi r6, r2, 8
        mrf r7, f4
        mulf f6, f5, f3
        sw r3, res1(r0)
        sw r5, res2(r0)
        sf f5, res3(r0)
        halt
```

Figura 96. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

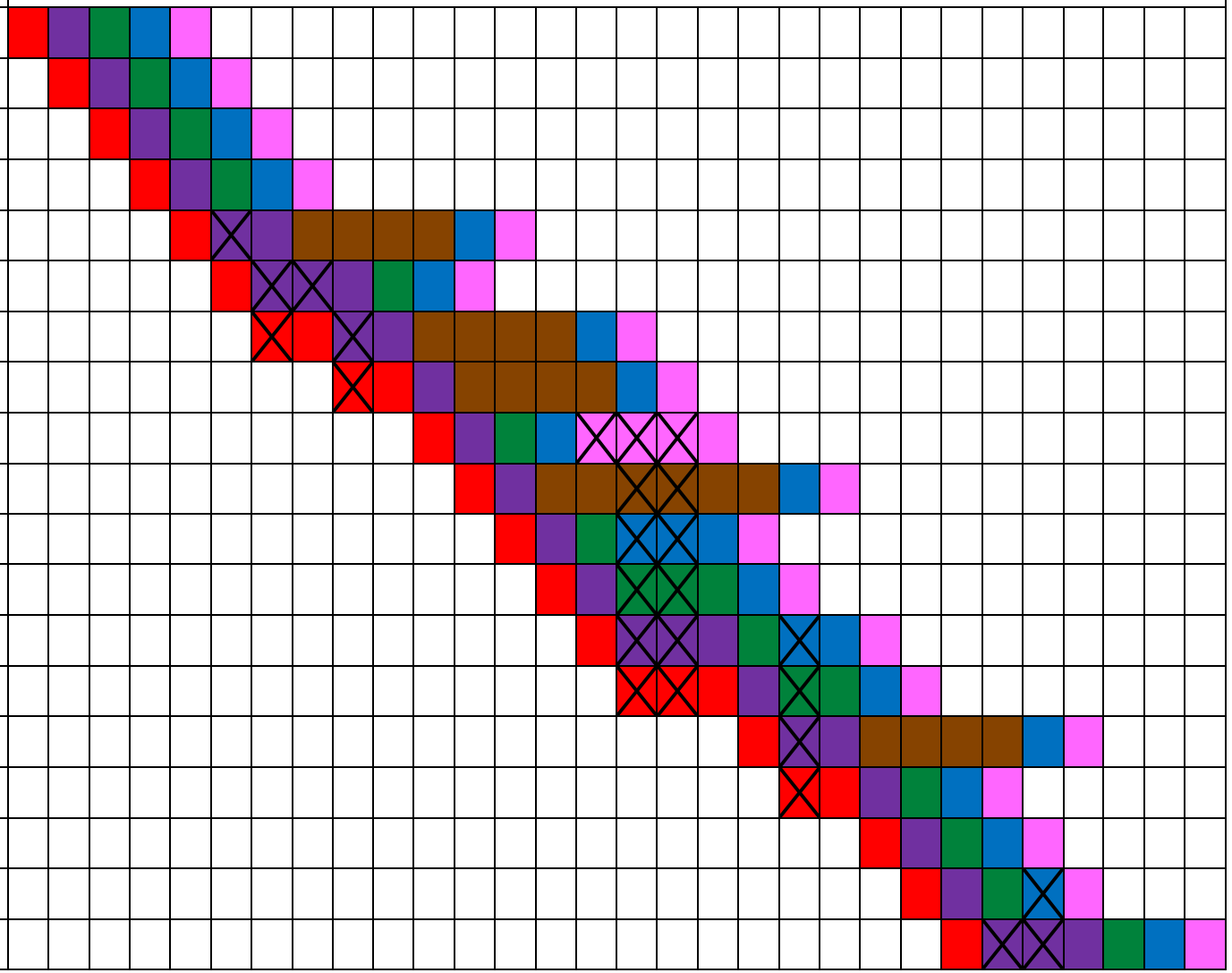
Instrucciones	Progreso de la ejecución del programa
lf f1, num3(r0)	
lf f2, num4(r0)	
lw r1, num1(r0)	
lw r2, num2(r0)	
addf f3, f1, f2	
ddiv r3, r2, r1	
ti f4, f2	
divf f5, f2, f1	
lf f5, num5(r0)	
ltf f1, f2	
orr r4, r3, r2	
dsub r5, r1, r3	
daddi r6, r2, 8	
mrf r7, f4	
mulf f6, f5, f3	
sw r3, res1(r0)	
sw r5, res2(r0)	
sf f5, res3(r0)	
halt	

Figura 97. Progreso de la ejecución segmentada de un programa dentro de un escenario distinto al de la figura anterior, cuyo atasco en el cauce del procesador en la última instrucción se debe ahora a otro tipo de incumplimiento de las condiciones mínimas de ejecución.

AGREGAR UN CUADRO CON UN RESUMEN DE LOS TIPOS DE RIESGOS REPRESENTADOS EN CADA FIGURA.

REVISAR QUE CADA ATASCO REPRESENTADO EN LAS FIGURAS COINCIDA CON EL EXPLICADO EN LA TEORÍA PARA CADA UNO EN LAS SUBSECCIONES ANTERIORES (3.1.8.1-3.1.8.6.)

SIMULAR TODOS LOS PROGRAMAS DE PRUEBA EN EL ACTIVE-HDL PARA COMPROBAR QUE ESTÉN BIEN ESCRITOS ANTES DE PASAR A LA SIGUIENTE SUBSECCIÓN.

3.1.9. Ejemplo de prueba completo

Una vez tenidas en cuenta todas las consideraciones previamente expresadas, se procederá a presentar el progreso teóricamente esperado a través del tiempo para la ejecución secuencial de un programa de prueba escrito en el lenguaje Assembler. La demostración de dicho progreso se asemejará en gran medida al ejemplo previamente ofrecido para mostrar la ejecución esperada a través del tiempo para la ejecución de algunas instrucciones del repertorio del procesador original (figura 34).

La diferencia más importante con respecto al caso ejemplificado en la figura 34 consiste en que, al encontrarse incompleta la descripción de aquel diseño (recordar que había varias funcionalidades y componentes fundamentales que no estaban incluidos en él), dichas instrucciones se ejecutaban en ese ejemplo en forma individual y aislada, sin poseer ninguna relación entre sí. En cambio, aquí todas las instrucciones ejecutadas pertenecen a un programa principal y cumplen una función particular dentro de él cuyo correcto cumplimiento depende

en varios casos del resultado obtenido a partir de la ejecución de una instrucción anterior. Esta dependencia adquirirá una importancia especial cuando llegue el momento de incorporar en la ejecución del programa la segmentación del cauce ya implementada en este procesador.

Además, aquí se ofrecerá el estado del contenido de cada uno de los dispositivos de almacenamiento incluidos en el diseño de esta computadora que pueden ser modificados durante la ejecución del programa: memoria de datos y banco de registros de uso general del procesador, para que el lector pueda visualizar el instante y la etapa en particular del ciclo de ejecución en la cual cada dispositivo es actualizado por la CPU mientras ejecuta una instrucción. También se incluirán los registros de uso particular que serán alterados por el mismo procesador ya que resultan necesarios para que la unidad de control identifique la denominación de la instrucción actualmente en ejecución (registro IR) y la unidad de búsqueda determine la dirección en memoria de la próxima instrucción a ejecutar (registro IP). No se tendrán en cuenta los registros cuyo contenido sea irrelevante para la ejecución de este programa y/o su funcionalidad se encuentre sin implementar o parcialmente implementada.

```
        .data
num1:   .word   20
num2:   .word   –80
res1:   .word   0
num3:   .float  –4.0
num4:   .float  1.5
res2:   .float  0.0
        .code
        lw r1, num1(r0)
        lw r2, num2(r0)
        lf f1, num3(r0)
        lf f2, num4(r0)
        dadd r3, r1, r2
        addf f3, f1, f2
        jmp fin
        subf f4, f1, f2
fin:    sw r3, res1(r0)
        sf f3, res2(r0)
```

halt

Figura 79. Programa de prueba propuesto para analizar la disposición de sus datos e instrucciones en la memoria principal de la nueva computadora.

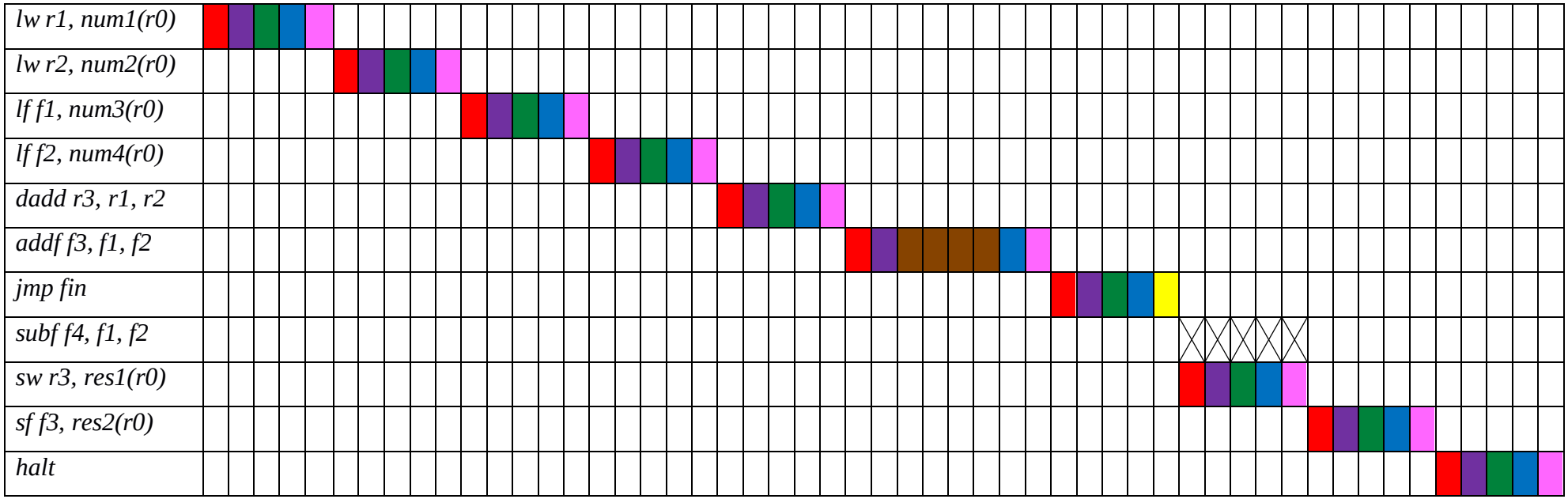


Figura 80. Progreso de la ejecución secuencial de las instrucciones del programa de prueba de la figura anterior en la nueva CPU a través del tiempo.

Observaciones:

- Cada columna de esta figura, obviamente exceptuando la inicial que contiene las denominaciones y operandos de las instrucciones ejecutadas del programa de prueba, representa el transcurso de un ciclo de reloj.
- Las celdas tachadas representan la cancelación de la ejecución de la instrucción "*subf f4, f1, f2*" debido a que previamente se ejecutó una instrucción de transferencia de control (salto incondicional), "*jmp fin*", la cual alteró el flujo de ejecución normal del programa, indicando que la próxima instrucción a ejecutar no sería aquella ubicada física e inmediatamente a continuación, es decir, "*subf f4, f1, f2*", sino la instrucción asociada a la etiqueta "fin", la cual vendría a ser "*sw r3, res1(r0)*".

Tabla 81. Colores utilizados para representar cada una de las etapas del ciclo de ejecución de las instrucciones del programa de prueba en la figura anterior.

Etapa	Color
1. Búsqueda	Rojo
2. Decodificación	Violeta
3.1. Ejecución	Verde
3.2. Ejecución en punto flotante	Marrón
4. Acceso a memoria	Azul
5. Almacenamiento en registro	Rosa

Como se habrá observado, la ejecución de las instrucciones del programa de prueba representada anteriormente se realizó en forma secuencial, es decir, cada instrucción se ejecutó en su totalidad una a continuación de la otra, sin tener en cuenta la división de su ciclo de ejecución en etapas separadas e independientes que permite la posibilidad de ejecutar múltiples instrucciones simultáneamente en distintas etapas. Cada instrucción se consideró como una única etapa en sí misma y se ejecutó como tal, desaprovechándose el potencial ofrecido por la segmentación del cauce para mejorar el rendimiento del procesador.

El próximo paso sería ejecutar nuevamente el mismo programa de prueba, pero ahora en forma segmentada, aprovechando la división del ciclo de ejecución de cada instrucción en etapas para comenzar en cada ciclo de reloj, si no existiera ningún inconveniente o conflicto (atascos), con la ejecución de una nue

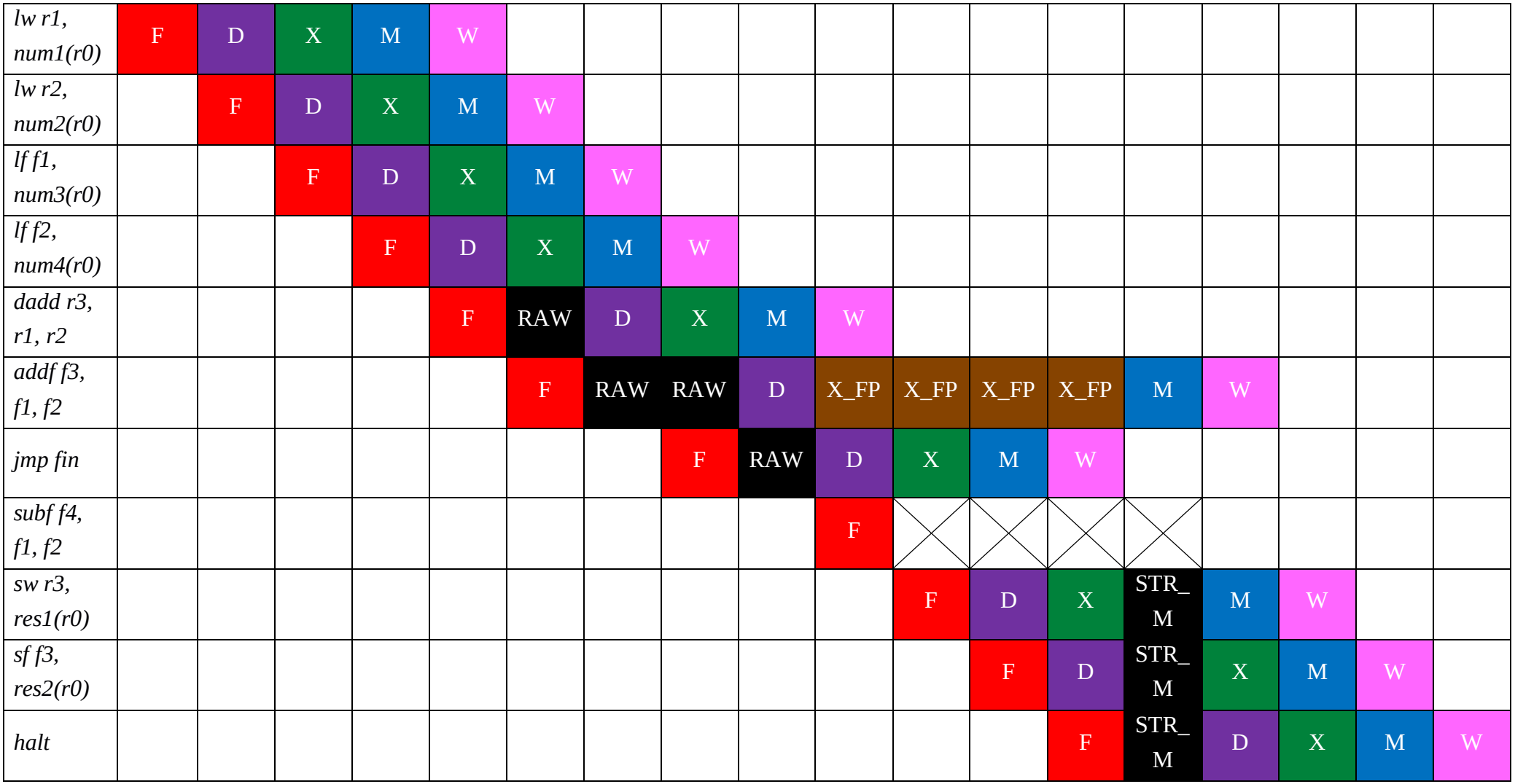


Figura 82. Progreso de la ejecución segmentada de las instrucciones del programa de prueba de la figura anterior en la nueva CPU a través del tiempo.

3.2. Diseño

3.2.1. Modelo jerárquico de entidades

3.2.2. Máquina de estados

Instrucciones canceladas con ejecución en punto flotante en la FPU: no usar el estado X_END sino los estados X_FP1_END, X_FP2_END, X_FP3_END y X_FP4_END. Implementar estas correciones en el proyecto VHDL.

3.3. Programación

3.3.1. Ejemplos

3.4. Simulación

3.4.1. Interfaz de usuario

3.4.2. Pruebas generales

3.4.3. Estructura de la memoria principal: Ejemplo

3.4.4. Riesgos de la segmentación del cauce: Ejemplos

3.4.5. Ejemplo de simulación final

4. Propuesta de enseñanza

4.1. Módulo 1

4.2. Módulo 2